

Preprocessing Security of (Nonzero) Short Schnorr Signatures and Hashed ElGamal KEM

Jeremiah Blocki¹ and Seunghoon Lee^{*2}

¹ Purdue University, West Lafayette, IN, USA
jblocki@purdue.edu

² University of Waterloo, Waterloo, ON, Canada
seunghoon.lee@uwaterloo.ca

Abstract. In modern cryptography, relatively few instantiations of foundational cryptographic primitives are used across most cryptographic protocols. This limited diversity raises concerns about preprocessing attacks, where an adversary with nation-state-level resources generates a hint that can later be exploited to break protocols built on these primitives. It is often notoriously challenging to analyze the advantage of a preprocessing attacker, even under idealized assumptions (e.g., random oracles, generic groups, etc.). Blocki and Lee (EUROCRYPT’22) analyzed the preprocessing security of *key-prefixed* short Schnorr signatures, where the random oracle is salted with the public key. However, their analysis did not extend to standardized implementations of Schnorr (e.g., ISO/IEC 14888-3), which do not adopt key-prefixing but disallow signatures that use a preimage of 0. The (in)security of such “(short) nonzero Schnorr signature” was left as an open question.

In this paper, we fully resolve this open question, demonstrating that (short) nonzero Schnorr signatures are also secure against preprocessing attacks in the Random Oracle Model (ROM) plus the Generic Group Model (GGM). We also introduce new techniques to analyze security of hashed ElGamal-based KEMs in the ROM+GGM and use these techniques to prove the CCA security of PSEC-KEM against preprocessing attacks. To support our analyses in the ROM+GGM, we extend Coretti et al.’s framework (CRYPTO/EUROCRYPT’18) to settings with multiple idealized primitives and use it in our proofs.

Keywords: Preprocessing Attacks · Short Schnorr Signatures · Hashed ElGamal KEM · Concrete Security · CCA Security · Bit-Fixing Model

1 Introduction

Many cryptographic protocols can be designed and analyzed from a handful of primitives. For example, the Schnorr signature scheme [Sch90] uses an elliptic curve group and a cryptographic hash function, while the simple oblivious transfer protocol [CO15] adds a block cipher. In practice, only a few instantiations of these primitives are widely used, e.g., AES, Triple DES, SHA-2/3, SHAKE, P-256/384, Curve25519/448. This raises the concern that an attacker with nation-state-level resources might attempt to execute a *preprocessing attack*, where a preprocessing attacker invests massive computational resources to generate a hint in the hope that this hint will help an online attacker quickly break cryptographic protocols that use the same instantiations of our primitives.

Blocki and Lee [BL22] analyzed the multi-user security³ of (short) Schnorr signatures against preprocessing attacks in the Random Oracle Model (ROM) [BR93] plus the Generic Group Model (GGM) [Nec94, Sho97], where they gained *short* signatures by truncating the hash output by

^{*} Work done partially while Seunghoon was at Purdue University.

³ We refer to the *1-out-of- N setting*, i.e., given N public keys, an adversary wins if it forges a valid signature under *any one* of these public keys.

half. In particular, they established the multi-user security of *key-prefixed* short Schnorr signatures⁴ against preprocessing attackers. However, their analysis involved a complicated compression argument and imposes a technical restriction: the preprocessing attacker can only examine the random oracle on a negligibly small fraction of inputs (e.g., $2^{3\lambda}$ out of the total possible $2^{4\lambda}$ input/output pairs). Furthermore, the analysis did not extend to the original Schnorr signature scheme *without key-prefixing* since this scheme is trivially broken by preprocessing attacks that can find a pre-image of 0. In particular, such an attacker can find (r, m) such that $H(g^r \| m) = 0$, making $\sigma = (r, 0)$ a universal forgery under *any* public key!

Standardized implementations of Schnorr signatures (e.g., BSI-TR-03111 [fIS18] and ISO/IEC 14888-3 [fSC18]) avoid this issue by disallowing signatures of form $\sigma = (\cdot, 0)$, and they do not adopt key-prefixing. We refer to such implementations as *nonzero Schnorr signatures* (see Definition 1 for the formal definition). Establishing the (in)security of (short) nonzero Schnorr signatures against preprocessing attacks was left as an open research direction [BL22].

The same ROM+GGM preprocessing setting arises for other cryptographic schemes that combine hashing with group operations. In particular, hashed ElGamal-style Key Encapsulation Mechanisms (KEMs) encapsulate by sending an ephemeral group element and deriving the shared secret by hashing a Diffie-Hellman value. Hashed ElGamal KEMs have been studied in the standard idealized model settings; for example, Ghoshal and Tessaro [GT20] showed the impossibility result of achieving memory-tight straight-line black-box reductions for the CCA security in the ROM+GGM. Nevertheless, to the best of our knowledge, prior work does not provide concrete preprocessing security bounds for hashed ElGamal-style KEMs in the ROM+GGM, i.e., bounds are explicitly parametrized by S and q , where S is a hint size generated during the preprocessing phase, and q is the number of online queries. One representative instantiation of this family is PSEC-KEM (Provably Secure Elliptic Curve-Key Encapsulation Mechanism; developed by NTT [Shi01, Sho01]).

This leads us to ask the following research questions:

- Are standardized implementations of Schnorr signatures (no key-prefixing, and signatures of form $(\cdot, 0)$ explicitly disallowed) secure against preprocessing attacks?
- Can we establish preprocessing security for hashed ElGamal KEMs in the ROM+GGM (e.g., PSEC-KEM)?

Coretti, Dodis, Guo, and Steinberger [CDGS18] introduced a comprehensive framework aimed at simplifying security analyses in the context of preprocessing in the ROM by proving improved bounds for Unruh’s pre-sampling technique [Unr07]. In particular, they showed how to translate the security in the Bit-Fixing ROM (BF-ROM) [Unr07] into the Auxiliary-Input ROM (AI-ROM). BF-ROM allows the preprocessing attacker to fix P random oracle (RO) input/output pairs and the remaining outputs are selected uniformly at random. This is less realistic, but it is often easier to upper-bound that attacker’s advantage⁵. In contrast, AI-ROM generates an S -bit hint during the preprocessing phase after arbitrary RO access. This is a more realistic model of real-world attacks, but proving query complexity lower bounds in this model often involves notoriously challenging compression arguments. Coretti et al. [CDGS18] showed that upper bounds for the attacker’s advantage in the BF-ROM translate to bounds in the AI-ROM with additive loss $O(Sq/P)$, where

⁴ Key-prefixed Schnorr signatures salt the random oracle with the user’s public key.

⁵ For example, consider the function inversion problem where we are given a random string $y \in \{0, 1\}^\lambda$ and the goal is to find an input x such that $f(x) = y$. In the BF-ROM, it is trivial to upper bound the attacker’s success probability as $(P + q)/2^\lambda$ where q denotes the number of RO queries made by the online attacker.

q denotes the number of online queries. As a concrete example, this immediately implies an upper bound of $O(\sqrt{Sq/2^\lambda})$ on the attacker’s success probability for the function inversion problem in the AI-ROM by setting $P = \sqrt{Sq2^\lambda}$. This matches state-of-the-art success probability bounds [ACDW20] for the function inversion problem obtained using complicated compression arguments.

Coretti et al. [CDGS18] extended the BF-ROM framework [CDGS18] to encompass other idealized primitives including the GGM, Ideal Cipher Model (ICM) [Sha49], and Random Permutation Model (RPM). They proved that BF-GGM (resp. BF-ICM/BF-RPM) bounds yield AI-GGM (resp. AI-ICM/AI-RPM) bounds with a similar additive loss $O(Sq/P)$.

Technically, the results in Coretti et al. [CDGS18, CDG18] are only stated for a *single* idealized primitive, while many cryptographic constructions utilize several primitives, e.g., (nonzero) Schnorr signatures and hashed ElGamal KEMs such as PSEC-KEM utilize elliptic curve groups and cryptographic hash functions. Thus, analyzing their security against preprocessing attacks requires extending the BF-to-AI framework to settings with *multiple* idealized primitives.

This leads us to ask the additional research question:

- Does Coretti et al.’s BF-to-AI framework extend to settings with multiple idealized models?

1.1 Our Contributions

1.1.1 Supporting Machinery: Extension of BF-to-AI Framework to Multiple Idealized Models.

To analyze our applications in multiple idealized models, such as ROM+GGM, we first verify that the results of Coretti et al. [CDGS18, CDG18] extend to settings with *multiple* idealized models. In particular, we verify that Bit-Fixing upper bounds yield upper bounds on the attacker’s success rate in the Auxiliary-Input model with an additive loss $O(Sq/P)$. Here, S denotes the size of the hint generated by the offline attacker in the Auxiliary-Input model, P denotes the total number of input/output pairs fixed by our Bit-Fixing attacker across all idealized primitives, and q denotes the total number of oracle queries made by the online attacker across all idealized models.

Note that in the Auxiliary-Input model, the S -bit hint could encode information about relationships between the various ideal oracles, e.g., the hint could identify two generic group elements that hash to the same value under our random oracle. Thus, we cannot simply apply the results of Coretti et al. [CDG18] separately to each idealized primitive in a black-box manner. Although conceptually close to prior analyses, this extension is not stated in prior work [CDGS18, CDG18] and is required for our applications that combine multiple idealized primitives.

Theorem 1 (informal). *If an application G is secure with probability ϵ' in the BF multiple idealized models (e.g., ROM+GGM+ICM) which fixes P coordinates of random oracles/generic group oracles/ideal ciphers in total, then it is also secure with probability ϵ in the corresponding AI multiple idealized model with an S -bit hint, with $\epsilon \leq \epsilon' + O(Sq/P)$ where q is the combined online query complexity corresponding to G across all idealized primitives used by G (see Theorem 5 and Theorem 12).*

While Theorem 1 applies more generally, Coretti et al. [CDGS18, CDG18] also established a tighter bound for *unpredictability applications* where a successful attacker must produce a particular target output (e.g., forged signature/MAC, preimage, hash collision, secret key) in contrast to a distinguishing attack where the attacker outputs a single bit. In unpredictability applications, we want to show that the attacker’s success probability is at most ϵ for some small/negligible value ϵ . By contrast, in an indistinguishability game, we want to show that the attacker guesses the

correct bit with probability at most $\frac{1}{2} + \varepsilon$ for some small/negligible ε . We verify that Coretti et al. [CDGS18, CDG18]’s result extends to our setting with multiple idealized models. This leads to tighter concrete bounds on the preprocessing security of short Schnorr signatures.

Theorem 2 (informal). *If an unpredictability application G is secure with probability ε' in the BF multiple idealized models (e.g., ROM+GGM+ICM) which fixes P coordinates of random oracles/generic group oracles/ideal ciphers in total, then it is also secure with probability ε in the corresponding AI multiple idealized model with an S -bit hint, with $\varepsilon \leq 2\varepsilon' + 2\gamma$, where γ satisfies $S + \log \gamma^{-1} \leq O(P/q)$ where q is the combined online query complexity corresponding to G across all idealized primitives used by G (see Theorem 6 and Theorem 13).*

In the formal statement of the theorem (Theorem 5, Theorem 6, Theorem 12, and Theorem 13), we establish slightly more refined bounds when we have separate upper bounds on the number of queries to each ideal oracle.

1.1.2 Multi-User Security of (Short) Nonzero Schnorr Signatures Against Preprocessing Attacks in the ROM+GGM. We resolve the open question of whether standardized (nonzero) short Schnorr signatures remain secure against preprocessing attacks by establishing concrete multi-user security bounds in the ROM+GGM. We first upper bound the success rate of a multi-user signature forgery attacker in the Bit-Fixing model (BF-ROM+GGM) and then leverage Theorem 2 to obtain upper bounds in the Auxiliary-Input model (AI-ROM+GGM) where the preprocessing attacker outputs an S -bit hint after performing an *unbounded* number of queries to the generic group oracles as well as the random oracles.

In the Bit-Fixing model, we are able to upper bound the attacker’s advantage as $O(qP/p + q/2^\lambda)$ where p is the size of our generic group and λ denotes the number of output bits for our random oracle. This bound holds for *both* key-prefixed short Schnorr signatures and for nonzero Schnorr signatures without key-prefixing. We note that this upper bound on the advantage of a bit-fixing attacker is *tight*, as a bit-fixing attacker can solve the discrete log problem with advantage $\Omega(qP/p)$. Now setting $P \approx Sq$ (which optimizes the BF-to-AI tradeoff), we can upper bound the advantage of an auxiliary-input attacker as $O(Sq^2/p + q/2^\lambda)$.

Theorem 3 (informal). *Any unbounded preprocessing attacker that outputs an S -bit hint during the preprocessing phase and makes at most q queries during the online phase wins the multi-user signature forgery game (chosen message attack) against the short nonzero Schnorr signature scheme with probability at most $O(Sq^2/p + q/2^\lambda)$ in the generic group model of order $p > 2^{2\lambda}$ plus programmable random oracle model (see Theorem 8). Essentially the same result holds for key-prefixed short Schnorr signatures as well (see Theorem 15).*

We remark that we analyze security under a *stronger multi-user unforgeability definition* where the attacker additionally has access to a corruption oracle $\text{Corrupt}(i)$ which reveals the secret key of user sk_i — to avoid triviality, we require that the forgery attacker produces a signature for an uncorrupted user.

We slightly oversimplified the above statement of Theorem 3 by hiding lower-order terms involving the number of users N^6 . Theorem 3 demonstrates that by suitably selecting parameters,

⁶ Terms involving N are lower-order terms unless the number of users is implausibly large, e.g., $N \geq \min\{2^\lambda, Sq\}$ where q is the total number of oracle queries made by the attacker.

these signatures can achieve λ bits of multi-user security⁷, even in the presence of preprocessing attacks. In particular, when setting $p \geq 2^{2\lambda}S$ and utilizing λ -bit hash outputs, the short nonzero Schnorr signatures maintain λ bits of multi-user preprocessing security since $Sq^2/p \leq q^2 2^{-2\lambda} \leq q 2^{-\lambda}$. For instance, if we choose $S = 2^{\lambda/2}$, then setting $p \approx 2^{2.5\lambda}$ results in signatures of length $\lambda + \log p \approx 3.5\lambda$ bits, which is shorter than that of regular Schnorr signatures (which require 4λ bits).

When it comes to key-prefixed short Schnorr signatures, previous work [BL22] gave an upper bound of $O(Sq^2 \log p/p + q/2^\lambda)$ for short key-prefixed Schnorr signatures. Compared to this bound, we stress that (1) our bound is *tighter*, and furthermore (2) our result is more general in that we do not impose any limitation on the running time of the offline attacker. We remark that one can actually achieve λ -bit security for slightly smaller group size, i.e., we require $p \approx 2^{2\lambda}S$ instead of requiring $p \approx 2^{2\lambda}S \log p$. This is due to the missing $\log p$ term, which reduces the required signature length to 3.5λ instead of $3.5\lambda + O(\log \lambda)$ [BL22].

1.1.3 CCA Security of Hashed ElGamal KEMs Against Preprocessing Attacks in the ROM+GGM:

PSEC-KEM as a Case Study. We introduce a new technique (the quadratic bridge-finding game) for analyzing CCA security of hashed ElGamal-style KEMs against preprocessing attacks in the ROM+GGM, and apply it to PSEC-KEM [Shi01, Sho01]. PSEC-KEM has several standardized implementations [Shi01, fSC06, 3rd05] and achieves provable security in the ROM plus the hardness assumption of the discrete logarithm problem [Men01, Shi01, CMV05], yet to the best of our knowledge, the preprocessing security of PSEC-KEM has not been explored. We prove that PSEC-KEM is CCA-secure in the BF-ROM+GGM and then lift the bound to AI-ROM+GGM via [Theorem 1](#). In the BF model, we upper bound the attacker's advantage as $\frac{1}{2} + O(qP/p + qP/2^{\lambda_1} + 1/2^{2\lambda})$ (see [Theorem 10](#)) where λ_1 specifies the length of certain hash inputs/outputs in PSEC-KEM and p is the size of our generic group. Setting $P \approx \sqrt{Sp}$ (which optimizes the BF-to-AI tradeoff), we obtain the following bound for the advantage of an AI attacker.

Theorem 4 (informal). *Any preprocessing attacker that is unbounded and outputs an S -bit hint during the preprocessing phase and makes at most q queries during the online phase wins the CCA indistinguishability game against PSEC-KEM with probability at most $\frac{1}{2} + O\left(q\sqrt{\frac{S}{p}} + \frac{q\sqrt{Sp}}{2^{\lambda_1}} + \frac{q\lambda \log p}{\sqrt{Sp}} + 2^{-2\lambda+1}\right)$, in the GGM of order $p > 2^{2\lambda}$ plus programmable ROM where λ_1 specifies the length of certain hash inputs/outputs in the PSEC-KEM protocol (see [Theorem 11](#)).*

A key ingredient in proving the preprocessing security of PSEC-KEM is a reduction to a new game called the *quadratic bridge-finding game*, where the goal of the attacker is to output a certain quadratic relationship between the unknown variables (see [Section 5.2](#)). We prove that the probability of winning the quadratic bridge-finding game for the bit-fixing attacker is upper bounded by $O(Pq/p)$ (see [Lemma 2](#)). [Theorem 4](#) implies that PSEC-KEM achieves λ bits of *preprocessing* security by setting $p \approx 2^{2\lambda}S$ and setting the hash output length $\lambda_1 \approx 2\lambda + \log S$. We expect this game to be useful for other hashed ElGamal-style KEMs with the same hashed-DH structure.

1.2 Related Work

Preprocessing Attacks and Lower Bounds. There has been extensive research on preprocessing attacks across various cryptographic primitives and problems, e.g., discrete-log problem [Che10, LCH11,

⁷ For “ λ -bit security” we require that for any running time parameter $t \leq 2^\lambda$ that any attacker running in time t succeeds with probability at most $O(t/2^\lambda)$.

BL13, CK18, CDG18, RS22], distributed discrete-log problem [HJK22], block ciphers [Hel80, Yao90, DTT10], Computational/Decisional Diffie-Hellman problem [CK18, RS22], salted Merkle-Damgård construction [CDGS18, ACDW20, AGL22, GK22, GT23], Sponge hashing [CDG18, FGK22], and Schnorr signatures [dR97, BL22]. For example, Corrigan-Gibbs and Kogan [CK18] proved that any preprocessing attacker succeeds in solving the discrete log problem with probability at most $\tilde{O}(Sq^2/p)$ where p is the size of the generic group, S is the size of the hint generated by the offline attacker, and q is the total number of online queries; they also gave a generic attack that matched this upper bound. The presampling framework of Coretti et al. [CDGS18, CDG18] is most directly related to our paper and is discussed in Section 3.

Short Schnorr Signatures. Schnorr’s original paper [Sch90] proposed the idea of shortening Schnorr signatures by truncating the hash output. The security of the short Schnorr signature scheme has been subsequently studied under a variety of assumptions/settings [BL22, NSW09, SJ00, KMP16], but only Blocki and Lee [BL22] consider preprocessing security. We compare our results with Blocki and Lee’s work [BL22] extensively throughout the paper.

Equivalence of ROM and ICM. The Ideal Cipher Model (ICM) [Sha49] and the Random Permutation Model (RPM) are widely used idealized models which often offer a simple framework to analyze the security of practical cryptographic protocols and constructions. It is known that the ROM and the ICM are equivalent by Coron et al. [CPS08] in a sense that one can build an ideal cipher from a random oracle and vice versa. In particular, Coron et al. proved that a 6-round Luby-Rackoff Feistel network is indistinguishable from a random permutation. Given that one can construct ideal ciphers from random oracles (or vice versa) one may wonder whether it is necessary to develop tools to analyze preprocessing security in multiple different idealized models. We note that the concrete security guarantee is not ideal as the distinguishing advantage is upper bounded as $O(q^{16}/2^\lambda)$. Dai and Steinburger [DS16] improved this bound to $O(q^8/2^\lambda)$, but one would still need to select a much larger security parameter, e.g., $\lambda = 8\lambda'$ achieves λ' -bit security. Furthermore, the indistinguishability results [CPS08, DS16] do not consider preprocessing attackers and it is unknown whether these indistinguishability results can be extended to the Bit-Fixing model or to the Auxiliary-Input model. Thus, it is still useful to develop tools to establish preprocessing security with multiple idealized models.

2 Preliminaries

Let $\mathbb{N}$ be the set of positive integers, and we define $[N] := \{1, \dots, N\}$ for $N \in \mathbb{N}$. Throughout the paper, we denote the security parameter by λ . We define $\mathbb{Z}_p$ to be a cyclic group of integers modulo p . We say that $\mathbf{x} = (x_1, \dots, x_N) \in \mathbb{Z}_p^N$ is an N -dimensional vector over $\mathbb{Z}_p^N$, and for each $i \in [N]$, we define $\hat{u}_i$ to be the i^{th} N -dimensional unit vector, i.e., the i^{th} element of $\hat{u}_i$ is 1, and all other elements are 0 elsewhere. For an N -dimensional vector $\mathbf{x} \in \mathbb{Z}_p^N$ and a subset $S \subseteq [N]$, $\mathbf{x}_S$ denotes a sub-vector of $\mathbf{x}$ of dimension $s := |S|$, i.e., if $\mathbf{x} = (x_1, \dots, x_N)$ then $\mathbf{x}_S = (x_{i_1}, \dots, x_{i_s})$ where $S = \{i_1, \dots, i_s\}$ with $i_1 < \dots < i_s$.

For simplicity, we let $\log(\cdot)$ be a log with base 2, i.e., $\log x := \log_2 x$. The notation $\leftarrow \$$ denotes a uniformly random sampling, e.g., we say $x \leftarrow \$ \mathbb{Z}_p$ when x is sampled uniformly at random from $\mathbb{Z}_p$.

The Random Oracle Model (ROM). The ROM is a type of an idealized model formalized by Bellare and Rogaway [BR93], in which we model a cryptographic hash function H as a truly random function. In particular, H can only be evaluated by *querying* an oracle that returns $H(x)$ when an input x is given. While there is no mathematically rigorous proof that a random oracle exists, the ROM remains an effective idealized model for analyzing schemes whose security crucially depends on how hash queries interact with the rest of the construction.

The Generic Group Model (GGM). The GGM is an idealized model proposed by Shoup [Sho97]. One key motivation of the GGM is that it can be used to analyze several computational hardness assumptions. In particular, Shoup [Sho97] proved that the discrete-log problem in a group of prime order p requires $\Omega(\sqrt{p})$ queries to solve, and the same lower bound holds for Computational Diffie-Hellman (CDH) problem and Decisional Diffie-Hellman (DDH) problem as well.

As the name suggests, the GGM only allows the adversary to have access to a random representation of a group which is done by a randomly chosen encoding (labeling) so that the adversary can only make the group operations in a black-box manner. More precisely, given a cyclic group $G = \langle g \rangle$ of prime order p , a random injective encoding (labeling) map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$, where $\mathbb{G}$ is the set of bit strings of length $\ell \geq \lceil \log p \rceil$, and we encode the discrete log of a group element instead of the group element itself for simplicity. In the GGM, the adversary is only given access to oracles $\text{Mult}(\cdot, \cdot)$ and $\text{Inv}(\cdot)$, which compute the group operation *indirectly* in G , as well as the encoding of the generator of G , i.e., $g = \tau(1)$. In particular, for $(a, b) \in \mathbb{G} \times \mathbb{G}$, the oracles work as $\text{Mult}(a, b) = \tau(\tau^{-1}(a) + \tau^{-1}(b))$ and $\text{Inv}(a) = \tau(-\tau^{-1}(a))$ if $a, b \in \tau(G)$.⁸

We remark that there are different GGMs; Schnorr and Jakobsson’s model [SJ00] where the attacker is not directly given a labeling map, Maurer’s model [Mau05] that uses pointers instead of random encoding of a group, and Kiltz et al.’s model [KMP16] in which the generic group oracle maintains a global counter. While those models are incomparable in general, Zhandry [Zha22] argued that Shoup’s model should be preferred as Maurer’s model fails to capture many generic techniques such as Merkle-Damgård transform, Feistel network, authenticated encryption, etc. Blocki and Lee [BL22] also noted that Schnorr and Jakobsson’s model and Kiltz et al.’s model do not provide a natural way to analyze preprocessing security since the attacker is never given bit encodings of group elements which could be processed outside of the generic group oracles. Because we are focused on preprocessing attacks, we will utilize Shoup’s GGM in this paper.

3 Background: Bit-Fixing/Auxiliary-Input ROM+GGM and Extension to Multiple Idealized Models

Coretti et al. [CDGS18, CDG18] introduced a breakthrough technique that generically translates the security bounds in the Bit-Fixing (BF) idealized model (e.g., ROM, GGM, ICM, or RPM) to the corresponding Auxiliary-Input (AI) model with optimal additive security loss. While prior work [CDGS18, CDG18] focused on a *single* idealized model, there are several cryptographic primitives that rely on *multiple* idealized models; for instance, the security analysis of short Schnorr signatures [BL22] relies on both ROM and GGM, and the security of a simple oblivious transfer protocol [CO15] even relies on three different idealized models — ROM, GGM, and ICM. Hence, it is useful to have a generic translation from the BF model with *multiple* idealized models to the

⁸ We can also define an oracle $\text{Pow}(a, n) := \tau(n\tau^{-1}(a))$ for convenience, which can be computed by making $O(\log n)$ oracle calls to Mult using the standard modular exponentiation algorithm.

AI model with the corresponding idealized models. In this section, we consider the ROM+GGM, which can be used to analyze the multi-user security of short nonzero Schnorr signatures against preprocessing attacks and the CCA security of a hashed ElGamal KEM (e.g., PSEC-KEM).

Informal Definition of the BF-ROM+GGM. We use $\text{BF-RO+GG}(P_1, P_2, m, \lambda, p, \ell)$ to denote the BF model with a random oracle $H : \{0, 1\}^m \rightarrow \{0, 1\}^\lambda$ and a generic group of size p with group elements encoded as ℓ -bit strings using a random injective map $\tau : \mathbb{Z}_p \rightarrow \{0, 1\}^\ell$ where $\ell \geq \lceil \log_2 p \rceil$. In this model, the preprocessing attacker $\mathcal{A}_{\text{pre}}$ may fix P_1 (resp. P_2) input/output pairs for the random oracle H (resp. injective map τ) and then output an S -bit hint for the online attacker $\mathcal{A}_{\text{on}}$ to use. However, the remaining input/output pairs are picked randomly (resp. randomly subject to the constraint that τ is still an injective map) and uncorrelated with the S -bit hint output by the preprocessing attacker. We say that an application is $(S, T_1, T_2, \varepsilon)$ -secure in the $\text{BF-RO+GG}(P_1, P_2, m, \lambda, p, \ell)$ if any attacker $\mathcal{A} = (\mathcal{A}_{\text{pre}}, \mathcal{A}_{\text{on}})$ wins the security game for our application with probability at most ε . Here, we constrain the online attacker to make at most T_1 (resp. T_2) random oracle (resp. generic group) queries. As before, we constrain the preprocessing attacker $\mathcal{A}_{\text{pre}}$ to fix at most P_1 (resp. P_2) points in the random oracle (resp. injective map τ) and to output a hint of length at most S bits. See [Section A.2](#) for a formal definition of the BF-ROM+GGM.

Informal Definition of the AI-ROM+GGM. We use $\text{AI-RO+GG}(m, \lambda, p, \ell)$ to denote the Auxiliary-Input model with a random oracle and a generic group that are defined in the same way as above in the BF-ROM+GGM. In this model, the preprocessing attacker $\mathcal{A}_{\text{pre}}$ may examine the entire input/output tables of H, τ and then output an S -bit hint for the online attacker $\mathcal{A}_{\text{on}}$. We say that an application is $(S, T_1, T_2, \varepsilon)$ -secure in the $\text{AI-RO+GG}(m, \lambda, p, \ell)$ if any preprocessing attacker $\mathcal{A} = (\mathcal{A}_{\text{pre}}, \mathcal{A}_{\text{on}})$ wins the security game for our application with probability at most ε . Here, we constrain the online attacker to make at most T_1 (resp. T_2) random oracle (resp. generic group) queries and, as before, we constrain the offline attacker $\mathcal{A}_{\text{pre}}$ to output an S -bit hint. See [Section A.2](#) for a formal definition of the AI-ROM+GGM.

Additive Error for Arbitrary Applications in the ROM+GGM. The main result considering BF-to-AI transition in the ROM+GGM is stated in [Theorem 5](#). The security bound references the combined query complexity of our application G . Intuitively, $(T_1)_G^{\text{comb}}$ (resp. $(T_2)_G^{\text{comb}}$) denotes the total number of random oracle (resp. generic group) queries made by the online attacker as well as by the challenger in the security game for G . In all of our applications, combined query complexity is not substantially larger than the query complexity of the online attacker, i.e., $(T_i)_G^{\text{comb}} = O(T_i)$ for $i = 1, 2$. The proof of [Theorem 5](#) can be found in [Appendix A](#).

Theorem 5. *For every $\gamma > 0$, if an application G is $(S, T_1, T_2, \varepsilon')$ -secure in $\text{BF-RO+GG}(P_1, P_2, m, \lambda, p, \ell)$ for any $P_1, P_2 \in \mathbb{N}$ such that $P_1 \lambda + P_2 \log(p/e) \leq \eta$, then it is $(S, T_1, T_2, \varepsilon)$ -secure in $\text{AI-RO+GG}(m, \lambda, p, \ell)$, for*

$$\varepsilon \leq \varepsilon' + \frac{2(S + \log \gamma^{-1}) ((T_1)_G^{\text{comb}} \lambda + 3(T_2)_G^{\text{comb}} \log p)}{\eta} + 2\gamma,$$

where e is the Euler constant, and $(T_1)_G^{\text{comb}}, (T_2)_G^{\text{comb}}$ are the combined query complexity corresponding to G across the random oracle and the generic group oracle.

Multiplicative Error for Unpredictability Applications in the ROM+GGM. Coretti et al. [[CDGS18](#), Theorem 6] further proved that for any *unpredictability* application, the security bound from the BF-ROM translates into the AI-ROM at the cost of a *multiplicative factor of 2*. This can lead to tighter

concrete preprocessing security bounds for unpredictability applications like the signature forgery game. Intuitively, an unpredictability application refers to security definitions where the attacker must generate a particular output (e.g., preimage, hash collision, forged signature/MAC) to win the game instead of simply outputting a single bit to win a distinguishing game. In an unpredictability game, the attacker's success probability (e.g., signature forgery) in the BF-ROM can typically be upper bounded by some small/negligible value ε . Thus, upper bounding probability of a successful attack in the AI-ROM as 2ε is quite meaningful. By contrast, in a distinguishing game, the attacker's success probability in the BF-ROM would be $\frac{1}{2} + \varepsilon$, so a multiplicative upper bound on the attacker's success rate is not meaningful. We verify that a similar BF-ROM+GGM to the AI-ROM+GGM translation can be applied to unpredictability applications — see [Theorem 6](#) and the proof in [Appendix A](#).

Theorem 6. *For every $\gamma > 0$, if an unpredictability application G is $(S, T_1, T_2, \varepsilon')$ -secure in BF-RO+GG $(P_1, P_2, m, \lambda, p, \ell)$ for any $P_1, P_2 \in \mathbb{N}$ satisfying $(S + \log \gamma^{-1}) \cdot ((T_1)_G^{\text{comb}} \lambda + 3(T_2)_G^{\text{comb}} \log p) \leq P_1 \lambda + P_2 \log(p/e)$, then it is $(S, T_1, T_2, \varepsilon)$ -secure in AI-RO+GG (m, λ, p, ℓ) , for $\varepsilon \leq 2\varepsilon' + 2\gamma$, where e is the Euler constant.*

We also extend this transition to leverage additional idealized models (e.g., ROM+GGM+ICM). See [Appendix B](#) for details.

4 Multi-User Security of Short Nonzero Schnorr Signatures Against Preprocessing Attacks

We apply the results from [Section 3](#) ([Theorem 5](#) and [Theorem 6](#)) to analyze the preprocessing security of short Schnorr signatures utilizing multiple idealized models. While Blocki and Lee [[BL22](#)] analyzed preprocessing security of *key-prefixed* short Schnorr signatures, several standardized implementations of Schnorr signatures, e.g., BSI-TR-03111 [[fIS18](#)] and ISO/IEC 14888-3 [[fSC18](#)], do not adopt key-prefixing but explicitly disallow $e = 0$ signatures (which we call *nonzero Schnorr signatures*, see [Definition 1](#)). In this section, we analyze the multi-user security of (short) nonzero Schnorr signatures.

Kg(1^λ):	Sign(sk, m):	Vfy(pk, m, σ):
1 : sk $\leftarrow \mathbb{Z}_p$	1 : r $\leftarrow \mathbb{Z}_p$	1 : Parse $\sigma = (s, e)$
2 : pk $\leftarrow g^{\text{sk}}$	2 : $I \leftarrow g^r$	2 : if $e = 0$ then
3 : return (pk, sk)	3 : $e \leftarrow H(I m)$	3 : return 0
	4 : while $e = 0$ do	4 : $R \leftarrow g^s \cdot \text{pk}^{-e}$
	5 : Resample $r \leftarrow \mathbb{Z}_p$	5 : if $H(R m) = e$ then
	6 : $I \leftarrow g^r$	6 : return 1
	7 : $e \leftarrow H(I m)$	7 : else return 0
	8 : $s \leftarrow r + \text{sk} \cdot e \pmod p$	
	9 : return $\sigma = (s, e)$	

Figure 1. The *nonzero* Schnorr signature scheme.

Definition 1. Given a cyclic group $G = \langle g \rangle$ and a prime p , the nonzero Schnorr signature scheme is a signature scheme that consists of a tuple of probabilistic polynomial-time algorithms $\Pi_{\setminus\{0\}} = (\text{Kg}, \text{Sign}, \text{Vfy})$, where each algorithm works as in Figure 1.

One could alternatively remove the loop in the signing procedure from Figure 1 and simply terminate and output $\perp$ with negligible probability. On the other hand, a more efficient way to generate a nonzero Schnorr signature is by outputting $e \leftarrow H(I\|m) + 1$ on line 3 of $\text{Sign}(\text{sk}, m)$ in Figure 1. By doing this, the range of e lies in $\{1, \dots, 2^{\lambda_1}\}$ and we have zero chance of $e = 0$. Then we do not even need to check if $e = 0$ in the verification. And since $p > 2^{2\lambda} \gg 2^{\lambda_1}$, we do not need to worry about the overflow when adding 1 to $H(I\|m)$.

Multi-User Security of Nonzero Schnorr Signatures under Corruption. We consider a stronger multi-user security notion than that considered in previous work [BL22]. In particular, we extend the signature forgery game to allow the attacker to access a *corruption oracle* $\text{Corrupt}(i)$ which reveals the secret key sk_i of user i . To win the game, the attacker needs to forge a signature for an *uncorrupted* user. We stress that adding the corruption oracle *only strengthens* the security notion, as the attacker could always choose to ignore it. Formally, fixing the signature scheme $\Pi = (\text{Kg}, \text{Sign}, \text{Vfy})$, the injective mapping $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ for a generic group oracle, a random oracle H , and an adversary $\mathcal{A}$, we consider the following security game:

The 1-out-of- N Generic Signature Forgery Game under Corruption $\text{SigForgeCor}_{\mathcal{A}, \Pi}^{\tau, H, N}(k)$:

1. $\text{Kg}(1^k)$ is run N times to obtain the public and the secret keys $(\text{pk}_i, \text{sk}_i)$ for each $i \in [N]$. Here, for each $i \in [N]$, sk_i is chosen randomly from the group $\mathbb{Z}_p$, where p is a $2k$ -bit prime, and $\text{pk}_i = \tau(\text{sk}_i)$.
2. Adversary $\mathcal{A}$ is given $(g = \tau(1), \text{pk}_1, \dots, \text{pk}_N, p)$, and initializes a set of corruption $C = \emptyset$. $\mathcal{A}$ has access to the following oracles:
 - the generic group oracles $\text{G0} = (\text{Mult}(\cdot, \cdot), \text{Inv}(\cdot))$,
 - the random oracle $H(\cdot)$,
 - the signing oracle $\text{Signing}(\cdot, \cdot)$ (without the secret key) where $\text{Signing}(i, m)$ returns $\text{Sign}(\text{sk}_i, m)$, and
 - the corruption oracle $\text{Corrupt}(\cdot)$ where $\text{Corrupt}(i)$ returns sk_i . In this case, we update the corruption set $C \leftarrow C \cup \{i\}$.

The experiment ends when the adversary outputs $(m, \sigma = (s, e))$.
3. $\mathcal{A}$ succeeds in forging a signature if and only if there exists some $j \in [N]$ such that (1) $j \in [N] \setminus C$, (2) $\text{Vfy}(\text{pk}_j, m, \sigma) = 1$, and (3) the query m was never submitted to the oracle $\text{Signing}(j, \cdot)$.

The output of the experiment is 1 when $\mathcal{A}$ succeeds; otherwise output 0.

Definition 2. Consider the generic group model with a labeling map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$. A signature scheme $\Pi = (\text{Kg}, \text{Sign}, \text{Vfy})$ is $(N, q_H, q_G, q_S, \varepsilon)$ -MU-UF-CMA secure (multi-user unforgeable against chosen message attack) under corruption if for every adversary $\mathcal{A}$ making at most q_H (resp. q_G, q_S) queries to the random (resp. generic group, signing) oracle, we have $\Pr[\text{SigForgeCor}_{\mathcal{A}, \Pi}^{\tau, H, N}(k) = 1] \leq \varepsilon$, where the randomness is taken over the selection of τ , the random coins of $\mathcal{A}$, the random coins of Kg , and the selection of random oracle H .

When it comes to the *preprocessing security*, we say that a signature scheme Π achieves the multi-user security against preprocessing attacks if for every adversary $\mathcal{A} = (\mathcal{A}_{\text{pre}}, \mathcal{A}_{\text{on}})$, Π is $(N, q_H, q_G, q_S, \varepsilon)$ -MU-UF-CMA secure against the *online* attacker $\mathcal{A}_{\text{on}}$. In the BF model, $\mathcal{A}_{\text{on}}$ will receive the information of P pre-fixed points from $\mathcal{A}_{\text{pre}}$, and in the AI model, $\mathcal{A}_{\text{on}}$ will receive an S -bit hint from $\mathcal{A}_{\text{pre}}$. Hence, to prove the preprocessing security, it is sufficient to upper bound the probability of the event $\text{SigForgeCor}_{\mathcal{A}_{\text{on}}, \Pi}^{\tau, H, N}(k) = 1$ for the online attacker $\mathcal{A}_{\text{on}}$.

Multi-User Bridge-Finding Game Under Corruption in the BF-GGM. Blocki and Lee [BL22] established the multi-user security of short Schnorr signatures against preprocessing attackers by providing a reduction from a game called the 1-out-of- N generic BRIDGE ^{N} -finding game. Given N inputs $\tau(x_1), \dots, \tau(x_N)$, the goal of the game is to find a non-trivial linear dependence among those inputs, i.e., find $a_1, \dots, a_N, b$ such that $\sum_{i=1}^N a_i x_i = b$, and $a_i \neq 0$ for at least one $i \in [N]$. The intuition is that the winning probability for the 1-out-of- N discrete log game is (approximately) the same as winning the 1-out-of- N generic BRIDGE ^{N} -finding game [BL22, Corollary 1]. Since the odds of winning the 1-out-of- N discrete log game is crucial to analyze the multi-user security of short Schnorr signatures, it is significant to analyze the probability to win the multi-user bridge-finding game.

In particular, analyzing the probability of winning the 1-out-of- N generic BRIDGE ^{N} -finding game in the bit-fixing generic group model is essential when we do the reduction in analyzing the multi-user security of short Schnorr signatures in the Bit-Fixing model. Hence, we consider the 1-out-of- N generic BRIDGE ^{N} -finding game under corruption in the BF-GGM, which is described formally below. This strengthens the definition of [BL22] BridgeChalCor by adding an oracle $\text{Reveal}(\cdot)$ to model the corruption/compromise of the secret value x_i of party i . Again, we note that our security definition is strictly stronger, as the attacker could simply ignore this new oracle.

The 1-out-of- N Generic BRIDGE ^{N} -Finding Game under Corruption $\text{BridgeChalCor}_{\mathcal{A}}^{\tau, N}(\lambda, \mathbf{x})$ with a Bit-Fixing Preprocessing Attacker $\mathcal{A} = (\mathcal{A}_{\text{pre}}^{\text{BF-GG}(P)}, \mathcal{A}_{\text{on}}^{\text{BF-GG}(P)})$:

Preprocessing Phase:

1. $\mathcal{A}_{\text{pre}}^{\text{BF-GG}(P)}$ takes as input $\mathbf{g} = \tau(1)$.
2. $\mathcal{A}_{\text{pre}}^{\text{BF-GG}(P)}$ fixes P input/output pairs of the map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$, i.e., $(t_1, \tau(t_1)), \dots, (t_P, \tau(t_P))$. (Note: the rest of the map τ is chosen uniformly at random as long as it remains to be injective.)

Online Phase:

3. The challenger initializes the list $\mathcal{L} = \{(\tau(1), \mathbf{0}, 1), (\tau(x_1), \hat{u}_1, 0), \dots, (\tau(x_N), \hat{u}_N, 0), (\tau(t_1), \mathbf{0}, t_1), \dots, (\tau(t_P), \mathbf{0}, t_P)\}$, $\mathbf{x} = (x_1, \dots, x_N)$, as well as the set of revealed indices $\text{Revealed} = \emptyset$.
4. The adversary $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$ is given $\mathbf{g} = \tau(1)$ and $\tau(x_i)$ for each $i \in [N]$.
5. $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$ is allowed to query the usual generic group oracles (Mult , Inv) as well as a “reveal” query $\text{Reveal}(\cdot)$ where $\text{Reveal}(i)$ reveals x_i .
 - (a) If $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$ ever submits any fresh element η which does not appear in $\mathcal{L}$ as input to a generic group oracle, then the challenger immediately queries $b_y = \text{DLog}(\eta)$, and adds the tuple $(\eta, \mathbf{0}, b_y)$ to the list $\mathcal{L}$.

- (b) Whenever $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$ submits a query η_1, η_2 to $\text{Mult}(\cdot, \cdot)$, we are ensured that there exist tuples $(\eta_1, \mathbf{a}_1, b_1), (\eta_2, \mathbf{a}_2, b_2) \in \mathcal{L}$. The challenger adds the tuple $(\text{Mult}(\eta_1, \eta_2), \mathbf{a}_1 + \mathbf{a}_2, b_1 + b_2)$ to $\mathcal{L}$.
 - (c) Whenever $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$ submits a query η to $\text{Inv}(\cdot)$, we are ensured that some tuple $(\eta, \mathbf{a}_y, b_y) \in \mathcal{L}$. The challenger adds the tuple $(\text{Inv}(\eta), -\mathbf{a}_y, -b_y)$ to $\mathcal{L}$.
 - (d) Whenever $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$ submits a query $i \in [N]$ to $\text{Reveal}(\cdot)$, the oracle returns x_i and update $\text{Revealed} \leftarrow \text{Revealed} \cup \{i\}$. Update each tuple $(\eta, \mathbf{a}, b) \in \mathcal{L}$ to $(\eta, \mathbf{a}', b')$ where $b' = b + a_i x_i$, $a'_i = 0$, and $a'_j = a_j$ for $j \neq i$ to ensure that $a_i = 0$ since x_i is now known.
6. If at any point in time we have a collision, i.e., two distinct tuples $(\eta, \mathbf{a}_1, b_1), (\eta, \mathbf{a}_2, b_2) \in \mathcal{L}$ with $(\mathbf{a}_1, b_1) \neq (\mathbf{a}_2, b_2)$, then the event BRIDGE^N occurs, and the output of the game is 1. If BRIDGE^N never occurs, then the output of the game is 0.

Lemma 1 gives the upper bound of the probability of winning the BRIDGE^N -finding game for a bit-fixing preprocessing attacker. One big advantage to analyzing the multi-user bridge game *with preprocessing* in the BF-GGM is that we can avoid the complicated compression argument as shown in prior work [BL22, Theorem 7] and follow a similar proof technique that was used in the analysis *without preprocessing* [BL22, Theorem 5] which is much simpler. One notable difference is that in the online phase, there are P extra tuples $(\tau(t_1), \mathbf{0}, t_1), \dots, (\tau(t_P), \mathbf{0}, t_P)$ in the list $\mathcal{L}$ when the challenger initializes it⁹. The bit-fixing attacker can only pre-fix P input/output pairs in the preprocessing phase and the hint for the online phase can only be dependent to those pairs, which means that after adding P extra tuples in the initialization of the online phase, it works identical to the bridge-finding game without preprocessing.

While we can largely utilize the same proof technique described in [BL22, Theorem 5], it is important to emphasize that it cannot be applied in a black-box manner. We follow similar proof technique in [BL22, Theorem 5] with two differences: (1) the initial size of the list $\mathcal{L}$ becomes $N + P + 1$ (instead of $N + 1$ in the prior case [BL22, Theorem 5]) due to P pre-fixed points, and (2) the $\text{Reveal}(\cdot)$ query was not part of definition of the game BridgeChalCor in [BL22]. However, we remark that the query $\text{Reveal}(\cdot)$ only removes variables from the linear constraints in $\mathcal{L}$ and does not introduce additional constraints on unrevealed variables. Thus, the final security bound in Equation (1) is not even impacted by this modification and we could derive

$$\Pr \left[\text{BridgeChalCor}_{\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}}^{\tau, N}(\lambda) = 1 \right] \leq \frac{q_G^{\text{on}}(N + P) + 3q_G^{\text{on}}(q_G^{\text{on}} + 1)/2}{p - (N + P + 3q_G^{\text{on}} + 1)^2 - N}. \quad (1)$$

However, an immediate concern arises as the group size p becomes larger compared to the prior work [BL22], resulting in longer signature lengths. Notably, there is a P^2 term in the denominator, indicating that the group size p should be at least P^2 . Considering the security loss of $O((S + \log \gamma^{-1})q_G^{\text{on}}/P)$ [CDG18, Theorem 1] that appears in the transition from the Bit-Fixing to the Auxiliary-Input model, we find that P must be $\Omega(2^\lambda S)$ to achieve λ bits of security. Consequently, $p = \Omega(P^2) = \Omega(2^{2\lambda} S^2)$. Hence, we come up with a longer signature length of $\log p + \lambda \geq 3\lambda + 2 \log S$ if we use the bound in Equation (1), when compared to the prior work that achieves λ bits of security with a Schnorr signature length of $3\lambda + \log S$ bits [BL22].

⁹ Note that the challenger picks $(x_1, \dots, x_N) \in \mathbb{Z}_p^N$ after such P pairs are fixed.

Consequently, a more sophisticated analysis is required to maintain the same signature length as demonstrated in [BL22] while ensuring the same security level. Recall that the bridge event occurs if we find two distinct tuples $(\eta_1, \mathbf{a}_1, b_1), (\eta_2, \mathbf{a}_2, b_2)$ such that $\eta_1 = \eta_2$ but $(\mathbf{a}_1, b_1) \neq (\mathbf{a}_2, b_2)$ which yields a non-trivial linear dependency on $\mathbf{x}$. Intuitively, we exploit the observation that if we pick two distinct tuples $(\eta_1, \mathbf{a}_1, b_1), (\eta_2, \mathbf{a}_2, b_2)$ such that $\mathbf{a}_1 = \mathbf{a}_2 = \mathbf{0}$ then we cannot produce the bridge event since otherwise we have a contradiction¹⁰. Since all the pre-fixed P tuples $(t_1, \mathbf{0}, \tau(t_1)), \dots, (t_P, \mathbf{0}, \tau(t_P))$ are of the form $(\cdot, \mathbf{0}, \cdot)$, we can get a tighter bound by splitting the list set $\mathcal{L}$ into two subsets $\mathcal{L}_0$ and $\mathcal{L}_1$ where $\mathcal{L}_0 := \{(\eta, \mathbf{a}, b) \in \mathcal{L} : \mathbf{a} = \mathbf{0}\}$ and $\mathcal{L}_1 := \{(\eta, \mathbf{a}, b) \in \mathcal{L} : \mathbf{a} \neq \mathbf{0}\}$ when analyzing the probability of the bridge event in the BF-GGM.

Lemma 1. *Let $p > 2^{2\lambda}$ be a prime number and $N \in \mathbb{N}$ be a parameter. Let $\mathcal{A} := (\mathcal{A}_{\text{pre}}^{\text{BF-GG}(P)}, \mathcal{A}_{\text{on}}^{\text{BF-GG}(P)})$ be a pair of bit-fixing generic algorithms with a labeling map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ such that $\mathcal{A}_{\text{pre}}^{\text{BF-GG}(P)}$ fixes P input/output pairs of the labeling map τ and $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$ makes at most $q_{\mathbb{G}}^{\text{on}} := q_{\mathbb{G}}^{\text{on}}(\lambda)$ queries to the generic group oracles. Then*

$$\Pr \left[\text{BridgeChalCor}_{\mathcal{A}}^{\tau, N}(\lambda) = 1 \right] \leq \frac{q_{\mathbb{G}}^{\text{on}}(N + P) + 3q_{\mathbb{G}}^{\text{on}}(q_{\mathbb{G}}^{\text{on}} + 1)/2}{p - 2P(N + 3q_{\mathbb{G}}^{\text{on}} + 1) - (N + 3q_{\mathbb{G}}^{\text{on}} + 1)^2 - N},$$

in the GGM of prime order p , where the randomness is taken over the selection of $x_1, \dots, x_N, \tau$ as well as any random coins of $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$.

Proof. Intuitively, we first consider the probability that the bridge event occurs, conditioning on the event that the bridge event has not yet occurred until the previous query, i.e., let $\overline{\text{BRIDGE}}_{<i}^N$ be the event that the event BRIDGE^N has not occurred until the $(i-1)^{\text{th}}$ query.

Before we even receive the output η_i , we already know the values $\mathbf{a}_i, b_i$ such that the tuple $(\eta_i, \mathbf{a}_i, b_i)$ will be added to $\mathcal{L}$. If $\mathcal{L}$ does already contain this *exact* tuple, then outputting η_i will not produce the event BRIDGE^N . If $\mathcal{L}$ does not already contain this tuple $(\eta_i, \mathbf{a}_i, b_i)$, then we are interested in the event B_i that some other tuple $(\eta_i, \mathbf{a}'_i, b'_i)$ has been recorded with $(\mathbf{a}'_i, b'_i) \neq (\mathbf{a}_i, b_i)$. Observe that B_i occurs if and only if there exists a tuple of the form $(\cdot, \mathbf{a}, b)$ with $(\mathbf{a} - \mathbf{a}_i) \cdot \mathbf{x} = b_i - b$ and $(\mathbf{a}, b) \neq (\mathbf{a}_i, b_i)$. Let $\overline{\text{Revealed}} = [N] \setminus \text{Revealed}$ denote the set of unrevealed variables. For each $r \in \overline{\text{Revealed}}$ we can almost view $x_r = \mathbf{x}[r]$ as a uniformly at random element from $\mathbb{Z}_p$ that has yet to be selected. Note that for all $r \in \overline{\text{Revealed}}$ and all $(\eta_i, \mathbf{a}'_i, b'_i) \in \mathcal{L}$ we have $\mathbf{a}'_i[r] = 0$. Thus, if we pick $\mathbf{x}_{\overline{\text{Revealed}}}$ randomly¹¹, the probability that $(\mathbf{a} - \mathbf{a}_i) \cdot \mathbf{x} = b_i - b$ would be $1/p$. However, we cannot quite view $\mathbf{x}_{\overline{\text{Revealed}}}$ as uniformly random due to the restrictions, i.e., because we condition on the event $\overline{\text{BRIDGE}}_{<i}^N$ we know that for any distinct tuples $(\eta_j, \mathbf{a}_j, b_j)$ and $(\eta_k, \mathbf{a}_k, b_k)$ that $\mathbf{a}_j \cdot \mathbf{x} + b_j \neq \mathbf{a}_k \cdot \mathbf{x} + b_k$ because $\eta_j \neq \eta_k$.

Consider sampling $\mathbf{x}_{\overline{\text{Revealed}}}$ uniformly at random subject to this restriction. Let $r \in \overline{\text{Revealed}}$ be an index such that $\mathbf{a}[r] - \mathbf{a}_i[r] \neq 0$ and suppose that $x_r = \mathbf{x}[r]$ is the last value sampled. We also define two subsets $\mathcal{L}_0, \mathcal{L}_1 \subseteq \mathcal{L}$ where $\mathcal{L}_0 := \{(\eta, \mathbf{a}, b) \in \mathcal{L} : \mathbf{a} = \mathbf{0}\}$ and $\mathcal{L}_1 := \{(\eta, \mathbf{a}, b) \in \mathcal{L} : \mathbf{a} \neq \mathbf{0}\}$. Now we observe that $\mathcal{L}_0 \cup \mathcal{L}_1 = \mathcal{L}$ and if we pick two distinct pairs $(\eta_1, \mathbf{a}_1, b_1)$ and $(\eta_2, \mathbf{a}_2, b_2)$ both from $\mathcal{L}_0$ we are guaranteed to have that $\mathbf{a}_1 \cdot \mathbf{x} + b_1 \neq \mathbf{a}_2 \cdot \mathbf{x} + b_2$ since $\mathbf{a}_1 = \mathbf{a}_2 = \mathbf{0}$, which implies that we are guaranteed to not cause any bridge event.

¹⁰ In particular, suppose that $(\eta_1, \mathbf{0}, b_1)$ and $(\eta_2, \mathbf{0}, b_2)$ cause a bridge event. Then it should be the case that $\eta_1 = \eta_2$ but $(\mathbf{0}, b_1) \neq (\mathbf{0}, b_2)$. But this is a contradiction because $\eta_1 = \eta_2$ implies that $\mathbf{0} \cdot \mathbf{x} + b_1 = \mathbf{0} \cdot \mathbf{x} + b_2$ which tells us that $b_1 = b_2$, resulting that $(\mathbf{0}, b_1) = (\mathbf{0}, b_2)$.

¹¹ Recall from Section 2 that $\mathbf{x}_{\overline{\text{Revealed}}}$ only contains x_j 's where x_j has not been revealed.

At this point, for each $r \in \overline{\text{Revealed}}$ we can view x_r as being drawn uniformly at random from a set of at least $p - (|\mathcal{L}|^2 - |\mathcal{L}_0|^2) - (N - 1)$ remaining values, subject to all of the restrictions. To see this, we observe the following:

- x_r must be distinct from x_j for all $j \in [r] \cap \overline{\text{Revealed}}$ since otherwise we would have had a bridge event, and
- each pair of distinct elements $(\eta_j, \mathbf{a}_j, b_j) \in \mathcal{L}$ and $(\eta_k, \mathbf{a}_k, b_k) \in \mathcal{L}$, we have the following cases:
 - if $(\eta_j, \mathbf{a}_j, b_j) \in \mathcal{L}_1$ and $(\eta_k, \mathbf{a}_k, b_k) \in \mathcal{L}_1$, then this pair eliminates at most one possible value of x_r , i.e., since η_j and η_k are distinct, we have $\mathbf{a}_j \cdot \mathbf{x} + b_j \neq \mathbf{a}_k \cdot \mathbf{x} + b_k$, which implies that $x_r \neq \left((b_k - b_j) - \sum_{i=1, i \neq j}^N (\mathbf{a}_k[i] - \mathbf{a}_j[i])x_i \right) (\mathbf{a}_j[r] - \mathbf{a}_k[r])^{-1}$,
 - if $(\eta_j, \mathbf{a}_j, b_j) \in \mathcal{L}_1$ and $(\eta_k, \mathbf{a}_k, b_k) \in \mathcal{L}_0$, then this pair eliminates at most one possible value of x_r , i.e., since η_j and η_k are distinct, we have $\mathbf{a}_j \cdot \mathbf{x} + b_j \neq \mathbf{0} \cdot \mathbf{x} + b_k$, which implies that $x_r \neq \left((b_k - b_j) + \sum_{i=1, i \neq j}^N \mathbf{a}_j[i]x_i \right) \mathbf{a}_j[r]^{-1}$,
 - if $(\eta_j, \mathbf{a}_j, b_j) \in \mathcal{L}_0$ and $(\eta_k, \mathbf{a}_k, b_k) \in \mathcal{L}_1$, then this pair eliminates at most one possible value of x_r , i.e., since η_j and η_k are distinct, we have $\mathbf{0} \cdot \mathbf{x} + b_j \neq \mathbf{a}_k \cdot \mathbf{x} + b_k$, which implies that $x_r \neq \left((b_k - b_j) - \sum_{i=1, i \neq j}^N \mathbf{a}_k[i]x_i \right) (-\mathbf{a}_k[r])^{-1}$, and
 - if both $(\eta_j, \mathbf{a}_j, b_j) \in \mathcal{L}_0$ and $(\eta_k, \mathbf{a}_k, b_k) \in \mathcal{L}_0$, then since both $\mathbf{a}_j = \mathbf{a}_k = \mathbf{0}$, it is a contradiction that $\mathbf{a}_j[r] - \mathbf{a}_k[r] \neq 0$ and therefore we do not eliminate any value of x_r .
- Hence, the total number of values that we remove when we draw x_r is at most $(r - 1) + |\mathcal{L}_1|^2 + 2|\mathcal{L}_0||\mathcal{L}_1| \leq (N - 1) + (|\mathcal{L}|^2 - |\mathcal{L}_0|^2)$.
- We remark that queries to the oracle $\text{Reveal}(\cdot)$ do not increase the size of $\mathcal{L}$ and do not introduce new exclusions on x_r for each $r \in \overline{\text{Revealed}}$.

We also observe that $|\mathcal{L}| \leq N + P + 3q_G^{\text{on}} + 1$ and $|\mathcal{L}_0| \geq P$ since the list already contains P additional pre-fixed pairs and each generic group oracle query adds *at most* three new tuples to $\mathcal{L}$ — exactly three in the case that we query $\text{Mult}(\eta_1, \eta_2)$ on two fresh elements. Thus, the probability that $(\mathbf{a} - \mathbf{a}_i) \cdot \mathbf{x} = b_i - b$ is at most $\frac{1}{p - (N + P + 3q_G^{\text{on}} + 1)^2 + P^2 - (N - 1)}$. Union bounding over all tuples $(\cdot, \mathbf{a}, b) \in \mathcal{L}$, we have

$$\begin{aligned} \Pr \left[B_i : \overline{\text{BRIDGE}}_{<i}^N \right] &\leq \frac{N + P + 3i}{p - (N + P + 3q_G^{\text{on}} + 1)^2 + P^2 - N} \\ &= \frac{N + P + 3i}{p - 2P(N + 3q_G^{\text{on}} + 1) - (N + 3q_G^{\text{on}} + 1)^2 - N}. \end{aligned}$$

To complete the proof, we observe that

$$\begin{aligned} \Pr \left[\text{BridgeChalCor}_{\mathcal{A}}^{\tau, N}(\lambda) = 1 \right] &= \sum_{i \leq q_G^{\text{on}}} \Pr \left[B_i : \overline{\text{BRIDGE}}_{<i}^N \right] \\ &\leq \sum_{i \leq q_G^{\text{on}}} \frac{N + P + 3i}{p - 2P(N + 3q_G^{\text{on}} + 1) - (N + 3q_G^{\text{on}} + 1)^2 - N} \\ &= \frac{q_G^{\text{on}}(N + P) + 3q_G^{\text{on}}(q_G^{\text{on}} + 1)/2}{p - 2P(N + 3q_G^{\text{on}} + 1) - (N + 3q_G^{\text{on}} + 1)^2 - N}. \quad \square \end{aligned}$$

4.1 Multi-User Security of Nonzero Schnorr Signatures in the BF-ROM+GGM

Theorem 7. Let $\Pi_{\setminus\{0\}} = (\text{Kg}, \text{Sign}, \text{Vfy})$ be a nonzero Schnorr signature scheme and $p > 2^{2\lambda}$ be a prime number. Let $N \in \mathbb{N}$ be a parameter and $(\mathcal{A}_{\text{Sig.pre}}^{\text{BF-ROM+GG}(P_1, P_2)}, \mathcal{A}_{\text{Sig.on}}^{\text{BF-ROM+GG}(P_1, P_2)})$ be a pair of bit-fixing generic algorithms with a labeling map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ such that $\mathcal{A}_{\text{Sig.pre}}^{\text{BF-ROM+GG}(P_1, P_2)}$ fixes P_1 input/output pairs of a random oracle $H : \{0, 1\}^* \rightarrow \{0, 1\}^{\lambda_1}$ and P_2 input/output pairs of the map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ such that $2P_1 + P_2 = P$, and the hint $\text{str}_{\tau, H}$ is only dependent on those P points. If $\mathcal{A}_{\text{Sig.on}}^{\text{BF-ROM+GG}(P_1, P_2)}$ makes at most $q_G^{\text{on}} := q_G^{\text{on}}(\lambda)$ queries to the generic group oracles, q_H^{on} queries to the random oracle, and q_S^{on} queries to the signing oracle, then $\Pr \left[\text{SigForgeCor}_{\mathcal{A}_{\text{Sig.on}, \text{str}_{\tau, H}}^{\text{BF-ROM+GG}(P_1, P_2)}, \Pi_{\setminus\{0\}}^{\text{short}}}(\lambda) = 1 \right] \leq \varepsilon$, with

$$\begin{aligned} \varepsilon = & \frac{q_G^{\text{on}}(N + P) + 3q_G^{\text{on}}(q_G^{\text{on}} + 1)/2}{p - 2P(N + 3q_G^{\text{on}} + 1) - (N + 3q_G^{\text{on}} + 1)^2 - N} + \frac{q_S^{\text{on}}(q_H^{\text{on}} + q_S^{\text{on}} + P)}{p} \\ & + \frac{q_H^{\text{on}} + q_S^{\text{on}} + P}{p - (N + P + 3q_G^{\text{on}} + 1)} + \frac{2q_H^{\text{on}} + 1}{2^{\lambda_1}}, \end{aligned}$$

where the randomness is taken over the selection of τ and the random coins of $\mathcal{A}_{\text{Sig.on}}^{\text{BF-ROM+GG}(P_1, P_2)}$.

Proof Sketch. Here, we only give a proof sketch (see [Appendix G.1](#) for the full proof). Given a bit-fixing adversary $\mathcal{A} = (\mathcal{A}_{\text{pre}}, \mathcal{A}_{\text{on}})$ attacking $\Pi_{\setminus\{0\}}$, we build an efficient algorithm $\mathcal{B} = (\mathcal{B}_{\text{pre}}, \mathcal{B}_{\text{on}})$ which tries to win the 1-out-of- N generic BRIDGE^N-finding game $\text{BridgeChalCor}_{\mathcal{B}}^{\tau, N}(\lambda)$ in the BF model. Here, $\mathcal{B}_{\text{pre}}$ can simply run $\mathcal{A}_{\text{pre}}$ and outputs the same P_1 input/output pairs of H and P_2 input/output pairs of τ being fixed by $\mathcal{A}_{\text{pre}}$. For each fixed RO input/output pair $(h_i, H(h_i))$ for $i \in [P_1]$, $\mathcal{A}_{\text{pre}}$ can parse $h_i = I'_i \| m'_i$ where I'_i is a bitstring of length ℓ . If I'_i is a fresh ℓ -bit string (i.e., $I'_i \neq \tau(t_j)$ for all $j \in [P_2]$), then $\mathcal{A}_{\text{pre}}$ picks a value for $\tau^{-1}(I'_i) =: r'_i$ since the labeling map τ has not been sampled yet in the preprocessing phase. Then $\mathcal{A}_{\text{pre}}$ also outputs the fixed input/output pairs of τ for those values. Then $P_1 + (P_1 + P_2) = P$ points have been fixed by $\mathcal{A}_{\text{pre}}$ in total. Now the challenger initializes the list $\mathcal{L}$ that consists of tuples of the form $(\tau(y), \mathbf{a}, b) \in \mathbb{G} \times \mathbb{Z}_p^N \times \mathbb{Z}_p$ including the initial N tuples of unknowns and incorporating the information about P fixed points.

Now the online attacker $\mathcal{B}_{\text{on}}$ receives this information as hint and run $\mathcal{A}_{\text{on}}$ with access to the random oracle H , the generic group oracles (Mult , Inv), and the signing oracle. $\mathcal{L}$ is updated as $\mathcal{A}_{\text{on}}$ makes queries to those oracles.

If $\mathcal{A}_{\text{on}}$ submits a query $\text{Corrupt}(i)$, then $\mathcal{B}_{\text{on}}$ obtains $x_i = \text{Reveal}(i)$ from the bridge challenger, forwards the response to $\mathcal{A}_{\text{on}}$ and updates every tuple $(\eta, \mathbf{a}, b) \in \mathcal{L}$ to $(\eta, \mathbf{a}', b')$ where $b' = b + a_i x_i$, $a'_i = 0$, and $a'_j = a_j$ for $j \neq i$ to ensure that $a_i = 0$ since the secret key x_i is now known.

Whenever $\mathcal{A}_{\text{on}}$ submits a query m_i to the signing oracle $\text{Signing}(j, \cdot)$, the attacker $\mathcal{B}_{\text{on}}$ first checks to see if user $j \in \text{Revealed}$ (or equivalently $j \in \mathcal{C}$). If j has been compromised then $\mathcal{B}_{\text{on}}$ simply uses the secret key x_j to run the honest signing algorithm and obtain a signatures $\sigma_i = (s_i, e_i)$ to send to $\mathcal{A}_{\text{on}}$ i.e., pick r_i randomly, compute $I_i = \text{Pow}(\mathbf{g}, r_i) = \tau(r_i)$, $e_i = H(I_i \| m_i)$ and $s_i = r_i + x_j e_i$. If $j \notin \text{Revealed}$ then $\mathcal{B}_{\text{on}}$ simulates the signing oracle without knowledge of the secret key x_j using the programmability of the random oracle, i.e., to sign a message m_i , we can pick s_i and e_i randomly, compute $I_i = \tau(s_i - x_j e_i) = \text{Mult}(\text{Pow}(\mathbf{g}, s_i), \text{Pow}(\tau(x_i), -e_i))$ and see if the random oracle has previously queried at $H(I_i \| m_i)$. If not, then we can program the random oracle as $H(I_i \| m_i) = e_i$ and output the signature (s_i, e_i) . Otherwise, the reduction simply outputs $\perp$ for failure. Since s_i and e_i are selected randomly, the probability for outputting $\perp$ is at most $\frac{q_H^{\text{on}} + q_S^{\text{on}} + P_1}{p}$.

because I_i can be viewed as a uniformly random string chosen from a set of size p and there are at most $q_H^{\text{on}} + q_S^{\text{on}} + P_1$ fixed input/output pairs — each query to the signing oracle generates one query to H . Union bounding over all q_S^{on} queries to the signing oracle the probability of outputting $\perp$ during any signing query is at most $\frac{q_S^{\text{on}}(q_H^{\text{on}} + q_S^{\text{on}} + P_1)}{p}$.

If $\mathcal{A}_{\text{on}}$ wins the signature forgery game then it must output a message m_{i^*} and a forged signature $\sigma_{i^*} = (s_{i^*}, e_{i^*})$ for this message which validates for some user $i^* \notin C$. Once $\mathcal{A}_{\text{on}}$ outputs σ_{i^*} and m_{i^*} our algorithm $\mathcal{B}_{\text{on}}$ will compute $\tau(-e_{i^*}x_{i^*}) = \text{Pow}(\tau(x_{i^*}), -e_{i^*})$, $s_{i^*} = \text{Pow}(g, s_{i^*})$, and $I_{i^*} = \text{Mult}(s_{i^*}, \tau(-e_{i^*}x_{i^*}))$, which ensures that $(I_{i^*}, -e_{i^*}\hat{u}_{i^*}, s_{i^*}) \in \mathcal{L}$ and see if we have a bridge event. One failure event is if I_{i^*} was not previously recorded in $\mathcal{L}$ so that we do not produce a bridge event. Another failure event is if I_{i^*} was previously recorded in $\mathcal{L}$ but the exact same tuple $(I_{i^*}, -e_{i^*}\hat{u}_{i^*}, s_{i^*})$ was in $\mathcal{L}$. We can prove that the probability of both failure events is also small, i.e., $\leq \frac{q_H^{\text{on}} + q_S^{\text{on}} + P}{p - (N + P + 3q_G^{\text{on}} + 1)} + \frac{2q_H^{\text{on}} + 1}{2^{\lambda_1}}$.

If those failure events do not occur, then BRIDGE^N occurs and $\mathcal{B}$ wins the multi-user bridge-finding game. By applying [Lemma 1](#), we get the desired result. $\square$

4.2 From Bit-Fixing Model to Auxiliary-Input Model

By applying [Theorem 6](#), we can address the multi-user security of *nonzero Schnorr signatures* against preprocessing attacks in the AI-ROM+GGM.

Theorem 8. Let $\Pi_{\setminus\{0\}} = (\text{Kg}, \text{Sign}, \text{Vfy})$ be a nonzero Schnorr signature scheme, $p > 2^{2\lambda}$ be a prime number, and $\gamma > 0$ be a parameter. Let $N \in \mathbb{N}$ be a parameter and $(\mathcal{A}_{\text{Sig.pre}}^{\text{AI-ROM+GG}}, \mathcal{A}_{\text{Sig.on}}^{\text{AI-ROM+GG}})$ be a pair of generic algorithms with a labeling map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ such that $\mathcal{A}_{\text{Sig.pre}}^{\text{AI-ROM+GG}}$ outputs an S -bit hint $\text{str}_{H,\tau}$. If $\mathcal{A}_{\text{Sig.on}}^{\text{AI-ROM+GG}}$ takes $\text{str}_{H,\tau}$ as input and makes at most $q_G^{\text{on}} := q_G^{\text{on}}(\lambda)$ queries to the generic group oracles, q_H^{on} queries to the random oracle, and q_S^{on} queries to the signing oracle, then $\Pr \left[\text{SigForgeCor}_{\mathcal{A}_{\text{Sig.on}}^{\text{AI-ROM+GG}}, \text{str}_{H,\tau}, \Pi_{\setminus\{0\}}^{\text{short}}}^{\tau, H, N}(\lambda) = 1 \right] \leq \varepsilon$, with

$$\varepsilon = \frac{2q_G^{\text{on}}(N + Sq) + 3q_G^{\text{on}}(q_G^{\text{on}} + 1)}{p - 2Sq(N + 3q_G^{\text{on}} + 1) - (N + 3q_G^{\text{on}} + 1)^2 - N} + \frac{2q_S^{\text{on}}(q_H^{\text{on}} + q_S^{\text{on}} + Sq)}{p} + \frac{2q_H^{\text{on}} + 2q_S^{\text{on}} + 2Sq}{p - (N + Sq + 3q_G^{\text{on}} + 1)} + \frac{2q_H^{\text{on}} + 1}{2^{\lambda_1 - 1}} + 2^{-2\lambda + 1},$$

where $q := q_H^{\text{on}} + q_G^{\text{on}} + q_S^{\text{on}}$ and the randomness is taken over the selection of τ and the random coins of $\mathcal{A}_{\text{Sig.on}}^{\text{AI-ROM+GG}}$.

Proof. This is straightforward by combining [Theorem 7](#) with [Theorem 6](#) with setting $P \approx Sq$ and $\gamma = 2^{-2\lambda}$. $\square$

Tightness of the Bound. We emphasize that the bound in [Theorem 8](#) is *tight*. Under the reasonable assumption that $N \ll q$ and $6Sq^2 \ll p$, the dominating terms are $O(Sq^2/p)$ and $O(q_H^{\text{on}}/2^{\lambda_1})$. Blocki and Lee [BL22] observed that a preprocessing attacker can solve one of the N discrete-log challenges with probability at least $\Omega(S(q_G^{\text{on}})^2/p)$ which enables recovery of a secret key and trivial signature forgery. This matches the $O(Sq^2/p)$ term from our bounds. Furthermore, one could also consider the following attack: pick random nonzero values of s, e then search for a message m for which (s, e) is a valid forgery, i.e., fix s, e , compute $R = g^s \cdot \text{pk}^{-e}$, and search for m such that $H(R||m) = e$. This attack succeeds with probability $\Omega(q_H^{\text{on}}/2^{\lambda_1})$ showing that the $O(q_H^{\text{on}}/2^{\lambda_1})$ term is necessary.

Signature Length for Nonzero Short Schnorr Signatures. To achieve λ bits of multi-user security for short Schnorr signatures with preprocessing, we can fix p such that $p \approx 2^{2\lambda}S$, and set the length of our hash output to be $\lambda_1 = \lambda + 2$. With these parameters, [Theorem 8](#) tells us that a preprocessing attacker in the AI-ROM+GGM wins the signature forgery game with probability at most $\varepsilon = O(q/2^\lambda)$. The length of the signatures we obtain will be $\lambda + 2 + \log p = 3\lambda + 2 + \log S$. As a concrete example, if $S \leq 2^{9\lambda/16}$, then we obtain signatures of length $\approx 3.5625\lambda$ while regular Schnorr signatures are 4λ bits. If we want to have $\lambda \geq 128$ bits of security, then the assumption that $S \leq 2^{9\lambda/16}$ seems quite reasonable since 2^{72} bits exceeds the current storage capacity of Meta’s data warehouse. As a second example, if we take $S \leq 2^{80}$ as an even more conservative upper bound on the storage capacity of any nation-state attacker, then we obtain signatures of length $\approx 3\lambda + 80$.

4.3 Practical Parameters for Short Nonzero Schnorr Signatures

Practical Parameters for Fixed Implementations. Consider a fixed implementation of a nonzero Schnorr signature with an elliptic group of size $p \approx 2^{384}$, e.g., P-384. Suppose that a preprocessing attacker utilizes a hint of size $S \approx 2^{72}$. We remark that 2^{72} plausibly exceeds the storage capacity of Meta’s data warehouse which was 300 petabytes in 2014¹² and has now reached the exabytes scale¹³. If an online attacker makes $\leq q \approx 2^{80}$ queries, [Theorem 8](#) tells us that the success probability of forging a signature will be $\approx 2Sq^2/p \approx 2^{-151}$. [Table 1](#) shows the (approximate) upper bound on the probability that a preprocessing attacker wins the 1-out-of- N signature forgery game for nonzero Schnorr signatures. [Table 1](#) highlights that to break 128-bit security for nonzero Schnorr signatures, one would need to consider preprocessing attacks with the hint size unreasonably large which grows to the yottabyte scale ($\approx 10^{24} \approx 2^{80}$) and making 2^{90} online queries with massive computational power. However, we could even prevent this vulnerability by using larger group sizes (e.g., $p \geq 2^{384}$) which ensures that even nation-state-level attackers with significant computational resources cannot compromise nonzero Schnorr signatures except for negligibly small probability, even considering preprocessing.

S	2^{64}			2^{72}			2^{80}		
q	2^{80}	2^{85}	2^{90}	2^{80}	2^{85}	2^{90}	2^{80}	2^{85}	2^{90}
Attacker Success Probability (Approximate)									
$p = 2^{256}$	2^{-31}	2^{-21}	2^{-11}	2^{-23}	2^{-13}	2^{-3}	2^{-15}	2^{-5}	≈ 1
$p = 2^{384}$	2^{-159}	2^{-149}	2^{-139}	2^{-151}	2^{-141}	2^{-131}	2^{-143}	2^{-133}	2^{-123}
$p = 2^{448}$	2^{-223}	2^{-213}	2^{-203}	2^{-215}	2^{-205}	2^{-195}	2^{-207}	2^{-197}	2^{-187}
$p = 2^{512}$	2^{-287}	2^{-277}	2^{-267}	2^{-279}	2^{-269}	2^{-259}	2^{-271}	2^{-261}	2^{-251}

Table 1. Security bounds of a nonzero Schnorr signature for different values of S (size of the hint), q (total query complexity), and p (group size).

¹² See the link: <https://engineering.fb.com/2014/04/10/core-data/scaling-the-facebook-data-warehouse-to-300-pb/>

¹³ See the link: <https://medium.com/@AnalyticsAtMeta/data-engineering-at-meta-high-level-overview-of-the-internal-tech-stack-a200460a44fe>

Elliptic Curve Groups for Desired Security Levels. To achieve λ bits of security for nonzero short Schnorr signatures against preprocessing attackers, we need to instantiate with bigger elliptic curve groups than against attackers without preprocessing. While NIST recommends using elliptic curve groups with size 2λ bits (i.e., $p \approx 2^{2\lambda}$) to achieve λ bits of security, one would need to fix the group size p such that $p \approx 2^{2\lambda}S$ for λ bits of security against *preprocessing attackers*. For example, to achieve 192 bits of security, NIST recommends to use elliptic curves of size ≥ 384 bits; for preprocessing security against an attacker with an $S \leq 2^{64}$ -bit hint, we suggest instantiate with an elliptic curve group of size ≥ 448 bits, e.g., use W-448, Curve448, Edwards448, E448, or P-521 (using a prime field $\mathbb{F}_p$ with prime p of size 521 bits), based on the NIST SP 800-186 [CMR⁺23]. If we further aim to achieve 224 bits of *preprocessing* security against an attacker with a $S \leq 2^{72}$ -bit hint, we recommend using an elliptic curve of size ≥ 520 bits, such as P-521. If we aim to achieve 256 bits of *preprocessing* security against an attacker with a $S \leq 2^{80}$ -bit hint, then it will be necessary to standardize new elliptic curve groups with larger sizes. Table 2 provides a summary of suggested elliptic curve group sizes for various NIST security levels to ensure preprocessing security for nonzero Schnorr signatures.

NIST Security Level (bits)	Current ECG (key size)	Suggested ECG for Preprocessing Security		
		$S = 2^{64}$	$S = 2^{72}$	$S = 2^{80}$
112	$224 \leq$	$288 \leq$	$296 \leq$	$304 \leq$
128	$256 \leq$	$320 \leq$	$328 \leq$	$336 \leq$
192	$384 \leq$	$448 \leq$	$456 \leq$	$464 \leq$
224	$448 \leq$	$512 \leq$	$520 \leq$	$528 \leq$
256	$512 \leq$	$576 \leq$	$584 \leq$	$592 \leq$

Table 2. Suggested minimum elliptic curve group (ECG) sizes for nonzero Schnorr signatures to achieve various NIST security levels against preprocessing attacks.

5 CCA Security of Hashed ElGamal KEMs with Preprocessing: A Case Study of PSEC-KEM

In this section, we develop techniques to analyze the CCA security of hashed ElGamal-style KEMs against preprocessing attacks in the ROM+GGM. Given a cyclic group $G = \langle g \rangle$, a *hashed ElGamal-style KEM* refers to schemes in which the encapsulation computes an ephemeral group element $U = g^\alpha$ (for some exponent α , possibly derived from randomness via hashing) and the shared secret is derived by hashing a Diffie-Hellman term of the form $Z = pk^\alpha = g^{sk \cdot \alpha}$, possibly together with U . We instantiate our analysis for a key encapsulation mechanism called PSEC-KEM (Provably Secure Elliptic Curve-Key Encapsulation Mechanism). PSEC-KEM [Shi01, Sho01] is an ElGamal-based key encapsulation mechanism first proposed by NTT. PSEC-KEM has several standardized implementation such as NESSIE [Shi01] and ISO/IEC 18033-2 [fSC06]. PSEC-KEM was also certified by IETF (Internet Engineering Task Force) for the XML security of URIs [3rd05]. PSEC-KEM has provable security in the ROM and under the hardness assumption of the elliptic curve discrete logarithm [Men01, Shi01, CMV05], but the preprocessing security of PSEC-KEM has never been established. To the best of our knowledge, our work is the first to prove

preprocessing security of PSEC-KEM in the ROM+GGM. We first formally define the PSEC-KEM scheme in the ROM+GGM below. Here, we stress that in the scheme, three separate random oracles $H_0 : \{0, 1\}^* \rightarrow \mathbb{Z}_p$, $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$, and $H_2 : \{0, 1\}^* \rightarrow \{0, 1\}^{\lambda_1}$ (where λ_1 is a parameter which will be set later) are used that is consistent with the PSEC-KEM specification.

The PSEC-KEM Scheme.

Consider the GGM with labeling map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ for a prime p and the set of bitstrings $\mathbb{G} = \{0, 1\}^{\lceil \log p \rceil}$. The PSEC-KEM scheme consists of three algorithms (Gen, Encaps, Decaps), where each algorithm works as follows:

- Gen(1^λ): the key-generation algorithm taking as input the security parameter 1^λ and outputs a public/secret-key pair $(pk, sk) := (\tau(x), x)$ for a random $x \in \mathbb{Z}_p$.
- Encaps($pk, 1^\lambda$): the encapsulation algorithm taking as input $g = \tau(1)$, $pk = \tau(x)$, and 1^λ and outputs a key k and a ciphertext c as follows:
 - Pick a random $r \in \{0, 1\}^{\lambda_1}$.
 - Compute $H_0(r) = \alpha$ and $H_1(r) = k$.
 - Compute $c_1 = r \oplus H_2(\tau(\alpha) \parallel \tau(\alpha x))$ and $c := (\tau(\alpha), c_1)$.
(Note: $\tau(\alpha) \leftarrow \text{Pow}(g, \alpha)$ and $\tau(\alpha x) \leftarrow \text{Pow}(pk, \alpha)$)
 - Output a key-ciphertext pair (k, c) . We write $k \leftarrow \text{Encaps}(pk, 1^\lambda).\text{key}$ and $c \leftarrow \text{Encaps}(pk, 1^\lambda).\text{ctxt}$.
- Decaps(sk, c): the deterministic decapsulation algorithm taking as input a secret key $sk = x$ and a ciphertext c , and outputs a key k or $\perp$ denoting failure as follows:
 - Parse $c = (h, c_1)$.
 - Compute $r = c_1 \oplus H_2(h \parallel \text{Pow}(h, x))$.
 - Compute $H_0(r) = \alpha$ and $H_1(r) = k$.
 - Verify that $h = \tau(\alpha)$. If verified, output k ; otherwise, output $\perp$.

The Generic CPA/CCA Indistinguishability Game for a KEM. Let $\Pi = (\text{Gen}, \text{Encaps}, \text{Decaps})$ be a KEM and $\mathcal{A}$ be an adversary attacking Π . We define the following CPA/CCA indistinguishability game in the ROM+GGM, in which the attacker tries to distinguish between the real key k and a random key.

Intuitively, the CPA indistinguishability game works as follows: the attacker is given the public key pk and a key-ciphertext pair $(\hat{k}, c)$. Ultimately, the attacker is asked to distinguish whether $\hat{k}$ is the challenger's real encapsulated key $k \leftarrow \text{Encaps}(pk, 1^\lambda).\text{key}$ or a randomly selected string. The challenger picks a uniform bit $b \in \{0, 1\}$ and selects the real encapsulated key if $b = 0$ and selects a random key if $b = 1$. The attacker is given access to the “encapsulation” oracle $\text{Encaps}_b(\cdot)$, where $\text{Encaps}_b(pk)$ first runs $(k, c) \leftarrow \text{Encaps}(pk, 1^\lambda)$ and then outputs (k_b, c) where $k_0 = k$ and $k_1 \leftarrow \{0, 1\}^\lambda$ is a uniformly random key unrelated to the ciphertext c . The attacker is also given access to the random oracles and the generic group oracle ($\text{Mult}(\cdot, \cdot), \text{Inv}(\cdot)$).

In the CCA security game, the attacker is additionally given access to the “decapsulation” oracle $\text{Decaps}'_{sk}(\cdot)$ as long as the input ciphertext c' is neither the challenge ciphertext c nor the output of prior Encaps_b queries (since otherwise, we have $\perp \leftarrow \text{Decaps}(sk, \text{Encaps}_b(pk))$ if and only if $b = 1$, so that the attacker would trivially be able to distinguish between the real and a random key and win the security game). After multiple queries to the oracles above, the attacker finally outputs a bit b' , and wins the security game if $b' = b$. The KEM is said to be CPA-secure (resp.

CCA-secure) if distinguishing between the two in the CPA (resp. CCA) indistinguishability game is not more efficient than a random guess, i.e., the distinguishing probability is not greater than $1/2 + \text{negl}(\lambda)$ for some negligible function $\text{negl}(\lambda)$.

The Generic CPA/CCA Indistinguishability Game $\text{KEM}_{\mathcal{A}, \Pi}^{\tau, \text{H}, \text{cpa}/\text{cca}}(\lambda)$:

1. The challenger runs $\text{Gen}(1^\lambda)$ to obtain a public key pk and a secret key sk . Here, sk is chosen randomly from the group $\mathbb{Z}_p$ where p is a 2λ -bit prime, and $\text{pk} = \tau(\text{sk})$.
2. The challenger invokes $\text{Encaps}(\text{pk}, 1^\lambda)$ to obtain a key-ciphertext pair (k^*, c^*) .
3. The challenger chooses a uniform bit $b \in \{0, 1\}$. If $b = 0$, set $\hat{k} := k^*$; if $b = 1$, choose a random $\hat{k} \leftarrow_{\$} \{0, 1\}^\lambda$.
4. $\mathcal{A}$ gets a tuple $(\text{pk}, \hat{k}, c^*)$. The adversary is given access to the oracle $\text{Encaps}_b(\cdot)$, the random oracles H , and the generic group oracle $(\text{Mult}(\cdot, \cdot), \text{Inv}(\cdot))$. Here, $\text{Encaps}_b(\text{pk})$ works as follows:

$$\text{Encaps}_b(\text{pk}) := \begin{cases} \text{Encaps}(\text{pk}, 1^\lambda) & \text{if } b = 0, \\ (R \leftarrow_{\$} \{0, 1\}^\lambda, \text{Encaps}(\text{pk}, 1^\lambda).\text{ctxt}) & \text{if } b = 1. \end{cases}$$

If this is a CCA indistinguishability game, then $\mathcal{A}$ is additionally given access to the oracle $\text{Decaps}'_{\text{sk}}(\cdot)$ which works as follows: suppose that we keep track of the responses (in particular ciphertexts) of the adversary's queries to the oracle $\text{Encaps}_b(\cdot)$ by initializing the set $\text{resp}_{\text{pk}} = \{c^*\}$ and updating the set $\text{resp}_{\text{pk}} \leftarrow \text{resp}_{\text{pk}} \cup \{\text{Encaps}(\text{pk}, 1^\lambda).\text{ctxt}\}$ whenever $\mathcal{A}$ makes a query $\text{Encaps}_b(\text{pk})$. Then

$$\text{Decaps}'_{\text{sk}}(c') := \begin{cases} \text{Decaps}(\text{sk}, c') & \text{if } c' \notin \text{resp}_{\text{pk}}, \\ \perp & \text{if } c' \in \text{resp}_{\text{pk}}. \end{cases}$$

After multiple queries, $\mathcal{A}$ outputs a bit b' .

5. $\mathcal{A}$ wins if $b' = b$, which we write $\text{KEM}_{\mathcal{A}, \Pi}^{\tau, \text{H}, \text{cpa}/\text{cca}}(\lambda) = 1$, and the output of the game is 0 if $\mathcal{A}$ fails to guess b correctly.

Definition 3. Consider the generic group model with the labeling map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$. A KEM $\Pi = (\text{Gen}, \text{Encaps}, \text{Decaps})$ is said to be $(q_{\text{H}}, q_{\text{G}}, q_{\text{E}}, \varepsilon)$ -CPA secure (secure against chosen-plaintext attack) if for every adversary $\mathcal{A}$ making at most q_{H} , q_{G} , and q_{E} queries to the random oracles, generic group oracles, and $\text{Encaps}_b(\cdot)$, respectively, the following bound holds:

$$\Pr \left[\text{KEM}_{\mathcal{A}, \Pi}^{\tau, \text{H}, \text{cpa}}(\lambda) = 1 \right] \leq \frac{1}{2} + \varepsilon,$$

where the randomness is taken over the selection of τ , the random coins of $\mathcal{A}$, the random coins of Gen , the selection of random oracles $\text{H}_0, \text{H}_1, \text{H}_2$, and the random coins of Encaps_b .

Furthermore, we say a KEM Π is said to be $(q_{\text{H}}, q_{\text{G}}, q_{\text{E}}, q_{\text{D}}, \varepsilon)$ -CCA secure (secure against chosen-ciphertext attack) if for every adversary $\mathcal{A}$ making additionally at most q_{D} queries to the $\text{Decaps}(\text{sk}, \cdot)$, together with former q_{H} , q_{G} , and q_{E} queries as described before, the following bound holds:

$$\Pr \left[\text{KEM}_{\mathcal{A}, \Pi}^{\tau, \text{H}, \text{cca}}(\lambda) = 1 \right] \leq \frac{1}{2} + \varepsilon,$$

where the randomness is the same as the CPA security but additionally taken over the random coins of Decaps.

When it comes to the *preprocessing security*, we say that a KEM Π is CPA-secure (resp. CCA-secure) against preprocessing attacks if for every adversary $\mathcal{A} = (\mathcal{A}_{\text{pre}}, \mathcal{A}_{\text{on}})$, Π is $(q_H, q_G, q_E, \varepsilon)$ -CPA secure (resp. $(q_H, q_G, q_E, q_D, \varepsilon)$ -CCA secure) against the *online* attacker $\mathcal{A}_{\text{on}}$. In the BF model, $\mathcal{A}_{\text{on}}$ will receive the information of P pre-fixed points from $\mathcal{A}_{\text{pre}}$, and in the AI model, $\mathcal{A}_{\text{on}}$ will receive an S -bit hint from $\mathcal{A}_{\text{pre}}$. Hence, to prove the preprocessing security, it is sufficient to upper bound the probability of the event $\text{KEM}_{\mathcal{A}_{\text{on}}, \Pi}^{\tau, H, \text{cpa}}(\lambda) = 1$ (resp. $\text{KEM}_{\mathcal{A}_{\text{on}}, \Pi}^{\tau, H, \text{cca}}(\lambda) = 1$) for $\mathcal{A}_{\text{on}}$.

As a warmup, we first analyze the CPA/CCA security of PSEC-KEM in the ROM+GGM without preprocessing in [Appendix D](#). For the rest of the paper, we will focus on the *preprocessing security* of PSEC-KEM in the ROM+GGM.

5.1 The Quadratic Bridge-Finding Game

We first introduce an important tool that helps analyze the security of hashed ElGamal KEM (e.g., PSEC-KEM) in the ROM+GGM — a game called the *quadratic bridge-finding game*.

The Quadratic Bridge-Finding Game in the BF-GGM. We define the *quadratic bridge-finding game* $\text{QDBridgeChal}_{\mathcal{A}}^{\tau}(\lambda)$ in the BF-GGM. The game is run by a challenger and an attacker $\mathcal{A} = (\mathcal{A}_{\text{pre}}, \mathcal{A}_{\text{on}})$. In a preprocessing phase, the preprocessing attacker fixes P input/output pairs of a labeling map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$, i.e., $(t_1, \tau(t_1)), \dots, (t_P, \tau(t_P))$. Then the rest of the map τ is chosen uniformly at random subject to the constraint that the map remains injective. In an online phase, the challenger receives P pre-fixed points, picks a secret $x \leftarrow \$ \mathbb{Z}_p$, and sends $\tau(1)$ and $\tau(x)$ to the online attacker $\mathcal{A}_{\text{on}}$. $\mathcal{A}_{\text{on}}$ is also given P pre-fixed points of the map τ as a hint from $\mathcal{A}_{\text{pre}}$, and has access to the generic group oracle ($\text{Mult}(\cdot, \cdot)$, $\text{Inv}(\cdot)$) and a generic oracle $\mathcal{O}$ which selects a new random unknown variable $x_j \leftarrow \$ \mathbb{Z}_p$ and outputs $\tau(x_j)$.

The attacker's goal is to find a *quadratic relationship* between the unknown variables; it is not required for the attacker to solve for those unknown variables. In particular, the attacker needs to output a tuple $(i, j, \mathbf{a}, b)$ which satisfies $x_i x_j = \mathbf{a} \cdot \mathbf{x} + b$ to win the game, denoted by $\text{QDBridgeChal}_{\mathcal{A}}^{\tau}(\lambda) = 1$, where the first two elements in the tuple denote the indices of the unknown variables that create a quadratic term, and the vector $\mathbf{a}$ and the scalar b create the linear relationship between the unknowns.¹⁴

There are two important things to note. (1) As the attacker makes query to the oracles, the challenger maintains the list $\mathcal{L}$ that consists of tuples of the form $(\eta, \mathbf{a}, b)$ which indicates the relationship $\eta = \tau(\mathbf{a} \cdot \mathbf{x} + b)$ where $\mathbf{x}$ is the vector of unknowns. Initially, $\mathcal{L} = \{(\tau(1), 0, 1), (\tau(x), 1, 0), (\tau(t_1), 0, t_1), \dots, (\tau(t_P), 0, t_P)\}$ since there is only one unknown variable x and the challenger has the information of P pre-fixed points. (2) We allow the unknown variables to *expand* as the game progresses. There are several occasions when this happens (suppose $\dim(\mathbf{x}) = j$). A few examples are:

¹⁴ For example, suppose there are 3 unknown variables x_1, x_2, x_3 . The attacker wins the quadratic bridge-finding game if s/he outputs a tuple $(2, 3, (5, -3, 2), -1)$, which indicates the quadratic relationship $x_2 x_3 = (5, -3, 2) \cdot (x_1, x_2, x_3) - 1 = 5x_1 - 3x_2 + 2x_3 - 1$. Note that we can always make the coefficient of the quadratic term 1 as p is prime, e.g., if the attacker ever found the quadratic relationship $c x_i x_j = \mathbf{a} \cdot \mathbf{x} + b$ for $c \neq 0$ then s/he can output $(i, j, c^{-1} \mathbf{a}, c^{-1} b)$ since c^{-1} always exists in $\mathbb{Z}_p$ if $c \neq 0$.

- When the attacker makes a query to the oracle $\mathcal{O}$, then the oracle picks a new random unknown $x_{j+1} \leftarrow \$ \mathbb{Z}_p$ and outputs $\tau(x_{j+1})$. The challenger updates $\mathbf{x} \leftarrow \mathbf{x} \circ x_{j+1}$ and updates all the entries of the list $\mathcal{L}$ from $(\mathfrak{s}, \mathbf{a}, b)$ to $(\mathfrak{s}, \mathbf{a} \circ 0, b)$ as the number of unknowns is increased by 1. Finally, the challenger adds $(\tau(x_{j+1}), \mathbf{0} \circ 1, 0)$ to the list $\mathcal{L}$.
- When the attacker queries $\text{Mult}(\eta_1, \eta_2)$, and if $(\eta_1, \mathbf{a}_1, b_1) \in \mathcal{L}$ but η_2 does not appear in $\mathcal{L}$, then the challenger updates $\mathbf{x} \leftarrow \mathbf{x} \circ x_{j+1}$ where x_{j+1} denotes a new unknown satisfying $\tau(x_{j+1}) = \eta_2$. Similarly, the challenger also updates all the entries of the list $\mathcal{L}$ from $(\mathfrak{s}, \mathbf{a}, b)$ to $(\mathfrak{s}, \mathbf{a} \circ 0, b)$, and adds $(\eta_2, \mathbf{0} \circ 1, 0)$ and $(\text{Mult}(\eta_1, \eta_2), \mathbf{a}_1 \circ 0 + \mathbf{0} \circ 1, b_1)$ to $\mathcal{L}$.

See [Appendix E](#) for the formal description of the quadratic bridge-finding game, which handles all the possible cases when the attacker queries $\text{Mult}(\eta_1, \eta_2)$ or $\text{Inv}(\eta)$.

Lemma 2 upper bounds the probability of winning the quadratic bridge-finding game for a bit-fixing preprocessing attacker. The proof works largely the same as the proof of [Lemma 5](#) in [Appendix D](#), which upper bounds the odds of winning the quadratic bridge-finding game without preprocessing. Intuitively, the proof works by maintaining a list $\mathcal{L}$ of tuples $(\tau(y), \mathbf{a}, b) \in \mathbb{G} \times \mathbb{Z}_p^{\dim(\mathbf{x})} \times \mathbb{Z}_p$ which satisfies the constraint $y = \mathbf{a} \cdot \mathbf{x} + b$. We consider the event $\overline{\text{BRIDGE}}$ which denotes the event that the attacker finds some linear dependency on $\mathbf{x}$. Conditioning on $\overline{\text{BRIDGE}}$, we can essentially view $\mathbf{x}$ selected uniformly at random subject to the restriction that for any distinct tuples $(\eta, \mathbf{a}, b)$ and $(\eta', \mathbf{a}', b')$ we have $\mathbf{a} \cdot \mathbf{x} + b \neq \mathbf{a}' \cdot \mathbf{x} + b'$. For $x_r = \mathbf{x}[r]$ such that $\mathbf{a}[r] - \mathbf{a}'[r] \neq 0$, if the attacker outputs a tuple $(i_1, i_2, \mathbf{a}, b)$ such that $x_{i_1}x_{i_2} = \mathbf{a} \cdot \mathbf{x} + b$ then there are at most 2 roots in $\mathbb{Z}_p$ since this is a quadratic equation at the worst. This allows us to upper bound the probability $\Pr[\text{QDBridgeChal}_{\mathcal{A}}^{\tau}(\lambda) = 1 | \overline{\text{BRIDGE}}]$. The analysis of $\Pr[\text{BRIDGE}]$ follows a similar approach to the proof of [Lemma 1](#). Now we obtain our result by upper bounding the probability $\Pr[\text{BRIDGE}] + \Pr[\text{QDBridgeChal}_{\mathcal{A}}^{\tau}(\lambda) = 1 | \overline{\text{BRIDGE}}]$ which upper bounds the desired probability $\Pr[\text{QDBridgeChal}_{\mathcal{A}}^{\tau}(\lambda) = 1]$. The full proof of [Lemma 2](#) can be found in [Appendix G.2](#).

Lemma 2. *Let $p > 2^{2\lambda}$ is a prime number. Let $\mathcal{A} := (\mathcal{A}_{\text{pre}}^{\text{BF-GG}(P)}, \mathcal{A}_{\text{on}}^{\text{BF-GG}(P)})$ be a pair of bit-fixing generic algorithms with a labeling map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ such that $\mathcal{A}_{\text{pre}}^{\text{BF-GG}(P)}$ fixes P input/output pairs of the labeling map τ and $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$ makes at most $q_{\mathbb{G}}^{\text{on}} := q_{\mathbb{G}}^{\text{on}}(\lambda)$ queries to the generic group oracles and $q_{\mathcal{O}}^{\text{on}} := q_{\mathcal{O}}^{\text{on}}(\lambda)$ queries to the oracle $\mathcal{O}$. Then $\Pr[\text{QDBridgeChal}_{\mathcal{A}}^{\tau}(\lambda) = 1] \leq \varepsilon$, where*

$$\varepsilon := \frac{3(q_{\mathbb{G}}^{\text{on}} + q_{\mathcal{O}}^{\text{on}})^2 + (5 + 2P)(q_{\mathbb{G}}^{\text{on}} + q_{\mathcal{O}}^{\text{on}}) + 4}{2p - 4P(3q_{\mathbb{G}}^{\text{on}} + q_{\mathcal{O}}^{\text{on}} + 1) - 2(3q_{\mathbb{G}}^{\text{on}} + q_{\mathcal{O}}^{\text{on}} + 1)^2 - 2(2q_{\mathbb{G}}^{\text{on}} + q_{\mathcal{O}}^{\text{on}})},$$

in the GGM of prime order p , where the randomness is taken over the selection of $x_1, \dots, x_N, \tau$ as well as any random coins of $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$.

While we focus on analyzing PSEC-KEM using the quadratic bridge-finding game in this work, we believe this game provides a useful abstraction for analyzing other hashed ElGamal-style KEMs with a similar hashed-DH structure.

5.2 The CCA Security of PSEC-KEM in the BF-ROM+GGM

We are now ready to prove the CCA security of PSEC-KEM against preprocessing attacks in the BF-ROM+GGM. We first prove the CPA security of PSEC-KEM against preprocessing attacks in the BF-ROM+GGM, and the CCA security will be reduced from it.

Theorem 9 (CPA Security of PSEC-KEM with Preprocessing). Let $\Pi = (\text{Gen}, \text{Encaps}, \text{Decaps})$ be PSEC-KEM and $p > 2^{2\lambda}$ be a prime number. Let $\mathcal{A} = (\mathcal{A}_{\text{pre}}^{\text{BF-RO+GG}(P_1, P_2)}, \mathcal{A}_{\text{on}}^{\text{BF-RO+GG}(P_1, P_2)})$ be a pair of bit-fixing generic algorithms with a labeling map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ such that $\mathcal{A}_{\text{pre}}^{\text{BF-RO+GG}(P_1, P_2)}$ fixes $P_{1,1}$ (resp. $P_{1,2}, P_{1,3}$) input/output pairs of a random oracle $H_0 : \{0, 1\}^* \rightarrow \mathbb{Z}_p$ (resp. $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$, $H_2 : \{0, 1\}^* \rightarrow \{0, 1\}^{\lambda_1}$), and P_2 input/output pairs of the map τ such that $P_{1,1} + P_{1,2} + P_{1,3} = P_1$ and $2P_1 + P_2 = P$, and the hint $\text{str}_{\tau, H_0, H_1, H_2}$ is only dependent on those P points. If $\mathcal{A}_{\text{on}}^{\text{BF-RO+GG}(P_1, P_2)}$ makes at most q_H^{on} (resp. $q_G^{\text{on}}, q_E^{\text{on}}$) queries to the random oracle (resp. generic group oracle, encapsulation oracle), then $\Pr \left[\text{KEM}_{\mathcal{A}_{\text{on}, \text{str}_{\tau, H_0, H_1, H_2}}^{\tau, (H_0, H_1, H_2), \text{cpa}}^{\text{BF-RO+GG}(P_1, P_2)}, \Pi}(\lambda) = 1 \right] \leq \frac{1}{2} + \varepsilon$, with

$$\varepsilon = \frac{3(q_G^{\text{on}} + 2q_E^{\text{on}})^2 + (5 + 2P)(q_G^{\text{on}} + 2q_E^{\text{on}}) + 4}{p - 2P(3q_G^{\text{on}} + 2q_E^{\text{on}} + 1) - (3q_G^{\text{on}} + 2q_E^{\text{on}} + 1)^2 - 2(q_G^{\text{on}} + q_E^{\text{on}})} + \frac{3(q_H^{\text{on}} + P)q_E^{\text{on}}}{2^{\lambda_1}} + \frac{(q_H^{\text{on}} + P)q_E^{\text{on}}}{p},$$

where the randomness is taken over the selection of τ and the random coins of $\mathcal{A}_{\text{on}}^{\text{BF-RO+GG}(P_1, P_2)}$.

Proof Sketch. We provide only a proof sketch here, and the full proof is deferred to [Appendix G.3](#). We use a hybrid argument. In the first hybrid (hybrid H_0), the distinguisher $\mathcal{D}$ is given the CPA indistinguishability game for PSEC-KEM with the challenge bit $b = 0$, and in the last hybrid (hybrid H_3), $\mathcal{D}$ is given the CPA indistinguishability game with $b = 1$. The intermediate hybrids are defined as follows. Hybrid H_1 is the same as H_0 except that the encapsulation oracle is modified so that $\tau(\alpha)$ and $\tau(\alpha x)$ are replaced with random elements in $\tau(\mathbb{Z}_p)$ by querying the oracle $\mathcal{O}$ twice. Hybrid H_2 is the same as H_1 except that the encapsulation oracle is further modified so that the key k is sampled uniformly at random from $\{0, 1\}^\lambda$.

We show that H_0, H_1 (resp. H_2, H_3) are perfectly indistinguishable unless (1) the random oracle query $H_2(\tau(\alpha) \parallel \tau(\alpha x))$ has been made, or (2) the adversary makes query r to H_0 (which is not one of the fixed points of H_0) but has not queried $H_2(\tau(\alpha) \parallel \tau(\alpha x))$, or (3) r is one of the fixed points of H_0 . We show that Case (1) reduces to winning the quadratic bridge-finding game so that we can apply [Lemma 2](#) with $q_{\mathcal{O}}^{\text{on}} \leq 2q_E^{\text{on}}$. Since there are at most P fixed points, by union bound we can show that Case (2) has probability $\leq \frac{q_H^{\text{on}} \cdot q_E^{\text{on}}}{2^{\lambda_1}}$ and Case (3) has probability $\leq \frac{Pq_E^{\text{on}}}{2^{\lambda_1}}$. We also prove that the distinguishing probability between H_1, H_2 is at most $q_E^{\text{on}} \left(\frac{q_H^{\text{on}} + P}{p} + \frac{q_H^{\text{on}} + P}{2^{\lambda_1}} \right)$ since two hybrids are perfectly indistinguishable unless $H_1(r)$ is queried (which is not one of the fixed points) or r is one of the fixed points. Taken together, we can conclude the proof. $\square$

Theorem 10 (CCA Security of PSEC-KEM with Preprocessing). Let $\Pi = (\text{Gen}, \text{Encaps}, \text{Decaps})$ be PSEC-KEM and $p > 2^{2\lambda}$ be a prime number. Let $\mathcal{A} = (\mathcal{A}_{\text{pre}}^{\text{BF-RO+GG}(P_1, P_2)}, \mathcal{A}_{\text{on}}^{\text{BF-RO+GG}(P_1, P_2)})$ be a pair of bit-fixing generic algorithms with a labeling map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ such that $\mathcal{A}_{\text{pre}}^{\text{BF-RO+GG}(P_1, P_2)}$ fixes $P_{1,1}$ (resp. $P_{1,2}, P_{1,3}$) input/output pairs of a random oracle $H_0 : \{0, 1\}^* \rightarrow \mathbb{Z}_p$ (resp. $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$, $H_2 : \{0, 1\}^* \rightarrow \{0, 1\}^{\lambda_1}$), and P_2 input/output pairs of the map τ such that $P_{1,1} + P_{1,2} + P_{1,3} = P_1$ and $2P_1 + P_2 = P$, and the hint $\text{str}_{\tau, H_0, H_1, H_2}$ is only dependent on those P points. If $\mathcal{A}_{\text{on}}^{\text{BF-RO+GG}(P_1, P_2)}$ makes at most q_H^{on} (resp. $q_G^{\text{on}}, q_E^{\text{on}}, q_D^{\text{on}}$) queries to the random (resp. generic group, encapsulation, decapsulation)

oracle, then $\Pr \left[\text{KEM}_{\mathcal{A}_{\text{on}}, \text{str}_{\tau, H_0, H_1, H_2}}^{\tau, (H_0, H_1, H_2), \text{cca}}_{\text{BF-RO+GG}(P_1, P_2), \Pi}(\lambda) = 1 \right] \leq \frac{1}{2} + \varepsilon$, with

$$\varepsilon = \frac{3(q_G^{\text{on}} + 2q_E^{\text{on}})^2 + (5 + 2P)(q_G^{\text{on}} + 2q_E^{\text{on}}) + 4}{p - 2P(3q_G^{\text{on}} + 2q_E^{\text{on}} + 1) - (3q_G^{\text{on}} + 2q_E^{\text{on}} + 1)^2 - 2(q_G^{\text{on}} + q_E^{\text{on}})} + \frac{2q_D^{\text{on}}(q_H^{\text{on}} + P) + 3q_E^{\text{on}}(q_H^{\text{on}} + q_D^{\text{on}} + P)}{2^{\lambda_1}} + \frac{(q_H^{\text{on}} + q_D^{\text{on}} + P)q_E^{\text{on}} + 2q_D^{\text{on}}}{p},$$

where the randomness is taken over the selection of τ and the random coins of $\mathcal{A}_{\text{on}}^{\text{BF-RO+GG}(P_1, P_2)}$.

Proof Sketch. We provide only a proof sketch here, and the full proof is deferred to [Appendix G.3](#). We use a hybrid argument to prove the CCA security. In the first hybrid (H_0), the distinguisher $\mathcal{D} = (\mathcal{D}_{\text{pre}}, \mathcal{D}_{\text{on}})$ is given the CCA indistinguishability game for PSEC-KEM in the BF-ROM+GGM with challenge bit $b = 0$, and in the last hybrid (H_3), $\mathcal{D}$ is given the CCA game with $b = 1$. The intermediate hybrids are defined as follows. Hybrid H_1 is the same as H_0 , but it replaces the decapsulation oracle with a “simulated” decapsulation oracle that tries to output the key without knowledge of the secret key. To do so, it keeps track of the database of input/output pairs of the random oracle queries H_0 and H_2 made by $\mathcal{D}_{\text{on}}$ along with all the fixed points of H_0 and H_2 (which we denote D_{H_0} and D_{H_2} , respectively). Then, for an input ciphertext c , the oracle checks if (1) c is neither the challenge ciphertext nor the output of previous encapsulation oracle queries, and (2) whether we can simulate the decaps oracle from known random oracle input/output pairs i.e., if there exists a tuple (r, α, z, c_1) such that $(r, \alpha) \in D_{H_0}$, $((\tau(\alpha) \parallel \tau(\alpha x)), z) \in D_{H_2}$, $c = (\tau(\alpha), c_1)$, and $r = c_1 \oplus z$. If both conditions are satisfied, then the oracle outputs $H_1(r)$; otherwise, it outputs $\perp$. Hybrid H_2 is the same as H_1 but with the challenge bit $b = 1$.

We show that hybrids H_0 and H_1 are perfectly indistinguishable unless one of the two bad events occurs for some round $i \leq q_D^{\text{on}}$: (1) the decapsulation oracle does not return $\perp$, but $(r_i, \alpha_i) \notin D_{H_0}$ where $c_i = (c_i^1, c_i^2)$, $z_i = H_2(c_i^1 \parallel \text{Pow}(c_i^1, x))$, $r_i = c_i^2 \oplus z_i$, and $\alpha_i = H_0(r_i)$ as computed by the decapsulation oracle (denoted as Lucky₁); or (2) the decapsulation oracle does not return $\perp$ and $(r_i, \alpha_i) \in D_{H_0}$ but $((\tau(\alpha_i) \parallel \tau(\alpha_i x)), z_i) \notin D_{H_2}$ where α_i and z_i were computed as in the previous case (denoted as Lucky₂). We observe that if neither event occurs, then the “simulated” oracle will perfectly simulate the decaps oracle (see [Appendix G](#) for details). Hence, the distinguishing probability between two hybrids is upper bounded by $\Pr[\text{Lucky}_1] + \Pr[\text{Lucky}_2]$. For each $i \in [q_D^{\text{on}}]$, $H_0(r_i) \in \mathbb{Z}_p$ can be viewed as a uniformly random integer that is selected after the query to the decaps oracle is fixed (i.e., once $\tau^{-1}(c_i^1)$ is fixed), which can be matched with $\tau^{-1}(c_i^1)$ with probability $1/p$. Union bounding over q_D^{on} rounds, we have $\Pr[\text{Lucky}_1] \leq q_D^{\text{on}}/p$. Similarly, $z_i = H_2(c_i^1 \parallel \text{Pow}(c_i^1, x))$ (and thus $r_i = c_i^2 \oplus z_i$) can be viewed as uniformly random strings that are selected after the query to Decaps' since we had $((\tau(\alpha_i) \parallel \tau(\alpha_i x)), z_i) \notin D_{H_2}$. However, for the event to occur, we must also have $(r_i, \alpha_i) \in D_{H_0}$. Since $\Pr[(r_i, \alpha_i) \in D_{H_0}] \leq |D_{H_0}|/2^{\lambda_1} \leq (P + q_H^{\text{on}})/2^{\lambda_1}$, union bounding over q_D^{on} rounds, we have $\Pr[\text{Lucky}_2] \leq q_D^{\text{on}}(P + q_H^{\text{on}})/2^{\lambda_1}$.

Next, if an attacker $\mathcal{D}$ can distinguish between H_1 and H_2 , then $\mathcal{D}$ can also be used to win the CPA game for PSEC-KEM in the BF-ROM+GGM since $\mathcal{D}$ never queries the *real* decaps oracle — it only queries the *simulated* oracle. The algorithm $\mathcal{D}'$ will simulate $\mathcal{D}$ and use the “simulated” oracle to respond to any decapsulation query. The advantage of $\mathcal{D}'$ in the CPA indistinguishability game is equivalent to the advantage of distinguisher $\mathcal{D}$ in distinguishing H_1 and H_2 . If $\mathcal{D}$ makes q_H^{on} RO queries and q_D^{on} decapsulation queries, then $\mathcal{D}'$ will make at most $(q_H^{\text{on}})' = q_H^{\text{on}} + q_D^{\text{on}}$ RO queries

and 0 decapsulation queries. Hence, we can invoke [Theorem 9](#) to upper bound the advantage as $\frac{3(q_G^{\text{on}} + 2q_E^{\text{on}})^2 + (5 + 2P)(q_G^{\text{on}} + 2q_E^{\text{on}}) + 4}{p - 2P(3q_G^{\text{on}} + 2q_E^{\text{on}} + 1) - (3q_G^{\text{on}} + 2q_E^{\text{on}} + 1)^2 - 2(q_G^{\text{on}} + q_E^{\text{on}})} + \frac{3(q_H^{\text{on}} + q_D^{\text{on}} + P)q_E^{\text{on}}}{2^{\lambda_1}} + \frac{(q_H^{\text{on}} + q_D^{\text{on}} + P)q_E^{\text{on}}}{p}$.

The distinguishing probability between H_2 and H_3 is the same as the distinguishing probability between H_0 and H_1 , since the only difference between H_2 and H_3 is that we replace the simulated oracle with the original decaps oracle. Since the challenge bit is now $b = 1$ in H_2 and H_3 , the encapsulation oracle will now generate random keys. However, because both simulated and original decaps oracles will reject outputs of the encapsulation oracle, the change in the challenge bit will not affect the analysis. We can finish the proof by combining these bounds together. $\square$

5.3 The CCA Security of PSEC-KEM in the AI-ROM+GGM

By applying [Theorem 5](#), we can address the CCA security of PSEC-KEM against preprocessing attacks in the AI-ROM+GGM.

Theorem 11. *Let $\Pi = (\text{Gen}, \text{Encaps}, \text{Decaps})$ be PSEC-KEM and $p > 2^{2\lambda}$ be a prime number. Let $\mathcal{A} = (\mathcal{A}_{\text{pre}}^{\text{AI-ROM+GG}}, \mathcal{A}_{\text{on}}^{\text{AI-ROM+GG}})$ be a pair of (auxiliary-input) generic algorithms with a labeling map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ such that $\mathcal{A}_{\text{pre}}^{\text{AI-ROM+GG}}$ outputs an S -bit hint $\text{str}_{\tau, H_0, H_1, H_2}$. If $\mathcal{A}_{\text{on}}^{\text{AI-ROM+GG}}$ makes at most q_H^{on} (resp. $q_G^{\text{on}}, q_E^{\text{on}}$) queries to the random oracles $H_0 : \{0, 1\}^* \rightarrow \mathbb{Z}_p$ and $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ and $H_2 : \{0, 1\}^* \rightarrow \{0, 1\}^{\lambda_1}$ in total (resp. generic group oracle, encapsulation oracle), then $\Pr \left[\text{KEM}_{\mathcal{A}_{\text{on}}^{\text{AI-ROM+GG}}, \text{str}_{\tau, H_0, H_1, H_2}, \Pi}^{\tau, (H_0, H_1, H_2), \text{cca}}(\lambda) = 1 \right] \leq \frac{1}{2} + \varepsilon$, with*

$$\begin{aligned} \varepsilon = & \frac{3(q_G^{\text{on}} + 2q_E^{\text{on}})^2 + (5 + 2\sqrt{Sp})(q_G^{\text{on}} + 2q_E^{\text{on}}) + 4}{p - 2\sqrt{Sp}(3q_G^{\text{on}} + 2q_E^{\text{on}} + 1) - (3q_G^{\text{on}} + 2q_E^{\text{on}} + 1)^2 - 2(q_G^{\text{on}} + q_E^{\text{on}})} \\ & + \frac{2q_D^{\text{on}}(q_H^{\text{on}} + \sqrt{Sp}) + 3q_E^{\text{on}}(q_H^{\text{on}} + q_D^{\text{on}} + \sqrt{Sp})}{2^{\lambda_1}} + \frac{(q_H^{\text{on}} + q_D^{\text{on}} + \sqrt{Sp})q_E^{\text{on}} + 2q_D^{\text{on}}}{p} \\ & + \frac{2(S + 2\lambda)((q_H^{\text{on}} + 3(q_E^{\text{on}} + q_D^{\text{on}}))\lambda + 3(q_G^{\text{on}} + 2\lceil \log p \rceil(q_E^{\text{on}} + q_D^{\text{on}}))\log p)}{\sqrt{Sp}(\lambda + \log(p/e))} + 2^{-2\lambda+1}, \end{aligned}$$

where the randomness is taken over the selection of τ and the random coins of $\mathcal{A}_{\text{on}}^{\text{AI-ROM+GG}}$.

Proof. This is straightforward by combining [Theorem 10](#) with [Theorem 5](#) with setting $P = \sqrt{Sp}$ and $\gamma = 2^{-2\lambda}$. Since the dominant term in the bound from [Theorem 10](#) is $O(qP/p)$ (where $q := q_H^{\text{on}} + q_G^{\text{on}} + q_E^{\text{on}} + q_D^{\text{on}}$ denotes the total number of *online* queries made by a preprocessing attacker) and the additive error term from [Theorem 5](#) is $\frac{2(S + \log \gamma^{-1})((T_1)_G^{\text{comb}}\lambda + 3(T_2)_G^{\text{comb}}\log p)}{\eta} + 2\gamma = O(qS/P)$, we can easily see that the optimized bound is achieved when $qP/p = qS/P$, i.e., $P = \sqrt{Sp}$. Moreover, η should satisfy $P_1\lambda + P_2\log(p/e) \leq \eta$, so we can choose η to be $\eta := P(\lambda + \log(p/e)) = \sqrt{Sp}(\lambda + \log(p/e))$.

We also note that $(T_1)_G^{\text{comb}} = q_H^{\text{on}} + 3(q_E^{\text{on}} + q_D^{\text{on}})$ since each encaps/decaps query generates three random oracle queries, respectively. Similarly, $(T_2)_G^{\text{comb}} = q_G^{\text{on}} + 2\lceil \log p \rceil(q_E^{\text{on}} + q_D^{\text{on}})$ since each encaps/decaps query generates two $\text{Pow}(\cdot, \cdot)$ oracle queries, respectively, and each $\text{Pow}(\cdot, \cdot)$ oracle can be implemented using at most $\lceil \log p \rceil$ calls to the Mult oracle [BL22]. Taken together, we get the desired result. $\square$

Instantiating PSEC-KEM with Concrete Elliptic Curves. We would like to have the distinguishing probability in [Theorem 11](#) bounded by $(1/2) + O(q/2^\lambda)$ for any $q \leq 2^\lambda$. To achieve λ bits of CCA security for PSEC-KEM with preprocessing, we can fix p such that $p \approx 2^{2\lambda}S$ and set the length of the output of H_2 to be $\lambda_1 = 2\lambda + \log_2(10S)$. Then [Theorem 11](#) tells us that a preprocessing attacker in the AI-ROM+GGM wins the CCA security game with probability at most $(1/2) + O(q/2^\lambda)$. For example, the curve P-384 [[CMR⁺23](#)] and SHA-384 would suffice for ≈ 152 -bit security against an attacker with an $S = 2^{80}$ -bit hint. Because the group size $p \approx 2^{2\lambda}S$ to achieve λ -bit security against an attacker with an S -bit hint is identical for Schnorr signatures, we refer the reader to [Table 2](#) in [Section 4.3](#) for more concrete examples.

References

- 3rd05. Donald E. Eastlake 3rd. Additional XML Security Uniform Resource Identifiers (URIs). RFC 4051, April 2005. [1.1.3](#), [5](#)
- ACDW20. Akshima, David Cash, Andrew Drucker, and Hoeteck Wee. Time-space tradeoffs and short collisions in merkle-damgård hash functions. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part I*, volume 12170 of *LNCS*, pages 157–186. Springer, Cham, August 2020. [1](#), [1.2](#)
- AGL22. Akshima, Siyao Guo, and Qipeng Liu. Time-space lower bounds for finding collisions in merkle-Damgård hash functions. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part III*, volume 13509 of *LNCS*, pages 192–221. Springer, Cham, August 2022. [1.2](#)
- BL13. Daniel J. Bernstein and Tanja Lange. Non-uniform cracks in the concrete: The power of free precomputation. In Kazue Sako and Palash Sarkar, editors, *ASIACRYPT 2013, Part II*, volume 8270 of *LNCS*, pages 321–340. Springer, Berlin, Heidelberg, December 2013. [1.2](#)
- BL22. Jeremiah Blocki and Seunghoon Lee. On the multi-user security of short schnorr signatures with preprocessing. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 614–643. Springer, Cham, May / June 2022. [1](#), [1.1.2](#), [1.2](#), [2](#), [3](#), [4](#), [4](#), [4](#), [4](#), [4.2](#), [5.3](#), [C](#), [F](#)
- BR93. Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby, editors, *ACM CCS 93*, pages 62–73. ACM Press, November 1993. [1](#), [2](#)
- CDG18. Sandro Coretti, Yevgeniy Dodis, and Siyao Guo. Non-uniform bounds in the random-permutation, ideal-cipher, and generic-group models. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 693–721. Springer, Cham, August 2018. [1](#), [1.1.1](#), [1.1.1](#), [1.2](#), [3](#), [4](#), [2](#), [15](#), [A.2](#), [A.2](#), [A.2](#), [2](#), [16](#), [B.2](#), [B.2](#), [F](#)
- CDGS18. Sandro Coretti, Yevgeniy Dodis, Siyao Guo, and John P. Steinberger. Random oracles and non-uniformity. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part I*, volume 10820 of *LNCS*, pages 227–258. Springer, Cham, April / May 2018. [1](#), [1.1.1](#), [1.1.1](#), [1.2](#), [3](#), [3](#), [A.2](#), [A.2](#), [A.2](#), [A.2](#), [B.2](#), [B.2](#), [B.2](#)
- Che10. Jung Hee Cheon. Discrete logarithm problems with auxiliary inputs. *Journal of Cryptology*, 23(3):457–476, July 2010. [1.2](#)
- CK18. Henry Corrigan-Gibbs and Dmitry Kogan. The discrete-logarithm problem with preprocessing. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 415–447. Springer, Cham, April / May 2018. [1.2](#)
- CMR⁺23. Lily Chen, Dustin Moody, Karen Randall, Andrew Regenscheid, and Angela Robinson. Recommendations for discrete logarithm-based cryptography: Elliptic curve domain parameters, 2023-02-02 05:02:00 2023. [4.3](#), [5.3](#)
- CMV05. D. Galindo Chacon, Sébastien Martin, and Jorge Luis Villar. The security of PSEC-KEM versus ECIES-KEM, 2005. [1.1.3](#), [5](#)
- CO15. Tung Chou and Claudio Orlandi. The simplest protocol for oblivious transfer. In *Proceedings of the 4th International Conference on Progress in Cryptology – LATINCRYPT 2015 - Volume 9230*, page 40–58, Berlin, Heidelberg, 2015. Springer-Verlag. [1](#), [3](#)
- CPS08. Jean-Sébastien Coron, Jacques Patarin, and Yannick Seurin. The random oracle model and the ideal cipher model are equivalent. In David Wagner, editor, *CRYPTO 2008*, volume 5157 of *LNCS*, pages 1–20. Springer, Berlin, Heidelberg, August 2008. [1.2](#)
- dR97. Peter de Rooij. On schnorr’s preprocessing for digital signature schemes. *J. Cryptol.*, 10(1):1–16, dec 1997. [1.2](#)

- DS16. Yuanxi Dai and John P. Steinberger. Indifferentiability of 8-round Feistel networks. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part I*, volume 9814 of *LNCS*, pages 95–120. Springer, Berlin, Heidelberg, August 2016. [1.2](#)
- DTT10. Anindya De, Luca Trevisan, and Madhur Tulsiani. Time space tradeoffs for attacks against one-way functions and PRGs. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 649–665. Springer, Berlin, Heidelberg, August 2010. [1.2](#)
- Dwo15. Morris Dworkin. SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions, 2015-08-04 2015. [H](#)
- EM93. Shimon Even and Yishay Mansour. A construction of a cipher from a single pseudorandom permutation. In Hideki Imai, Ronald L. Rivest, and Tsutomu Matsumoto, editors, *ASIACRYPT'91*, volume 739 of *LNCS*, pages 210–224. Springer, Berlin, Heidelberg, November 1993. [H](#)
- FGK22. Cody Freitag, Ashrujit Ghoshal, and Ilan Komargodski. Time-space tradeoffs for sponge hashing: Attacks and limitations for short collisions. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part III*, volume 13509 of *LNCS*, pages 131–160. Springer, Cham, August 2022. [1.2](#)
- fIS18. Federal Office for Information Security. Elliptic Curve Cryptography, Version 2.1. *Technical Guideline BSI TR-03111*, Jun 2018. [1](#), [4](#)
- fSC06. International Organization for Standardization and International Electrotechnical Commission. IT Security techniques – Encryption algorithms – Part 2: Asymmetric ciphers. *ISO/IEC 18033-2*, 2006. [1.1.3](#), [5](#)
- fSC18. International Organization for Standardization and International Electrotechnical Commission. IT Security techniques – Digital signatures with appendix – Part 3: Discrete logarithm based mechanisms. *ISO/IEC 14888-3*, Nov 2018. [1](#), [4](#)
- GK22. Ashrujit Ghoshal and Ilan Komargodski. On time-space tradeoffs for bounded-length collisions in merkle-Damgård hashing. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part III*, volume 13509 of *LNCS*, pages 161–191. Springer, Cham, August 2022. [1.2](#)
- GT20. Ashrujit Ghoshal and Stefano Tessaro. On the memory-tightness of hashed ElGamal. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part II*, volume 12106 of *LNCS*, pages 33–62. Springer, Cham, May 2020. [1](#)
- GT23. Ashrujit Ghoshal and Stefano Tessaro. The query-complexity of preprocessing attacks. In Helena Hand-schuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part II*, volume 14082 of *LNCS*, pages 482–513. Springer, Cham, August 2023. [1.2](#)
- Hel80. M. Hellman. A cryptanalytic time-memory trade-off. *IEEE Transactions on Information Theory*, 26(4):401–406, 1980. [1.2](#)
- HJK22. Pavel Hubáček, Ľubica Jančová, and Veronika Králová. On the distributed discrete logarithm problem with preprocessing. Cryptology ePrint Archive, Report 2022/521, 2022. [1.2](#)
- KMP16. Eike Kiltz, Daniel Masny, and Jiaxin Pan. Optimal security proofs for signatures from identification schemes. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part II*, volume 9815 of *LNCS*, pages 33–61. Springer, Berlin, Heidelberg, August 2016. [1.2](#), [2](#)
- LCH11. Hyung Tae Lee, Jung Hee Cheon, and Jin Hong. Accelerating ID-based encryption based on trapdoor DL using pre-computation. Cryptology ePrint Archive, Report 2011/187, 2011. [1.2](#)
- Mau05. Ueli Maurer. Abstract models of computation in cryptography. In *Proceedings of the 10th International Conference on Cryptography and Coding*, IMA'05, page 1–12, Berlin, Heidelberg, 2005. Springer-Verlag. [2](#)
- Men01. A. Menezes. Evaluation of security level of cryptography: The revised version of PSEC-2 (PSEC-KEM). Technical report, CRYPTREC, 2001. [1.1.3](#), [5](#)
- Nec94. V. I. Nechaev. Complexity of a determinate algorithm for the discrete logarithm. *Math Notes*, 55:165, 1994. [1](#)
- NSW09. Gregory Neven, Nigel Smart, and Bogdan Warinschi. Hash function requirements for schnorr signatures. *Journal of Mathematical Cryptology*, 3, 05 2009. [1.2](#)
- oSN14. National Institute of Standards and Technology (NIST). FIPS 202. sha-3 standard: Permutation-based hash and extendable-output functions. Technical report, US Department of Commerce, April 2014. [H](#)
- oST01. National Institute of Standards and Technology. Advanced Encryption Standard. *NIST FIPS PUB 197*, 2001. [H](#)
- RS22. Lior Rotem and Gil Segev. A fully-constructive discrete-logarithm preprocessing algorithm with an optimal time-space tradeoff. Cryptology ePrint Archive, Report 2022/583, 2022. [1.2](#)
- Sch90. Claus-Peter Schnorr. Efficient identification and signatures for smart cards. In Gilles Brassard, editor, *CRYPTO'89*, volume 435 of *LNCS*, pages 239–252. Springer, New York, August 1990. [1](#), [1.2](#)
- Sha49. Claude E. Shannon. Communication theory of secrecy systems. *Bell Systems Technical Journal*, 28(4):656–715, 1949. [1](#), [1.2](#), [H](#)

- Shi01. R. Shipsey. PSEC-KEM. Technical Report NES/DOC/EHU/WP5/001/a, NESSIE, 2001. [1](#), [1.1.3](#), [5](#)
- Sho97. Victor Shoup. Lower bounds for discrete logarithms and related problems. In Walter Fumy, editor, *EURO-CRYPT'97*, volume 1233 of *LNCS*, pages 256–266. Springer, Berlin, Heidelberg, May 1997. [1](#), [2](#)
- Sho01. Victor Shoup. A proposal for an ISO standard for public key encryption. Cryptology ePrint Archive, Report 2001/112, 2001. [1](#), [1.1.3](#), [5](#)
- SJ00. Claus-Peter Schnorr and Markus Jakobsson. Security of signed ElGamal encryption. In Tatsuaki Okamoto, editor, *ASIACRYPT 2000*, volume 1976 of *LNCS*, pages 73–89. Springer, Berlin, Heidelberg, December 2000. [1.2](#), [2](#)
- Unr07. Dominique Unruh. Random oracles and auxiliary input. In Alfred Menezes, editor, *CRYPTO 2007*, volume 4622 of *LNCS*, pages 205–223. Springer, Berlin, Heidelberg, August 2007. [1](#)
- Yao90. A. C.-C. Yao. Coherent functions and program checkers. In *Proceedings of the Twenty-Second Annual ACM Symposium on Theory of Computing, STOC '90*, page 84–94, New York, NY, USA, 1990. Association for Computing Machinery. [1.2](#)
- Zha22. Mark Zhandry. To label, or not to label (in generic groups). In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part III*, volume 13509 of *LNCS*, pages 66–96. Springer, Cham, August 2022. [2](#)

A Formal Definition of Bit-Fixing/Auxiliary-Input ROM+GGM

A.1 Replacing Auxiliary Information by Bit-Fixing in the ROM+GGM

We first define an (H, τ) -source for a random oracle H and a generic-group random encoding map τ as follows.

Definition 4. An (H, τ) -source is a pair of random variables (X, Y) where X corresponds to the function table of a random oracle $H : \{0, 1\}^m \rightarrow \{0, 1\}^\lambda$, with range $(\{0, 1\}^\lambda)^{2^m}$, and Y corresponds to the function table of an injection $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$. A source (X, Y) is called

- $(P_1, P_2, 1 - \delta)$ -dense if for $(P_1, P_2) \in \mathbb{Z}_{2^m} \times \mathbb{Z}_p$ if H is fixed on at most P_1 coordinates, τ is fixed on at most P_2 coordinates, and if for all pairs of subsets $(I_1, I_2) \subseteq \{0, 1\}^m \times \mathbb{Z}_p$ of non-fixed coordinates,

$$H_\infty(X_{I_1}, Y_{I_2}) \geq (1 - \delta) \left[|I_1| \lambda + \log(p - P_2)^{|I_2|} \right],$$

where $a^b := a!/(a-b)!$ and X_I is X restricted to the coordinates in I .

- $(1 - \delta)$ -dense if it is $(0, 0, 1 - \delta)$ -dense, and
- (P_1, P_2) -Bit-Fixing if it is fixed on at most P_1 coordinates on H and P_2 coordinates on τ and uniform on the rest.

Lemma 3. Let (X_1, X_2) be a pair of random variables such that X_1 is distributed uniformly over $(\{0, 1\}^\lambda)^{2^m}$ and X_2 is a uniformly random injection that takes on as value function tables corresponding to an injection $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$. Let $Z := f(X_1, X_2)$, where $f : (\{0, 1\}^\lambda)^{2^m} \times \mathbb{G}^p \rightarrow \{0, 1\}^S$ is an arbitrary function. For any $\gamma > 0$, there exists a family $\{(Y_1, Y_2)^{Z, \mathcal{Y}}\}$, indexed by $Z \in \{0, 1\}^S$ and size- p subsets $\mathcal{Y}$ of $\mathbb{G}$, of convex combinations of (P_1, P_2) -Bit-Fixing (H, τ) -sources satisfying $P_1 \lambda + P_2 \log(p/e) \leq W$ such that for any distinguisher $\mathcal{D}$ taking an S -bit input and querying at most $T_1 < P_1$ coordinates and $T_2 < P_2$ coordinates of each oracle,

$$\begin{aligned} & \left| \Pr [\mathcal{D}^{X_1, X_2}(f(X_1, X_2)) = 1] - \Pr [\mathcal{D}^{(Y_1, Y_2)^{f(X_1, X_2), \text{im}(X_2)}}(f(X_1, X_2)) = 1] \right| \\ & \leq \frac{(S + \log \gamma^{-1})(T_1 \lambda + T_2 \log p)}{W} + \gamma, \end{aligned}$$

and

$$\begin{aligned} & \Pr [\mathcal{D}^{X_1, X_2}(f(X_1, X_2)) = 1] \\ & \leq 2^{(S + \log \gamma^{-1})(T_1 \lambda + T_2 \log p)/W} \cdot \Pr [\mathcal{D}^{(Y_1, Y_2)^{f(X_1, X_2), \text{im}(X_2)}}(f(X_1, X_2)) = 1] + 2\gamma, \end{aligned}$$

where e is the Euler's number.

Proof. Fix an arbitrary $z \in \{0, 1\}^S$ and let (X_1^z, X_2^z) be the distribution of (X_1, X_2) conditioned on $f(X_1, X_2) = z$. Let $S_z = 2^m \lambda + \log p! - H_\infty(X_1, X_2)$ be the min-entropy deficiency of (X_1^z, X_2^z) . Let $\gamma > 0$ be arbitrary.

Claim 1. For every $\delta > 0$, any leaky source (X_1^z, X_2^z) is γ -close to a source (Y_1^z, Y_2^z) which is a convex combination of sources such that each source in the convex combination is $(P'_1, P'_2, 1 - \delta)$ -dense sources for some P'_1 and P'_2 which satisfy

$$P'_1 \lambda + P'_2 \log(p/e) \leq \frac{S_z + \log \gamma^{-1}}{\delta}.$$

Proof of Claim 1. Without loss of generality, we can assume that (X_1^z, X_2^z) is not $(1 - \delta)$ -dense since if it is $(1 - \delta)$ -dense then we can trivially find such $(P'_1, P'_2, 1 - \delta)$ -dense sources. Let $(Y_1^z, Y_2^z) := (X_1^z, X_2^z)$ and $I_1 \subseteq \{0, 1\}^m$ and $I_2 \subseteq \mathbb{Z}_p$ be the largest subset for which there exists a violation, i.e., there exists $y_1^{I_1} \in (\{0, 1\}^\lambda)^{|I_1|}$ and $y_2^{I_2} \in \mathbb{G}^{|I_2|}$ such that

$$\Pr \left[(Y_1^z)_{I_1} = y_1^{I_1} \wedge (Y_2^z)_{I_2} = y_2^{I_2} \right] > 2^{-(1-\delta) \left[|I_1| \lambda + \log p^{|I_2|} \right]}.$$

Let $(\tilde{Y}_1^z, \tilde{Y}_2^z)$ be the distribution of (Y_1^z, Y_2^z) conditioned on the event $(Y_1^z)_{I_1} = y_1^{I_1} \wedge (Y_2^z)_{I_2} = y_2^{I_2}$.

- (1) Now we claim that $(\tilde{Y}_1^z, \tilde{Y}_2^z)$ is $(P'_1, P'_2, 1 - \delta)$ -dense with $P'_1 = |I_1|$ and $P'_2 = |I_2|$. Suppose that $(\tilde{Y}_1^z, \tilde{Y}_2^z)$ is not $(P'_1, P'_2, 1 - \delta)$ -dense. Then there exists a pair of non-empty sets $J_1 \subseteq \overline{I_1}$ and $J_2 \subseteq \overline{I_2}$ and $(y_1^{J_1}, y_2^{J_2})$ such that

$$\begin{aligned} & \Pr \left[(\tilde{Y}_1^z)_{J_1} = y_1^{J_1} \wedge (\tilde{Y}_2^z)_{J_2} = y_2^{J_2} \right] \\ &= \Pr \left[(Y_1^z)_{J_1} = y_1^{J_1} \wedge (Y_2^z)_{J_2} = y_2^{J_2} \mid (Y_1^z)_{I_1} = y_1^{I_1} \wedge (Y_2^z)_{I_2} = y_2^{I_2} \right] \\ &> 2^{-(1-\delta) \left[|J_1| \lambda + \log p^{|J_2|} \right]}. \end{aligned}$$

The set $I_1 \cup J_1$ and $I_2 \cup J_2$ now form subsets for which

$$\begin{aligned} & \Pr \left[(Y_1^z)_{I_1 \cup J_1} = y_1^{I_1 \cup J_1} \wedge (Y_2^z)_{I_2 \cup J_2} = y_2^{I_2 \cup J_2} \right] \\ &= \Pr \left[(Y_1^z)_{I_1} = y_1^{I_1} \wedge (Y_1^z)_{J_1} = y_1^{J_1} \wedge (Y_2^z)_{I_2} = y_2^{I_2} \wedge (Y_2^z)_{J_2} = y_2^{J_2} \right] \\ &= \Pr \left[(Y_1^z)_{I_1} = y_1^{I_1} \wedge (Y_2^z)_{I_2} = y_2^{I_2} \right] \\ &\quad \cdot \Pr \left[(Y_1^z)_{J_1} = y_1^{J_1} \wedge (Y_2^z)_{J_2} = y_2^{J_2} \mid (Y_1^z)_{I_1} = y_1^{I_1} \wedge (Y_2^z)_{I_2} = y_2^{I_2} \right] \\ &> 2^{-(1-\delta) \left[|I_1| \lambda + \log p^{|I_2|} \right]} \cdot 2^{-(1-\delta) \left[|J_1| \lambda + \log p^{|J_2|} \right]} \\ &= 2^{-(1-\delta) \left[|I_1 \cup J_1| \lambda + \log p^{|I_2 \cup J_2|} \right]}, \end{aligned}$$

since I_i 's and J_i 's are disjoint for $i = 1, 2$. However, this contradicts the maximality of I_1 and I_2 .

- (2) Next, we claim that $|I_1| \lambda + |I_2| \log(p/e) \leq S_z / \delta$. Let $\mathcal{B}_{y_2^{I_2}}$ be the set of vectors $y_2^{\overline{I_2}}$ such that $y_2^{I_2}$ and $y_2^{\overline{I_2}}$ are a valid injection. Since $H_\infty(Y_1^z, Y_2^z) \geq 2^m \lambda + \log p! - S_z$, we observe that for any $y_1^{I_1}$ and $y_2^{I_2}$,

$$\begin{aligned} & \Pr \left[(Y_1^z)_{I_1} = y_1^{I_1} \wedge (Y_2^z)_{I_2} = y_2^{I_2} \right] \\ &= \sum_{\substack{y_1^{\overline{I_1}} \in (\{0,1\}^\lambda)^{2^m - |I_1|}, \\ y_2^{\overline{I_2}} \in \mathcal{B}_{y_2^{I_2}}}} \Pr \left[(Y_1^z)_{I_1} = y_1^{I_1} \wedge (Y_2^z)_{I_2} = y_2^{I_2} \wedge (Y_1^z)_{\overline{I_1}} = y_1^{\overline{I_1}} \wedge (Y_2^z)_{\overline{I_2}} = y_2^{\overline{I_2}} \right] \\ &\leq 2^{(2^m - |I_1|) \lambda} \cdot 2^{\log(p - |I_2|)!} \cdot 2^{-(2^m \lambda + \log p! - S_z)} \end{aligned}$$

$$= 2^{-(|I_1|\lambda + \log p^{|I_2|} - S_z)},$$

and, hence,

$$H_\infty((Y_1^z)_{I_1}, (Y_2^z)_{I_2}) \geq |I_1|\lambda + \log p^{|I_2|} - S_z. \quad (2)$$

On the other hand, because $((Y_1^z)_{I_1}, (Y_2^z)_{I_2})$ is not $(1 - \delta)$ -dense, we have

$$H_\infty((Y_1^z)_{I_1}, (Y_2^z)_{I_2}) < (1 - \delta) \left[|I_1|\lambda + \log p^{|I_2|} \right]. \quad (3)$$

Combining equation (2) and (3) together, we have

$$\begin{aligned} S_z &\geq \delta \left[|I_1|\lambda + \log p^{|I_2|} \right] \\ &\geq \delta [|I_1|\lambda + |I_2| \log(p/e)], \end{aligned}$$

where the last inequality comes from [CDG18, Proposition 38]¹⁵. Divided by $\delta > 0$, we get the result that $|I_1|\lambda + |I_2| \log(p/e) \leq S_z/\delta$.

Hence, $(\tilde{Y}_1^z, \tilde{Y}_2^z)$ is a $(P'_1, P'_2, 1 - \delta)$ -dense source such that $P'_1\lambda + P'_2 \log(p/e) \leq S_z/\delta$. Set (Y_1^z, Y_2^z) now to be $(\tilde{Y}_1^z, \tilde{Y}_2^z)$ conditioned on $(Y_1^z)_{I_1} \neq y_1^{I_1}$ and $(Y_2^z)_{I_2} \neq y_2^{I_2}$ and recursively decompose (Y_1^z, Y_2^z) as long as

$$\Pr[X_1^z \in \text{supp}(Y_1^z) \wedge X_2^z \in \text{supp}(Y_2^z)] > \gamma.$$

Observe that $H_\infty(Y_1^z, Y_2^z) \geq 2^m \lambda + \log p! - (S_z + \log \gamma^{-1})$ at any point in this decomposition process since

$$\begin{aligned} \Pr[Y_1^z = y_1 \wedge Y_2^z = y_2] &= \Pr[X_1^z = y_1 \wedge X_2^z = y_2 | X_1^z \in \text{supp}(Y_1^z) \wedge X_2^z \in \text{supp}(Y_2^z)] \\ &\leq \frac{\Pr[X_1^z = y_1 \wedge X_2^z = y_2]}{\Pr[\text{supp}(Y_1^z) \wedge X_2^z \in \text{supp}(Y_2^z)]} \\ &\leq \frac{2^{-(2^m \lambda + \log p! - S_z)}}{\gamma} \\ &= 2^{-(2^m \lambda + \log p! - (S_z + \log \gamma^{-1}))}. \end{aligned}$$

Note that $|\text{supp}(Y_1^z)|$ and $|\text{supp}(Y_2^z)|$ decreases in every step, and since $\text{supp}(X_1^z)$ and $\text{supp}(X_2^z)$ are finite, after finitely many steps, this process ends with $(Y_{1,\text{final}}^z, Y_{2,\text{final}}^z)$ with

$$\Pr[X_1^z \in \text{supp}(Y_{1,\text{final}}^z) \wedge X_{2,\text{final}}^z \in \text{supp}(Y_2^z)] > \gamma.$$

Hence, (X_1^z, X_2^z) is a convex combination of finitely many $(P'_1, P'_2, 1 - \delta)$ -dense sources and $(Y_{1,\text{final}}^z, Y_{2,\text{final}}^z)$, where P'_1 and P'_2 satisfy the inequality

$$P'_1\lambda + P'_2 \log(p/e) \leq (S_z + \log \gamma^{-1})/\delta.$$

This implies that (X_1^z, X_2^z) is γ -close to a convex combination of $(P'_1, P'_2, 1 - \delta)$ -dense sources where $P'_1\lambda + P'_2 \log(p/e) \leq (S_z + \log \gamma^{-1})/\delta$. $\square$

¹⁵ [CDG18, Proposition 38] says that $N^j \geq (N/e)^j$, where $a^b := a!/(a-b)!$.

Let $(\tilde{X}_1^z, \tilde{X}_2^z)$ be the convex combination of $(P'_1, P'_2, 1 - \delta)$ -dense sources that is γ -close to (X_1^z, X_2^z) for a $\delta = \delta_z$ to be determined later. For every $(P'_1, P'_2, 1 - \delta)$ source $(\tilde{X}_1, \tilde{X}_2)$ in said convex combination, let $(\tilde{Y}_1, \tilde{Y}_2)$ be the corresponding (P'_1, P'_2) -Bit-Fixing source, i.e., $(\tilde{X}_1, \tilde{X}_2)$ and $(\tilde{Y}_1, \tilde{Y}_2)$ are fixed on the same coordinates to the same values. The following claim bounds the distinguishing advantage between $(\tilde{X}_1, \tilde{X}_2)$ and $(\tilde{Y}_1, \tilde{Y}_2)$ for any (T_1, T_2) -query distinguisher.

Claim 2. For any $(P'_1, P'_2, 1 - \delta)$ -dense source $(\tilde{X}_1, \tilde{X}_2)$ and its corresponding (P'_1, P'_2) -Bit-Fixing source $(\tilde{Y}_1, \tilde{Y}_2)$, it holds that for any (adaptive) distinguisher $\mathcal{D}$ that queries at most T_1 coordinates of the random oracle H and makes at most T_2 forward or inverse queries to the injection map τ ,

$$\left| \Pr \left[\mathcal{D}^{\tilde{X}_1, \tilde{X}_2} = 1 \right] - \Pr \left[\mathcal{D}^{\tilde{Y}_1, \tilde{Y}_2} = 1 \right] \right| \leq \delta(T_1 \lambda + T_2 \log p),$$

and

$$\Pr \left[\mathcal{D}^{\tilde{X}_1, \tilde{X}_2} = 1 \right] \leq 2^{\delta(T_1 \lambda + T_2 \log p)} \cdot \Pr \left[\mathcal{D}^{\tilde{Y}_1, \tilde{Y}_2} = 1 \right].$$

Proof of Claim 2. Without loss of generality, assume that $\mathcal{D}$ is deterministic and does not query any of the fixed positions, make the same query twice, or make an inverse query after making the corresponding forward query or vice-versa. Let $T_{\tilde{X}_1, \tilde{X}_2}$ and $T_{\tilde{Y}_1, \tilde{Y}_2}$ be the random variables corresponding to the transcripts containing the query/answer pairs resulting from $\mathcal{D}$'s interaction with $(\tilde{X}_1, \tilde{X}_2)$ and $(\tilde{Y}_1, \tilde{Y}_2)$, respectively.

For a fixed transcript t , denote by $\mathbf{p}_{\tilde{X}_1, \tilde{X}_2}(t)$ and $\mathbf{p}_{\tilde{Y}_1, \tilde{Y}_2}(t)$ the probabilities that $(\tilde{X}_1, \tilde{X}_2)$ and $(\tilde{Y}_1, \tilde{Y}_2)$, respectively, produce the answers in t if the queries in t are asked. Since $\mathcal{D}$ is deterministic, $\Pr \left[T_{\tilde{X}_1, \tilde{X}_2} = t \right] \in \{0, \mathbf{p}_{\tilde{X}_1, \tilde{X}_2}(t)\}$, and similarly, $\Pr \left[T_{\tilde{Y}_1, \tilde{Y}_2} = t \right] \in \{0, \mathbf{p}_{\tilde{Y}_1, \tilde{Y}_2}(t)\}$. Denote by $\mathcal{T}_{X_1, X_2}$ the set of all transcripts t for which $\Pr \left[T_{\tilde{X}_1, \tilde{X}_2} = t \right] > 0$. For such t , $\Pr \left[T_{\tilde{X}_1, \tilde{X}_2} = t \right] = \mathbf{p}_{\tilde{X}_1, \tilde{X}_2}(t)$ and also $\Pr \left[T_{\tilde{Y}_1, \tilde{Y}_2} = t \right] = \mathbf{p}_{\tilde{Y}_1, \tilde{Y}_2}(t)$. Observe that for every transcript t ,

$$\mathbf{p}_{\tilde{X}_1, \tilde{X}_2}(t) \leq 2^{-(1-\delta) \left[T_1 \lambda + \log(p - P'_2) T_2 \right]} \text{ and } \mathbf{p}_{\tilde{Y}_1, \tilde{Y}_2}(t) \leq 2^{-\left[T_1 \lambda + \log(p - P'_2) T_2 \right]}, \quad (4)$$

as $(\tilde{X}_1, \tilde{X}_2)$ is $(P'_1, P'_2, 1 - \delta)$ -dense and $(\tilde{Y}_1, \tilde{Y}_2)$ is (P'_1, P'_2) -fixed.

Towards proving the first part of the claim, observe that $\mathcal{D}$'s output can be computed from the transcript (including whether a query was a forward for an inverse query) by just running $\mathcal{D}$ and providing the answers to its queries from the transcript. Hence,

$$\begin{aligned} \left| \Pr \left[\mathcal{D}^{\tilde{X}_1, \tilde{X}_2} = 1 \right] - \Pr \left[\mathcal{D}^{\tilde{Y}_1, \tilde{Y}_2} = 1 \right] \right| &\leq \text{SD} \left(T_{\tilde{X}_1, \tilde{X}_2}, T_{\tilde{Y}_1, \tilde{Y}_2} \right) \\ &= \sum_t \max \left\{ 0, \Pr \left[T_{\tilde{X}_1, \tilde{X}_2} = t \right] - \Pr \left[T_{\tilde{Y}_1, \tilde{Y}_2} = t \right] \right\} \\ &= \sum_{t \in \mathcal{T}_{X_1, X_2}} \max \left\{ 0, \mathbf{p}_{\tilde{X}_1, \tilde{X}_2}(t) - \mathbf{p}_{\tilde{Y}_1, \tilde{Y}_2}(t) \right\} \\ &= \sum_{t \in \mathcal{T}_{X_1, X_2}} \mathbf{p}_{\tilde{X}_1, \tilde{X}_2}(t) \cdot \max \left\{ 0, 1 - \frac{\mathbf{p}_{\tilde{Y}_1, \tilde{Y}_2}(t)}{\mathbf{p}_{\tilde{X}_1, \tilde{X}_2}(t)} \right\} \\ &\leq 1 - 2^{-\delta \left[T_1 \lambda + \log(p - P'_2) T_2 \right]} \end{aligned}$$

$$\leq 1 - 2^{-\delta(T_1\lambda + T_2 \log p)} \leq \delta(T_1\lambda + T_2 \log p),$$

where the first sum is over all possible transcripts and where the last inequality uses $2^{-x} \geq 1 - x$ for $x \geq 0$ and $a^b \leq a^b$ for $a, b \in \mathbb{N}$.

As for the second part of the claim, observe that due to the equation (4) and the support of $T_{\tilde{X}_1, \tilde{X}_2}$ being a subset of $T_{\tilde{Y}_1, \tilde{Y}_2}$,

$$\begin{aligned} \Pr [T_{\tilde{X}_1, \tilde{X}_2} = t] &\leq 2^{\delta(T_1\lambda + \log(p-P'_2)T_2)} \cdot \Pr [T_{\tilde{Y}_1, \tilde{Y}_2} = t] \\ &\leq 2^{\delta(T_1\lambda + T_2 \log p)} \cdot \Pr [T_{\tilde{Y}_1, \tilde{Y}_2} = t], \end{aligned}$$

for any transcript t . Let $\mathcal{T}_{\mathcal{D}}$ be the set of transcripts where $\mathcal{D}$ outputs 1. Then we have

$$\begin{aligned} \Pr [\mathcal{D}^{\tilde{X}_1, \tilde{X}_2} = 1] &= \sum_{t \in \mathcal{T}_{\mathcal{D}}} \Pr [T_{\tilde{X}_1, \tilde{X}_2} = t] \\ &\leq 2^{\delta(T_1\lambda + T_2 \log p)} \cdot \sum_{t \in \mathcal{T}_{\mathcal{D}}} \Pr [T_{\tilde{Y}_1, \tilde{Y}_2} = t] \\ &= 2^{\delta(T_1\lambda + T_2 \log p)} \cdot \Pr [\mathcal{D}^{\tilde{Y}_1, \tilde{Y}_2} = 1], \end{aligned}$$

which concludes the proof. $\square$

Let $(\tilde{Y}_1^z, \tilde{Y}_2^z)$ be obtained by replacing $(\tilde{X}_1, \tilde{X}_2)$ by $(\tilde{Y}_1, \tilde{Y}_2)$ in $(\tilde{X}_1^z, \tilde{X}_2^z)$. Setting

$$\delta_z = \frac{S_z + \log \gamma^{-1}}{P_1 k + P_2 \log(p/e)},$$

Claim 1 and **Claim 2** imply

$$\begin{aligned} &\left| \Pr [\mathcal{D}^{X_1^z, X_2^z}(z) = 1] - \Pr [\mathcal{D}^{\tilde{Y}_1^z, \tilde{Y}_2^z}(z) = 1] \right| \\ &\leq \frac{(S_z + \log \gamma^{-1})(T_1\lambda + T_2 \log p)}{(P_1\lambda + P_2 \log(p/e))} + \gamma, \end{aligned} \tag{5}$$

as well as

$$\begin{aligned} &\Pr [\mathcal{D}^{X_1^z, X_2^z}(z) = 1] \\ &\leq 2^{\frac{(S_z + \log \gamma^{-1})(T_1\lambda + T_2 \log p)}{(P_1\lambda + P_2 \log(p/e))}} \cdot \Pr [\mathcal{D}^{\tilde{Y}_1^z, \tilde{Y}_2^z}(z) = 1] + \gamma. \end{aligned} \tag{6}$$

Moreover, note that for the above choice of δ_z , we have $P'_1 = P_1$ and $P'_2 = P_2$, i.e., the sources $(\tilde{Y}_1^z, \tilde{Y}_2^z)$ are (P_1, P_2) -fixed, as desired.

Claim 3. $\mathbb{E}_z[S_z] \leq S$ and $\Pr[S_{f(X_1, X_2)} > S + \log \gamma^{-1}] \leq \gamma$.

Proof of Claim 3. Observe that $H_\infty(X_1^z, X_2^z) = H_\infty((X_1, X_2)|Z = z) = H((X_1, X_2)|Z = z)$ since, conditioned on $Z = z$, (X_1, X_2) is distributed uniformly over all values (x_1, x_2) with $f(x_1, x_2) = z$. Hence,

$$\mathbb{E}_z[S_z] = 2^m \lambda + \log p! - \mathbb{E}_z[H_\infty((X_1, X_2)|Z = z)]$$

$$\begin{aligned}
&= 2^m \lambda + \log p! - \mathbb{E}_z [H((X_1, X_2)|Z = z)] \\
&= 2^m \lambda + \log p! - H((X_1, X_2)|Z = z) \leq S.
\end{aligned}$$

Again, due to the uniformity of (X_1, X_2) , $\Pr[f(X_1, X_2) = z] = 2^{-S_z}$. Hence,

$$\begin{aligned}
\Pr[S_{f(X_1, X_2)} > S + \log \gamma^{-1}] &= \sum_{z \in \{0,1\}^S : S_z > S + \log \gamma^{-1}} \Pr[f(X_1, X_2) = z] \\
&\leq 2^S \cdot 2^{-(S + \log \gamma^{-1})} \leq \gamma.
\end{aligned}$$

□

Now the first part of the lemma follows (using $(Y_1^z, Y_2^z) := (\tilde{Y}_1^z, \tilde{Y}_2^z)$) by taking expectations over z of the equation (5) and applying the first part of [Claim 3](#). The second part of the lemma is proved as follows:

$$\begin{aligned}
&\Pr[\mathcal{D}^{X_1, X_2}(f(X_1, X_2)) = 1] \\
&\leq \Pr[\mathcal{D}^{X_1, X_2}(f(X_1, X_2)) = 1, S_{f(X_1, X_2)} \leq S + \log \gamma^{-1}] \\
&\quad + \Pr[S_{f(X_1, X_2)} > S + \log \gamma^{-1}] \\
&\leq \left(2^{(S + \log \gamma^{-1})(T_1 \lambda + T_2 \log p)/W} \cdot \Pr[\mathcal{D}^{X_1, X_2}(f(X_1, X_2)) = 1, \right. \\
&\quad \left. S_{f(X_1, X_2)} \leq S + \log \gamma^{-1}] + \gamma \right) + \gamma \\
&\leq 2^{(S + \log \gamma^{-1})(T_1 \lambda + T_2 \log p)/W} \cdot \Pr[\mathcal{D}^{X_1, X_2}(f(X_1, X_2)) = 1, \\
&\quad S_{f(X_1, X_2)} \leq S + \log \gamma^{-1}] + 2\gamma,
\end{aligned}$$

where the second inequality follows by taking expectations over z of the equation (6) (together with the condition $S_z \leq S + \log \gamma^{-1}$) and the second part of [Claim 3](#). □

A.2 From the BF-ROM+GGM to the AI-ROM+GGM

Capturing the Models. An oracle pair $(\mathcal{O}_1, \mathcal{O}_2)$ has two interfaces $(\mathcal{O}_1.\text{pre}, \mathcal{O}_2.\text{pre})$ and $(\mathcal{O}_1.\text{on}, \mathcal{O}_2.\text{on})$, where $(\mathcal{O}_1.\text{pre}, \mathcal{O}_2.\text{pre})$ is accessible only once before any calls to $(\mathcal{O}_1.\text{on}, \mathcal{O}_2.\text{on})$ are made. We consider the oracles in this work as follows:

- **Random Oracle + Generic Group Oracle RO+GG(m, λ, p, ℓ)**: Samples a random oracle function table $H \leftarrow \mathcal{H}_{m, \lambda}$, where $\mathcal{H}_{m, \lambda}$ is the set of all functions from $\{0, 1\}^m \rightarrow \{0, 1\}^\lambda$, and samples a random injection $\tau \leftarrow \mathcal{I}_{p, \ell}$, where $\mathcal{I}_{p, \ell}$ is the set of all injections from $\mathbb{Z}_p \rightarrow \mathbb{G}$ where $\mathbb{G}$ is the set of bitstrings of length $\ell \geq \log p$; offers no functionality at $(\mathcal{O}_1.\text{pre}, \mathcal{O}_2.\text{pre})$; answers queries $x \in \{0, 1\}^m$ to H at $\mathcal{O}_1.\text{on}$ by the corresponding value $H(x) \in \{0, 1\}^\lambda$; answers *forward* queries $x' \in \mathbb{Z}_p$ to τ at $\mathcal{O}_2.\text{on}$ by the corresponding value $\tau(x') \in \mathbb{G}$; answers *group-operation* queries (s, s') at $\mathcal{O}_2.\text{on}$ as follows: if $s = \tau(x)$ and $s' = \tau(x')$ for some x, x' , the oracle replies by $\tau(x + x')$ and by $\perp$ otherwise; answers *inverse* queries s at $\mathcal{O}_2.\text{on}$ by returning $\tau^{-1}(s)$ if s is in the range of τ and $\perp$ otherwise.
- **Auxiliary-Input Random Oracle + Generic Group Oracle AI-RO+GG(m, λ, p, ℓ)**: Samples a random oracle function table $H \leftarrow \mathcal{H}_{m, \lambda}$ and samples a random injection $\tau \leftarrow \mathcal{I}_{p, \ell}$ as explained in RO+GG(m, λ, p, ℓ); outputs all of H at $\mathcal{O}_1.\text{pre}$ and all of τ at $\mathcal{O}_2.\text{pre}$; $(\mathcal{O}_1.\text{on}, \mathcal{O}_2.\text{on})$ behaves the same as RO+GG(m, λ, p, ℓ). When the parameters (m, λ, p, ℓ) are clear in context, we sometimes abuse the notation and simply say AI-RO+GG.

- **Bit-Fixing Random Oracle + Generic Group Oracle** $\text{BF-RO+GG}(P_1, P_2, m, \lambda, p, \ell)$: Samples a random oracle function table $H \leftarrow \mathcal{H}_{m,\lambda}$ at $\mathcal{O}_1.\text{pre}$ and samples a random size- p subset $\mathcal{Y}$ of $\mathbb{G}$ and outputs $\mathcal{Y}$ at $\mathcal{O}_2.\text{pre}$; takes a list at $(\mathcal{O}_1.\text{pre}, \mathcal{O}_2.\text{pre})$ of at most P_1 query/answer pairs that override H in the corresponding positions and at most P_2 query/answer pairs without collisions and all answers in $\mathcal{Y}$; samples a random injection $\tau \leftarrow \mathcal{I}_{p,\ell}$ with range $\mathcal{Y}$ and consistent with said list; $(\mathcal{O}_1.\text{on}, \mathcal{O}_2.\text{on})$ behaves the same as $\text{RO+GG}(m, \lambda, p, \ell)$. When the parameters (m, λ, p, ℓ) are clear in context, we sometimes abuse the notation and simply say $\text{BF-RO+GG}(P_1, P_2)$.

Attackers with Oracle-Dependent Advice. We define the attackers $\mathcal{A} = (\mathcal{A}_{\text{pre}}, \mathcal{A}_{\text{on}})$ similar to that of prior work [CDGS18, CDG18], which consist of a preprocessing attacker $\mathcal{A}_{\text{pre}}$ and an online attacker $\mathcal{A}_{\text{on}}$, which carries out the actual attack using the output of $\mathcal{A}_{\text{pre}}$. The difference is that we consider a pair of oracles instead of a single oracle. More precisely, in the presence of an oracle pair $(\mathcal{O}_1, \mathcal{O}_2)$, $\mathcal{A}_{\text{pre}}$ interacts with $(\mathcal{O}_1.\text{pre}, \mathcal{O}_2.\text{pre})$ and $\mathcal{A}_{\text{on}}$ with $(\mathcal{O}_1.\text{on}, \mathcal{O}_2.\text{on})$.

Definition 5. An (S, T_1, T_2) -attacker $\mathcal{A} = (\mathcal{A}_{\text{pre}}, \mathcal{A}_{\text{on}})$ in the $(\mathcal{O}_1, \mathcal{O}_2)$ -model consists of two procedures

- $\mathcal{A}_{\text{pre}}$, which is computationally unbounded, interacts with $(\mathcal{O}_1.\text{pre}, \mathcal{O}_2.\text{pre})$, and outputs an S -bit string as hint, and
- $\mathcal{A}_{\text{on}}$, which takes an S -bit auxiliary input and makes at most T_1 queries to $\mathcal{O}_1.\text{on}$ and T_2 queries to $\mathcal{O}_2.\text{on}$.

For an arbitrary oracle $\mathcal{O}$, Coretti et al. [CDGS18, CDG18] defined an *application* G in the $\mathcal{O}$ -model by specifying a challenger C that has access to $\mathcal{O}.\text{on}$, interacts with the online attacker $\mathcal{A}_{\text{on}}$ of an attacker $\mathcal{A} = (\mathcal{A}_{\text{pre}}, \mathcal{A}_{\text{on}})$, and outputs a bit at the end of the interaction. Then they defined the *success* of $\mathcal{A}$ on G in the $\mathcal{O}$ -model as follows:

$$\text{Succ}_{G,\mathcal{O}}(\mathcal{A}) := \Pr [\mathcal{A}_{\text{on}}^{\mathcal{O}.\text{on}}(\mathcal{A}_{\text{pre}}^{\mathcal{O}.\text{pre}}) \leftrightarrow C^{\mathcal{O}.\text{on}} = 1],$$

where $\mathcal{A}_{\text{on}}^{\mathcal{O}.\text{on}}(\mathcal{A}_{\text{pre}}^{\mathcal{O}.\text{pre}}) \leftrightarrow C^{\mathcal{O}.\text{on}}$ denotes the bit output by the challenger C after its interaction with $\mathcal{A}$. It can be extended to the oracle pair setting in the same manner when $\mathcal{O} = (\mathcal{O}_1, \mathcal{O}_2)$, by defining $\mathcal{O}.\text{pre} := (\mathcal{O}_1.\text{pre}, \mathcal{O}_2.\text{pre})$ and $\mathcal{O}.\text{on} := (\mathcal{O}_1.\text{on}, \mathcal{O}_2.\text{on})$.

Definition 6. For an indistinguishability application G in the $(\mathcal{O}_1, \mathcal{O}_2)$ -model, the advantage of an attacker $\mathcal{A}$ is defined as

$$\text{Adv}_{G,(\mathcal{O}_1,\mathcal{O}_2)}(\mathcal{A}) := 2 \left| \text{Succ}_{G,(\mathcal{O}_1,\mathcal{O}_2)}(\mathcal{A}) - \frac{1}{2} \right|.$$

In the case of an unpredictability application G , the advantage is defined as $\text{Adv}_{G,(\mathcal{O}_1,\mathcal{O}_2)}(\mathcal{A}) := \text{Succ}_{G,(\mathcal{O}_1,\mathcal{O}_2)}(\mathcal{A})$. An application G is said to be $(S, T_1, T_2, \varepsilon)$ -secure in the $(\mathcal{O}_1, \mathcal{O}_2)$ -model if for every (S, T_1, T_2) -attacker $\mathcal{A}$,

$$\text{Adv}_{G,(\mathcal{O}_1,\mathcal{O}_2)}(\mathcal{A}) \leq \varepsilon.$$

Additive Error for Arbitrary Applications in the ROM+GGM. Using Lemma 3, one can extend [CDGS18, Theorem 5] and [CDG18, Theorem 1] to the translation from the Bit-Fixing model with multiple idealized models (ROM+GGM) to the corresponding Auxiliary-Input model at the cost of an additive term which is similar to those from [CDGS18, Theorem 5] and [CDG18, Theorem 1].

Reminder of Theorem 5. For every $\gamma > 0$, if an application G is $(S, T_1, T_2, \varepsilon')$ -secure in BF-RO+GG $(P_1, P_2, m, \lambda, p, \ell)$ for any $P_1, P_2 \in \mathbb{N}$ such that $P_1\lambda + P_2 \log(p/e) \leq \eta$, then it is $(S, T_1, T_2, \varepsilon)$ -secure in AI-RO+GG (m, λ, p, ℓ) , for

$$\varepsilon \leq \varepsilon' + \frac{2(S + \log \gamma^{-1}) ((T_1)_G^{\text{comb}} \lambda + 3(T_2)_G^{\text{comb}} \log p)}{\eta} + 2\gamma,$$

where e is the Euler constant, and $(T_1)_G^{\text{comb}}, (T_2)_G^{\text{comb}}$ are the combined query complexity corresponding to G across the random oracle and the generic group oracle.

Proof. Fix P_1, P_2 as well as γ . Let G be an arbitrary application in the (BF-RO, BF-GG)-model (or in the (AI-RO, AI-GG)-model) and C be the corresponding challenger. Moreover, fix an (S, T_1, T_2) -attacker $\mathcal{A} = (\mathcal{A}_{\text{pre}}, \mathcal{A}_{\text{on}})$, and let $\{(Y_1, Y_2)^{z, \mathcal{Y}}\}$, indexed by $z \in \{0, 1\}^S$ and size- p subsets $\mathcal{Y} \subseteq \mathbb{G}$, be the family of distributions guaranteed to exist by Lemma 3. Consider the following (S, T_1, T_2) -attacker $\mathcal{A}' = (\mathcal{A}'_{\text{pre}}, \mathcal{A}'_{\text{on}})$ (expecting to interact with BF-RO+GG):

- $\mathcal{A}'_{\text{pre}}$ obtains the set $\mathcal{Y}$ from BF-GG.pre and internally simulates $\mathcal{A}_{\text{pre}}$ on a uniformly random input $X_1 \in \{0, 1\}^m$ and a uniformly random injection $X_2 \in \mathbb{Z}_p$ with range $\mathcal{Y}$ to obtain $z \leftarrow \mathcal{A}_{\text{pre}}^{\text{AI-RO+GG.pre}}$. Then, it samples one of the (P_1, P_2) -bit-fixing sources (Y'_1, Y'_2) making up $(Y_1, Y_2)^{z, \mathcal{Y}}$ and presets BF-RO+GG to match (Y'_1, Y'_2) on the at most P_1 (resp. P_2) points where Y'_1 (resp. Y'_2) is fixed. The output of $\mathcal{A}'_{\text{pre}}$ is z .
- $\mathcal{A}'_{\text{on}}$ works exactly the same as $\mathcal{A}_{\text{on}}$.

Let $\mathcal{D}$ be a distinguisher — making forward, group-operation, and inverse queries to a (H, τ) -source — that internally runs the combination of $\mathcal{A}_{\text{on}} = \mathcal{A}'_{\text{on}}$ and C . It answers their queries as follows:

- (1) It passes forward queries to and back from its own oracle.
- (2) It answers group-operation queries (s, s') (to the generic group oracle) by making two backward queries to its own oracle for s and s' , obtaining i and j , respectively, making a forward query $i + j$, and passing the answer to $\mathcal{A}_{\text{on}}$ or C (unless one of the answers to the backward queries was $\perp$, in which case $\perp$ is returned).

Note that $\mathcal{D}^{X_1, X_2}(f(X_1, X_2))$ is identical to

$$\mathcal{A}_{\text{on}}^{\text{AI-RO+GG.on}}(\mathcal{A}_{\text{pre}}^{\text{AI-RO+GG.pre}}) \leftrightarrow C^{\text{AI-RO+GG.on}},$$

and $\mathcal{D}^{(Y_1, Y_2)^{f(X_1, X_2), \text{im}(X_2)}}(f(X_1, X_2))$ is identical to

$$\mathcal{A}'_{\text{on}}^{\text{BF-RO+GG.on}}(\mathcal{A}'_{\text{pre}}^{\text{BF-RO+GG.pre}}) \leftrightarrow C^{\text{BF-RO+GG.on}}.$$

Furthermore, $\mathcal{D}$ is a distinguisher taking an S -bit input and making at most $(T_1)_G^{\text{comb}}$ to its random oracle and $3(T_2)_G^{\text{comb}}$ to its generic group oracle. Therefore, by Lemma 3,

$$\begin{aligned} \text{Succ}_{G, \text{AI-RO+GG}}(\mathcal{A}) &\leq \text{Succ}_{G, \text{BF-RO+GG}}(\mathcal{A}') \\ &\quad + \frac{(S + \log \gamma^{-1}) ((T_1)_G^{\text{comb}} \lambda + 3(T_2)_G^{\text{comb}} \log p)}{\eta} + \gamma, \end{aligned}$$

for any P_1, P_2 satisfying $P_1\lambda + P_2 \log(p/e)$. Since there is only an additive term between the two success probabilities, the above inequality implies

$$\begin{aligned} \text{Adv}_{G, \text{AI-RO+GG}}(\mathcal{A}) &\leq \text{Adv}_{G, \text{BF-RO+GG}}(\mathcal{A}') \\ &\quad + \frac{2(S + \log \gamma^{-1}) ((T_1)_G^{\text{comb}} \lambda + 3(T_2)_G^{\text{comb}} \log p)}{\eta} + 2\gamma, \end{aligned}$$

for both indistinguishability and unpredictability applications. Note that the extra factor of 2 is technically only necessary for indistinguishability applications [CDGS18]. $\square$

Reminder of Theorem 6. For every $\gamma > 0$, if an unpredictability application G is $(S, T_1, T_2, \varepsilon')$ -secure in $\text{BF-RO+GG}(P_1, P_2, m, \lambda, p, \ell)$ for any $P_1, P_2 \in \mathbb{N}$ satisfying $(S + \log \gamma^{-1}) \cdot ((T_1)_G^{\text{comb}} \lambda + 3(T_2)_G^{\text{comb}} \log p) \leq P_1 \lambda + P_2 \log(p/e)$, then it is $(S, T_1, T_2, \varepsilon)$ -secure in $\text{AI-RO+GG}(m, \lambda, p, \ell)$, for $\varepsilon \leq 2\varepsilon' + 2\gamma$, where e is the Euler constant.

Proof. Using the same attacker $\mathcal{A}'$ as in the proof of Theorem 5 and applying the second part of Lemma 3, for any $P_1, P_2 \in \mathbb{N}$ such that $(S + \log \gamma^{-1}) ((T_1)_G^{\text{comb}} \lambda + 3(T_2)_G^{\text{comb}} \log p) \leq P_1 \lambda + P_2 \log(p/e) \leq \eta$, we have

$$\begin{aligned} \text{Succ}_{G, \text{AI-RO+GG}}(\mathcal{A}) &\leq 2 \frac{(S + \log \gamma^{-1}) ((T_1)_G^{\text{comb}} \lambda + 3(T_2)_G^{\text{comb}} \log p)}{\eta} \cdot \text{Succ}_{G, \text{BF-RO+GG}}(\mathcal{A}') + 2\gamma \\ &\leq 2 \cdot \text{Succ}_{G, \text{BF-RO+GG}}(\mathcal{A}') + 2\gamma, \end{aligned}$$

which translates into

$$\text{Adv}_{G, \text{AI-RO+GG}}(\mathcal{A}) \leq 2 \cdot \text{Adv}_{G, \text{BF-RO+GG}}(\mathcal{A}') + 2\gamma$$

for unpredictability applications. $\square$

B Bit-Fixing/Auxiliary-Input Multiple Idealized Models

In Section 3, we studied the Bit-Fixing to Auxiliary-Input transition with two idealized models at once — the Random Oracle Model plus the Generic Group Model. We can further extend this approach and consider multiple idealized models, including the Ideal Cipher Model and the Random Permutation Model as well.

B.1 Replacing Auxiliary Information by Bit-Fixing

Definition 7. A (n_1, n_2, n_3) -source is a tuple of vector random variables $(\mathbf{X}, \mathbf{Y}, \mathbf{Z})$, where

- $\mathbf{X} = (X_1, \dots, X_{n_1})$ is an n_1 -dimensional vector where for each $i \in [n_1]$, an element X_i is a random variable that corresponds to the function table of a random oracle $H_i : [M_i] \rightarrow [N_i]$ with range $[N_i]^{M_i}$,
- $\mathbf{Y} = (Y_1, \dots, Y_{n_2})$ is an n_2 -dimensional vector where for each $i \in [n_2]$, an element Y_i is a random variable that corresponds to the function table of an injection $\tau_i : \mathbb{Z}_{p_i} \rightarrow \mathbb{G}_i$, where $\mathbb{G}_i$ is the set of bitstrings of length ℓ_i (with $\ell_i \geq \log p_i$), and
- $\mathbf{Z} = (Z_1, \dots, Z_{n_3})$ is an n_3 -dimensional vector where for each $i \in [n_3]$, an element Z_i is a random variable that corresponds to the function table of a cipher $F_i : [K_i] \times [C_i] \rightarrow [C_i]$.

Further, a (n_1, n_2, n_3) -source is called:

- $(\mathbf{P}, \mathbf{Q}, \mathbf{R}, 1 - \delta)$ -dense if for vectors $\mathbf{P} = (P_1, \dots, P_{n_1}) \in \mathbb{Z}_{M_1} \times \dots \times \mathbb{Z}_{M_{n_1}}$, $\mathbf{Q} = (Q_1, \dots, Q_{n_2}) \in \mathbb{Z}_{p_1} \times \dots \times \mathbb{Z}_{p_{n_2}}$, and $\mathbf{R} = (\mathbf{R}_1, \dots, \mathbf{R}_{n_3}) \in (\mathbb{Z}_{C_1})^{K_1} \times \dots \times (\mathbb{Z}_{C_{n_3}})^{K_{n_3}}$ where $\mathbf{R}_k = (R_{k,1}, \dots, R_{k,K_k})$ for each $k \in [n_3]$, if $\mathbf{H}_i$ is fixed on at most P_i coordinates for each $i \in [n_1]$, τ_j is fixed on at most Q_j coordinates for each $j \in [n_2]$, and for each cipher F_k with $k \in [n_3]$, it is fixed on at most $R_{k,s}$ coordinates for each $s \in [K_k]$, and if for all tuples of vectors of subsets $(\mathbf{U}, \mathbf{V}, \mathbf{W})$ of non-fixed coordinates where $\mathbf{U} = (U_1, \dots, U_{n_1}) \subseteq \mathbb{Z}_{M_1} \times \dots \times \mathbb{Z}_{M_{n_1}}$, $\mathbf{V} = (V_1, \dots, V_{n_2}) \subseteq \mathbb{Z}_{p_1} \times \dots \times \mathbb{Z}_{p_{n_2}}$, and $\mathbf{W} = (\mathbf{W}_1, \dots, \mathbf{W}_{n_3}) \subseteq (\mathbb{Z}_{C_1})^{K_1} \times \dots \times (\mathbb{Z}_{C_{n_3}})^{K_{n_3}}$ where $\mathbf{W}_k = (W_{k,1}, \dots, W_{k,K_k})$ for each $k \in [n_3]$,

$$H_\infty(\mathbf{X}_\mathbf{U}, \mathbf{Y}_\mathbf{V}, \mathbf{Z}_\mathbf{W}) \geq (1 - \delta) \left[\sum_{i=1}^{n_1} |U_i| \log N_i + \sum_{j=1}^{n_2} \log(p_j - Q_j)^{\frac{|V_j|}{p_j}} + \sum_{k=1}^{n_3} \sum_{s=1}^{K_k} \log(C_k - R_{k,s})^{\frac{|W_{k,s}|}{C_k}} \right],$$

where $a^b := a!/(a-b)!$ and $\mathbf{X}_\mathbf{U}$ is $\mathbf{X}$ restricted to the coordinates in $\mathbf{U}$,

- $(1 - \delta)$ -dense if it is $(\mathbf{0}, \mathbf{0}, \mathbf{0}, 1 - \delta)$ -dense, and
- $(\mathbf{P}, \mathbf{Q}, \mathbf{R})$ -bit-fixing if it is $(\mathbf{P}, \mathbf{Q}, \mathbf{R}, 1)$ -dense, i.e., $(\mathbf{P}, \mathbf{Q}, \mathbf{R}, 1 - \delta)$ -dense with $\delta = 0$.

In [Definition 7](#), we remark that $\mathbf{X}$ captures a source with multiple random oracles, $\mathbf{Y}$ captures a source with multiple generic groups, and $\mathbf{Z}$ captures a source with multiple ideal ciphers. In the case of $\mathbf{Z}$, if $K_i = 1$ for some $i \in [n_3]$, it can be considered as a random permutation. Hence, [Definition 7](#) captures the usage of multiple idealized models at once, including the random oracle model, generic group model, ideal cipher model, and random permutation model.

Lemma 4. Let $(\mathbf{X}, \mathbf{Y}, \mathbf{Z})$ be a tuple of vector random variables which satisfies the following.

- $\mathbf{X} = (X_1, \dots, X_{n_1})$ is an n_1 -dimensional vector where X_i is distributed uniformly over $[N_i]^{M_i}$ for each $i \in [n_1]$,
- $\mathbf{Y} = (Y_1, \dots, Y_{n_2})$ is an n_2 -dimensional vector where Y_i is a uniformly random injection that takes on as value function tables corresponding to an injection $\tau_i : \mathbb{Z}_{p_i} \rightarrow \mathbb{G}_i$ for each $i \in [n_2]$, and
- $\mathbf{Z} = (Z_1, \dots, Z_{n_3})$ is an n_3 -dimensional vector where Z_i takes on as value function tables corresponding to a cipher $F_i : [K_i] \times [C_i] \rightarrow [C_i]$ for each $i \in [n_3]$.

Let $W = f(\mathbf{X}, \mathbf{Y}, \mathbf{Z})$, where

$$f : \left(\prod_{i=1}^{n_1} [M_i] \right) \times \left(\prod_{j=1}^{n_2} \mathbb{Z}_{p_j} \right) \times \left(\prod_{k=1}^{n_3} [K_k] \times [C_k] \right) \rightarrow \{0, 1\}^S$$

is an arbitrary function. For any $\gamma > 0$ and $\mathbf{P}, \mathbf{Q}, \mathbf{R}$ that were defined as in [Definition 7](#), there exists a family $\{(\mathbf{X}', \mathbf{Y}', \mathbf{Z}')\}_{W, \mathcal{Y}}$, indexed by $W \in \{0, 1\}^S$ and $\mathcal{Y} = (\mathcal{Y}_1, \dots, \mathcal{Y}_{n_2})$ where $\mathcal{Y}_i$ is a size- p_i subset of $\mathbb{G}_i$ for each $i \in [n_2]$, of convex combinations of $(\mathbf{P}, \mathbf{Q}, \mathbf{R})$ -bit-fixing (n_1, n_2, n_3) -sources satisfying

$$\sum_{i=1}^{n_1} P_i \log N_i + \sum_{j=1}^{n_2} Q_j \log(p_j/e) + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} R_{i,j} \log(C_i/e) \leq \eta,$$

such that for any distinguisher $\mathcal{D}$ taking an S -bit input and querying at most $\mathbf{S} = (S_1, \dots, S_{n_1})$ coordinates of each oracle that corresponds to $\mathbf{X}$ where $S_i < P_i$ for each $i \in [n_1]$, $\mathbf{T} = (T_1, \dots, T_{n_2})$ coordinates of each

oracle that corresponds to $\mathbf{Y}$ where $T_i < Q_i$ for each $i \in [n_2]$, and $\mathbf{L} = (\mathbf{L}_1, \dots, \mathbf{L}_{n_3})$ coordinates of each oracle that corresponds to $\mathbf{Z}$ where $\mathbf{L}_k = (L_{k,1}, \dots, L_{k,K_k})$ and $L_{i,j} < R_{i,j}$ for each $j \in [K_i]$ and $i \in [n_3]$, we have (here, e is the Euler's number)

$$\begin{aligned} & \left| \Pr [\mathcal{D}^{\mathbf{X}, \mathbf{Y}, \mathbf{Z}}(f(\mathbf{X}, \mathbf{Y}, \mathbf{Z})) = 1] - \Pr [\mathcal{D}^{(\mathbf{X}', \mathbf{Y}', \mathbf{Z}')}_{f(\mathbf{X}, \mathbf{Y}, \mathbf{Z}), \text{im}(\mathbf{Y})}(f(\mathbf{X}, \mathbf{Y}, \mathbf{Z})) = 1] \right| \\ & \leq \frac{(S + \log \gamma^{-1}) \left(\sum_{i=1}^{n_1} S_i \log N_i + \sum_{i=1}^{n_2} T_i \log p_i + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} L_{i,j} \log C_i \right)}{\eta} + \gamma, \end{aligned}$$

and

$$\begin{aligned} & \Pr [\mathcal{D}^{\mathbf{X}, \mathbf{Y}, \mathbf{Z}}(f(\mathbf{X}, \mathbf{Y}, \mathbf{Z})) = 1] \\ & \leq 2 \frac{(S + \log \gamma^{-1}) \left(\sum_{i=1}^{n_1} S_i \log N_i + \sum_{i=1}^{n_2} T_i \log p_i + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} L_{i,j} \log C_i \right)}{\eta} \cdot \Pr [\mathcal{D}^{(\mathbf{X}', \mathbf{Y}', \mathbf{Z}')}_{f(\mathbf{X}, \mathbf{Y}, \mathbf{Z}), \text{im}(\mathbf{Y})}(f(\mathbf{X}, \mathbf{Y}, \mathbf{Z})) = 1] + 2\gamma. \end{aligned}$$

Proof. Fix an arbitrary $w \in \{0, 1\}^S$ and let $(\mathbf{X}^w, \mathbf{Y}^w, \mathbf{Z}^w)$ be the distribution of $(\mathbf{X}, \mathbf{Y}, \mathbf{Z})$ conditioned on $f(\mathbf{X}, \mathbf{Y}, \mathbf{Z}) = w$. Let $S_z = \sum_{i=1}^{n_1} M_i \log N_i + \sum_{i=1}^{n_2} \log p_i! + \sum_{i=1}^{n_3} K_i \log C_i! - H_\infty(\mathbf{X}, \mathbf{Y}, \mathbf{Z})$ be the min-entropy deficiency of $(\mathbf{X}^w, \mathbf{Y}^w, \mathbf{Z}^w)$. Let $\gamma > 0$ be arbitrary.

Claim 4. For every $\delta > 0$, any leaky source $(\mathbf{X}^w, \mathbf{Y}^w, \mathbf{Z}^w)$ is γ -close to a source $(\mathbf{X}'^w, \mathbf{Y}'^w, \mathbf{Z}'^w)$ which is a convex combination of $(\mathbf{P}', \mathbf{Q}', \mathbf{R}', 1 - \delta)$ -dense sources for

$$\sum_{i=1}^{n_1} P'_i \log N_i + \sum_{i=1}^{n_2} Q'_i \log(p_i/e) + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} R'_{i,j} \log(C_i/e) \leq \frac{S_z + \log \gamma^{-1}}{\delta}.$$

Proof of Claim 4. Without loss of generality, we can assume that $(\mathbf{X}^w, \mathbf{Y}^w, \mathbf{Z}^w)$ is not $(1 - \delta)$ -dense since if it is $(1 - \delta)$ -dense then we can trivially find such $(\mathbf{P}', \mathbf{Q}', \mathbf{R}', 1 - \delta)$ -dense sources. Let $(\mathbf{X}'^w, \mathbf{Y}'^w, \mathbf{Z}'^w) := (\mathbf{X}^w, \mathbf{Y}^w, \mathbf{Z}^w)$ and $\mathbf{U} = (U_1, \dots, U_{n_1}) \in \mathbb{Z}_{M_1} \times \dots \times \mathbb{Z}_{M_{n_1}}$, $\mathbf{V} = (V_1, \dots, V_{n_2}) \in \mathbb{Z}_{p_1} \times \dots \times \mathbb{Z}_{p_{n_2}}$, and $\mathbf{W} = (\mathbf{W}_1, \dots, \mathbf{W}_{n_3}) \in (\mathbb{Z}_{C_1})^{K_1} \times \dots \times (\mathbb{Z}_{C_{n_3}})^{K_{n_3}}$ (where $\mathbf{W}_k = (W_{k,1}, \dots, W_{k,K_k})$ for each $k \in [n_3]$) be the largest subset for which there exists a violation, i.e., there exists $\mathbf{x}'_{\mathbf{U}}, \mathbf{y}'_{\mathbf{V}}, \mathbf{z}'_{\mathbf{W}}$ such that

$$\begin{aligned} & \Pr [(\mathbf{X}'^w)_{\mathbf{U}} = \mathbf{x}'_{\mathbf{U}} \wedge (\mathbf{Y}'^w)_{\mathbf{V}} = \mathbf{y}'_{\mathbf{V}} \wedge (\mathbf{Z}'^w)_{\mathbf{W}} = \mathbf{z}'_{\mathbf{W}}] \\ & > 2^{-(1-\delta) \left[\sum_{i=1}^{n_1} |U_i| \log N_i + \sum_{j=1}^{n_2} \log p_j^{|V_j|} + \sum_{k=1}^{n_3} \sum_{j=1}^{K_k} \log C_k^{|W_{k,j}|} \right]}. \end{aligned}$$

Let $(\widetilde{\mathbf{X}}'^w, \widetilde{\mathbf{Y}}'^w, \widetilde{\mathbf{Z}}'^w)$ be the distribution of $(\mathbf{X}'^w, \mathbf{Y}'^w, \mathbf{Z}'^w)$ conditioned on the event $(\mathbf{X}'^w)_{\mathbf{U}} = \mathbf{x}'_{\mathbf{U}} \wedge (\mathbf{Y}'^w)_{\mathbf{V}} = \mathbf{y}'_{\mathbf{V}} \wedge (\mathbf{Z}'^w)_{\mathbf{W}} = \mathbf{z}'_{\mathbf{W}}$.

(1) Now we claim that $(\widetilde{\mathbf{X}}'^w, \widetilde{\mathbf{Y}}'^w, \widetilde{\mathbf{Z}}'^w)$ is $(\mathbf{P}', \mathbf{Q}', \mathbf{R}', 1 - \delta)$ -dense with $P'_i = |U_i|$ for $i \in [n_1]$, $Q'_j = |V_j|$ for $j \in [n_2]$, and $R'_{k,j} = |W_{k,j}|$ for $k \in [n_3], j \in [K_k]$. Suppose that $(\widetilde{\mathbf{X}}'^w, \widetilde{\mathbf{Y}}'^w, \widetilde{\mathbf{Z}}'^w)$ is not $(\mathbf{P}', \mathbf{Q}', \mathbf{R}', 1 - \delta)$ -dense. Then there exists a pair of non-empty of sets $\mathbf{U}' \subseteq \overline{\mathbf{U}}$ (here, $\subseteq$ refers to an pairwise subset notation), $\mathbf{V}' \subseteq \overline{\mathbf{V}}$, and $\mathbf{W}' \subseteq \overline{\mathbf{W}}$ such that

$$\Pr [(\widetilde{\mathbf{X}}'^w)_{\mathbf{U}'} = \mathbf{x}'_{\mathbf{U}'} \wedge (\widetilde{\mathbf{Y}}'^w)_{\mathbf{V}'} = \mathbf{y}'_{\mathbf{V}'} \wedge (\widetilde{\mathbf{Z}}'^w)_{\mathbf{W}'} = \mathbf{z}'_{\mathbf{W}'}]$$

$$\begin{aligned}
&= \Pr \left[(\mathbf{X}'^w)_{\mathbf{U}'} = \mathbf{x}'_{\mathbf{U}'} \wedge (\mathbf{Y}'^w)_{\mathbf{V}'} = \mathbf{y}'_{\mathbf{V}'} \wedge (\mathbf{Z}'^w)_{\mathbf{W}'} = \mathbf{z}'_{\mathbf{W}'} \right. \\
&\quad \left. (\mathbf{X}'^w)_{\mathbf{U}} = \mathbf{x}'_{\mathbf{U}} \wedge (\mathbf{Y}'^w)_{\mathbf{V}} = \mathbf{y}'_{\mathbf{V}} \wedge (\mathbf{Z}'^w)_{\mathbf{W}} = \mathbf{z}'_{\mathbf{W}} \right] \\
&\geq 2^{-(1-\delta) \left[\sum_{i=1}^{n_1} |U'_i| \log N_i + \sum_{j=1}^{n_2} \log p_j^{|V'_j|} + \sum_{k=1}^{n_3} \sum_{j=1}^{K_k} \log C_k^{|W'_{k,j}|} \right]}.
\end{aligned}$$

The set $\mathbf{U} \cup \mathbf{U}'$ (also pairwise), $\mathbf{V} \cup \mathbf{V}'$, and $\mathbf{W} \cup \mathbf{W}'$ now form subsets for which

$$\begin{aligned}
&\Pr \left[(\mathbf{X}'^w)_{\mathbf{U} \cup \mathbf{U}'} = \mathbf{x}'_{\mathbf{U} \cup \mathbf{U}'} \wedge (\mathbf{Y}'^w)_{\mathbf{V} \cup \mathbf{V}'} = \mathbf{y}'_{\mathbf{V} \cup \mathbf{V}'} \wedge (\mathbf{Z}'^w)_{\mathbf{W} \cup \mathbf{W}'} = \mathbf{z}'_{\mathbf{W} \cup \mathbf{W}'} \right] \\
&= \Pr \left[(\mathbf{X}'^w)_{\mathbf{U}} = \mathbf{x}'_{\mathbf{U}} \wedge (\mathbf{X}'^w)_{\mathbf{U}'} = \mathbf{x}'_{\mathbf{U}'} \wedge (\mathbf{Y}'^w)_{\mathbf{V}} = \mathbf{y}'_{\mathbf{V}} \wedge (\mathbf{Y}'^w)_{\mathbf{V}'} = \mathbf{y}'_{\mathbf{V}'} \right. \\
&\quad \left. \wedge (\mathbf{Z}'^w)_{\mathbf{W}} = \mathbf{z}'_{\mathbf{W}} \wedge (\mathbf{Z}'^w)_{\mathbf{W}'} = \mathbf{z}'_{\mathbf{W}'} \right] \\
&= \Pr \left[(\mathbf{X}'^w)_{\mathbf{U}} = \mathbf{x}'_{\mathbf{U}} \wedge (\mathbf{Y}'^w)_{\mathbf{V}} = \mathbf{y}'_{\mathbf{V}} \wedge (\mathbf{Z}'^w)_{\mathbf{W}} = \mathbf{z}'_{\mathbf{W}} \right] \\
&\quad \cdot \Pr \left[(\mathbf{X}'^w)_{\mathbf{U}'} = \mathbf{x}'_{\mathbf{U}'} \wedge (\mathbf{Y}'^w)_{\mathbf{V}'} = \mathbf{y}'_{\mathbf{V}'} \wedge (\mathbf{Z}'^w)_{\mathbf{W}'} = \mathbf{z}'_{\mathbf{W}'} \right] \\
&\quad \left. (\mathbf{X}'^w)_{\mathbf{U}} = \mathbf{x}'_{\mathbf{U}} \wedge (\mathbf{Y}'^w)_{\mathbf{V}} = \mathbf{y}'_{\mathbf{V}} \wedge (\mathbf{Z}'^w)_{\mathbf{W}} = \mathbf{z}'_{\mathbf{W}} \right] \\
&\geq 2^{-(1-\delta) \left[\sum_{i=1}^{n_1} |U_i| \log N_i + \sum_{j=1}^{n_2} \log p_j^{|V_j|} + \sum_{k=1}^{n_3} \sum_{j=1}^{K_k} \log C_k^{|W_{k,j}|} \right]} \\
&\quad \cdot 2^{-(1-\delta) \left[\sum_{i=1}^{n_1} |U'_i| \log N_i + \sum_{j=1}^{n_2} \log p_j^{|V'_j|} + \sum_{k=1}^{n_3} \sum_{j=1}^{K_k} \log C_k^{|W'_{k,j}|} \right]} \\
&= 2^{-(1-\delta) \left[\sum_{i=1}^{n_1} |U_i \cup U'_i| \log N_i + \sum_{j=1}^{n_2} \log p_j^{|V_j \cup V'_j|} + \sum_{k=1}^{n_3} \sum_{j=1}^{K_k} \log C_k^{|W_{k,j} \cup W'_{k,j}|} \right]}.
\end{aligned}$$

since $(\mathbf{U}, \mathbf{V}, \mathbf{W})$ and $(\mathbf{U}', \mathbf{V}', \mathbf{W}')$ are pairwise disjoint. However, this contradicts the maximality of $(\mathbf{U}, \mathbf{V}, \mathbf{W})$.

(2) Next, we claim that

$$\sum_{i=1}^{n_1} |U_i| \log N_i + \sum_{j=1}^{n_2} |V_j| \log(p_j/e) + \sum_{k=1}^{n_3} \sum_{j=1}^{K_k} |W_{k,j}| \log(C_k/e) \leq S_z/\delta.$$

Let $\mathcal{B}_{\mathbf{y}'_{\mathbf{V}}}$ be the set of vectors $\mathbf{y}'_{\mathbf{V}}$ such that $\mathbf{y}'_{\mathbf{V}}$ and $\mathbf{y}'_{\mathbf{V}}$ are a valid injection. Similarly, let $\mathcal{C}_{\mathbf{z}'_{\mathbf{W}}}$ be the set of vectors $\mathbf{z}'_{\mathbf{W}}$ such that $\mathbf{z}'_{\mathbf{W}}$ and $\mathbf{z}'_{\mathbf{W}}$ are a valid IC. Since $H_{\infty}(\mathbf{X}'^w, \mathbf{Y}'^w, \mathbf{Z}'^w) \geq \sum_{i=1}^{n_1} M_i \log N_i + \sum_{j=1}^{n_2} \log p_j! + \sum_{k=1}^{n_3} \sum_{j=1}^{K_k} \log C_k! - S_z$, we observe that for any $\mathbf{x}'_{\mathbf{U}}, \mathbf{y}'_{\mathbf{V}}, \mathbf{z}'_{\mathbf{W}}$,

$$\begin{aligned}
&\Pr \left[(\mathbf{X}'^w)_{\mathbf{U}} = \mathbf{x}'_{\mathbf{U}} \wedge (\mathbf{Y}'^w)_{\mathbf{V}} = \mathbf{y}'_{\mathbf{V}} \wedge (\mathbf{Z}'^w)_{\mathbf{W}} = \mathbf{z}'_{\mathbf{W}} \right] \\
&= \sum_{\mathbf{x}'_{\mathbf{U}} \in \prod_{i=1}^{n_1} [N_i]^{M_i - |U_i|}} \sum_{\mathbf{y}'_{\mathbf{V}} \in \mathcal{B}_{\mathbf{y}'_{\mathbf{V}}}} \sum_{\mathbf{z}'_{\mathbf{W}} \in \mathcal{C}_{\mathbf{z}'_{\mathbf{W}}}} \Pr \left[(\mathbf{X}^w)_{\mathbf{U}} = \mathbf{x}'_{\mathbf{U}} \right. \\
&\quad \left. \wedge (\mathbf{Y}^w)_{\mathbf{V}} = \mathbf{y}'_{\mathbf{V}} \wedge (\mathbf{Z}^w)_{\mathbf{W}} = \mathbf{z}'_{\mathbf{W}} \wedge (\mathbf{X}^w)_{\mathbf{U}} = \mathbf{x}'_{\mathbf{U}} \right. \\
&\quad \left. \wedge (\mathbf{Y}^w)_{\mathbf{V}} = \mathbf{y}'_{\mathbf{V}} \wedge (\mathbf{Z}^w)_{\mathbf{W}} = \mathbf{z}'_{\mathbf{W}} \right]
\end{aligned}$$

$$\begin{aligned}
&\leq 2^{\sum_{i=1}^{n_1} (M_i - |U_i|) \log N_i} \cdot 2^{\sum_{j=1}^{n_2} \log(p_j - |V_j|)!} \cdot 2^{\sum_{k=1}^{n_3} \sum_{j=1}^{K_k} \log(C_k - |W_{j,k}|)!} \\
&\quad \cdot 2^{-(\sum_{i=1}^{n_1} M_i \log N_i + \sum_{j=1}^{n_2} \log p_j! + \sum_{k=1}^{n_3} \sum_{j=1}^{K_k} \log C_k! - S_z)} \\
&= 2^{-(\sum_{i=1}^{n_1} |U_i| \log N_i + \sum_{j=1}^{n_2} \log p_j^{|V_j|} + \sum_{k=1}^{n_3} \sum_{j=1}^{K_k} \log C_k^{|W_{j,k}|} - S_z)},
\end{aligned}$$

and hence,

$$\begin{aligned}
&H_\infty((\mathbf{X}'^w)_U, (\mathbf{Y}'^w)_V, (\mathbf{Z}'^w)_W) \\
&\geq \sum_{i=1}^{n_1} |U_i| \log N_i + \sum_{j=1}^{n_2} \log p_j^{|V_j|} + \sum_{k=1}^{n_3} \sum_{j=1}^{K_k} \log C_k^{|W_{j,k}|} - S_z.
\end{aligned}$$

On the other hand, because $((\mathbf{X}'^w)_U, (\mathbf{Y}'^w)_V, (\mathbf{Z}'^w)_W)$ is not $(1 - \delta)$ -dense, we have

$$\begin{aligned}
&H_\infty((\mathbf{X}'^w)_U, (\mathbf{Y}'^w)_V, (\mathbf{Z}'^w)_W) \\
&\leq (1 - \delta) \left[\sum_{i=1}^{n_1} |U_i| \log N_i + \sum_{j=1}^{n_2} \log p_j^{|V_j|} + \sum_{k=1}^{n_3} \sum_{j=1}^{K_k} \log C_k^{|W_{j,k}|} \right].
\end{aligned}$$

Combining those two equations together, we have

$$\begin{aligned}
S_z &\geq \delta \left[\sum_{i=1}^{n_1} |U_i| \log N_i + \sum_{j=1}^{n_2} \log p_j^{|V_j|} + \sum_{k=1}^{n_3} \sum_{j=1}^{K_k} \log C_k^{|W_{j,k}|} \right] \\
&\geq \delta \left[\sum_{i=1}^{n_1} |U_i| \log N_i + \sum_{j=1}^{n_2} |V_j| \log(p_j/e) + \sum_{k=1}^{n_3} \sum_{j=1}^{K_k} |W_{j,k}| \log(C_k/e) \right],
\end{aligned}$$

where the last inequality comes from [CDG18, Proposition 38]¹⁶. Divided by $\delta > 0$, we get the desired result.

Hence, $(\widetilde{\mathbf{X}}'^w, \widetilde{\mathbf{Y}}'^w, \widetilde{\mathbf{Z}}'^w)$ is a $(\mathbf{P}', \mathbf{Q}', \mathbf{R}', 1 - \delta)$ -dense source such that

$$\sum_{i=1}^{n_1} P'_i \log N_i + \sum_{j=1}^{n_2} Q'_j \log(p_j/e) + \sum_{k=1}^{n_3} \sum_{j=1}^{K_k} R'_{k,j} \log(C_k/e) \leq S_z/\delta.$$

Set $(\mathbf{X}'^w, \mathbf{Y}'^w, \mathbf{Z}'^w)$ now to be $(\mathbf{X}'^w, \mathbf{Y}'^w, \mathbf{Z}'^w)$ conditioned on $(\mathbf{X}'^w)_U \neq \mathbf{x}'_U$, $(\mathbf{Y}'^w)_V \neq \mathbf{y}'_V$, and $(\mathbf{Z}'^w)_W \neq \mathbf{z}'_W$ and recursively decompose $(\mathbf{X}'^w, \mathbf{Y}'^w, \mathbf{Z}'^w)$ as long as

$$\Pr[\mathbf{X}^w \in \text{supp}(\mathbf{X}'^w) \wedge \mathbf{Y}^w \in \text{supp}(\mathbf{Y}'^w) \wedge \mathbf{Z}^w \in \text{supp}(\mathbf{Z}'^w)] > \gamma.$$

Observe that

$$\begin{aligned}
&H_\infty(\mathbf{X}'^w, \mathbf{Y}'^w, \mathbf{Z}'^w) \\
&\geq \sum_{i=1}^{n_1} M_i \log N_i + \sum_{j=1}^{n_2} \log p_j! + \sum_{k=1}^{n_3} \sum_{j=1}^{K_k} \log C_k! - (S_z + \log \gamma^{-1})
\end{aligned}$$

¹⁶ [CDG18, Proposition 38] says that $N^{\underline{a}} \geq (N/e)^{\underline{b}}$, where $a^{\underline{b}} := a!/(a-b)!$.

at any point in this decomposition process since

$$\begin{aligned}
& \Pr [\mathbf{X}'^w = \mathbf{x}' \wedge \mathbf{Y}'^w = \mathbf{y}' \wedge \mathbf{Z}'^w = \mathbf{z}'] \\
&= \Pr [\mathbf{X}^w = \mathbf{x}' \wedge \mathbf{Y}^w = \mathbf{y}' \wedge \mathbf{Z}^w = \mathbf{z}' \mid \\
&\quad \mathbf{X}^w \in \text{supp}(\mathbf{X}'^w) \wedge \mathbf{Y}^w \in \text{supp}(\mathbf{Y}'^w) \wedge \mathbf{Z}^w \in \text{supp}(\mathbf{Z}'^w)] \\
&\leq \frac{\Pr [\mathbf{X}^w = \mathbf{x}' \wedge \mathbf{Y}^w = \mathbf{y}' \wedge \mathbf{Z}^w = \mathbf{z}']}{\Pr [\mathbf{X}^w \in \text{supp}(\mathbf{X}'^w) \wedge \mathbf{Y}^w \in \text{supp}(\mathbf{Y}'^w) \wedge \mathbf{Z}^w \in \text{supp}(\mathbf{Z}'^w)]} \\
&\leq \frac{2^{-(\sum_{i=1}^{n_1} M_i \log N_i + \sum_{j=1}^{n_2} \log p_j! + \sum_{k=1}^{n_3} \sum_{j=1}^{K_k} \log C_k! - S_z)}}{\gamma} \\
&= 2^{-(\sum_{i=1}^{n_1} M_i \log N_i + \sum_{j=1}^{n_2} \log p_j! + \sum_{k=1}^{n_3} \sum_{j=1}^{K_k} \log C_k! - (S_z + \log \gamma^{-1}))}.
\end{aligned}$$

Note that $\text{supp}(\mathbf{X}'^w)$, $\text{supp}(\mathbf{Y}'^w)$, $\text{supp}(\mathbf{Z}'^w)$ decreases in every step, and since $\text{supp}(\mathbf{X}^w)$, $\text{supp}(\mathbf{Y}^w)$, $\text{supp}(\mathbf{Z}^w)$ are finite, after finitely many steps, this process ends with $(\mathbf{X}'_{\text{final}}^w, \mathbf{Y}'_{\text{final}}^w, \mathbf{Z}'_{\text{final}}^w)$ with

$$\Pr [\mathbf{X}^w \in \text{supp}(\mathbf{X}'_{\text{final}}^w) \wedge \mathbf{Y}^w \in \text{supp}(\mathbf{Y}'_{\text{final}}^w) \wedge \mathbf{Z}^w \in \text{supp}(\mathbf{Z}'_{\text{final}}^w)] > \gamma.$$

Hence, $(\mathbf{X}^w, \mathbf{Y}^w, \mathbf{Z}^w)$ is a convex combination of finitely many $(\mathbf{P}', \mathbf{Q}', \mathbf{R}', 1 - \delta)$ -dense sources and $(\mathbf{X}'_{\text{final}}^w, \mathbf{Y}'_{\text{final}}^w, \mathbf{Z}'_{\text{final}}^w)$, where $\mathbf{P}', \mathbf{Q}', \mathbf{R}'$ satisfy the inequality

$$\sum_{i=1}^{n_1} P'_i \log N_i + \sum_{i=1}^{n_2} Q'_i \log(p_i/e) + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} R'_{i,j} \log(C_i/e) \leq \frac{S_z + \log \gamma^{-1}}{\delta}.$$

This implies that $(\mathbf{X}^w, \mathbf{Y}^w, \mathbf{Z}^w)$ is γ -close to a convex combination of $(\mathbf{P}', \mathbf{Q}', \mathbf{R}', 1 - \delta)$ -dense sources where $\sum_{i=1}^{n_1} P'_i \log N_i + \sum_{i=1}^{n_2} Q'_i \log(p_i/e) + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} R'_{i,j} \log(C_i/e) \leq (S_z + \log \gamma^{-1})/\delta$. $\square$

Let $(\tilde{\mathbf{X}}^w, \tilde{\mathbf{Y}}^w, \tilde{\mathbf{Z}}^w)$ be the convex combination of $(\mathbf{P}', \mathbf{Q}', \mathbf{R}', 1 - \delta)$ -dense sources that is γ -close to $(\mathbf{X}^w, \mathbf{Y}^w, \mathbf{Z}^w)$ for a $\delta = \delta_z$ to be determined later. For every $(\mathbf{P}', \mathbf{Q}', \mathbf{R}', 1 - \delta)$ source $(\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}})$ in said convex combination, let $(\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}')$ be the corresponding $(\mathbf{P}', \mathbf{Q}', \mathbf{R}')$ -bit-fixing source, i.e., $(\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}})$ and $(\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}')$ are fixed on the same coordinates to the same values. The following claim bounds the distinguishing advantage between $(\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}})$ and $(\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}')$ for any $(\mathbf{S}, \mathbf{T}, \mathbf{L})$ -query distinguisher.

Claim 5. *For any $(\mathbf{P}', \mathbf{Q}', \mathbf{R}', 1 - \delta)$ -dense source $(\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}})$ and its corresponding $(\mathbf{P}', \mathbf{Q}', \mathbf{R}')$ -bit-fixing source $(\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}')$, it holds that for any (adaptive) distinguisher $\mathcal{D}$ that queries at most $\mathbf{S} = (S_1, \dots, S_{n_1})$ coordinates of each oracle that corresponds to $\tilde{\mathbf{X}}$ where $S_i < P'_i$ for each $i \in [n_1]$, $\mathbf{T} = (T_1, \dots, T_{n_2})$ coordinates of each oracle that corresponds to $\tilde{\mathbf{Y}}$ where $T_i < Q'_i$ for each $i \in [n_2]$, and $\mathbf{L} = (\mathbf{L}_1, \dots, \mathbf{L}_{n_3})$ coordinates of each oracle that corresponds to $\tilde{\mathbf{Z}}$ where $\mathbf{L}_k = (L_{k,1}, \dots, L_{k,K_k})$ and $L_{i,j} < R'_{i,j}$ for each $j \in [K_i]$ and $i \in [n_3]$,*

$$\begin{aligned}
& \left| \Pr [\mathcal{D}^{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}} = 1] - \Pr [\mathcal{D}^{\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}'} = 1] \right| \\
& \leq \delta \left(\sum_{i=1}^{n_1} S_i \log N_i + \sum_{i=1}^{n_2} T_i \log p_i + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} L_{i,j} \log C_i \right),
\end{aligned}$$

and

$$\begin{aligned} & \Pr \left[\mathcal{D}^{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}} = 1 \right] \\ & \leq 2^{\delta \left(\sum_{i=1}^{n_1} S_i \log N_i + \sum_{i=1}^{n_2} T_i \log p_i + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} L_{i,j} \log C_i \right)} \\ & \quad \cdot \Pr \left[\mathcal{D}^{\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}'} = 1 \right]. \end{aligned}$$

Proof of Claim 5. Without loss of generality, assume that $\mathcal{D}$ is deterministic and does not query any of the fixed positions, make the same query twice, or make an inverse query after making the corresponding forward query or vice-versa. Let $T_{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}}$ and $T_{\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}'}$ be the random variables corresponding to the transcripts containing the query/answer pairs resulting from $\mathcal{D}$'s interaction with $(\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}})$ and $(\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}')$, respectively.

For a fixed transcript t , denote by $p_{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}}(t)$ and $p_{\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}'}(t)$ the probabilities that $(\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}})$ and $(\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}')$, respectively, produce the answers in t if the queries in t are asked. Since $\mathcal{D}$ is deterministic, $\Pr \left[T_{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}} = t \right] \in \{0, p_{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}}(t)\}$, and similarly, $\Pr \left[T_{\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}'} = t \right] \in \{0, p_{\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}'}(t)\}$. Denote by $\mathcal{T}_{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}}$ the set of all transcripts t for which $\Pr \left[T_{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}} = t \right] > 0$. For such t , $\Pr \left[T_{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}} = t \right] = p_{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}}(t)$ and also $\Pr \left[T_{\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}'} = t \right] = p_{\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}'}(t)$. Observe that for every transcript t ,

$$\begin{aligned} & p_{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}}(t) \\ & \leq 2^{-(1-\delta) \left[\sum_{i=1}^{n_1} S_i \log N_i + \sum_{i=1}^{n_2} \log(p_i - Q'_i) \frac{T_i}{p_i} + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} \log(C_i - R'_{i,j}) \frac{L_{i,j}}{C_i} \right]}, \end{aligned} \quad (7)$$

and

$$\begin{aligned} & p_{\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}'}(t) \\ & \leq 2^{-\left[\sum_{i=1}^{n_1} S_i \log N_i + \sum_{i=1}^{n_2} \log(p_i - Q'_i) \frac{T_i}{p_i} + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} \log(C_i - R'_{i,j}) \frac{L_{i,j}}{C_i} \right]}, \end{aligned} \quad (8)$$

as $(\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}})$ is $(\mathbf{P}', \mathbf{Q}', \mathbf{R}', 1 - \delta)$ -dense and $(\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}')$ is $(\mathbf{P}', \mathbf{Q}', \mathbf{R}')$ -fixed.

Towards proving the first part of the claim, observe that $\mathcal{D}$'s output can be computed from the transcript (including whether a query was a forward for an inverse query) by just running $\mathcal{D}$ and providing the answers to its queries from the transcript. Hence,

$$\begin{aligned} & \left| \Pr \left[\mathcal{D}^{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}} = 1 \right] - \Pr \left[\mathcal{D}^{\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}'} = 1 \right] \right| \leq \text{SD} \left(T_{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}}, T_{\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}'} \right) \\ & = \sum_t \max \left\{ 0, \Pr \left[T_{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}} = t \right] - \Pr \left[T_{\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}'} = t \right] \right\} \\ & = \sum_{t \in \mathcal{T}_{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}}} \max \left\{ 0, p_{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}}(t) - p_{\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}'}(t) \right\} \\ & = \sum_{t \in \mathcal{T}_{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}}} p_{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}}(t) \cdot \max \left\{ 0, 1 - \frac{p_{\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}'}(t)}{p_{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}}(t)} \right\} \\ & \leq 1 - 2^{-\delta \left[\sum_{i=1}^{n_1} S_i \log N_i + \sum_{i=1}^{n_2} \log(p_i - Q'_i) \frac{T_i}{p_i} + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} \log(C_i - R'_{i,j}) \frac{L_{i,j}}{C_i} \right]} \end{aligned}$$

$$\begin{aligned}
&\leq 1 - 2^{-\delta \left[\sum_{i=1}^{n_1} S_i \log N_i + \sum_{i=1}^{n_2} T_i \log p_i + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} L_{i,j} \log C_i \right]} \\
&\leq -\delta \left[\sum_{i=1}^{n_1} S_i \log N_i + \sum_{i=1}^{n_2} T_i \log p_i + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} L_{i,j} \log C_i \right],
\end{aligned}$$

where the first sum is over all possible transcripts and where the last inequality uses $2^{-x} \geq 1 - x$ for $x \geq 0$ and $a^b \leq a^b$ for $a, b \in \mathbb{N}$.

As for the second part of the claim, observe that due to the equation (7),(8), and the support of $T_{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}}$ being a subset of $T_{\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}'}$,

$$\begin{aligned}
&\Pr \left[T_{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}} = t \right] \\
&\leq 2^{\delta \left[\sum_{i=1}^{n_1} S_i \log N_i + \sum_{i=1}^{n_2} \log(p_i - Q'_i) T_i + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} \log(C_i - R'_{i,j}) L_{i,j} \right]} \\
&\quad \cdot \Pr \left[T_{\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}'} = t \right] \\
&\leq 2^{\delta \left[\sum_{i=1}^{n_1} S_i \log N_i + \sum_{i=1}^{n_2} T_i \log p_i + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} L_{i,j} \log C_i \right]} \cdot \Pr \left[T_{\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}'} = t \right],
\end{aligned}$$

for any transcript t . Let $\mathcal{T}_{\mathcal{D}}$ be the set of transcripts where $\mathcal{D}$ outputs 1. Then we have

$$\begin{aligned}
\Pr \left[\mathcal{D}^{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}} = 1 \right] &= \sum_{t \in \mathcal{T}_{\mathcal{D}}} \Pr \left[T_{\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}}} = t \right] \\
&\leq 2^{\delta \left[\sum_{i=1}^{n_1} S_i \log N_i + \sum_{i=1}^{n_2} T_i \log p_i + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} L_{i,j} \log C_i \right]} \\
&\quad \cdot \sum_{t \in \mathcal{T}_{\mathcal{D}}} \Pr \left[T_{\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}'} = t \right] \\
&= 2^{\delta \left[\sum_{i=1}^{n_1} S_i \log N_i + \sum_{i=1}^{n_2} T_i \log p_i + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} L_{i,j} \log C_i \right]} \cdot \Pr \left[\mathcal{D}^{\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}'} = 1 \right],
\end{aligned}$$

which concludes the proof. $\square$

Let $(\tilde{\mathbf{X}}'^w, \tilde{\mathbf{Y}}'^w, \tilde{\mathbf{Z}}'^w)$ be obtained by replacing $(\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \tilde{\mathbf{Z}})$ by $(\tilde{\mathbf{X}}', \tilde{\mathbf{Y}}', \tilde{\mathbf{Z}}')$ in $(\tilde{\mathbf{X}}^w, \tilde{\mathbf{Y}}^w, \tilde{\mathbf{Z}}^w)$. Setting

$$\delta_z = \frac{S_z + \log \gamma^{-1}}{\sum_{i=1}^{n_1} S_i \log N_i + \sum_{i=1}^{n_2} T_i \log p_i + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} L_{i,j} \log C_i},$$

Claim 4 and Claim 5 imply

$$\left| \Pr \left[\mathcal{D}^{\mathbf{X}^w, \mathbf{Y}^w, \mathbf{Z}^w}(z) = 1 \right] - \Pr \left[\mathcal{D}^{\tilde{\mathbf{X}}'^w, \tilde{\mathbf{Y}}'^w, \tilde{\mathbf{Z}}'^w}(z) = 1 \right] \right| \leq \Lambda + \gamma, \quad (9)$$

as well as

$$\Pr \left[\mathcal{D}^{\mathbf{X}^w, \mathbf{Y}^w, \mathbf{Z}^w}(z) = 1 \right] \leq 2^\Lambda \cdot \Pr \left[\mathcal{D}^{\tilde{\mathbf{X}}'^w, \tilde{\mathbf{Y}}'^w, \tilde{\mathbf{Z}}'^w}(z) = 1 \right] + \gamma, \quad (10)$$

where

$$\Lambda = \frac{(S + \log \gamma^{-1}) \left(\sum_{i=1}^{n_1} S_i \log N_i + \sum_{i=1}^{n_2} T_i \log p_i + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} L_{i,j} \log C_i \right)}{\sum_{i=1}^{n_1} P_i \log N_i + \sum_{i=1}^{n_2} Q_i \log(p_i/e) + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} R_{i,j} \log(C_i/e)}.$$

Moreover, note that for the above choice of δ_z , we have $(\mathbf{P}', \mathbf{Q}', \mathbf{R}') = (\mathbf{P}, \mathbf{Q}, \mathbf{R})$, i.e., the sources $(\tilde{\mathbf{X}}'^w, \tilde{\mathbf{Y}}'^w, \tilde{\mathbf{Z}}'^w)$ are $(\mathbf{P}, \mathbf{Q}, \mathbf{R})$ -fixed, as desired.

Claim 6. $\mathbb{E}_z[S_z] \leq S$ and $\Pr[S_{f(\mathbf{X}, \mathbf{Y}, \mathbf{Z})} > S + \log \gamma^{-1}] \leq \gamma$.

Proof of Claim 6. Observe that $H_\infty(\mathbf{X}^w, \mathbf{Y}^w, \mathbf{Z}^w) = H_\infty((\mathbf{X}, \mathbf{Y}, \mathbf{Z})|W = w) = H((\mathbf{X}, \mathbf{Y}, \mathbf{Z})|W = w)$ since, conditioned on $W = w$, $(\mathbf{X}, \mathbf{Y}, \mathbf{Z})$ is distributed uniformly over all values $(\mathbf{x}, \mathbf{y}, \mathbf{z})$ with $f(\mathbf{x}, \mathbf{y}, \mathbf{z}) = w$. Hence,

$$\begin{aligned} \mathbb{E}_z[S_z] &= \sum_{i=1}^{n_1} M_i \log N_i + \sum_{i=1}^{n_2} \log p_i! + \sum_{i=1}^{n_3} K_i \log C_i! - \mathbb{E}_z[H_\infty((\mathbf{X}, \mathbf{Y}, \mathbf{Z})|W = w)] \\ &= \sum_{i=1}^{n_1} M_i \log N_i + \sum_{i=1}^{n_2} \log p_i! + \sum_{i=1}^{n_3} K_i \log C_i! - \mathbb{E}_z[H((\mathbf{X}, \mathbf{Y}, \mathbf{Z})|W = w)] \\ &= \sum_{i=1}^{n_1} M_i \log N_i + \sum_{i=1}^{n_2} \log p_i! + \sum_{i=1}^{n_3} K_i \log C_i! - H((\mathbf{X}, \mathbf{Y}, \mathbf{Z})|W = w) \leq S. \end{aligned}$$

Again, due to the uniformity of $(\mathbf{X}, \mathbf{Y}, \mathbf{Z})$, $\Pr[f(\mathbf{X}, \mathbf{Y}, \mathbf{Z}) = z] = 2^{-S_z}$. Hence,

$$\begin{aligned} \Pr[S_{f(\mathbf{X}, \mathbf{Y}, \mathbf{Z})} > S + \log \gamma^{-1}] &= \sum_{w \in \{0,1\}^S : S_z > S + \log \gamma^{-1}} \Pr[f(\mathbf{X}, \mathbf{Y}, \mathbf{Z}) = w] \\ &\leq 2^S \cdot 2^{-(S + \log \gamma^{-1})} \leq \gamma. \end{aligned} \quad \square$$

Now the lemma follows (using $(\mathbf{X}'^w, \mathbf{Y}'^w, \mathbf{Z}'^w) := (\widetilde{\mathbf{X}}'^w, \widetilde{\mathbf{Y}}'^w, \widetilde{\mathbf{Z}}'^w)$) by taking expectations over w of the equation (9) and applying the first part of Claim 6. The second part of the Lemma is proved as follows:

$$\begin{aligned} &\Pr[\mathcal{D}^{\mathbf{X}, \mathbf{Y}, \mathbf{Z}}(f(\mathbf{X}, \mathbf{Y}, \mathbf{Z})) = 1] \\ &\leq \Pr[\mathcal{D}^{\mathbf{X}, \mathbf{Y}, \mathbf{Z}}(f(\mathbf{X}, \mathbf{Y}, \mathbf{Z})) = 1, S_{f(\mathbf{X}, \mathbf{Y}, \mathbf{Z})} \leq S + \log \gamma^{-1}] + \Pr[S_{f(\mathbf{X}, \mathbf{Y}, \mathbf{Z})} > S + \log \gamma^{-1}] \\ &\leq \left(2^{\frac{(S + \log \gamma^{-1}) \left(\sum_{i=1}^{n_1} S_i \log N_i + \sum_{i=1}^{n_2} T_i \log p_i + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} L_{i,j} \log C_i \right)}{\eta}} \cdot \Pr[\mathcal{D}^{\mathbf{X}, \mathbf{Y}, \mathbf{Z}}(f(\mathbf{X}, \mathbf{Y}, \mathbf{Z})) = 1, \right. \\ &\quad \left. S_{f(\mathbf{X}, \mathbf{Y}, \mathbf{Z})} \leq S + \log \gamma^{-1}] + \gamma \right) \\ &\leq 2^{\frac{(S + \log \gamma^{-1}) \left(\sum_{i=1}^{n_1} S_i \log N_i + \sum_{i=1}^{n_2} T_i \log p_i + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} L_{i,j} \log C_i \right)}{\eta}} \cdot \Pr[\mathcal{D}^{\mathbf{X}, \mathbf{Y}, \mathbf{Z}}(f(\mathbf{X}, \mathbf{Y}, \mathbf{Z})) = 1, \\ &\quad S_{f(\mathbf{X}, \mathbf{Y}, \mathbf{Z})} \leq S + \log \gamma^{-1}] + 2\gamma, \end{aligned}$$

where the second inequality follows by taking expectations over w of the equation (10) (together with the condition $S_z \leq S + \log \gamma^{-1}$) and the second part of Claim 6. $\square$

B.2 From the Bit-Fixing Model to the Auxiliary-Input Model

Capturing the Models. A tuple of oracles $(\{\mathcal{O}_{1,i}\}_{i=1}^{n_1}, \{\mathcal{O}_{2,j}\}_{j=1}^{n_2}, \{\mathcal{O}_{3,k}\}_{k=1}^{n_3})$ has two interfaces $(\{\mathcal{O}_{1,i}.\text{pre}\}_{i=1}^{n_1}, \{\mathcal{O}_{2,j}.\text{pre}\}_{j=1}^{n_2}, \{\mathcal{O}_{3,k}.\text{pre}\}_{k=1}^{n_3})$ that is accessible during a preprocessing phase, and $(\{\mathcal{O}_{1,i}.\text{on}\}_{i=1}^{n_1}, \{\mathcal{O}_{2,j}.\text{on}\}_{j=1}^{n_2}, \{\mathcal{O}_{3,k}.\text{on}\}_{k=1}^{n_3})$.

$\{\mathcal{O}_{3,k}.\text{on}\}_{k=1}^{n_3}$) that is accessible during an online phase, where $(\{\mathcal{O}_{1,i}.\text{pre}\}_{i=1}^{n_1}, \{\mathcal{O}_{2,j}.\text{pre}\}_{j=1}^{n_2}, \{\mathcal{O}_{3,k}.\text{pre}\}_{k=1}^{n_3})$ is accessible only once before any calls to $(\{\mathcal{O}_{1,i}.\text{on}\}_{i=1}^{n_1}, \{\mathcal{O}_{2,j}.\text{on}\}_{j=1}^{n_2}, \{\mathcal{O}_{3,k}.\text{on}\}_{k=1}^{n_3})$ are made. We consider the oracles as follows:

- **Random Oracle + Generic Group Oracle + Ideal Cipher RO+GG+IC** $(\{M_i, N_i\}_{i=1}^{n_1}, \{p_j, \ell_j\}_{j=1}^{n_2}, \{K_k, C_k\}_{k=1}^{n_3})$: Samples random oracle function tables $H_i \leftarrow \mathcal{H}_{M_i, N_i}$ for each $i \in [n_1]$, where $\mathcal{H}_{M_i, N_i}$ is the set of all functions from $[M_i] \rightarrow [N_i]$, samples random injections $\tau_j \leftarrow \mathcal{I}_{p_j, \ell_j}$ for each $j \in [n_2]$, where $\mathcal{I}_{p_j, \ell_j}$ is the set of all injections from $\mathbb{Z}_{p_j} \rightarrow \mathbb{G}_j$ where $\mathbb{G}_j$ is the set of bitstrings of length $\ell_j \geq \log p_j$, and samples random permutations $\pi_s \leftarrow \mathcal{P}_{C_k}$ for each $s \in [K_k]$, where $\mathcal{P}_{C_k}$ is the set of all permutations from $[C_k] \rightarrow [C_k]$; offers no functionality at $(\{\mathcal{O}_{1,i}.\text{pre}\}_{i=1}^{n_1}, \{\mathcal{O}_{2,j}.\text{pre}\}_{j=1}^{n_2}, \{\mathcal{O}_{3,k}.\text{pre}\}_{k=1}^{n_3})$; for the online queries,
 - for each $i \in [n_1]$, answers queries $x \in [M_i]$ to H_i at $\mathcal{O}_{1,i}.\text{on}$ by the corresponding value $H_i(x) \in [N_i]$,
 - for each $j \in [n_2]$, answers *forward* queries $x' \in \mathbb{Z}_{p_j}$ to τ_j at $\mathcal{O}_{2,j}.\text{on}$ by the corresponding value $\tau_j(x') \in \mathbb{G}_j$; answers *group-operation* queries (s, s') at $\mathcal{O}_{2,j}.\text{on}$ as follows: if $s = \tau_j(x)$ and $s' = \tau_j(x')$ for some x, x' , the oracle replies by $\tau_j(x + x')$ and by $\perp$ otherwise; answers *inverse* queries s at $\mathcal{O}_{2,j}.\text{on}$ by returning $\tau_j^{-1}(s)$ if s is in the range of τ_j and $\perp$ otherwise, and
 - for each $k \in [n_3]$, answers both *forward* and *backward* queries $(s, x) \in [K_k] \times [C_k]$ at $\mathcal{O}_{3,k}.\text{on}$ by the corresponding value $\pi_s(x) \in [C_k]$ or $\pi_s^{-1}(x) \in [C_k]$, respectively.
- **Auxiliary-Input Random Oracle + Generic Group Oracle + Ideal Cipher AI-RO+GG+IC** $(\{M_i, N_i\}_{i=1}^{n_1}, \{p_j, \ell_j\}_{j=1}^{n_2}, \{K_k, C_k\}_{k=1}^{n_3})$: Samples H_i, τ_j, π_s for each $i \in [n_1], j \in [n_2]$, and $s \in [K_k]$ where $k \in [n_3]$ as explained in RO+GG+IC $(\{M_i, N_i\}_{i=1}^{n_1}, \{p_j, \ell_j\}_{j=1}^{n_2}, \{K_k, C_k\}_{k=1}^{n_3})$ above; outputs all of H_i, τ_j , and π_s at $(\{\mathcal{O}_{1,i}.\text{pre}\}_{i=1}^{n_1}, \{\mathcal{O}_{2,j}.\text{pre}\}_{j=1}^{n_2}, \{\mathcal{O}_{3,k}.\text{pre}\}_{k=1}^{n_3})$; $(\{\mathcal{O}_{1,i}.\text{on}\}_{i=1}^{n_1}, \{\mathcal{O}_{2,j}.\text{on}\}_{j=1}^{n_2}, \{\mathcal{O}_{3,k}.\text{on}\}_{k=1}^{n_3})$ behaves the same as RO+GG+IC $(\{M_i, N_i\}_{i=1}^{n_1}, \{p_j, \ell_j\}_{j=1}^{n_2}, \{K_k, C_k\}_{k=1}^{n_3})$. When the parameters $(\{M_i, N_i\}_{i=1}^{n_1}, \{p_j, \ell_j\}_{j=1}^{n_2}, \{K_k, C_k\}_{k=1}^{n_3})$ are clear in context, we sometimes abuse the notation and simply say AI-RO+GG+IC.
- **Bit-Fixing Random Oracle + Generic Group Oracle + Ideal Cipher BF-RO+GG+IC** $(\mathbf{P}, \mathbf{Q}, \mathbf{R}, \{M_i, N_i\}_{i=1}^{n_1}, \{p_j, \ell_j\}_{j=1}^{n_2}, \{K_k, C_k\}_{k=1}^{n_3})$: Samples a random oracle function table $H_i \leftarrow \mathcal{H}_{M_i, N_i}$ at $\mathcal{O}_{1,i}.\text{pre}$ for each $i \in [n_1]$, where $\mathcal{H}_{M_i, N_i}$ is the set of all functions from $[M_i] \rightarrow [N_i]$, samples a random size- p_j subset $\mathcal{Y}_j$ of $\mathbb{G}_j$ for each $j \in [n_2]$ and outputs $\mathcal{Y}_j$ at $\mathcal{O}_{2,j}.\text{pre}$, and samples a random permutation $\pi_s \leftarrow \mathcal{P}_{C_k}$ at $\mathcal{O}_{3,k}.\text{pre}$ for each $s \in [K_k]$, where $\mathcal{P}_{C_k}$ is the set of all permutations from $[C_k] \rightarrow [C_k]$.
 - Given $\mathbf{P} = (P_1, \dots, P_{n_1}) \in \mathbb{Z}_{M_1} \times \dots \times \mathbb{Z}_{M_{n_1}}$, for each $i \in [n_1]$, takes a list at $\mathcal{O}_{1,i}.\text{pre}$ of at most P_i query/answer pairs that override H_i in the corresponding positions,
 - given $\mathbf{Q} = (Q_1, \dots, Q_{n_2}) \in \mathbb{Z}_{p_1} \times \dots \times \mathbb{Z}_{p_{n_2}}$, for each $j \in [n_2]$, takes a list at $\mathcal{O}_{2,j}.\text{pre}$ of at most Q_j query/answer pairs without collisions and all answers in $\mathcal{Y}_j$; samples a random injection $\tau_j \leftarrow \mathcal{I}_{p_j, \ell_j}$ with range $\mathcal{Y}_j$ and consistent with said list, and
 - given $\mathbf{R} = (\mathbf{R}_1, \dots, \mathbf{R}_{n_3}) \in (\mathbb{Z}_{C_1})^{K_1} \times \dots \times (\mathbb{Z}_{C_{n_3}})^{K_{n_3}}$ where $\mathbf{R}_k = (R_{k,1}, \dots, R_{k,K_k})$ for each $k \in [n_3]$, takes a list at $\mathcal{O}_{3,k}.\text{pre}$ of at most $R_{k,s}$ query/answer pairs (without collisions for each $s \in [K_k]$); samples a random permutation $\pi_s \leftarrow \mathcal{P}_{C_k}$ consistent with said list for each $s \in [K_k]$.
 - $(\{\mathcal{O}_{1,i}.\text{on}\}_{i=1}^{n_1}, \{\mathcal{O}_{2,j}.\text{on}\}_{j=1}^{n_2}, \{\mathcal{O}_{3,k}.\text{on}\}_{k=1}^{n_3})$ behaves the same as RO+GG+IC $(\{M_i, N_i\}_{i=1}^{n_1}, \{p_j, \ell_j\}_{j=1}^{n_2}, \{K_k, C_k\}_{k=1}^{n_3})$. When the parameters $(\{M_i, N_i\}_{i=1}^{n_1}, \{p_j, \ell_j\}_{j=1}^{n_2}, \{K_k, C_k\}_{k=1}^{n_3})$ are clear in context, we sometimes abuse the notation and simply say BF-RO+GG+IC $(\mathbf{P}, \mathbf{Q}, \mathbf{R})$.

Attackers with Oracle-Dependent Advice. We define the attackers $\mathcal{A} = (\mathcal{A}_{\text{pre}}, \mathcal{A}_{\text{on}})$ similar to that of prior work [CDGS18, CDG18], which consist of a preprocessing attacker $\mathcal{A}_{\text{pre}}$ and an online attacker $\mathcal{A}_{\text{on}}$, which carries out the actual attack using the output of $\mathcal{A}_{\text{pre}}$. The difference is that we consider a tuple of oracles instead of a single oracle. More precisely, in the presence of a tuple of oracles $(\{\mathcal{O}_{1,i}\}_{i=1}^{n_1}, \{\mathcal{O}_{2,j}\}_{j=1}^{n_2}, \{\mathcal{O}_{3,k}\}_{k=1}^{n_3})$, $\mathcal{A}_{\text{pre}}$ interacts with $(\{\mathcal{O}_{1,i}.\text{pre}\}_{i=1}^{n_1}, \{\mathcal{O}_{2,j}.\text{pre}\}_{j=1}^{n_2}, \{\mathcal{O}_{3,k}.\text{pre}\}_{k=1}^{n_3})$ and $\mathcal{A}_{\text{on}}$ with $(\{\mathcal{O}_{1,i}.\text{on}\}_{i=1}^{n_1}, \{\mathcal{O}_{2,j}.\text{on}\}_{j=1}^{n_2}, \{\mathcal{O}_{3,k}.\text{on}\}_{k=1}^{n_3})$.

Definition 8. Given $\mathbf{S} = (S_1, \dots, S_{n_1})$, $\mathbf{T} = (T_1, \dots, T_{n_2})$, and $\mathbf{L} = (\mathbf{L}_1, \dots, \mathbf{L}_{n_3})$ where $\mathbf{L}_k = (L_{k,1}, \dots, L_{k,K_k})$ for $k \in [n_3]$, an $(S, \mathbf{S}, \mathbf{T}, \mathbf{L})$ -attacker $\mathcal{A} = (\mathcal{A}_{\text{pre}}, \mathcal{A}_{\text{on}})$ in the $(\{\mathcal{O}_{1,i}\}_{i=1}^{n_1}, \{\mathcal{O}_{2,j}\}_{j=1}^{n_2}, \{\mathcal{O}_{3,k}\}_{k=1}^{n_3})$ -model consists of two procedures

- $\mathcal{A}_{\text{pre}}$, which is computationally unbounded, interacts with $(\{\mathcal{O}_{1,i}.\text{pre}\}_{i=1}^{n_1}, \{\mathcal{O}_{2,j}.\text{pre}\}_{j=1}^{n_2}, \{\mathcal{O}_{3,k}.\text{pre}\}_{k=1}^{n_3})$, and outputs an S -bit string as hint, and
- $\mathcal{A}_{\text{on}}$, which takes an S -bit auxiliary input and makes at most $(\mathbf{S}, \mathbf{T}, \mathbf{L})$ queries to $(\{\mathcal{O}_{1,i}.\text{on}\}_{i=1}^{n_1}, \{\mathcal{O}_{2,j}.\text{on}\}_{j=1}^{n_2}, \{\mathcal{O}_{3,k}.\text{on}\}_{k=1}^{n_3})$. That is, $\mathcal{A}_{\text{on}}$ makes at most S_i queries to $\mathcal{O}_{1,i}.\text{on}$ for each $i \in [n_1]$, T_j queries to $\mathcal{O}_{2,j}.\text{on}$ for each $j \in [n_2]$, and $L_{k,s}$ queries to $\mathcal{O}_{3,k}.\text{on}$ for each $k \in [n_3]$ and for each key $s \in [K_k]$.

The definition of an application G and the success of $\mathcal{A}$ on G (see Definition 5) can be easily extended by defining $\mathcal{O} := (\{\mathcal{O}_{1,i}\}_{i=1}^{n_1}, \{\mathcal{O}_{2,j}\}_{j=1}^{n_2}, \{\mathcal{O}_{3,k}\}_{k=1}^{n_3})$, $\mathcal{O}.\text{pre} := (\{\mathcal{O}_{1,i}.\text{pre}\}_{i=1}^{n_1}, \{\mathcal{O}_{2,j}.\text{pre}\}_{j=1}^{n_2}, \{\mathcal{O}_{3,k}.\text{pre}\}_{k=1}^{n_3})$, and $\mathcal{O}.\text{on} := (\{\mathcal{O}_{1,i}.\text{on}\}_{i=1}^{n_1}, \{\mathcal{O}_{2,j}.\text{on}\}_{j=1}^{n_2}, \{\mathcal{O}_{3,k}.\text{on}\}_{k=1}^{n_3})$. The advantage of an attacker $\mathcal{A}$ for an indistinguishability/unpredictability application G can also be defined accordingly for $\mathcal{O} = (\{\mathcal{O}_{1,i}\}_{i=1}^{n_1}, \{\mathcal{O}_{2,j}\}_{j=1}^{n_2}, \{\mathcal{O}_{3,k}\}_{k=1}^{n_3})$ as defined in Definition 6.

Additive Error for Arbitrary Applications in the ROM+GGM+ICM. Using Lemma 4, one can extend [CDGS18, Theorem 5] and [CDG18, Theorem 1] to the translation from bit-fixing model with multiple idealized models (ROM+GGM+ICM) to the corresponding auxiliary-input model at the cost of an additive term which is similar to those from [CDGS18, Theorem 5] and [CDG18, Theorem 1].

Theorem 12. For every $\gamma > 0$, if an application G is $(S, \mathbf{S}, \mathbf{T}, \mathbf{L}, \varepsilon')$ -secure in the BF-RO+GG+IC $(\mathbf{P}, \mathbf{Q}, \mathbf{R}, \{M_i, N_i\}_{i=1}^{n_1}, \{p_i, \ell_i\}_{i=1}^{n_2}, \{K_i, C_i\}_{i=1}^{n_3})$ -model for any $\mathbf{P}, \mathbf{Q}, \mathbf{R}$ as introduced in Definition 7 which satisfies

$$\sum_{i=1}^{n_1} P_i \log N_i + \sum_{i=1}^{n_2} Q_i \log(p_i/e) + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} R_{i,j} \log(C_i/e) \leq \eta,$$

then it is $(S, \mathbf{S}, \mathbf{T}, \mathbf{L}, \varepsilon)$ -secure in the AI-RO+GG+IC $(\{M_i, N_i\}_{i=1}^{n_1}, \{p_i, \ell_i\}_{i=1}^{n_2}, \{K_i, C_i\}_{i=1}^{n_3})$ -model, for $\varepsilon \leq \varepsilon' + \Lambda + 2\gamma$, where

$$\Lambda = \frac{2(S + \log \gamma^{-1}) \left(\sum_{i=1}^{n_1} (S_i)_G^{\text{comb}} \log N_i + \sum_{i=1}^{n_2} 3(T_i)_G^{\text{comb}} \log p_i + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} 2(L_{i,j})_G^{\text{comb}} \log C_i \right)}{\eta},$$

e is the Euler constant, and $(S_i)_G^{\text{comb}}$, $(T_i)_G^{\text{comb}}$ and $(L_{i,j})_G^{\text{comb}}$ are the combined query complexity corresponding to G that corresponds to the random oracles, the generic group oracles, and the ideal ciphers, respectively.

Proof. Fix $\mathbf{P}, \mathbf{Q}, \mathbf{R}$ as well as γ . Let G be an arbitrary application and C be the corresponding challenger. Moreover, fix an $(S, \mathbf{S}, \mathbf{T}, \mathbf{L})$ -attacker $\mathcal{A} = (\mathcal{A}_{\text{pre}}, \mathcal{A}_{\text{on}})$, and let $\{(\mathbf{X}', \mathbf{Y}', \mathbf{Z}')^{w, \mathcal{Y}}\}$, indexed by $w \in \{0, 1\}^S$ and $\mathcal{Y} = (\mathcal{Y}_1, \dots, \mathcal{Y}_{n_2})$ where $\mathcal{Y}_i$ is a size- p_i subset of $\mathbb{G}_i$ for each $i \in [n_2]$, be the family of distributions guaranteed to exist by [Lemma 4](#). Consider the following $(S, \mathbf{S}, \mathbf{T}, \mathbf{L})$ -attacker $\mathcal{A}' = (\mathcal{A}'_{\text{pre}}, \mathcal{A}'_{\text{on}})$ that is expected to interact with BF-RO+GG+IC:

- $\mathcal{A}'_{\text{pre}}$ obtains the set $\mathcal{Y}$ from $\{\text{BF-GG.pre}\}_{i=1}^{n_2}$ and internally simulates $\mathcal{A}_{\text{pre}}$ on a uniformly random input $\mathbf{X} \in \prod_{i=1}^{n_1} [M_i]$, a uniformly random injection $\mathbf{Y} \in \prod_{j=1}^{n_2} \mathbb{Z}_{p_j}$ with range $\mathcal{Y}$, a uniformly random IC $\mathbf{Z} \in \prod_{k=1}^{n_3} [K_k] \times [C_k]$ to obtain $w \leftarrow \mathcal{A}_{\text{pre}}^{\text{AI-RO+GG+IC.pre}}$. Then, it samples one of the $(\mathbf{P}, \mathbf{Q}, \mathbf{R})$ -bit-fixing sources $(\mathbf{X}'', \mathbf{Y}'', \mathbf{Z}'')$ making up $(\mathbf{X}', \mathbf{Y}', \mathbf{Z}')^{w, \mathcal{Y}}$ and presets BF-RO+GG+IC to match $(\mathbf{X}'', \mathbf{Y}'', \mathbf{Z}'')$ on the at most $(\mathbf{P}, \mathbf{Q}, \mathbf{R})$ points (pairwise and element-wise) where $(\mathbf{X}'', \mathbf{Y}'', \mathbf{Z}'')$ is fixed. The output of $\mathcal{A}'_{\text{pre}}$ is w .
- $\mathcal{A}'_{\text{on}}$ works exactly the same as $\mathcal{A}_{\text{on}}$.

Let $\mathcal{D}$ be a distinguisher — making forward, group-operation, inverse, and backward queries to an (n_1, n_2, n_3) -source — that internally runs the combination of $\mathcal{A}_{\text{on}} = \mathcal{A}'_{\text{on}}$ and C . It answers their queries as follows:

- (1) It passes forward and backward queries to and back from its own oracle.
- (2) It answers group-operation queries (s, s') (to the generic group oracle) by making two backward queries to its own oracle for s and s' , obtaining i and j , respectively, making a forward query $i + j$, and passing the answer to $\mathcal{A}_{\text{on}}$ or C (unless one of the answers to the backward queries was $\perp$, in which case $\perp$ is returned).

Note that $\mathcal{D}^{\mathbf{X}, \mathbf{Y}, \mathbf{Z}}(f(\mathbf{X}, \mathbf{Y}, \mathbf{Z}))$ is identical to

$$\mathcal{A}_{\text{on}}^{\text{AI-RO+GG+IC.on}}(\mathcal{A}_{\text{pre}}^{\text{AI-RO+GG+IC.pre}}) \leftrightarrow C^{\text{AI-RO+GG+IC.on}},$$

and $\mathcal{D}^{(\mathbf{X}', \mathbf{Y}', \mathbf{Z}')^{f(\mathbf{X}, \mathbf{Y}, \mathbf{Z}), \text{im}(\mathbf{Y})}}(f(\mathbf{X}, \mathbf{Y}, \mathbf{Z}))$ is identical to

$$\mathcal{A}'_{\text{on}}^{\text{BF-RO+GG+IC.on}}(\mathcal{A}'_{\text{pre}}^{\text{BF-RO+GG+IC.pre}}) \leftrightarrow C^{\text{BF-RO+GG+IC.on}}.$$

Furthermore, $\mathcal{D}$ is a distinguisher taking an S -bit input and making at most $(S_i)_G^{\text{comb}}$ queries to the i^{th} random oracle for $i \in [n_1]$, $3(T_j)_G^{\text{comb}}$ queries to the j^{th} generic group oracle for $j \in [n_2]$, and $2(L_{k,j})_G^{\text{comb}}$ queries to the k^{th} ideal cipher for $k \in [n_3]$ and for each key $j \in [K_k]$. Therefore, by

$$\begin{aligned} & \text{Succ}_{G, \text{AI-RO+GG+IC}}(\mathcal{A}) \\ & \leq \text{Succ}_{G, \text{BF-RO+GG+IC}}(\mathcal{A}') \\ \text{Lemma 4,} \quad & + \frac{(S + \log \gamma^{-1}) \left(\sum_{i=1}^{n_1} (S_i)_G^{\text{comb}} \log N_i + \sum_{j=1}^{n_2} 3(T_j)_G^{\text{comb}} \log p_i + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} 2(L_{k,j})_G^{\text{comb}} \log C_i \right)}{\eta} \\ & + \gamma, \end{aligned}$$

for any $\mathbf{P}, \mathbf{Q}, \mathbf{R}$ satisfying

$$\sum_{i=1}^{n_1} P_i \log N_i + \sum_{i=1}^{n_2} Q_i \log(p_i/e) + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} R_{i,j} \log(C_i/e) \leq \eta.$$

Since there is only an additive term between the two success probabilities, the above inequality implies

$$\begin{aligned}
& \text{Adv}_{G, \text{AI-RO+GG+IC}}(\mathcal{A}) \\
& \leq \text{Adv}_{G, \text{BF-RO+GG+IC}}(\mathcal{A}') \\
& \quad + \frac{(S + \log \gamma^{-1}) \left(\sum_{i=1}^{n_1} (S_i)_G^{\text{comb}} \log N_i + \sum_{i=1}^{n_2} 3(T_i)_G^{\text{comb}} \log p_i + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} 2(L_{i,j})_G^{\text{comb}} \log C_i \right)}{\eta} \\
& \quad + 2\gamma,
\end{aligned}$$

for both indistinguishability and unpredictability applications. Note that the extra factor of 2 is technically only necessary for indistinguishability applications [CDGS18]. $\square$

Multiplicative Error for Unpredictability Applications in the ROM+GGM+ICM.

Theorem 13. For every $\gamma > 0$, if an unpredictability application G is $(S, \mathbf{S}, \mathbf{T}, \mathbf{L}, \varepsilon')$ -secure in the $\text{BF-RO+GG+IC}(\mathbf{P}, \mathbf{Q}, \mathbf{R}, \{M_i, N_i\}_{i=1}^{n_1}, \{p_i, \ell_i\}_{i=1}^{n_2}, \{K_i, C_i\}_{i=1}^{n_3})$ -model for any $\mathbf{P}, \mathbf{Q}, \mathbf{R}$ as introduced in Definition 7 satisfying

$$\begin{aligned}
& (S + \log \gamma^{-1}) \left(\sum_{i=1}^{n_1} (S_i)_G^{\text{comb}} \log N_i + \sum_{i=1}^{n_2} 3(T_i)_G^{\text{comb}} \log p_i \right. \\
& \quad \left. + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} 2(L_{i,j})_G^{\text{comb}} \log C_i \right) \\
& \leq \sum_{i=1}^{n_1} P_i \log N_i + \sum_{i=1}^{n_2} Q_i \log(p_i/e) + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} R_{i,j} \log(C_i/e),
\end{aligned}$$

then it is $(S, \mathbf{S}, \mathbf{T}, \mathbf{L}, \varepsilon)$ -secure in the $\text{AI-RO+GG+IC}(\{M_i, N_i\}_{i=1}^{n_1}, \{p_i, \ell_i\}_{i=1}^{n_2}, \{K_i, C_i\}_{i=1}^{n_3})$ -model, for

$$\varepsilon \leq 2\varepsilon' + 2\gamma,$$

where e is the Euler's number, and $(S_i)_G^{\text{comb}}, (T_i)_G^{\text{comb}}$ and $(L_{i,j})_G^{\text{comb}}$ are the combined query complexity corresponding to G that corresponds to the random oracles, the generic group oracles, and the ideal ciphers, respectively.

Proof. Using the same attacker $\mathcal{A}'$ as in the proof of Theorem 12 and applying the second part of Lemma 4, for any $\mathbf{P}, \mathbf{Q}, \mathbf{R}$ such that $\sum_{i=1}^{n_1} P_i \log N_i + \sum_{i=1}^{n_2} Q_i \log(p_i/e) + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} R_{i,j} \log(C_i/e) \leq \eta$, we have

$$\begin{aligned}
& \text{Succ}_{G, \text{AI-RO+GG+IC}}(\mathcal{A}) \\
& \leq 2 \frac{(S + \log \gamma^{-1}) \left(\sum_{i=1}^{n_1} S_i \log N_i + \sum_{i=1}^{n_2} T_i \log p_i + \sum_{i=1}^{n_3} \sum_{j=1}^{K_i} L_{i,j} \log C_i \right)}{\eta} \cdot \text{Succ}_{G, \text{BF-RO+GG+IC}}(\mathcal{A}') + 2\gamma \\
& \leq 2 \cdot \text{Succ}_{G, \text{BF-RO+GG+IC}}(\mathcal{A}') + 2\gamma,
\end{aligned}$$

which translates into

$$\text{Adv}_{G,\text{AI-RO+GG+IC}}(\mathcal{A}) \leq 2 \cdot \text{Adv}_{G,\text{BF-RO+GG+IC}}(\mathcal{A}') + 2\gamma$$

for unpredictability applications. $\square$

C Multi-User Security of Key-Prefixed Short Schnorr Signatures

For completeness, we employ the bit-fixing-to-auxiliary-input technique utilized in [Theorem 8](#) to examine the multi-user security of a *key-prefixed* Schnorr signature scheme. We then compare our findings with the prior work [\[BL22\]](#). First, we analyze the multi-user security of key-prefixed Schnorr signatures within the bit-fixing ROM+GGM framework.

Theorem 14. *Let $\Pi^* = (\text{Kg}, \text{Sign}, \text{Vfy})$ be a key-prefixed Schnorr signature scheme and $p > 2^{2\lambda}$ be a prime number. Let $N \in \mathbb{N}$ be a parameter and $(\mathcal{A}_{\text{Sig.pre}}^{\text{BF-RO+GG}(P_1, P_2)}, \mathcal{A}_{\text{Sig.on}}^{\text{BF-RO+GG}(P_1, P_2)})$ be a pair of bit-fixing generic algorithms with a labeling map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ such that $\mathcal{A}_{\text{Sig.pre}}^{\text{BF-RO+GG}(P_1, P_2)}$ fixes P_1 input/output pairs of a random oracle $H : \{0, 1\}^* \rightarrow \{0, 1\}^{\lambda_1}$ and P_2 input/output pairs of a generic group oracle $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ such that $P_1 + P_2 = P$. If $\mathcal{A}_{\text{Sig.on}}^{\text{BF-RO+GG}(P_1, P_2)}$ makes at most $q_G^{\text{on}} := q_G^{\text{on}}(\lambda)$ queries to the generic group oracles, at most q_H^{on} queries to the random oracle, and at most q_S^{on} queries to the signing oracle, then*

$$\Pr \left[\text{SigForgeCor}_{\mathcal{A}_{\text{Sig.on, str}_{\tau, H}}^{\text{BF-RO+GG}(P_1, P_2)}, \Pi^*}^{\tau, N}(\lambda) = 1 \right] \leq \varepsilon,$$

with

$$\begin{aligned} \varepsilon = & \frac{q_G^{\text{on}}(N + P) + 3q_G^{\text{on}}(q_G^{\text{on}} + 1)/2}{p - 2P(N + 3q_G^{\text{on}} + 1) - (N + 3q_G^{\text{on}} + 1)^2 - N} + \frac{q_S^{\text{on}}(q_H^{\text{on}} + q_S^{\text{on}} + P)}{p} \\ & + \frac{q_H^{\text{on}} + q_S^{\text{on}} + P}{p - (N + P + 3q_G^{\text{on}} + 1)} + \frac{q_H^{\text{on}} + 1}{2^{\lambda_1}}, \end{aligned}$$

where the randomness is taken over the selection of τ and the random coins of $\mathcal{A}_{\text{Sig.on}}^{\text{BF-RO+GG}(P_1, P_2)}$.

Proof. Given a bit-fixing generic adversary $(\mathcal{A}_{\text{Sig.pre}}^{\text{BF-RO+GG}(P_1, P_2)}, \mathcal{A}_{\text{Sig.on}}^{\text{BF-RO+GG}(P_1, P_2)})$ that attacks the short Schnorr signature scheme, we construct the following efficient generic algorithm $\mathcal{A}_{\text{bridge}} = (\mathcal{A}_{\text{bridge.pre}}^{\text{BF-RO+GG}(P_1, P_2)}, \mathcal{A}_{\text{bridge.on}}^{\text{BF-RO+GG}(P_1, P_2)})$ in the bit-fixing model which tries to succeed in the 1-out-of- N generic BRIDGE^N-finding game $\text{BridgeChalCor}_{\mathcal{A}_{\text{bridge}}}^{\tau, N}(\lambda)$:

Algorithm $(\mathcal{A}_{\text{bridge.pre}}^{\text{BF-RO+GG}(P_1, P_2)}, \mathcal{A}_{\text{bridge.on}}^{\text{BF-RO+GG}(P_1, P_2)})$:

The algorithm is given $p, g = \tau(1), \text{pk}_i = \tau(x_i), 1 \leq i \leq N$ as input.

1. $\mathcal{A}_{\text{bridge.pre}}^{\text{BF-RO+GG}(P_1, P_2)}$ simply runs $\mathcal{A}_{\text{Sig.pre}}^{\text{BF-RO+GG}(P_1, P_2)}$ and fixes the same P_1 input/output pairs of H ($\{(h_1, H(h_1)), \dots, (h_{P_1}, H(h_{P_1}))\}$) and P_2 input/output pairs of τ ($\{(t_1, \tau(t_1)), \dots, (t_{P_2}, \tau(t_{P_2}))\}$) as being fixed by $\mathcal{A}_{\text{Sig.pre}}^{\text{BF-RO+GG}(P_1, P_2)}$.
2. Initialize the set $H_{\text{resp}} = \{\}$ which stores the random oracle input/output pairs observed during online processing. It also maintains the set $H_{\text{Fixed}} = \{(h_1, H(h_1)), \dots, (h_{P_1}, H(h_{P_1}))\}$ that $\mathcal{A}_{\text{Sig.pre}}^{\text{BF-RO+GG}(P_1, P_2)}$ has already fixed in Step 1. Let $h_i = \text{pk}_{j_i} \| I'_i \| m'_i$ for each $i \in [P_1]$ and for some $j_i \in [N]$.
3. Initialize the list $\mathcal{L} = \{(\tau(1), \mathbf{0}, 1), (\text{pk}_i, \hat{u}_i, 0) \text{ for } i \in [N], (\tau(t_j), \mathbf{0}, t_j) \text{ for } j \in [P_2], (I'_i, \mathbf{0}, r'_i := \text{DLog}(I'_i)) \text{ for } i \in [P_1]\}$ that contains $1 + N + P$ tuples, and runs $\mathcal{A}_{\text{Sig.on}}^{\text{BF-RO+GG}(P_1, P_2)}$ with a number of access to the generic group oracles $\text{GO} = (\text{Mult}(\cdot, \cdot), \text{Inv}(\cdot), \text{DLog}(\cdot), \text{Sign}_i(\cdot))$ for $1 \leq i \leq N$, and the random oracle $H(\cdot)$. We maintain the invariant that every output of a generic group query during the online phase appears in the list $\mathcal{L}$. We consider the following cases:
 - (a) Whenever $\mathcal{A}_{\text{Sig.on}}^{\text{BF-RO+GG}(P_1, P_2)}$ submits a query w to the random oracle H :
 - If there is a pair $(w, R) \in H_{\text{resp}}$ for some string $R \in \{0, 1\}^{\lambda_1}$ then return R .
 - Otherwise, check if $(w, H(w)) \in H_{\text{Fixed}}$. If so, then return $H(w)$.
 - Otherwise, select $R \leftarrow \{0, 1\}^{\lambda_1}$ and add (w, R) to H_{resp} .
 - If w has the form $w = (\text{pk}_j \| \mathbf{a} \| m_i)$ where the value $\mathbf{a}$ has not been observed previously (i.e., is not in the list $\mathcal{L}$ then we query $b = \text{DLog}(\mathbf{a})$ and add $(\mathbf{a}, \mathbf{0}, b)$ to $\mathcal{L}$.
 - (b) Whenever $\mathcal{A}_{\text{Sig.on}}^{\text{BF-RO+GG}(P_1, P_2)}$ submits a query $\mathbf{a}$ to the generic group oracle $\text{Inv}(\cdot)$:
 - If $\mathbf{a}$ is not in $\mathcal{L}$ then we immediately query $b = \text{DLog}(\mathbf{a})$ and add $(\mathbf{a}, \mathbf{0}, b)$ to $\mathcal{L}$.
 - Otherwise, $(\mathbf{a}, \mathbf{a}, b) \in \mathcal{L}$ for some $\mathbf{a}$ and b . Then we query $\text{Inv}(\mathbf{a}) = \tau(-\mathbf{a} \cdot \mathbf{x} - b)$, output the result and add $(\tau(-\mathbf{a} \cdot \mathbf{x} - b), -\mathbf{a}, -b)$ to $\mathcal{L}$.
 - (c) Whenever $\mathcal{A}_{\text{Sig.on}}^{\text{BF-RO+GG}(P_1, P_2)}$ submits a query $\mathbf{a}, \mathbf{b}$ to the generic group oracle $\text{Mult}(\cdot, \cdot)$:
 - If the element $\mathbf{a}$ (resp. $\mathbf{b}$) is not in $\mathcal{L}$ then query $b_0 = \text{DLog}(\mathbf{a})$ (resp. $b_1 = \text{DLog}(\mathbf{b})$) and add the element $(\mathbf{a}, \mathbf{0}, b_0)$ (resp. $(\mathbf{b}, \mathbf{0}, b_1)$) to $\mathcal{L}$.
 - Otherwise both elements $(\mathbf{a}, \mathbf{a}_0, b_0), (\mathbf{b}, \mathbf{a}_1, b_1) \in \mathcal{L}$. Then we return $\text{Mult}(\mathbf{a}, \mathbf{b}) = \tau((\mathbf{a}_0 + \mathbf{a}_1) \cdot \mathbf{x} + b_0 + b_1)$ and add $(\tau((\mathbf{a}_0 + \mathbf{a}_1) \cdot \mathbf{x} + b_0 + b_1), \mathbf{a}_0 + \mathbf{a}_1, b_0 + b_1) \in \mathcal{L}$.
 - (d) Whenever $\mathcal{A}_{\text{Sig.on}}^{\text{BF-RO+GG}(P_1, P_2)}$ submits a query m_i to the signing oracle $\text{Sign}(x_j, \cdot)$:
 - The attacker tries to forge a signature without knowledge of the secret key x_j , relying on the ability to program the random oracle as follows:
 - i. Pick s_i, e_i randomly and compute $I_i = \tau(s_i - x_j e_i) = \text{Mult}(\mathbf{s}_i, \text{Pow}(\text{pk}_j, -e_i))$ where $\mathbf{s}_i = \text{Pow}(\mathbf{g}, s_i)$.
 - ii. If $H(\text{pk}_j \| I_i \| m_i)$ has been previously queried or if it is in the set H_{Fixed} , then return $\perp$.
 - iii. Otherwise, program $H(\text{pk}_j \| I_i \| m_i) := e_i$ and return $\sigma_i = (s_i, e_i)$.
 - We remark that a side effect of querying the Sign_j oracle is the addition of the tuples $(\tau(s_i), \mathbf{0}, s_i), (\tau(x_j e_i), e_i \hat{u}_i, 0)$ and $(\tau(s_i - x_j e_i), -e_i \hat{u}_i, s_i)$ to $\mathcal{L}$, since these values are computed using the generic group oracles Inv, Mult .

4. After $\mathcal{A}_{\text{sig}}^{\text{on}}$ outputs $\sigma_{i^*} = (s_{i^*}, e_{i^*})$ and m_{i^*} , identify the index $i^* \in [N]$ such that $\text{Vf}(\text{pk}_{i^*}, m_{i^*}, \sigma_{i^*}) = 1$. Without loss of generality, we can assume that m_{i^*} is not a part of pre-fixed points H_{Fixed} in Step 2 since if the attacker forges a signature (s_{i^*}, e_{i^*}) for a message m_{i^*} which involves the pre-fixed point $(\text{pk}_{i^*} \| I_{i^*} \| m_{i^*}, \text{H}(\text{pk}_{i^*} \| I_{i^*} \| m_{i^*})) \in \text{H}_{\text{Fixed}}$ then we can directly force the bridge event to occur, i.e., $(I_{i^*}, \mathbf{0}, r_{i^*} = \text{DLog}(I_{i^*})) \in \mathcal{L}$ and $(I_{i^*}, -e_{i^*} \hat{u}_{i^*}, s_{i^*}) \in \mathcal{L}$ (see Step 7 below) since $\mathbf{0} \neq -e_{i^*} \hat{u}_{i^*}$.
5. Compute $\tau(-e_{i^*} x_{i^*}) = \text{Inv}(\text{Pow}(\tau(x_{i^*}), e_{i^*}))$. This will ensure that the elements $(\tau(-e_{i^*} x_{i^*}), -e_{i^*} \hat{u}_{i^*}, \mathbf{0})$ and $(\tau(e_{i^*} x_{i^*}), e_{i^*} \hat{u}_{i^*}, \mathbf{0})$ are both added to $\mathcal{L}$.
6. Compute $s_{i^*} = \text{Pow}(\mathbf{g}, s_{i^*})$ to ensure that $(s_{i^*}, \mathbf{0}, s_{i^*}) \in \mathcal{L}$.
7. Finally, compute $I_{i^*} = \text{Mult}(s_{i^*}, \tau(-e_{i^*} x_{i^*})) = \tau(s_{i^*} - x_{i^*} e_{i^*})$ which ensures that $(I_{i^*}, -e_{i^*} \hat{u}_{i^*}, s_{i^*}) \in \mathcal{L}$ and check to see if we previously had any tuple of the form $(I_{i^*}, \mathbf{a}, b) \in \mathcal{L}$.

Analysis. We first remark that if the signature is valid then we must have $e_{i^*} = \text{H}(\text{pk}_{i^*} \| I_{i^*} \| m_{i^*})$ and $\text{DLog}(I_{i^*}) = s_{i^*} - x_{i^*} e_{i^*} = \mathbf{a} \cdot \mathbf{x} + b$. Moreover, without loss of generality, we can assume that each string η occurs at most once in the list $\mathcal{L}$ in Step 3 because if at any point we have some string η such that $(\eta, \mathbf{a}, b) \in \mathcal{L}$ and $(\eta, \mathbf{c}, d) \in \mathcal{L}$ for $(\mathbf{a}, b) \neq (\mathbf{c}, d)$ then we can immediately have a BRIDGE^N instance $(\tau((\mathbf{a} - \mathbf{c}) \cdot \mathbf{x}), \mathbf{a} - \mathbf{c}, \mathbf{0}) \in \mathcal{L}$ and $(\tau(d - b), \mathbf{0}, d - b) \in \mathcal{L}$ since $\tau((\mathbf{a} - \mathbf{c}) \cdot \mathbf{x}) = \tau(d - b)$.

We now consider the failure events that our algorithm outputs $\perp$ for failure, either before $\mathcal{A}_{\text{Sig}}^{\text{on}}$ outputs a signature or after $\mathcal{A}_{\text{Sig}}^{\text{on}}$ outputs a valid signature.

- (1) The first failure event we consider is when our algorithm outputs $\perp$ before $\mathcal{A}_{\text{Sig}}^{\text{on}}$ outputs a signature. This event is called FailtoSign when our reduction outputs $\perp$ in Step 3.(d) due to the signing oracle failure, i.e., since $\text{H}(\text{pk}_j \| I_i \| m_i)$ has been previously queried or it is contained in the set H_{Fixed} . We observe that every time the attacker queries the signing oracle, we would generate a query to the random oracle H . Thus, we would have at most $q_{\text{H}}^{\text{on}} + q_{\text{S}}^{\text{on}} + P_1$ input/output pairs recorded for the random oracle, including the fixed points during the pre-processing phase. Since $I_i = \tau(s_i - x_i e_i)$ represents a fresh/randomly selected group element of size p , the probability that (pk_j, I_i, m_i) is one of the inputs is at most $(q_{\text{H}}^{\text{on}} + q_{\text{S}}^{\text{on}} + P_1)/p$. Applying union bound over q_{S}^{on} queries to the signing oracle, we have that

$$\Pr[\text{FailtoSign}] \leq \frac{q_{\text{S}}^{\text{on}}(q_{\text{H}}^{\text{on}} + q_{\text{S}}^{\text{on}} + P_1)}{p}.$$

- (2) Our algorithm can also output $\perp$ even after $\mathcal{A}_{\text{Sig}}^{\text{on}}$ outputs a valid signature. We define the event $\text{FailtoFind}(I_{i^*})$ that we find that the signature is valid but I_{i^*} was not previously recorded in our list $\mathcal{L}$ before we computed $\text{Mult}(s_{i^*}, \tau(-e_{i^*} x_{i^*}))$ in the last step so that we cannot find the bridge event. We observe that if $I_{i^*} \notin \mathcal{L}$ then we can view $I_{i^*} = \tau(s_{i^*} - x_{i^*} e_{i^*})$ as a uniformly random binary string from a set of size at least $p - |\mathcal{L}|$ which had not yet been selected at the time $\mathcal{A}_{\text{Sig}}^{\text{on}}$ outputs σ_{i^*} . Thus, the probability that the query $\text{H}(\text{pk}_{i^*} \| I_{i^*} \| m_{i^*})$ was previously recorded is at most $(q_{\text{H}}^{\text{on}} + q_{\text{S}}^{\text{on}} + P_1)/(p - |\mathcal{L}|)$. If the query $\text{H}(\text{pk}_{i^*} \| I_{i^*} \| m_{i^*})$ was *not* previously recorded then the probability of a successful forgery $\text{H}(\text{pk}_{i^*} \| I_{i^*} \| m_{i^*}) = e_{i^*}$ is at most 2^{-k_1} since we can view $\text{H}(\text{pk}_{i^*} \| I_{i^*} \| m_{i^*})$ chosen uniformly at random from the possible 2^{λ_1} options. Hence, we have that

$$\Pr[\text{FailtoFind}(I_{i^*})] \leq \frac{q_{\text{H}}^{\text{on}} + q_{\text{S}}^{\text{on}} + P_1}{p - |\mathcal{L}|} + \frac{1}{2^{\lambda_1}}.$$

- (3) Finally, there is another failure event called `BadQuery` after $\mathcal{A}_{\text{Sig}}^{\text{on}}$ outputs a valid signature, which denotes the event that the signature is valid but for the only prior tuple $(I_{i*}, \mathbf{a}, b) \in \mathcal{L}$ we have that $\mathbf{a} = -e_{i*} \hat{u}_{i*}$ so that we still cannot find the bridge event. We observe that by construction we ensure that the tuple $(I, \mathbf{a}, b)$ will always be recorded in $\mathcal{L}$ before a query of the form $H(\text{pk}_i \| I \| m)$ is ever issued — if I is new then we call $\text{DLog}(I)$ before querying the random oracle. Now define a subset $\hat{\mathcal{L}} \subset \mathcal{L}$ as the set of tuples $(\mathbf{a}, \mathbf{a}, b) \in \mathbb{G} \times \mathbb{Z}_p^N \times \mathbb{Z}_p$ such that $\mathbf{a}$ has exactly *one* nonzero element. Now we call a random oracle query $x = (\text{pk}_i \| I \| m)$ “bad” if $H(x) = -\tilde{a}$ where the tuple $(I, \mathbf{a}, b) \in \hat{\mathcal{L}}$ has already been recorded and the nonzero element of $\mathbf{a}$ is $\tilde{a}$ (Recall that if there were two recorded tuples $(I, \mathbf{a}, b)$ and $(I, \mathbf{c}, d)$ then our algorithm would have already found a BRIDGE^N instance). Thus, the probability each individual query is “bad” is at most $1/2^{\lambda_1}$ and we can use union bounds to upper bound the probability of any “bad” query as

$$\Pr[\text{BadQuery}] \leq \frac{q_{\text{H}}^{\text{on}}}{2^{\lambda_1}}.$$

Now we have shown that

$$\begin{aligned} & \Pr \left[\text{BridgeChalCor}_{\mathcal{A}_{\text{bridge}}}^{\tau, N}(\lambda) = 1 \right] \\ & \geq \Pr \left[\text{SigForgeCor}_{\mathcal{A}_{\text{Sig.on, str}_{\tau, \text{H}}}, \Pi_{\setminus \{0\}}^{\text{short}}}^{\tau, N}(\lambda) = 1 \right] - \Pr[\text{FailtoSign}] \\ & \quad - \Pr[\text{FailtoFind}(I_{i*})] - \Pr[\text{BadQuery}] \\ & \geq \Pr \left[\text{SigForgeCor}_{\mathcal{A}_{\text{Sig.on, str}_{\tau, \text{H}}}, \Pi_{\setminus \{0\}}^{\text{short}}}^{\tau, N}(\lambda) = 1 \right] - \frac{q_{\text{S}}^{\text{on}}(q_{\text{H}}^{\text{on}} + q_{\text{S}}^{\text{on}} + P_1)}{p} \\ & \quad - \frac{q_{\text{H}}^{\text{on}} + q_{\text{S}}^{\text{on}} + P_1}{p - |\mathcal{L}|} - \frac{q_{\text{H}}^{\text{on}} + 1}{2^{\lambda_1}}. \end{aligned}$$

Finally, by applying [Lemma 1](#), we can conclude that

$$\begin{aligned} & \Pr \left[\text{SigForgeCor}_{\mathcal{A}_{\text{Sig.on, str}_{\tau, \text{H}}}, \Pi_{\setminus \{0\}}^{\text{short}}}^{\tau, N}(\lambda) = 1 \right] \\ & \leq \left[\text{BridgeChalCor}_{\mathcal{A}_{\text{bridge}}}^{\tau, N}(\lambda) = 1 \right] + \frac{q_{\text{S}}^{\text{on}}(q_{\text{H}}^{\text{on}} + q_{\text{S}}^{\text{on}} + P_1)}{p} + \frac{q_{\text{H}}^{\text{on}} + q_{\text{S}}^{\text{on}} + P_1}{p - |\mathcal{L}|} \\ & \quad + \frac{q_{\text{H}}^{\text{on}} + 1}{2^{\lambda_1}} \\ & \leq \frac{q_{\text{G}}^{\text{on}}(N + P) + 3q_{\text{G}}^{\text{on}}(q_{\text{G}}^{\text{on}} + 1)/2}{p - 2P(N + 3q_{\text{G}}^{\text{on}} + 1) - (N + 3q_{\text{G}}^{\text{on}} + 1)^2 - N} + \frac{q_{\text{S}}^{\text{on}}(q_{\text{H}}^{\text{on}} + q_{\text{S}}^{\text{on}} + P)}{p} \\ & \quad + \frac{q_{\text{H}}^{\text{on}} + q_{\text{S}}^{\text{on}} + P}{p - (N + P + 3q_{\text{G}}^{\text{on}} + 1)} + \frac{q_{\text{H}}^{\text{on}} + 1}{2^{\lambda_1}}. \quad \square \end{aligned}$$

Combining with [Theorem 6](#), we get the following result in the AI-ROM+GGM.

Theorem 15. Let $\Pi^* = (\text{Kg}, \text{Sign}, \text{Vfy})$ be a key-prefixed Schnorr signature scheme, $p > 2^{2\lambda}$ be a prime number, and $\gamma > 0$ be a parameter. Let $N \in \mathbb{N}$ be a parameter and $(\mathcal{A}_{\text{Sig.pre}}^{\text{AI-RO+GG}}, \mathcal{A}_{\text{Sig.on}}^{\text{AI-RO+GG}})$ be a pair of generic algorithms with a labeling map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ such that $\mathcal{A}_{\text{Sig.pre}}^{\text{AI-RO+GG}}$ outputs an S -bit hint $\text{str}_{\text{H}, \tau}$. If $\mathcal{A}_{\text{Sig.on}}^{\text{AI-RO+GG}}$ takes $\text{str}_{\text{H}, \tau}$ as input and makes at most $q_{\text{G}}^{\text{on}} := q_{\text{G}}^{\text{on}}(\lambda)$ queries to the generic

group oracles, at most q_H^{on} queries to the random oracle, and at most q_S^{on} queries to the signing oracle, then

$$\Pr \left[\text{SigForgeCor}_{\mathcal{A}_{\text{Sig.on, str}_{\tau, H}}^{\text{AI-RO+GG}}, \Pi^*}^{\tau, N}(\lambda) = 1 \right] \leq \varepsilon, \text{ with}$$

$$\begin{aligned} \varepsilon = & \frac{2q_G^{\text{on}}(N + Sq) + 3q_G^{\text{on}}(q_G^{\text{on}} + 1)}{p - 2Sq(N + 3q_G^{\text{on}} + 1) - (N + 3q_G^{\text{on}} + 1)^2 - N} \\ & + \frac{2q_S^{\text{on}}(q_H^{\text{on}} + q_S^{\text{on}} + Sq)}{p} \\ & + \frac{2(q_H^{\text{on}} + q_S^{\text{on}} + Sq)}{p - (N + Sq + 3q_G^{\text{on}} + 1)} + \frac{q_H^{\text{on}} + 1}{2^{\lambda_1 - 1}} + 2^{-2\lambda + 1}, \end{aligned}$$

where $q = q_H^{\text{on}} + 3q_G^{\text{on}} + q_S^{\text{on}}$ and the randomness is taken over the selection of τ and the random coins of $\mathcal{A}_{\text{Sig.on}}^{\text{AI-RO+GG}}$.

Proof. This is straightforward by combining [Theorem 14](#) with [Theorem 5](#) and by setting an optimal $P \approx Sq$ and $\gamma = 2^{-2\lambda}$. $\square$

D Warmup: Security of PSEC-KEM without Preprocessing in the ROM+GGM

The Generic Quadratic Bridge-Finding Game. We establish the CPA security of PSEC-KEM (without preprocessing) via reduction from a new game called *the generic quadratic bridge-finding game*. As the name suggests, we will consider the game in the GGM with the labeling map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$. In this game, the attacker is given $\tau(1)$ and $\tau(x)$ for a random unknown $x \in \mathbb{Z}_p$. The attacker is given access to the generic group oracle (Mult, Inv), along with an additional oracle $\mathcal{O}$ which returns some $\tau(\alpha)$ where the value $\alpha \in \mathbb{Z}_p$ is unknown to the attacker. In a nutshell, as the attacker makes multiple queries to those oracles, the number of unknowns can be increased by at most 2 per query, and the attacker's goal is to find a specific form of quadratic relationship between these unknowns. One important note is that the attacker *must output* the very quadratic relationship. It is not required for the attacker to solve the equation for the unknowns, but the attacker should specify which unknown values satisfy which quadratic relationship. We formally define the game below.

The Generic Quadratic Bridge-Finding Game $\text{QDBridgeChal}_{\mathcal{A}}^{\tau}(\lambda)$:

1. The challenger picks a random $x \leftarrow \$ \mathbb{Z}_p$ and initializes the list $\mathcal{L} = \{(\tau(1), 0, 1), (\tau(x), 1, 0)\}$.
2. The adversary $\mathcal{A}$ is given $\tau(1)$ and $\tau(x)$ as input. Here, $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ is a labeling map where $\mathbb{G}$ is a set of bitstrings of length $\ell \geq \log p$, and p is a 2λ -bit prime.
3. The challenger maintains a vector $\mathbf{x} = (x_1, \dots, x_n) \in \mathbb{Z}_p^n$ which is a vector of unknowns such that $x_1 = x$, where the dimension n satisfies the following:
 - initially, $n = 1$ since the only unknown is x , and
 - the number of unknowns n increases by at most 2 as $\mathcal{A}$ makes query to the generic group oracle (Mult, Inv) or a generic oracle $\mathcal{O}$ that selects a new random unknown variable $x_j \leftarrow \$ \mathbb{Z}_p$ and outputs $\tau(x_j)$.
4. $\mathcal{A}$ is allowed to query the generic group oracle ($\text{Mult}(\cdot, \cdot), \text{Inv}(\cdot)$) and the oracle $\mathcal{O}$.
 - (a) If $\mathcal{A}$ submits a query to $\mathcal{O}$, $\mathcal{A}$ gets some bitstring $\eta \in \tau(\mathbb{Z}_p)$. If η does not appear in $\mathcal{L}$ and $\dim(\mathbf{x}) = j$, then the challenger updates $\mathbf{x} \leftarrow \mathbf{x} \circ x_{j+1}$ (i.e., increase the dimension

- of $\mathbf{x}$ by 1 and add the element x_{j+1} at the end), where x_{j+1} denotes an unknown variable satisfying the constraint $\tau(x_{j+1}) = \eta$, and updates all of the entries of the list $\mathcal{L}$ from $(s, \mathbf{a}, b)$ to $(s, \mathbf{a} \circ 0, b)$. Then the challenger adds $(\eta, \mathbf{0} \circ 1, 0)$ to $\mathcal{L}$.
- (b) If $\mathcal{A}$ submits a query η_1, η_2 to $\text{Mult}(\cdot, \cdot)$:
- If there exist tuples $(\eta_1, \mathbf{a}_1, b_1), (\eta_2, \mathbf{a}_2, b_2) \in \mathcal{L}$ for both η_1 and η_2 , then the challenger adds the tuple $(\text{Mult}(\eta_1, \eta_2), \mathbf{a}_1 + \mathbf{a}_2, b_1 + b_2)$ to the list $\mathcal{L}$.
 - If $(\eta_1, \mathbf{a}_1, b_1) \in \mathcal{L}$ but η_2 does not appear in $\mathcal{L}$ and if $\eta_2 \in \tau(\mathbb{Z}_p)$, and if $\dim(\mathbf{x}) = j$, then the challenger first updates $\mathbf{x} \leftarrow \mathbf{x} \circ x_{j+1}$ where x_{j+1} denotes an unknown variable satisfying the constraint $\tau(x_{j+1}) = \eta_2$ and updates all of the entries of the list $\mathcal{L}$ from $(s, \mathbf{a}, b)$ to $(s, \mathbf{a} \circ 0, b)$. Then the challenger adds tuples $(\eta_2, \mathbf{0} \circ 1, 0)$ and $(\text{Mult}(\eta_1, \eta_2), \mathbf{a}_1 \circ 0 + \mathbf{0} \circ 1, b_1)$ to $\mathcal{L}$. Do it similarly if η_2 appears in $\mathcal{L}$ and η_1 does not.
 - If both η_1 and η_2 does not appear in $\mathcal{L}$, and if both $\eta_1, \eta_2 \in \tau(\mathbb{Z}_p)$, and if $\dim(\mathbf{x}) = j$, then the challenger first updates $\mathbf{x} \leftarrow \mathbf{x} \circ x_{j+1} \circ x_{j+2}$ where x_{j+1} and x_{j+2} are unknown values such that $\tau(x_{j+i}) = \eta_i$ for $i = 1, 2$, and updates all the entries of the list $\mathcal{L}$ from $(s, \mathbf{a}, b)$ to $(s, \mathbf{a} \circ 0^2, b)$. Then the challenger adds tuples $(\eta_1, \mathbf{0} \circ 1 \circ 0, 0), (\eta_2, \mathbf{0} \circ 0 \circ 1, 0)$, and $(\text{Mult}(\eta_1, \eta_2), \mathbf{0} \circ 1 \circ 1, 0)$ to $\mathcal{L}$.
- (c) If $\mathcal{A}$ submits a query η to $\text{Inv}(\cdot)$:
- If there exists a tuple $(\eta, \mathbf{a}, b) \in \mathcal{L}$, then the challenger adds the tuple $(\text{Inv}(\eta), -\mathbf{a}, -b)$ to the list $\mathcal{L}$.
 - If η does not appear in $\mathcal{L}$ and $\eta \in \tau(\mathbb{Z}_p)$, and if $\dim(\mathbf{x}) = j$, then the challenger first updates $\mathbf{x} \leftarrow \mathbf{x} \circ x_{j+1}$ where x_{j+1} is an unknown value that satisfies $\tau(x_{j+1}) = \eta$ and updates all of the entries of the list $\mathcal{L}$ from $(s, \mathbf{a}, b)$ to $(s, \mathbf{a} \circ 0, b)$. Then the challenger adds tuples $(\eta, \mathbf{0} \circ 1, 0)$ and $(\text{Inv}(\eta), \mathbf{0} \circ (-1), 0)$ to $\mathcal{L}$.
5. If at any point in time $\mathcal{A}$ outputs a tuple $(i_1, i_2, \mathbf{a}, b)$ such that (1) $1 \leq i_1, i_2 \leq \dim(\mathbf{x})$, (2) $(\mathbf{a}, b) \in \mathbb{Z}_p^{\dim(\mathbf{x})} \times \mathbb{Z}_p$, and (3) $x_{i_1} x_{i_2} = \mathbf{a} \cdot \mathbf{x} + b \pmod p$, then we say that the event QDBridge occurs. In this case, the output of the game is 1 and we write $\text{QDBridgeChal}_{\mathcal{A}}^{\tau}(\lambda) = 1$. If QDBridge never occurs, then the output of the game is 0.

Lemma 5 upper bounds the probability to win the generic quadratic bridge-finding game $\text{QDBridgeChal}_{\mathcal{A}}^{\tau}(\lambda)$.

Lemma 5. *For any attacker $\mathcal{A}$ making at most $q_{\mathcal{G}}$ generic group oracle queries and $q_{\mathcal{O}}$ queries to the oracle $\mathcal{O}$,*

$$\Pr[\text{QDBridgeChal}_{\mathcal{A}}^{\tau}(\lambda) = 1] \leq \frac{3(q_{\mathcal{G}} + q_{\mathcal{O}})^2 + 5(q_{\mathcal{G}} + q_{\mathcal{O}}) + 4}{2p - 2(3q_{\mathcal{G}} + q_{\mathcal{O}} + 1)^2 - 2(2q_{\mathcal{G}} + q_{\mathcal{O}})},$$

in the generic group model of prime order p where the randomness is taken over the selection of x and τ , as well as any random coins of $\mathcal{A}$.

Proof. The proof works by maintaining a list $\mathcal{L}$ of tuples $(\tau(y), \mathbf{a}, b)$ where $\tau(y) \in \mathbb{G}$, $\mathbf{a} \in \mathbb{Z}_p^{\dim(\mathbf{x})}$, and $b \in \mathbb{Z}_p$ such that $y = \mathbf{a} \cdot \mathbf{x} + b$ for every oracle output $\tau(y)$. To bound the probability that the attacker wins the quadratic bridge-finding game, we consider the case where the event QDBridge occurs after the attacker makes $q_{\mathcal{G}}$ generic group queries and $q_{\mathcal{O}}$ queries to $\mathcal{O}$. Suppose that $\dim(\mathbf{x}) = j$ at the point in time when QDBridge occurs. We observe that $j \leq 2q_{\mathcal{G}} + q_{\mathcal{O}} + 1$ since $\dim(\mathbf{x})$ can be increased by at most 2 whenever $\mathcal{A}$ makes a generic group query and at most

1 whenever $\mathcal{A}$ makes a query to $\mathcal{O}$. We further observe that in this case, $|\mathcal{L}| \leq 3q_{\mathcal{G}} + q_{\mathcal{O}} + 1$ since for each query to $\mathcal{O}$, at most 1 item can be added to $\mathcal{L}$ whereas each generic group oracle query can add at most 3 items to $\mathcal{L}$ (exactly three in the case when $\mathcal{A}$ queries $\text{Mult}(\eta_1, \eta_2)$ on two fresh elements).

Now consider the event BRIDGE where the list $\mathcal{L}$ contains two distinct tuples $(\eta, \mathbf{a}, b)$ and $(\eta', \mathbf{a}', b')$ such that $\mathbf{a} \cdot \mathbf{x} + b = \mathbf{a}' \cdot \mathbf{x} + b'$. To upper bound $\Pr[\text{BRIDGE}]$, consider the event $\overline{\text{BRIDGE}}_{<i}$ that the event BRIDGE has not occurred until the $(i-1)^{\text{th}}$ query. Conditioning on the event $\overline{\text{BRIDGE}}_{<i}$, we are now interested in the event BRIDGE_i where the i^{th} query makes the event BRIDGE occur, i.e., a tuple $(\eta_i, \mathbf{a}_i, b_i)$ has been recorded to $\mathcal{L}$ and there exists a tuple of the form $(\cdot, \mathbf{a}, b)$ such that $\mathbf{a}_i \cdot \mathbf{x} + b_i = \mathbf{a} \cdot \mathbf{x} + b$. We can essentially view $\mathbf{x}$ sampled uniformly at random subject to some restrictions due to $\overline{\text{BRIDGE}}_{<i}$, from which we know that for any distinct pair $(\eta, \mathbf{a}, b)$ and $(\eta', \mathbf{a}', b')$ we have $\mathbf{a} \cdot \mathbf{x} + b \neq \mathbf{a}' \cdot \mathbf{x} + b'$.

Let $r \leq \dim(\mathbf{x}) = j$ be an index such that $\mathbf{a}[r] - \mathbf{a}'[r] \neq 0$ and suppose that $x_r = \mathbf{x}[r]$ is the last value sampled. At this point, we can view x_r as being drawn uniformly at random from a set of at least $p - |\mathcal{L}|^2 - (j-1)$ remaining values. To see this, we observe that x_r must be distinct from $\{x_1, \dots, x_j\} \setminus \{x_r\}$ since otherwise we would have had the event $\text{BRIDGE}_{<i}$, and for each pair of distinct elements $(\eta, \mathbf{a}, b)$ and $(\eta', \mathbf{a}', b')$, we can eliminate one possible value of x_r due to the fact that $\mathbf{a} \cdot \mathbf{x} + b \neq \mathbf{a}' \cdot \mathbf{x} + b'$ and $\mathbf{a}[r] \neq \mathbf{a}'[r]$, which implies $x_r \neq \left[(b' - b) + \sum_{i \neq r} x_i (\mathbf{a}'[i] - \mathbf{a}[i]) \right] \cdot (\mathbf{a}[r] - \mathbf{a}'[r])^{-1}$. Thus, the probability that $\mathbf{a}_i \cdot \mathbf{x} + b_i = \mathbf{a} \cdot \mathbf{x} + b$ is at most $\frac{1}{p - (3q_{\mathcal{G}} + q_{\mathcal{O}} + 1)^2 - (2q_{\mathcal{G}} + 2q_{\mathcal{O}})}$. Union bounding over all tuples in $\mathcal{L}$, we have

$$\begin{aligned} \Pr[\text{BRIDGE}] &= \sum_{i \leq q_{\mathcal{G}} + q_{\mathcal{O}}} \Pr[\text{BRIDGE}_i | \overline{\text{BRIDGE}}_{<i}] \\ &\leq \sum_{i \leq q_{\mathcal{G}} + q_{\mathcal{O}}} \frac{1 + 3i}{p - (3q_{\mathcal{G}} + q_{\mathcal{O}} + 1)^2 - (2q_{\mathcal{G}} + 2q_{\mathcal{O}})} \\ &\leq \frac{3(q_{\mathcal{G}} + q_{\mathcal{O}})^2 + 5(q_{\mathcal{G}} + q_{\mathcal{O}})}{2p - 2(3q_{\mathcal{G}} + q_{\mathcal{O}} + 1)^2 - 2(2q_{\mathcal{G}} + 2q_{\mathcal{O}})}. \end{aligned}$$

Now, consider the case where the event BRIDGE never occurs. Then we would like to upper bound the probability

$$\Pr[\text{QDBridgeChal}_{\mathcal{A}}^{\tau}(\lambda) = 1 | \overline{\text{BRIDGE}}].$$

The argument works similarly; conditioning on the event $\overline{\text{BRIDGE}}$, we know that for any distinct pair $(\eta, \mathbf{a}, b)$ and $(\eta', \mathbf{a}', b')$ we have $\mathbf{a} \cdot \mathbf{x} + b \neq \mathbf{a}' \cdot \mathbf{x} + b'$. Let $r \leq \dim(\mathbf{x}) \leq 2q_{\mathcal{G}} + q_{\mathcal{O}} + 1$ be an index such that $\mathbf{a}[r] - \mathbf{a}'[r] \neq 0$ and suppose that $x_r = \mathbf{x}[r]$ is the last value sampled. At this point, we can view x_r as being drawn uniformly at random from a set of at least $p - |\mathcal{L}|^2 - (2q_{\mathcal{G}} + q_{\mathcal{O}})$ remaining values as before. Now, the attacker wins if and only if s/he outputs a tuple $(i_1, i_2, \mathbf{a}, b)$ such that $x_{i_1} x_{i_2} = \mathbf{a} \cdot \mathbf{x} + b$. Since this equation is at most degree 2 in terms of x_r (in case $i_1 = i_2 = r$), there are at most 2 roots in $\mathbb{Z}_p$. Thus, we have that

$$\begin{aligned} \Pr[\text{QDBridgeChal}_{\mathcal{A}}^{\tau}(\lambda) = 1 | \overline{\text{BRIDGE}}] &\leq \frac{2}{p - |\mathcal{L}|^2 - (2q_{\mathcal{G}} + q_{\mathcal{O}})} \\ &\leq \frac{2}{p - (3q_{\mathcal{G}} + q_{\mathcal{O}} + 1)^2 - (2q_{\mathcal{G}} + q_{\mathcal{O}})}. \end{aligned}$$

Taken together, we have

$$\begin{aligned}
& \Pr [\text{QDBridgeChal}_{\mathcal{A}}^{\tau}(\lambda) = 1] \\
& \leq \Pr [\text{BRIDGE}] + \Pr [\text{QDBridgeChal}_{\mathcal{A}}^{\tau}(\lambda) = 1 \mid \overline{\text{BRIDGE}}] \\
& \leq \frac{(3/2)(q_{\mathcal{G}} + q_{\mathcal{O}})^2 + (5/2)(q_{\mathcal{G}} + q_{\mathcal{O}})}{p - (3q_{\mathcal{G}} + q_{\mathcal{O}} + 1)^2 - (2q_{\mathcal{G}} + q_{\mathcal{O}})} \\
& \quad + \frac{2}{p - (3q_{\mathcal{G}} + q_{\mathcal{O}} + 1)^2 - (2q_{\mathcal{G}} + q_{\mathcal{O}})} \\
& = \frac{3(q_{\mathcal{G}} + q_{\mathcal{O}})^2 + 5(q_{\mathcal{G}} + q_{\mathcal{O}}) + 4}{2p - 2(3q_{\mathcal{G}} + q_{\mathcal{O}} + 1)^2 - 2(2q_{\mathcal{G}} + q_{\mathcal{O}})}. \quad \square
\end{aligned}$$

D.1 The CPA Security of PSEC-KEM in the ROM+GGM

Now we are ready to prove the CPA security of PSEC-KEM in the ROM+GGM by reduction from the quadratic bridge-finding game.

Theorem 16. *The PSEC-KEM scheme $\Pi = (\text{Gen}, \text{Encaps}, \text{Decaps})$ is $(q_{\mathcal{H}}, q_{\mathcal{G}}, q_{\mathcal{E}}, \varepsilon)$ -CPA secure with*

$$\varepsilon = \frac{3(q_{\mathcal{G}} + 2q_{\mathcal{E}})^2 + 5(q_{\mathcal{G}} + 2q_{\mathcal{E}}) + 4}{p - (3q_{\mathcal{G}} + 2q_{\mathcal{E}} + 1)^2 - 2(q_{\mathcal{G}} + q_{\mathcal{E}})} + \frac{3q_{\mathcal{H}}q_{\mathcal{E}}}{2^{\lambda_1}} + \frac{q_{\mathcal{H}}q_{\mathcal{E}}}{p},$$

in the generic group model of order $p \approx 2^{2\lambda}$ and the programmable random oracle model.

Proof. To prove [Theorem 16](#), we use a hybrid argument to prove the CPA security of PSEC-KEM. In the first hybrid (hybrid H_0), the distinguisher $\mathcal{D}$ is given the CPA indistinguishability game for PSEC-KEM with the challenge bit $b = 0$, and in the last hybrid (hybrid H_3), $\mathcal{D}$ is given the CPA indistinguishability game with $b = 1$. Each hybrid is defined as follows:

Hybrid H_0 . This is the original CPA indistinguishability game $\text{KEM}_{\mathcal{A}, \Pi}^{\tau, \mathcal{H}, \text{cpa}}(\lambda)$ for PSEC-KEM with the challenge bit $b = 0$. In particular, $\mathcal{D}$ has access to the encapsulation oracle $\text{Encaps}'_0(\cdot) := \text{Encaps}(\cdot, 1^\lambda)$, i.e., whenever $\mathcal{D}$ submits a query $\text{Encaps}'_0(\text{pk})$, $\mathcal{D}$ always gets the output $\text{Encaps}(\text{pk}, 1^\lambda)$.

Hybrid H_1 . This is the same security game with H_0 except that the encapsulation oracle $\text{Encaps}'_0(\cdot)$ is replaced with a modified oracle $\text{Encaps}'_1(\cdot)$. In $\text{Encaps}'_1(\cdot)$, $\tau(\alpha)$ and $\tau(\alpha x)$ (where x is the secret key) are replaced with random elements in $\tau(\mathbb{Z}_p)$ by querying the oracle $\mathcal{O}$ twice. See [Figure 2](#) for the details.

Hybrid H_2 . This is the same security game with H_1 except that the encapsulation oracle $\text{Encaps}'_1(\cdot)$ is further replaced with a modified oracle $\text{Encaps}'_2(\cdot)$. $\text{Encaps}'_2(\cdot)$ is exactly the same as $\text{Encaps}'_1(\cdot)$ except that the key k is sampled uniformly at random from $\{0, 1\}^\lambda$.

Hybrid H_3 . This is the same security game with H_2 except that the encapsulation oracle $\text{Encaps}'_2(\cdot)$ is replaced with a modified oracle $\text{Encaps}'_3(\cdot) := \text{Encaps}_1(\cdot)$, which leads to the original CPA indistinguishability game $\text{KEM}_{\mathcal{A}, \Pi}^{\tau, \mathcal{H}, \text{cpa}}(\lambda)$ for PSEC-KEM with the challenge bit $b = 1$. In particular, $\text{Encaps}'_3(\cdot)$ is the same as $\text{Encaps}'_2(\cdot)$ but the random elements in $\tau(\mathbb{Z}_p)$ are reverted back to the honest computation $\tau(\alpha)$ and $\tau(\alpha x)$. This can also be interpreted as replacing an honest key k in $\text{Encaps}'_0(\cdot)$ with a uniformly random key of length λ . See [Figure 2](#) for the details.

$\text{Encaps}'_0(\text{pk})$	$\text{Encaps}'_1(\text{pk})$
1 : $r \leftarrow \{0, 1\}^{\lambda_1}$	1 : $r \leftarrow \{0, 1\}^{\lambda_1}$
2 : $\alpha \leftarrow H_0(r), k \leftarrow H_1(r)$	2 : $\eta_1 \leftarrow \mathcal{O}(\cdot), \eta_2 \leftarrow \mathcal{O}(\cdot), k \leftarrow H_1(r)$
3 : $c_1 \leftarrow r \oplus H_2(\tau(\alpha) \parallel \tau(\alpha x))$	3 : $c_1 \leftarrow r \oplus H_2(\eta_1 \parallel \eta_2)$
4 : $c \leftarrow (\tau(\alpha), c_1)$	4 : $c \leftarrow (\eta_1, c_1)$
5 : Return (k, c)	5 : Return (k, c)
$\text{Encaps}'_2(\text{pk})$	$\text{Encaps}'_3(\text{pk})$
1 : $r \leftarrow \{0, 1\}^{\lambda_1}$	1 : $r \leftarrow \{0, 1\}^{\lambda_1}$
2 : $\eta_1 \leftarrow \mathcal{O}(\cdot), \eta_2 \leftarrow \mathcal{O}(\cdot), k \leftarrow \{0, 1\}^\lambda$	2 : $\alpha \leftarrow H_0(r), k \leftarrow \{0, 1\}^\lambda$
3 : $c_1 \leftarrow r \oplus H_2(\eta_1 \parallel \eta_2)$	3 : $c_1 \leftarrow r \oplus H_2(\tau(\alpha) \parallel \tau(\alpha x))$
4 : $c \leftarrow (\eta_1, c_1)$	4 : $c \leftarrow (\tau(\alpha), c_1)$
5 : Return (k, c)	5 : Return (k, c)

Figure 2. Comparison between the oracles Encaps'_i for PSEC-KEM, which is used in Hybrid H_i for each $i \in \{0, 1, 2, 3\}$, where the highlighted parts denote the difference from the previous hybrid.

Indistinguishability of H_0 and H_1 . We observe that two hybrids are perfectly indistinguishable unless one of the two events occurs: for some round $i \leq q_E$,

- (1) the random oracle query $H_2(\tau(\alpha_i) \parallel \tau(\alpha_i x))$ has been made from the adversary, or
- (2) the adversary makes query r_i to H_0 but has not queried $(\tau(\alpha_i) \parallel \tau(\alpha_i x))$ to H_2 .

In Case (1), we argue that $\mathcal{D}$ can win the quadratic bridge-finding game. Once $\mathcal{D}$ ever makes a query of the form $H_2(\tau(\alpha_i) \parallel \tau(\alpha_i x))$, $\mathcal{D}$ can win the quadratic bridge-finding game as follows:

- We observe that $\mathcal{D}$ can maintain the list $\mathcal{L}$ as follows.
 - Initialize the list $\mathcal{L} = \{(\tau(1), 0, 1), (\text{pk} = \tau(x), 1, 0)\}$ and the vector of unknowns $\mathbf{x} := (x_1)$ (for now, $\dim \mathbf{x} = 1$) where x_0 is the unknown (secret key) that maps to pk , i.e., $\tau(x_1) = \text{pk}$. (Note: $x_1 = x$ but $\mathcal{D}$ does not know sk and simply name it as x_1 as an unknown.) $\mathcal{D}$ also initializes $H_i^{\text{resp}} = \{\}$ for $i = 0, 1, 2$, where H_i^{resp} stores the random oracle queries of H_i .
 - Whenever $\mathcal{D}$ submits a query w to the RO H_i for $i = 0, 1, 2$:
 - ♣ If there is a pair $(w, R) \in H_i^{\text{resp}}$ for some string R , then return R .
 - ♣ Otherwise, select a uniformly random $R \leftarrow \text{codomain}(H_i)$ and add (w, R) to the set H_i^{resp} .
 - ♣ If w has the form $w = (\tau(\alpha) \parallel \cdot)$ for some α and if $\tau(\alpha)$ does not appear in $\mathcal{L}$, then update the vector $\mathbf{x} \leftarrow \mathbf{x} \circ \alpha$ and update all of the entries in $\mathcal{L}$ from $(\eta, \mathbf{a}, b)$ to $(\eta, \mathbf{a} \circ 0, b)$. Finally, add a new tuple $(\tau(\alpha), \mathbf{0} \circ 1, 0)$ to $\mathcal{L}$.
 - Whenever $\mathcal{D}$ submits a query η_1, η_2 to $\text{Mult}(\cdot, \cdot)$:
 - ♣ If there exist tuples $(\eta_1, \mathbf{a}_1, b_1), (\eta_2, \mathbf{a}_2, b_2) \in \mathcal{L}$ for both η_1 and η_2 , then add the tuple $(\text{Mult}(\eta_1, \eta_2), \mathbf{a}_1 + \mathbf{a}_2, b_1 + b_2)$ to the list $\mathcal{L}$.
 - ♣ If $(\eta_1, \mathbf{a}_1, b_1) \in \mathcal{L}$ but η_2 does not appear in $\mathcal{L}$ and if $\eta_2 \in \tau(\mathbb{Z}_p)$, then first update $\mathbf{x} \leftarrow \mathbf{x} \circ y_2$ where y_2 is an unknown value that satisfies $\tau(y_2) = \eta_2$ and update all of the entries of the list $\mathcal{L}$ from $(\eta, \mathbf{a}, b)$ to $(\eta, \mathbf{a} \circ 0, b)$. Then add tuples $(\eta_2, \mathbf{0} \circ 1, 0)$ and $(\text{Mult}(\eta_1, \eta_2), \mathbf{a}_1 \circ 0 + \mathbf{0} \circ 1, b_1)$ to $\mathcal{L}$. Do it similarly if η_2 appears in $\mathcal{L}$ and η_1 does not.

- ♣ If both η_1, η_2 does not appear in $\mathcal{L}$, and if both $\eta_1, \eta_2 \in \tau(\mathbb{Z}_p)$, then first update $\mathbf{x} \leftarrow \mathbf{x} \circ y_1 \circ y_2$ where y_1 and y_2 are unknown values such that $\tau(y_i) = \eta_i$ for $i = 1, 2$, and update all the entries of the list $\mathcal{L}$ from $(\eta, \mathbf{a}, b)$ to $(\eta, \mathbf{a} \circ 0^2, b)$. Then add tuples $(\eta_1, \mathbf{0} \circ 1 \circ 0, 0)$, $(\eta_2, \mathbf{0} \circ 0 \circ 1, 0)$, and $(\text{Mult}(\eta_1, \eta_2), \mathbf{0} \circ 1 \circ 1, 0)$ to $\mathcal{L}$.
 - Whenever $\mathcal{D}$ submits a query η to $\text{Inv}(\cdot)$:
 - ♣ If there exists a tuple $(\eta, \mathbf{a}, b) \in \mathcal{L}$, then we adds the tuple $(\text{Inv}(\eta), -\mathbf{a}, -b)$ to the list $\mathcal{L}$.
 - ♣ If η does not appear in $\mathcal{L}$ and $\eta \in \tau(\mathbb{Z}_p)$, then first update $\mathbf{x} \leftarrow \mathbf{x} \circ y$ where y is an unknown value that satisfies $\tau(y) = \eta$ and update all of the entries of the list $\mathcal{L}$ from $(\eta, \mathbf{a}, b)$ to $(\eta, \mathbf{a} \circ 0^2, b)$. Then add tuples $(\eta, \mathbf{0} \circ 1, 0)$ and $(\text{Inv}(\eta), \mathbf{0} \circ (-1), 0)$ to $\mathcal{L}$.
 - Whenever $\mathcal{D}$ makes a query to $\mathcal{O}$, $\mathcal{D}$ gets some bitstring $\eta \in \tau(\mathbb{Z}_p)$. If η does not appear in $\mathcal{L}$ and $\dim(\mathbf{x}) = j$, then the challenger updates $\mathbf{x} \leftarrow \mathbf{x} \circ x_{j+1}$ (i.e., increase the dimension of $\mathbf{x}$ by 1 and add the element x_{j+1} at the end), where x_{j+1} denotes an unknown variable satisfying the constraint $\tau(x_{j+1}) = \eta$, and updates all of the entries of the list $\mathcal{L}$ from $(\mathbf{s}, \mathbf{a}, b)$ to $(\mathbf{s}, \mathbf{a} \circ 0, b)$. Then the challenger adds $(\eta, \mathbf{0} \circ 1, 0)$ to $\mathcal{L}$.
 - Whenever $\mathcal{D}$ makes a query to Encaps'_0 or Encaps'_1 , simulating those oracles include queries to random oracles and the oracle $\mathcal{O}$. Then $\mathcal{D}$ maintains the list $\mathcal{L}$ and the vector of unknowns $\mathbf{x}$ as described before per each oracle.
- Once $\mathcal{D}$ makes a query of the form $(\tau(\alpha_i) \parallel \tau(\alpha_i x))$ to H_2 , parse $\tau(\alpha)$ and $\tau(\alpha x)$. Look up the list $\mathcal{L}$ and find j such that $x_j = \alpha_i$ (this is possible since $\mathcal{D}$ has been maintaining both the list $\mathcal{L}$ and the vector $\mathbf{x} = (x_1, x_2, \dots, x_j, \dots)$). We now have two cases:
- If $(\tau(\alpha_i), \hat{u}_j, 0) \in \mathcal{L}$ where $\hat{u}_j$ denotes a unit vector with j^{th} element 1, and $(\tau(\alpha_i x), \mathbf{a}, b) \in \mathcal{L}$ for some $\mathbf{a}, b$, then $\mathcal{D}$ wins the quadratic bridge-finding game by outputting $(1, j, \mathbf{a}, b)$, and
 - If $(\tau(\alpha_i), \hat{u}_j, 0) \in \mathcal{L}$ but $\tau(\alpha_i x)$ does not appear in $\mathcal{L}$, then this implies that $\alpha_i x$ is a fresh unknown value. Increase $\dim \mathbf{x}$ by 1 and update all the entries in $\mathcal{L}$ from $(\eta, \mathbf{a}, b)$ to $(\eta, \mathbf{a} \circ 0, b)$. Add a new tuple $(\tau(\alpha_i x), \mathbf{0} \circ 1, 0)$ to $\mathcal{L}$ and $\mathcal{D}$ can win the quadratic bridge-finding game by outputting $(1, j, \mathbf{0} \circ 1, 0)$.

This proves that $\mathcal{D}$ can always win the quadratic bridge-finding game once $\mathcal{D}$ makes a query $H_2(\tau(\alpha_i) \parallel \tau(\alpha_i x))$.

In Case (2), let this event Lucky. Since $H_2(\tau(\alpha_i) \parallel \tau(\alpha_i x))$ was not queried, $c_{1,i}$ is basically random and unrelated to r_i . Hence, the adversary cannot have extra information to guess r_i from the information of (k_i, c_i) for $1 \leq i \leq q_E$. Since there are q_E different index i and every random oracle query is matched with $H_0(r_i)$ with probability at most $1/2^{\lambda_1}$, we observe that

$$\Pr[\text{Lucky}] \leq \frac{q_H \cdot q_E}{2^{\lambda_1}}.$$

Combining both cases with [Lemma 5](#), we have

$$\begin{aligned}
 & |\Pr[\mathcal{D}^{H_0} = 1] - \Pr[\mathcal{D}^{H_1} = 1]| \\
 & \leq \Pr[\text{QDBridgeChal}_{\mathcal{D}}^{\tau}(\lambda) = 1] + \Pr[\text{Lucky}] \\
 & \leq \frac{3(q_G + q_{\mathcal{O}})^2 + 5(q_G + q_{\mathcal{O}}) + 4}{2p - 2(3q_G + q_{\mathcal{O}} + 1)^2 - 2(2q_G + q_{\mathcal{O}})} + \frac{q_H \cdot q_E}{2^{\lambda_1}} \\
 & \leq \frac{3(q_G + 2q_E)^2 + 5(q_G + 2q_E) + 4}{2p - 2(3q_G + 2q_E + 1)^2 - 4(q_G + q_E)} + \frac{q_H \cdot q_E}{2^{\lambda_1}},
 \end{aligned}$$

since $q_{\mathcal{O}} \leq 2q_E$ and $f(q_{\mathcal{O}}) = \frac{3(q_G + q_{\mathcal{O}})^2 + 5(q_G + q_{\mathcal{O}}) + 4}{2p - 2(3q_G + q_{\mathcal{O}} + 1)^2 - 2(2q_G + q_{\mathcal{O}})}$ is an increasing function on $q_{\mathcal{O}}$ as long as the function is properly defined, i.e., $f(q_{\mathcal{O}}) \geq 0$.

Indistinguishability of H_1 and H_2 . We first observe that two hybrids are perfectly indistinguishable unless the random oracle query $H_1(r_i)$ is queried for some round $i \leq q_E$. Let bad_i be the event that the random oracle query $H_1(r_i)$ is queried in round i . Unless the random oracle query $H_2(\eta_{1,i} \parallel \eta_{2,i})$ was made, we can view r_i as chosen uniformly at random from $\{0, 1\}^{\lambda_1}$ (since otherwise, one can retrieve $r_i \leftarrow c_{1,i} \oplus H_2(\eta_{1,i} \parallel \eta_{2,i})$). Let Z_i be the event that the random oracle query $H_2(\eta_{1,i} \parallel \eta_{2,i})$ was made. In H_1 we have $\Pr[Z_i] \leq \frac{q_H}{p}$ as $\eta_{2,i}$ is uniformly random group element unknown to the attacker. Conditioning on the event that Z_i does not occur the probability that $H_1(r_i)$ is queried is at most $q_H 2^{-\lambda_1}$. Thus

$$\begin{aligned} |\Pr[\mathcal{D}^{H_1} = 1] - \Pr[\mathcal{D}^{H_2} = 1]| &\leq \sum_{i=1}^{q_E} \Pr[\text{bad}_i] \\ &\leq \sum_{i=1}^{q_E} \left\{ \Pr[Z_i] + \Pr[\text{bad}_i | \overline{Z_i}] \right\} \\ &\leq q_E \left(\frac{q_H}{p} + \frac{q_H}{2^{\lambda_1}} \right). \end{aligned}$$

Indistinguishability of H_2 and H_3 . Analyzing the distinguishing probability between the hybrid H_2 and H_3 works similar as the indistinguishability of H_0 and H_1 since the difference between Encaps'_2 and Encaps'_3 is the same as the difference between Encaps'_1 and Encaps'_0 . Hence,

$$\begin{aligned} |\Pr[\mathcal{D}^{H_2} = 1] - \Pr[\mathcal{D}^{H_3} = 1]| \\ \leq \frac{3(q_G + 2q_E)^2 + 5(q_G + 2q_E) + 4}{2p - 2(3q_G + 2q_E + 1)^2 - 4(q_G + q_E)} + \frac{q_H \cdot q_E}{2^{\lambda_1}}. \end{aligned}$$

Putting everything together, we have

$$\begin{aligned} \Pr \left[\text{KEM}_{\mathcal{A}, \Pi}^{\tau, H, \text{cpa}}(\lambda) = 1 \right] &\leq \frac{1}{2} + |\Pr[\mathcal{D}^{H_0} = 1] - \Pr[\mathcal{D}^{H_3} = 1]| \\ &\leq \frac{1}{2} + \sum_{i=0}^2 |\Pr[\mathcal{D}^{H_i} = 1] - \Pr[\mathcal{D}^{H_{i+1}} = 1]| \\ &\leq \frac{1}{2} + \frac{3(q_G + 2q_E)^2 + 5(q_G + 2q_E) + 4}{p - (3q_G + 2q_E + 1)^2 - 2(q_G + q_E)} + \frac{3q_H q_E}{2^{\lambda_1}} + \frac{q_H q_E}{p}. \end{aligned} \quad \square$$

D.2 The CCA Security of PSEC-KEM in the ROM+GGM

Theorem 17. *The PSEC-KEM scheme $\Pi = (\text{Gen}, \text{Encaps}, \text{Decaps})$ is $(q_H, q_G, q_E, q_D, \varepsilon)$ -CCA secure with*

$$\begin{aligned} \varepsilon = & \frac{3(q_G + 2q_E)^2 + 5(q_G + 2q_E) + 4}{p - (3q_G + 2q_E + 1)^2 - 2(q_G + q_E)} + \frac{2q_H q_D + 3q_E(q_H + q_D)}{2^{\lambda_1}} \\ & + \frac{(q_H + q_D)q_E + 2q_D}{p}, \end{aligned}$$

in the generic group model of order $p \approx 2^{2\lambda}$ and the programmable random oracle model.

Proof. To prove [Theorem 17](#), we also use a hybrid argument to prove the CCA security of PSEC-KEM. In the first hybrid (Hybrid H_0), the distinguisher $\mathcal{D}$ is given the CCA distinguishability game for PSEC-KEM with the challenge bit $b = 0$, and in the last hybrid (Hybrid H_5), $\mathcal{D}$ is given the CCA distinguishability game with $b = 1$. Each hybrid is defined as follows:

Hybrid H_0 .

This is the original CCA indistinguishability game $\text{KEM}_{\mathcal{A}, H}^{\tau, H, \text{cca}}(\lambda)$ for PSEC-KEM with the challenge bit $b = 0$. That is, whenever $\mathcal{D}$ submits a query $\text{Encaps}_b(\text{pk})$, $\mathcal{D}$ always gets the output $\text{Encaps}(\text{pk}, 1^\lambda)$. $\mathcal{D}$ is additionally given access to $\text{Decaps}'_{\text{sk}}(\cdot)$ as defined in Appendix ??.

Hybrid H_1 .

Hybrid H_1 works almost identically to H_0 , but it replaces the oracle $\text{Decaps}'_{\text{sk}}(\cdot)$ with an oracle named $\text{SimDecaps}_{\text{sk}}(\cdot)$. Let D_{H_0} (resp. D_{H_2}) be the database of input/output pairs of the random oracle H_0 (resp. H_2) that has been queried by $\mathcal{D}$. Also, recall that resp_{pk} denotes the set of all ciphertexts output by queries to $\text{Encaps}_b(\text{pk})$ along with the challenge ciphertext. Then, for an input ciphertext c , $\text{SimDecaps}_{\text{sk}}(c)$ works as follows:

- If $c \notin \text{resp}_{\text{pk}}$ and if there exists a tuple (r, α, z, c_1) such that
 - $(r, \alpha) \in D_{H_0}$,
 - $((\tau(\alpha) \parallel \tau(\alpha x)), z) \in D_{H_2}$,
 - $c = (\tau(\alpha), c_1)$, and
 - $r = c_1 \oplus z$,
 then return $H_1(r)$.
- Otherwise, return $\perp$.

Indistinguishability of H_0 and H_1 . Two hybrids are perfectly indistinguishable unless one of the following bad events occurs for some round $i \leq q_D$:

- (1) $\text{Decaps}'_{\text{sk}}(c_i)$ does not return $\perp$ but we have $(r_i, \alpha_i) \notin D_{H_0}$ where the values $c_i = (c_i^1, c_i^2)$, $z_i = H_2(c_i^1 \parallel \text{Pow}(c_i^1, x))$, $r_i = c_i^2 \oplus z_i$, and $\alpha_i = H_0(r_i)$ are as computed by $\text{Decaps}'_{\text{sk}}$.
- (2) $\text{Decaps}'_{\text{sk}}(c_i)$ does not return $\perp$ but we have $((\tau(\alpha_i) \parallel \tau(\alpha_i x)), z_i) \notin D_{H_2}$ where the values $c_i = (c_i^1, c_i^2)$, $z_i = H_2(c_i^1 \parallel \text{Pow}(c_i^1, x))$, $r_i = c_i^2 \oplus z_i$, and $\alpha_i = H_0(r_i)$ are as computed by $\text{Decaps}'_{\text{sk}}$.

Let event (1) be $\text{Lucky}_{1,i}$ (for round i) and event (2) be $\text{Lucky}_{2,i}$ (also for round i). And let $\text{Lucky}_1 := \bigcup_{i=1}^{q_D} \text{Lucky}_{1,i}$ (resp. $\text{Lucky}_2 := \bigcup_{i=1}^{q_D} \text{Lucky}_{2,i}$) be the event that the corresponding bad event occurs for *some* round. First observe that if neither event Lucky_1 or Lucky_2 occurs then $\text{SimDecaps}(c_i)$ will perfectly simulate $\text{Decaps}'_{\text{sk}}(c_i)$. If $c_i \notin \text{resp}_{\text{pk}}$ then both oracles output $\perp$. Otherwise $\text{Decaps}'_{\text{sk}}(c_i)$ computes $c_i = (c_i^1, c_i^2)$, $z_i = H_2(c_i^1 \parallel \text{Pow}(c_i^1, x))$, $r_i = c_i^2 \oplus z_i$, and $\alpha_i = H_0(r_i)$. If $\text{Decaps}'_{\text{sk}}(c_i)$ outputs $H_1(r)$ instead of $\perp$ (i.e., verification that $\text{Pow}(g, \alpha_i) = c_i^1$ passes) then we can conclude that both $((\tau(\alpha_i) \parallel \tau(\alpha_i x)), z_i) \in D_{H_2}$ and $(r_i, \alpha_i) \in D_{H_0}$ since neither event $\text{Lucky}_{1,i}$ or $\text{Lucky}_{2,i}$ occurs. But in this case $\text{SimDecaps}(c_i)$ will also output $H_1(r)$. Similarly, if $\text{Decaps}'_{\text{sk}}(c_i)$ outputs $\perp$ because $\text{Pow}(g, \alpha_i) \neq c_i^1$ then it is easy to verify that $\text{SimDecaps}(c_i)$ will also output $\perp$.

Thus, the distinguishing probability between two hybrids is upper-bounded by $\Pr[\text{Lucky}_1] + \Pr[\text{Lucky}_2]$.

First, let's upper bound $\Pr[\text{Lucky}_1]$. If $\text{Lucky}_{1,i}$ occurs, since $(r_i, \alpha_i) \notin D_{H_0}$, we observe that $H_0(r_i)$ was not queried. Hence, α_i is uniformly random and it is unlikely to have $c_i = (\tau(\alpha_i), c_i^2)$, except with probability $1/p$, i.e., $\Pr[\text{Lucky}_{1,i}] \leq 1/p$. Union bounding over q_D indices, we have

$$\Pr[\text{Lucky}_1] \leq \sum_{i=1}^{q_D} \Pr[\text{Lucky}_{1,i}] \leq \frac{q_D}{p}.$$

Next, let's upper bound $\Pr[\text{Lucky}_2]$. If the event $\text{Lucky}_{2,i}$ occurs then, since c_i^2 is fixed, z_i is uniformly random and $r_i = c_i^2 \oplus z_i$ is also uniformly random. However, since $\text{Lucky}_{2,i} \rightarrow \overline{\text{Lucky}_{1,i}}$, it must be the case r_i appears in D_{H_0} . Since r_i was random, $\Pr[(r_i, \cdot) \in D_{H_0}] \leq |D_{H_0}|/2^{\lambda_1} \leq q_H/2^{\lambda_1}$, which implies $\Pr[\text{Lucky}_{2,i}] \leq q_H/2^{\lambda_1}$. Now, union bounding over all q_D queries to Decaps' , we get

$$\Pr[\text{Lucky}_2] \leq \sum_{i=1}^{q_D} \Pr[\text{Lucky}_{2,i}] \leq \frac{q_D q_H}{2^{\lambda_1}}.$$

Taken together,

$$\begin{aligned} |\Pr[\mathcal{D}^{H_0} = 1] - \Pr[\mathcal{D}^{H_1} = 1]| &\leq \Pr[\text{Lucky}_1] + \Pr[\text{Lucky}_2] \\ &\leq q_D \left(\frac{1}{p} + \frac{q_H}{2^{\lambda_1}} \right). \end{aligned}$$

Hybrid H_2 .

This is the same as Hybrid H_1 but with the challenge bit $b = 1$.

Indistinguishability of H_1 and H_2 . Intuitively, if an attacker $\mathcal{D}$ can distinguish between H_1 and H_2 , then $\mathcal{D}$ can also be used to win the CPA indistinguishability game for PSEC-KEM since $\mathcal{D}$ never queries the real decapsulation oracle — it only gets to query the simulated oracle. The algorithm $\mathcal{D}'$ will simply simulate $\mathcal{D}$ and use the SimDecaps oracle to respond to any query to Decaps . The advantage of $\mathcal{D}'$ in CPA indistinguishability game is equivalent to the advantage of distinguisher $\mathcal{D}$ in distinguishing H_1 and H_2 . If $\mathcal{D}$ makes q_H random oracle queries and q_D decapsulation queries, then $\mathcal{D}'$ will make at most $q_H' = q_H + q_D$ random oracle queries and 0 decapsulation queries. Hence,

$$\begin{aligned} &|\Pr[\mathcal{D}^{H_1} = 1] - \Pr[\mathcal{D}^{H_2} = 1]| \\ &\leq \left| \Pr[\text{KEM}_{\mathcal{A},H}^{\tau,H,\text{cpa}}(\lambda) = 1] - \frac{1}{2} \right| \\ &\leq \frac{3(q_G + 2q_E)^2 + 5(q_G + 2q_E) + 4}{p - (3q_G + 2q_E + 1)^2 - 2(q_G + q_E)} + \frac{3(q_H + q_D)q_E}{2^{\lambda_1}} + \frac{(q_H + q_D)q_E}{p}. \end{aligned}$$

Hybrid H_3 .

This is the same as Hybrid H_0 but with the challenge bit $b = 1$. That is, this is the original CCA indistinguishability game $\text{KEM}_{\mathcal{A},H}^{\tau,H,\text{cca}}(\lambda)$ for PSEC-KEM with the challenge bit $b = 1$.

Indistinguishability of H_2 and H_3 . The distinguishing probability between H_2 and H_3 is the same as the distinguishing probability between H_0 and H_1 since the only difference between H_2 and H_3 is that we replace the simulated oracle SimDecaps with the original oracle Decaps' . Note: Since the challenge bit is now $b = 1$ in H_2 and H_3 the encapsulation oracle will now generate random keys. However, because both oracles SimDecaps and Decaps' will reject outputs of the encapsulation oracle the change in the challenge bit will not affect the analysis. Hence,

$$|\Pr[\mathcal{D}^{H_2} = 1] - \Pr[\mathcal{D}^{H_3} = 1]| = |\Pr[\mathcal{D}^{H_0} = 1] - \Pr[\mathcal{D}^{H_1} = 1]|$$

$$\leq q_D \left(\frac{1}{p} + \frac{q_H}{2^{\lambda_1}} \right).$$

Putting everything together, we have

$$\begin{aligned} \Pr \left[\text{KEM}_{\mathcal{A}, H}^{\tau, H, \text{cca}}(\lambda) = 1 \right] &\leq \frac{1}{2} + |\Pr[\mathcal{D}^{H_0} = 1] - \Pr[\mathcal{D}^{H_3} = 1]| \\ &\leq \frac{1}{2} + \sum_{i=0}^2 |\Pr[\mathcal{D}^{H_i} = 1] - \Pr[\mathcal{D}^{H_{i+1}} = 1]| \\ &\leq \frac{1}{2} + \frac{3(q_G + 2q_E)^2 + 5(q_G + 2q_E) + 4}{p - (3q_G + 2q_E + 1)^2 - 2(q_G + q_E)} + \frac{2q_H q_D + 3q_E(q_H + q_D)}{2^{\lambda_1}} \\ &\quad + \frac{(q_H + q_D)q_E + 2q_D}{p}. \end{aligned}$$

□

E Formal Description of the Quadratic Bridge-Finding Game with Preprocessing

The Generic Quadratic Bridge-Finding Game $\text{QDBridgeChal}_{\mathcal{A}}^{\tau}(\lambda)$ with a Bit-Fixing Attacker $\mathcal{A} = (\mathcal{A}_{\text{pre}}^{\text{BF-GG}(P)}, \mathcal{A}_{\text{on}}^{\text{BF-GG}(P)})$:

Preprocessing Phase:

1. $\mathcal{A}_{\text{pre}}^{\text{BF-GG}(P)}$ takes as input $\tau(1)$. Here, $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ is a labeling map where $\mathbb{G}$ is a set of bitstrings of length $\ell \geq \log p$, and $p > 2^{2\lambda}$ is a prime.
2. $\mathcal{A}_{\text{pre}}^{\text{BF-GG}(P)}$ fixes P input/output pairs of the map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$, i.e., $(t_1, \tau(t_1)), \dots, (t_P, \tau(t_P))$. (Note: the rest of the map τ is chosen uniformly at random as long as it remains to be injective.)

Online Phase:

1. The challenger receives P pre-fixed points, pick $x \leftarrow \$ \mathbb{Z}_p$ and initializes the list $\mathcal{L} = \{(\tau(1), 0, 1), (\tau(x), 1, 0), (\tau(t_1), 0, t_1), \dots, (\tau(t_P), 0, t_P)\}$.
2. The adversary $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$ is given $\tau(1)$ and $\tau(x)$ as input. And $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$ is additionally given P pre-fixed points of the map τ from $\mathcal{A}_{\text{pre}}^{\text{BF-GG}(P)}$.
3. The challenger maintains a vector $\mathbf{x} = (x_1, \dots, x_n) \in \mathbb{Z}_p^n$ which is a vector of unknowns such that $x_1 = x$, where the dimension n satisfies the following:
 - initially, $n = 1$ since the only unknown is x , and
 - the number of unknowns n increases by at most 2 as $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$ makes query to the generic group oracle (Mult, Inv) or a generic oracle $\mathcal{O}$ that selects a new random unknown variable $x_j \leftarrow \$ \mathbb{Z}_p$ and outputs $\tau(x_j)$.
4. $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$ is allowed to query the generic group oracle ($\text{Mult}(\cdot, \cdot), \text{Inv}(\cdot)$) and the oracle $\mathcal{O}$.
 - (a) If $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$ submits a query to $\mathcal{O}$, $\mathcal{A}$ gets some bitstring $\eta \in \tau(\mathbb{Z}_p)$. If η does not appear in $\mathcal{L}$ and $\dim(\mathbf{x}) = j$, then the challenger updates $\mathbf{x} \leftarrow \mathbf{x} \circ x_{j+1}$ (i.e., increase the dimension of $\mathbf{x}$ by 1 and add the element x_{j+1} at the end), where x_{j+1} denotes an unknown variable satisfying the constraint $\tau(x_{j+1}) = \eta$, and updates all of the entries of the list $\mathcal{L}$ from (s, a, b) to $(s, a \circ 0, b)$. Then the challenger adds $(\eta, \mathbf{0} \circ 1, 0)$ to $\mathcal{L}$.

- (b) If $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$ submits a query η_1, η_2 to $\text{Mult}(\cdot, \cdot)$:
- If there exist tuples $(\eta_1, \mathbf{a}_1, b_1), (\eta_2, \mathbf{a}_2, b_2) \in \mathcal{L}$ for both η_1 and η_2 , then the challenger adds the tuple $(\text{Mult}(\eta_1, \eta_2), \mathbf{a}_1 + \mathbf{a}_2, b_1 + b_2)$ to the list $\mathcal{L}$.
 - If $(\eta_1, \mathbf{a}_1, b_1) \in \mathcal{L}$ but η_2 does not appear in $\mathcal{L}$ and if $\eta_2 \in \tau(\mathbb{Z}_p)$, and if $\dim(\mathbf{x}) = j$, then the challenger first updates $\mathbf{x} \leftarrow \mathbf{x} \circ x_{j+1}$ where x_{j+1} denotes an unknown variable satisfying the constraint $\tau(x_{j+1}) = \eta_2$ and updates all of the entries of the list $\mathcal{L}$ from $(\mathbf{s}, \mathbf{a}, b)$ to $(\mathbf{s}, \mathbf{a} \circ 0, b)$. Then the challenger adds tuples $(\eta_2, \mathbf{0} \circ 1, 0)$ and $(\text{Mult}(\eta_1, \eta_2), \mathbf{a}_1 \circ 0 + \mathbf{0} \circ 1, b_1)$ to $\mathcal{L}$. Do it similarly if η_2 appears in $\mathcal{L}$ and η_1 does not.
 - If both η_1 and η_2 does not appear in $\mathcal{L}$, and if both $\eta_1, \eta_2 \in \tau(\mathbb{Z}_p)$, and if $\dim(\mathbf{x}) = j$, then the challenger first updates $\mathbf{x} \leftarrow \mathbf{x} \circ x_{j+1} \circ x_{j+2}$ where x_{j+1} and x_{j+2} are unknown values such that $\tau(x_{j+i}) = \eta_i$ for $i = 1, 2$, and updates all the entries of the list $\mathcal{L}$ from $(\mathbf{s}, \mathbf{a}, b)$ to $(\mathbf{s}, \mathbf{a} \circ 0^2, b)$. Then the challenger adds tuples $(\eta_1, \mathbf{0} \circ 1 \circ 0, 0)$, $(\eta_2, \mathbf{0} \circ 0 \circ 1, 0)$, and $(\text{Mult}(\eta_1, \eta_2), \mathbf{0} \circ 1 \circ 1, 0)$ to $\mathcal{L}$.
- (c) If $\mathcal{A}$ submits a query η to $\text{Inv}(\cdot)$:
- If there exists a tuple $(\eta, \mathbf{a}, b) \in \mathcal{L}$, then the challenger adds the tuple $(\text{Inv}(\eta), -\mathbf{a}, -b)$ to the list $\mathcal{L}$.
 - If η does not appear in $\mathcal{L}$ and $\eta \in \tau(\mathbb{Z}_p)$, and if $\dim(\mathbf{x}) = j$, then the challenger first updates $\mathbf{x} \leftarrow \mathbf{x} \circ x_{j+1}$ where x_{j+1} is an unknown value that satisfies $\tau(x_{j+1}) = \eta$ and updates all of the entries of the list $\mathcal{L}$ from $(\mathbf{s}, \mathbf{a}, b)$ to $(\mathbf{s}, \mathbf{a} \circ 0, b)$. Then the challenger adds tuples $(\eta, \mathbf{0} \circ 1, 0)$ and $(\text{Inv}(\eta), \mathbf{0} \circ (-1), 0)$ to $\mathcal{L}$.
5. If at any point in time $\mathcal{A}$ outputs a tuple $(i_1, i_2, \mathbf{a}, b)$ such that (1) $1 \leq i_1, i_2 \leq \dim(\mathbf{x})$, (2) $(\mathbf{a}, b) \in \mathbb{Z}_p^{\dim(\mathbf{x})} \times \mathbb{Z}_p$, and (3) $x_{i_1} x_{i_2} = \mathbf{a} \cdot \mathbf{x} + b \pmod p$, then we say that the event QDBridge occurs. In this case, the output of the game is 1 and we write $\text{QDBridgeChal}_{\mathcal{A}}^{\tau}(\lambda) = 1$. If QDBridge never occurs, then the output of the game is 0.

F The Quadratic Discrete-Log Game

The Quadratic Discrete-Log Game. We define a variant of the discrete-log game called *the quadratic discrete-log game* which might be crucial in analyzing the security of various cryptographic primitives or protocols.

The Generic Quadratic Discrete-Log Game $\text{QDLogChal}_{\mathcal{A}}^{\tau}(\lambda)$:

1. The adversary $\mathcal{A}$ is given $(\mathbf{g} = \tau(1), \tau(x))$ for a random value of $x \in \mathbb{Z}_p$. Here, $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ is a labeling map where $\mathbb{G}$ is a set of bitstrings of length $\ell \geq \log p$. Here, p is a 2λ -bit prime.
2. $\mathcal{A}$ is allowed to query the usual generic group oracles (Mult, Inv) and is additionally allowed to query $\text{DLog}(\tau(y))$, but *only* if $\tau(y)$ is “fresh”, i.e., $\tau(y)$ is not $\tau(x)$, and $\tau(y)$ has not been the output of a previous random generic group query.
3. After multiple queries, $\mathcal{A}$ outputs a tuple (a, b, η) where $a, b \in \mathbb{Z}_p$ with $a \neq 0$ and $\eta \in \mathbb{G}$.
4. The output of the game is defined to be $\text{QDLogChal}_{\mathcal{A}}^{\tau}(\lambda) = 1$ if $\eta = \tau(ax^2 + bx)$, and 0 otherwise.

Lemma 6. *The probability the attacker making at most q_G generic group oracle queries wins the generic quadratic discrete-log game $\text{QDLogChal}_{\mathcal{A}}^{\tau}(\lambda)$ (even with access to the restricted DLog oracle) is at most*

$$\Pr[\text{QDLogChal}_{\mathcal{A}}^{\tau}(\lambda) = 1] \leq \frac{3q_G^2 + 7q_G + 4}{2p - 2(3q_G + 2)^2},$$

in the generic group model of prime order p , where the randomness is taken over the selection of the labeling map τ , the challenge x , as well as any random coins of $\mathcal{A}$.

Proof. The proof largely borrows the same idea from the prior work [BL22, Lemma 1]. Intuitively, the proof works by maintaining a list $\mathcal{L}$ of tuples $(\tau(y), a, b)$ where $\tau(y) \in \mathbb{G}$ and $a, b \in \mathbb{Z}_p$ such that $y = ax + b$ for every oracle output $\tau(y)$. Initially, the list $\mathcal{L}$ contains two items $(\tau(1), 0, 1)$ and $(\tau(x), 1, 0)$. $\mathcal{L}$ is updated after every query to the generic group oracles as follows:

- if $(\tau(y_1), a_1, b_1) \in \mathcal{L}$ and $(\tau(y_2), a_2, b_2) \in \mathcal{L}$, then querying $\text{Mult}(\tau(y_1), \tau(y_2))$ will result in adding a tuple $(\tau(y_1 + y_2), a_1 + a_2, b_1 + b_2)$ to $\mathcal{L}$, and
- if $(\tau(y), a, b) \in \mathcal{L}$ then querying $\text{Inv}(\tau(y))$ will result in adding a tuple $(-\tau(y), -a, -b)$ to $\mathcal{L}$.

We say that the event **BRIDGE** occurs if $\mathcal{L}$ contains two tuples $(\tau(y_1), a_1, b_1)$ and $(\tau(y_2), a_2, b_2)$ such that $\tau(y_1) = \tau(y_2)$ but $(a_1, b_1) \neq (a_2, b_2)$. We can further use the restricted discrete-log oracle DLog to maintain the invariant that every output of the generic group oracles Mult and Inv can be added to $\mathcal{L}$. In particular, if we ever encounter an input $\eta = \tau(b)$ that *does not* already appear in $\mathcal{L}$, then η is considered *fresh* and we can simply query DLog to extract $b = \text{DLog}(\eta)$, ensuring that the tuple $(\eta, 0, b)$ is added to $\mathcal{L}$ before the generic group query is processed.

If the event **BRIDGE** occurs, it becomes trivial for the attacker to find η that satisfies $\eta = ax^2 + bx$ for some $a, b \in \mathbb{Z}_p$ with $a \neq 0$ because the attacker can indeed extract the secret x , i.e., if $\mathcal{L}$ contains two tuples $(\tau(y_1), a_1, b_1)$ and $(\tau(y_2), a_2, b_2)$ such that $\tau(y_1) = \tau(y_2)$ but $(a_1, b_1) \neq (a_2, b_2)$, then we know that $a_1x + b_1 = a_2x + b_2$ with $a_1 \neq a_2$ (since otherwise we have $b_1 = b_2$ which contradicts the fact that $(a_1, b_1) \neq (a_2, b_2)$) which allows us to solve for $x = (b_2 - b_1)(a_1 - a_2)^{-1}$.

If the event **BRIDGE** does not occur then after all queries have finished the attacker can still view x as an element yet to be sampled from a uniform distribution over a set of size at least $p - |\mathcal{L}|^2$. To see this, note that each tuple of distinct elements $(\tau(y_1), a_1, b_1)$ and $(\tau(y_2), a_2, b_2)$ eliminates at most one possible value of x , i.e., since $\tau(y_1)$ and $\tau(y_2)$ are distinct we have $x \neq (b_2 - b_1)(a_1 - a_2)^{-1}$. We further observe that $|\mathcal{L}| \leq 3q_G + 2$ since each query to the generic group oracles adds at most 3 elements to $\mathcal{L}$ — equality holds when both inputs to $\text{Mult}(\cdot, \cdot)$ are fresh. Now, the attacker wins if and only if s/he outputs a tuple (a, b, η) with nonzero a and $\eta = \tau(ax^2 + bx)$. Since we ensured that η appears in $\mathcal{L}$, we have that $\eta = \tau(a'x + b')$ for some known $a', b' \in \mathbb{Z}_p$, which implies that $ax^2 + bx = a'x + b' \pmod{p}$ where $a \neq 0$. Since this quadratic equation has at most 2 roots in $\mathbb{Z}_p$ there are at most 2 out of at least $p - (3q_G + 2)^2$ remaining choices of x which satisfy the equation $ax^2 + bx = a'x + b' \pmod{p}$. Thus, we have that

$$\begin{aligned} \Pr[\text{QDLogChal}_{\mathcal{A}}^{\tau}(\lambda) = 1 \mid \overline{\text{BRIDGE}}] &\leq \frac{2}{p - |\mathcal{L}|^2} \\ &\leq \frac{2}{p - (3q_G + 2)^2}. \end{aligned}$$

Next, we have that $\Pr[\text{BRIDGE}] \leq \frac{3q_G^2 + 7q_G}{2p - 2(3q_G + 2)^2}$ from [BL22, Lemma 1]. Hence, the probability the attacker succeeds is at most

$$\Pr[\text{QDLogChal}_{\mathcal{A}}^{\tau}(\lambda) = 1]$$

$$\begin{aligned}
&\leq \Pr[\text{BRIDGE}] + \Pr[\text{QDLogChal}_{\mathcal{A}}^{\tau}(\lambda) = 1 \mid \overline{\text{BRIDGE}}] \\
&\leq \frac{3q_{\mathbb{G}}^2 + 7q_{\mathbb{G}}}{2p - 2(3q_{\mathbb{G}} + 2)^2} + \frac{2}{p - (3q_{\mathbb{G}} + 2)^2} \\
&= \frac{3q_{\mathbb{G}}^2 + 7q_{\mathbb{G}} + 4}{2p - 2(3q_{\mathbb{G}} + 2)^2}.
\end{aligned}$$

□

The Quadratic Discrete-Log Game with Preprocessing. Now we consider a preprocessing setting in a similar manner. We first define the quadratic discrete-log game in the *bit-fixing* generic group model which is described formally below.

The Generic Quadratic Discrete-Log Game $\text{QDLogChal}_{\mathcal{A}}^{\tau}(\lambda)$ with a Bit-Fixing Preprocessing Attacker $\mathcal{A} := (\mathcal{A}_{\text{pre}}^{\text{BF-GG}(P)}, \mathcal{A}_{\text{on}}^{\text{BF-GG}(P)})$:

Preprocessing Phase:

1. $\mathcal{A}_{\text{pre}}^{\text{BF-GG}(P)}$ takes as input $\mathbf{g} = \tau(1)$.
2. $\mathcal{A}_{\text{pre}}^{\text{BF-GG}(P)}$ fixes P input/output pairs of the map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$, i.e., $(t_1, \tau(t_1)), \dots, (t_P, \tau(t_P))$. (Note: the rest of the map τ is chosen uniformly at random as long as it remains to be injective.)

Online Phase:

1. The adversary $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$ is given $\mathbf{g} = \tau(1)$ and $\tau(x)$ for a random value of $x \in \mathbb{Z}_p$.
2. $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$ is allowed to query the usual generic group oracles (Mult , Inv) and is additionally allowed to query $\text{DLog}(\tau(y))$, but *only* if $\tau(y)$ is “fresh”, i.e., $\tau(y)$ is not $\tau(x)$, and $\tau(y)$ has not been the output of a previous random generic group query.
3. After multiple queries, $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$ outputs a tuple (a, b, η) where $a, b \in \mathbb{Z}_p$ with $a \neq 0$ and $\eta \in \mathbb{G}$.
4. The output of the game is defined to be $\text{QDLogChal}_{\mathcal{A}}^{\tau}(\lambda) = 1$ if $\eta = \tau(ax^2 + bx)$, and 0 otherwise.

Lemma 7. Let $p > 2^{2\lambda}$ be a prime number and let $\mathcal{A} := (\mathcal{A}_{\text{pre}}^{\text{BF-GG}(P)}, \mathcal{A}_{\text{on}}^{\text{BF-GG}(P)})$ be a pair of bit-fixing generic algorithms with a labeling map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ such that $\mathcal{A}_{\text{pre}}^{\text{BF-GG}(P)}$ fixes P input/output pairs of the labeling map and $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$ makes at most $q_{\mathbb{G}}^{\text{on}} := q_{\mathbb{G}}^{\text{on}}(\lambda)$ queries to the generic group oracles. Then

$$\Pr[\text{QDLogChal}_{\mathcal{A}}^{\tau}(\lambda) = 1] \leq \frac{3(q_{\mathbb{G}}^{\text{on}})^2 + 5q_{\mathbb{G}}^{\text{on}} + 2q_{\mathbb{G}}^{\text{on}}P + 4}{2p - 4P(3q_{\mathbb{G}}^{\text{on}} + 2) - 2(3q_{\mathbb{G}}^{\text{on}} + 2)^2},$$

in the generic group model of prime order p , where the randomness is taken over the selection of the labeling map τ , the challenge x , as well as any random coins of $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$.

Proof. The proof works largely the same as [Lemma 6](#), except for the fact that

- the upper bound of the size of the list is increased by P : $|\mathcal{L}| \leq P + 3q_{\mathbb{G}}^{\text{on}} + 2$,
- if the event BRIDGE does not occur then after all queries have finished the attacker can still view x as an element yet to be sampled from a uniform distribution over a set of size at least $p - (|\mathcal{L}|^2 - P^2)$, and

- the probability of the event BRIDGE becomes higher, i.e., $\Pr[\text{BRIDGE}] \leq \frac{1.5(q_G^{\text{on}})^2 + 2.5q_G^{\text{on}} + q_G^{\text{on}}P}{p - 2P(3q_G^{\text{on}} + 2) - (3q_G^{\text{on}} + 2)^2}$ (see Lemma 1).

Hence, we have

$$\begin{aligned} \Pr[\text{QDLogChal}_{\mathcal{A}}^{\tau}(\lambda) = 1 \mid \overline{\text{BRIDGE}}] &\leq \frac{2}{p - (|\mathcal{L}|^2 - P^2)} \\ &\leq \frac{2}{p - 2P(3q_G^{\text{on}} + 2) - (3q_G^{\text{on}} + 2)^2}, \end{aligned}$$

and

$$\begin{aligned} &\Pr[\text{QDLogChal}_{\mathcal{A}}^{\tau}(\lambda) = 1] \\ &\leq \Pr[\text{BRIDGE}] + \Pr[\text{QDLogChal}_{\mathcal{A}}^{\tau}(\lambda) = 1 \mid \overline{\text{BRIDGE}}] \\ &\leq \frac{1.5(q_G^{\text{on}})^2 + 2.5q_G^{\text{on}} + q_G^{\text{on}}P}{p - 2P(3q_G^{\text{on}} + 2) - (3q_G^{\text{on}} + 2)^2} + \frac{2}{p - 2P(3q_G^{\text{on}} + 2) - (3q_G^{\text{on}} + 2)^2} \\ &= \frac{3(q_G^{\text{on}})^2 + 5q_G^{\text{on}} + 2q_G^{\text{on}}P + 4}{2p - 4P(3q_G^{\text{on}} + 2) - 2(3q_G^{\text{on}} + 2)^2}. \quad \square \end{aligned}$$

An immediate corollary of Lemma 7 can be obtained by applying [CDG18, Theorem 1] and by setting an optimal value of $P \approx Sq_G^{\text{on}}$ and $\gamma = 2^{-2\lambda}$.

Corollary 1. Let $p > 2^{2\lambda}$ be a prime number and $N \in \mathbb{N}$ be a parameter. Let $\mathcal{A} := (\mathcal{A}_{\text{pre}}^{\text{AI-GG}}, \mathcal{A}_{\text{on}}^{\text{AI-GG}})$ be a pair of auxiliary-input generic algorithms with a labeling map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ such that $\mathcal{A}_{\text{pre}}^{\text{AI-GG}}$ outputs an S -bit hint after looking at the entire map τ and $\mathcal{A}_{\text{on}}^{\text{AI-GG}}$ makes at most $q_G^{\text{on}} := q_G^{\text{on}}(\lambda)$ queries to the generic group oracles. Then

$$\begin{aligned} \Pr[\text{QDLogChal}_{\mathcal{A}}^{\tau}(\lambda) = 1] &\leq \frac{3(q_G^{\text{on}})^2 + 5q_G^{\text{on}} + 2S(q_G^{\text{on}})^2 + 4}{p - 2Sq_G^{\text{on}}(3q_G^{\text{on}} + 2) - (3q_G^{\text{on}} + 2)^2} + 2^{-2\lambda+1}. \end{aligned}$$

G Missing Proofs

G.1 Missing Proofs from Section 4

Reminder of Theorem 7. Let $\Pi_{\setminus\{0\}} = (\text{Kg}, \text{Sign}, \text{Vfy})$ be a nonzero Schnorr signature scheme and $p > 2^{2\lambda}$ be a prime number. Let $N \in \mathbb{N}$ be a parameter and $(\mathcal{A}_{\text{Sig.pre}}^{\text{BF-RO+GG}(P_1, P_2)}, \mathcal{A}_{\text{Sig.on}}^{\text{BF-RO+GG}(P_1, P_2)})$ be a pair of bit-fixing generic algorithms with a labeling map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ such that $\mathcal{A}_{\text{Sig.pre}}^{\text{BF-RO+GG}(P_1, P_2)}$ fixes P_1 input/output pairs of a random oracle $H : \{0, 1\}^* \rightarrow \{0, 1\}^{\lambda_1}$ and P_2 input/output pairs of the map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ such that $2P_1 + P_2 = P$, and the hint $\text{str}_{\tau, H}$ is only dependent on those P points. If $\mathcal{A}_{\text{Sig.on}}^{\text{BF-RO+GG}(P_1, P_2)}$ makes at most $q_G^{\text{on}} := q_G^{\text{on}}(\lambda)$ queries to the generic group oracles, q_H^{on} queries to the random oracle, and q_S^{on} queries to the signing oracle, then $\Pr[\text{SigForgeCor}_{\mathcal{A}_{\text{Sig.on}, \text{str}_{\tau, H}}^{\text{BF-RO+GG}(P_1, P_2)}, \Pi_{\setminus\{0\}}}^{\tau, H, N}(\lambda) = 1] \leq \varepsilon$, with

$$\varepsilon = \frac{q_G^{\text{on}}(N + P) + 3q_G^{\text{on}}(q_G^{\text{on}} + 1)/2}{p - 2P(N + 3q_G^{\text{on}} + 1) - (N + 3q_G^{\text{on}} + 1)^2 - N} + \frac{q_S^{\text{on}}(q_H^{\text{on}} + q_S^{\text{on}} + P)}{p}$$

$$+ \frac{q_H^{\text{on}} + q_S^{\text{on}} + P}{p - (N + P + 3q_G^{\text{on}} + 1)} + \frac{2q_H^{\text{on}} + 1}{2\lambda_1},$$

where the randomness is taken over the selection of τ and the random coins of $\mathcal{A}_{\text{Sig.on}}^{\text{BF-RO+GG}(P_1, P_2)}$.

Proof of Theorem 7. Given a bit-fixing adversary $(\mathcal{A}_{\text{Sig.pre}}^{\text{BF-RO+GG}(P_1, P_2)}, \mathcal{A}_{\text{Sig.on}}^{\text{BF-RO+GG}(P_1, P_2)})$ that attacks the short nonzero Schnorr signature scheme, we construct the following efficient generic algorithm $\mathcal{A}_{\text{bridge}} = (\mathcal{A}_{\text{bridge.pre}}^{\text{BF-RO+GG}(P_1, P_2)}, \mathcal{A}_{\text{bridge.on}}^{\text{BF-RO+GG}(P_1, P_2)})$ in the bit-fixing model which tries to succeed in the 1-out-of- N generic BRIDGE^{*N*}-finding game $\text{BridgeChalCor}_{\mathcal{A}_{\text{bridge}}}^{\tau, N}(\lambda)$:

Algorithm $(\mathcal{A}_{\text{bridge.pre}}^{\text{BF-RO+GG}(P_1, P_2)}, \mathcal{A}_{\text{bridge.on}}^{\text{BF-RO+GG}(P_1, P_2)})$:

Preprocessing Phase:

The algorithm is given $p, g = \tau(1)$ as input.

1. $\mathcal{A}_{\text{bridge.pre}}^{\text{BF-RO+GG}(P_1, P_2)}$ simply runs $\mathcal{A}_{\text{Sig.pre}}^{\text{BF-RO+GG}(P_1, P_2)}$ and outputs the same P_1 input/output pairs of $H(\{(h_1, H(h_1)), \dots, (h_{P_1}, H(h_{P_1}))\})$ and P_2 input/output pairs of $\tau(\{(t_1, \tau(t_1)), \dots, (t_{P_2}, \tau(t_{P_2}))\})$ as being fixed by $\mathcal{A}_{\text{Sig.pre}}^{\text{BF-RO+GG}(P_1, P_2)}$.
2. For each fixed RO input/output pair $(h_i, H(h_i))$ where $i \in [P_1]$, $\mathcal{A}_{\text{bridge.pre}}^{\text{BF-RO+GG}(P_1, P_2)}$ first parses $h_i = I'_i \| m'_i$ where I'_i is a bitstring of length ℓ while m'_i represents the remaining part of the bitstring. If I'_i is a fresh ℓ -bit string, i.e., $I'_i \neq \tau(t_j)$ for all $j \in [P_2]$, then $\mathcal{A}_{\text{bridge.pre}}^{\text{BF-RO+GG}(P_1, P_2)}$ picks a value for $\tau^{-1}(I'_i) =: r'_i$ where the labeling map τ will be sampled later. (Note: this is because the attacker can query a restricted discrete-log oracle for a fresh bitstring but the labeling map τ has not been sampled.) Finally, $\mathcal{A}_{\text{bridge.pre}}^{\text{BF-RO+GG}(P_1, P_2)}$ also outputs the fixed input/output pairs of $\tau\{(r'_1, I'_1), \dots, (r'_{P_1}, I'_{P_1})\}$.
3. Now the challenger initializes the list $\mathcal{L} = \mathcal{L}_1 \cup \mathcal{L}_2 \cup \mathcal{L}_3$, where
 - $\mathcal{L}_1 = \{(\tau(1), \mathbf{0}, 1), (pk_i, \hat{u}_i, 0) : i \in [N]\}$,
 - $\mathcal{L}_2 = \{(I'_i, \mathbf{0}, r'_i = \text{DLog}(I'_i)) : i \in [P_1]\}$, and
 - $\mathcal{L}_3 = \{(\tau(t_j), \mathbf{0}, t_j) : j \in [P_2]\}$.
 (Note that $|\mathcal{L}| \leq |\mathcal{L}_1| + |\mathcal{L}_2| + |\mathcal{L}_3| \leq 1 + N + P_1 + P_2 = N + P + 1$.)

Online Phase:

The algorithm is given $p, g = \tau(1), pk_i = \tau(x_i), 1 \leq i \leq N$ as input and initialize the set of revealed indices $\text{Revealed} = \emptyset$.

4. Initialize the set $H_{\text{resp}} = \{\}$ which stores the random oracle input/output pairs observed during online processing. It also maintains the set $H_{\text{Fixed}} = \{(h_1, H(h_1)), \dots, (h_{P_1}, H(h_{P_1}))\}$ that $\mathcal{A}_{\text{Sig.pre}}^{\text{BF-RO+GG}(P_1, P_2)}$ has already fixed in Step 1.
5. $\mathcal{A}_{\text{bridge.on}}^{\text{BF-RO+GG}(P_1, P_2)}$ runs $\mathcal{A}_{\text{Sig.on}}^{\text{BF-RO+GG}(P_1, P_2)}$ with a number of access to the generic group oracles $G_0 = (\text{Mult}(\cdot, \cdot), \text{Inv}(\cdot))$, the restricted discrete-log oracles $\text{DLog}(\cdot)$, the signing oracles $\text{Signing}(i, \cdot)$ for $1 \leq i \leq N$, and the random oracle $H(\cdot)$. We maintain the invariant that every output of a generic group query during the online phase appears in the list $\mathcal{L}$ that is kept in track on the challenger's side. We consider the following cases:
 - (a) Whenever $\mathcal{A}_{\text{Sig.on}}^{\text{BF-RO+GG}(P_1, P_2)}$ submits a query w to the random oracle H :

- If there is a pair $(w, R) \in H_{\text{resp}}$ for some string $R \in \{0, 1\}^{\lambda_1}$ then return R .
- Otherwise, check if $(w, H(w)) \in H_{\text{Fixed}}$. If so, then return $H(w)$.
- Otherwise, select $R \leftarrow \{0, 1\}^{\lambda_1}$ and add (w, R) to H_{resp} .
- If w has the form $w = (a \| m_i)$ where the value a has not been observed previously (i.e., is not in $\mathcal{L}$) then we query $b = \text{DLog}(a)$ which results in the challenger adding $(a, 0, b)$ to $\mathcal{L}$.
- (b) Whenever $\mathcal{A}_{\text{Sig.on}}^{\text{BF-RO+GG}(P_1, P_2)}$ submits a query a to the generic group oracle $\text{Inv}(\cdot)$:
 - If a is not in $\mathcal{L}$ then we immediately query $b = \text{DLog}(a)$ which results in the challenger adding $(a, 0, b)$ to $\mathcal{L}$.
 - Otherwise, $(a, a, b) \in \mathcal{L}$ for some a and b . Then we query $\text{Inv}(a) = \tau(-a \cdot x - b)$, output the result, which results in the challenger adding $(\tau(-a \cdot x - b), -a, -b)$ to $\mathcal{L}$.
- (c) Whenever $\mathcal{A}_{\text{Sig.on}}^{\text{BF-RO+GG}(P_1, P_2)}$ submits a query a, b to the generic group oracle $\text{Mult}(\cdot, \cdot)$:
 - If the element a (resp. b) is not in $\mathcal{L}$ then query $b_0 = \text{DLog}(a)$ (resp. $b_1 = \text{DLog}(b)$) which results in the challenger adding the element $(a, 0, b_0)$ (resp. $(b, 0, b_1)$) to $\mathcal{L}$.
 - Otherwise both elements $(a, a_0, b_0), (b, a_1, b_1) \in \mathcal{L}$. Then we return $\text{Mult}(a, b) = \tau((a_0 + a_1) \cdot x + b_0 + b_1)$ which results in the challenger adding $(\tau((a_0 + a_1) \cdot x + b_0 + b_1), a_0 + a_1, b_0 + b_1) \in \mathcal{L}$.
- (d) Whenever $\mathcal{A}_{\text{Sig.on}}^{\text{BF-RO+GG}(P_1, P_2)}$ submits a query $i \in [N]$ to $\text{Reveal}(\cdot)$, the oracle returns x_i and update $\text{Revealed} \leftarrow \text{Revealed} \cup \{i\}$. Update each tuple $(\eta, a, b) \in \mathcal{L}$ to (η, a', b') where $b' = b + a_i x_i$, $a'_i = 0$, and $a'_j = a_j$ for $j \neq i$ to ensure that $a_i = 0$ since x_i is now known.
- (e) Whenever $\mathcal{A}_{\text{Sig.on}}^{\text{BF-RO+GG}(P_1, P_2)}$ submits a query m_i to the signing oracle $\text{Signing}(j, \cdot)$:
 - It first checks to see if user $j \in \text{Revealed}$ (or equivalently, $j \in C$). If $j \in \text{Revealed}$, then the algorithm simply uses the secret key x_j to run the *honest* signing algorithm and obtain a signature $\sigma_i = (s_i, e_i)$ to respond, i.e.,
 - i. Pick r_i randomly and compute $I_i = \text{Pow}(g, r_i) = \tau(r_i)$.
 - ii. Compute $e_i = H(I_i \| m_i)$ and $s_i = r_i + x_j e_i$.
 - iii. Return $\sigma_i = (s_i, e_i)$.
 - If $j \notin \text{Revealed}$, then the algorithm *simulates* the signing oracle without knowledge of the secret key x_j , relying on the ability to program the random oracle as follows:
 - i. Pick s_i, e_i randomly and compute $I_i = \tau(s_i - x_j e_i) = \text{Mult}(s_i, \text{Pow}(\text{pk}_j, -e_i))$ where $s_i = \text{Pow}(g, s_i)$.
 - ii. If $H(I_i \| m_i)$ has been previously queried or if it is in the set H_{Fixed} , then return $\perp$.
 - iii. Otherwise, program $H(I_i \| m_i) := e_i$ and return $\sigma_i = (s_i, e_i)$.
 - We remark that a side effect of querying $\text{Signing}(j, \cdot)$ is the addition of the tuples $(\tau(s_i), 0, s_i)$, $(\tau(x_j e_i), e_i \hat{u}_i, 0)$ and $(\tau(s_i - x_j e_i), -e_i \hat{u}_i, s_i)$ to $\mathcal{L}$, since these values are computed using the generic group oracles Inv, Mult .
- 6. After $\mathcal{A}_{\text{Sig}}^{\text{on}}$ outputs $\sigma_{i^*} = (s_{i^*}, e_{i^*})$ and m_{i^*} , identify the index $i^* \in [N]$ such that $\text{Vf}(\text{pk}_{i^*}, m_{i^*}, \sigma_{i^*}) = 1$. If $i^* \in \text{Revealed}$ (i.e., the target index is corrupted), we discard the attempt. In the following, we assume $i^* \notin \text{Revealed}$. Without loss of generality, we can assume that m_{i^*} is not a part of pre-fixed points H_{Fixed} in Step 2 since if

the attacker forges a signature (s_{i*}, e_{i*}) for a message m_{i*} which involves the pre-fixed point $(I_{i*} \| m_{i*}, H(I_{i*} \| m_{i*})) \in H_{\text{Fixed}}$ then we can directly force the bridge event to occur, i.e., $(I_{i*}, \mathbf{0}, r_{i*} = \text{DLog}(I_{i*})) \in \mathcal{L}$ and $(I_{i*}, -e_{i*}\hat{u}_{i*}, s_{i*}) \in \mathcal{L}$ (see Step 7 below) where $\mathbf{0} \neq -e_{i*}\hat{u}_{i*}$.

7. Compute $\tau(-e_{i*}x_{i*}) = \text{Inv}(\text{Pow}(\tau(x_{i*}), e_{i*}))$. This will ensure that the elements $(\tau(-e_{i*}x_{i*}), -e_{i*}\hat{u}_{i*}, 0)$ and $(\tau(e_{i*}x_{i*}), e_{i*}\hat{u}_{i*}, 0)$ are both added to $\mathcal{L}$.
8. Compute $\mathfrak{s}_{i*} = \text{Pow}(\mathfrak{g}, s_{i*})$ to ensure that $(\mathfrak{s}_{i*}, \mathbf{0}, s_{i*}) \in \mathcal{L}$.
9. Finally, compute $I_{i*} = \text{Mult}(\mathfrak{s}_{i*}, \tau(-e_{i*}x_{i*})) = \tau(s_{i*} - x_{i*}e_{i*})$ which ensures that $(I_{i*}, -e_{i*}\hat{u}_{i*}, s_{i*}) \in \mathcal{L}$ and check to see if we previously had any tuple of the form $(I_{i*}, \mathbf{a}, b) \in \mathcal{L}$.

Analysis. We first remark that if the signature is valid then we must have $e_{i*} = H(I_{i*} \| m_{i*})$ and $\text{DLog}(I_{i*}) = s_{i*} - x_{i*}e_{i*} = \mathbf{a} \cdot \mathbf{x} + b$. Moreover, without loss of generality, we can assume that each string η occurs at most once in the list $\mathcal{L}$ in Step 5 because if at any point we have some string η such that $(\eta, \mathbf{a}, b) \in \mathcal{L}$ and $(\eta, \mathbf{c}, d) \in \mathcal{L}$ for $(\mathbf{a}, b) \neq (\mathbf{c}, d)$ then we can immediately have a BRIDGE^N instance $(\tau((\mathbf{a} - \mathbf{c}) \cdot \mathbf{x}), \mathbf{a} - \mathbf{c}, 0) \in \mathcal{L}$ and $(\tau(d - b), \mathbf{0}, d - b) \in \mathcal{L}$ since $\tau((\mathbf{a} - \mathbf{c}) \cdot \mathbf{x}) = \tau(d - b)$.

We now define the failure events FailtoSign , $\text{FailtoFind}(I_{i*})$, and BadQuery . The event FailtoSign occurs when our reduction outputs $\perp$ in Step 5.(d) due to the signing oracle failure, i.e., since $H(I_i \| m_i)$ has been previously queried or it is contained in the set H_{Fixed} . $\text{FailtoFind}(I_{i*})$ denotes the event that we find that the signature is valid but I_{i*} was not previously recorded in our list $\mathcal{L}$ before we computed $\text{Mult}(\mathfrak{s}_{i*}, \tau(-e_{i*}x_{i*}))$ in the last step so that we cannot find the bridge event. BadQuery is the event that the signature is valid but for the only prior tuple $(I_{i*}, \mathbf{a}, b) \in \mathcal{L}$ we have that $\mathbf{a} = -e_{i*}\hat{u}_{i*}$ so that we still cannot find the bridge event.

We observe that corruption has *no effect* on the analysis of these failure events. For FailtoSign , the signing oracle never fails if we query a signing oracle with a corrupted index, since we can run the honest signing algorithm with the revealed secret key. Furthermore, $\text{FailtoFind}(I_{i*})$ and BadQuery only count for the case once the online adversary outputs a forged signature σ_{i*} , where the forgery becomes immediately invalid if i^* is a corrupted index.

Claim 7, **Claim 8**, and **Claim 9** upper bound the probability of our events FailtoSign , FailtoFind , and BadQuery respectively.

Claim 7. $\Pr[\text{FailtoSign}] \leq \frac{q_{\text{S}}^{\text{on}}(q_{\text{H}}^{\text{on}} + q_{\text{S}}^{\text{on}} + P_1)}{p}$.

Proof of Claim 7. We observe that every time the attacker queries the signing oracle, we would generate a query to the random oracle H . Thus, we would have at most $q_{\text{H}}^{\text{on}} + q_{\text{S}}^{\text{on}} + P_1$ input/output pairs recorded for the random oracle, including the fixed points during the preprocessing phase. Since $I_i = \tau(s_i - x_i e_i)$ represents a fresh/randomly selected group element of size p , the probability that (I_i, m_i) is one of the inputs is at most $(q_{\text{H}}^{\text{on}} + q_{\text{S}}^{\text{on}} + P_1)/p$. Applying union bound over q_{S}^{on} queries to the signing oracle, we have that

$$\Pr[\text{FailtoSign}] \leq \frac{q_{\text{S}}^{\text{on}}(q_{\text{H}}^{\text{on}} + q_{\text{S}}^{\text{on}} + P_1)}{p}. \quad \square$$

Claim 8. $\Pr[\text{FailtoFind}(I_{i*})] \leq \frac{q_{\text{H}}^{\text{on}} + q_{\text{S}}^{\text{on}} + P_1}{p - |\mathcal{L}|} + \frac{1}{2^{\lambda_1}}$.

Proof of Claim 8. We observe that if $I_{i*} \notin \mathcal{L}$ then we can view $I_{i*} = \tau(s_{i*} - x_{i*}e_{i*})$ as a uniformly random binary string from a set of size at least $p - |\mathcal{L}|$ which had not yet been selected at the time $\mathcal{A}_{\text{Sig}}^{\text{on}}$ outputs σ_{i*} . Thus, the probability that the query $H(I_{i*}||m_{i*})$ was previously recorded is at most $(q_H^{\text{on}} + q_S^{\text{on}} + P_1)/(p - |\mathcal{L}|)$. If the query $H(I_{i*}||m_{i*})$ was *not* previously recorded then the probability of a successful forgery $H(I_{i*}||m_{i*}) = e_{i*}$ is at most 2^{-k_1} since we can view $H(I_{i*}||m_{i*})$ chosen uniformly at random from the possible 2^{λ_1} options. Hence, we have that

$$\Pr[\text{FailtoFind}(I_{i*})] \leq \frac{q_H^{\text{on}} + q_S^{\text{on}} + P_1}{p - |\mathcal{L}|} + \frac{1}{2^{\lambda_1}}. \quad \square$$

Claim 9. $\Pr[\text{BadQuery}] \leq \frac{q_H^{\text{on}}}{2^{\lambda_1}}$.

Proof of Claim 9. We observe that by construction we ensure that the tuple $(I, \mathbf{a}, b)$ will always be recorded in $\mathcal{L}$ before a query of the form $H(I||m)$ is ever issued — if I is new then we call $\text{DLog}(I)$ before querying the random oracle. Now define a subset $\hat{\mathcal{L}} \subset \mathcal{L}$ as the set of tuples $(\mathbf{a}, \mathbf{a}, b) \in \mathbb{G} \times \mathbb{Z}_p^N \times \mathbb{Z}_p$ such that $\mathbf{a}$ has exactly *one* nonzero element. Now we call a random oracle query $x = (I||m)$ “bad” if $H(x) = -\tilde{a}$ where the tuple $(I, \mathbf{a}, b) \in \hat{\mathcal{L}}$ has already been recorded and the nonzero element of $\mathbf{a}$ is $\tilde{a}$ (Recall that if there were two recorded tuples $(I, \mathbf{a}, b)$ and $(I, \mathbf{c}, d)$ then our algorithm would have already found a BRIDGE^N instance). Thus, the probability each individual query is “bad” is at most $2/2^{\lambda_1}$ and we can use union bounds to upper bound the probability of any “bad” query as

$$\Pr[\text{BadQuery}] \leq \frac{q_H^{\text{on}}}{2^{\lambda_1}}. \quad \square$$

Now we have shown that

$$\begin{aligned} & \Pr \left[\text{BridgeChalCor}_{\mathcal{A}_{\text{bridge}}}^{\tau, N}(\lambda) = 1 \right] \\ & \geq \Pr \left[\text{SigForgeCor}_{\mathcal{A}_{\text{Sig.on, str}_{\tau, H}}^{\text{BF-RO}}, \Pi_{\setminus \{0\}}^{\text{short}}}^{\tau, H, N}(\lambda) = 1 \right] - \Pr[\text{FailtoSign}] \\ & \quad - \Pr[\text{FailtoFind}(I_{i*})] - \Pr[\text{BadQuery}] \\ & \geq \Pr \left[\text{SigForgeCor}_{\mathcal{A}_{\text{Sig.on, str}_{\tau, H}}^{\text{BF-RO}}, \Pi_{\setminus \{0\}}^{\text{short}}}^{\tau, H, N}(\lambda) = 1 \right] - \frac{q_S^{\text{on}}(q_H^{\text{on}} + q_S^{\text{on}} + P_1)}{p} \\ & \quad - \frac{q_H^{\text{on}} + q_S^{\text{on}} + P_1}{p - |\mathcal{L}|} - \frac{q_H^{\text{on}} + 1}{2^{\lambda_1}}. \end{aligned}$$

Finally, by applying Lemma 1, we can conclude that

$$\begin{aligned} & \Pr \left[\text{SigForgeCor}_{\mathcal{A}_{\text{Sig.on, str}_{\tau, H}}^{\text{BF-RO}}, \Pi_{\setminus \{0\}}^{\text{short}}}^{\tau, H, N}(\lambda) = 1 \right] \\ & \leq \left[\text{BridgeChalCor}_{\mathcal{A}_{\text{bridge}}}^{\tau, N}(\lambda) = 1 \right] + \frac{q_S^{\text{on}}(q_H^{\text{on}} + q_S^{\text{on}} + P_1)}{p} \\ & \quad + \frac{q_H^{\text{on}} + q_S^{\text{on}} + P_1}{p - |\mathcal{L}|} + \frac{q_H^{\text{on}} + 1}{2^{\lambda_1}} \\ & \leq \frac{q_G^{\text{on}}(N + P) + 3q_G^{\text{on}}(q_G^{\text{on}} + 1)/2}{p - 2P(N + 3q_G^{\text{on}} + 1) - (N + 3q_G^{\text{on}} + 1)^2 - N} + \frac{q_S^{\text{on}}(q_H^{\text{on}} + q_S^{\text{on}} + P)}{p} \\ & \quad + \frac{q_H^{\text{on}} + q_S^{\text{on}} + P}{p - (N + P + 3q_G^{\text{on}} + 1)} + \frac{q_H^{\text{on}} + 1}{2^{\lambda_1}}. \quad \square \end{aligned}$$

G.2 Missing Proofs from Section 5.1

Reminder of Lemma 2. Let $p > 2^{2\lambda}$ is a prime number. Let $\mathcal{A} := (\mathcal{A}_{\text{pre}}^{\text{BF-GG}(P)}, \mathcal{A}_{\text{on}}^{\text{BF-GG}(P)})$ be a pair of bit-fixing generic algorithms with a labeling map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ such that $\mathcal{A}_{\text{pre}}^{\text{BF-GG}(P)}$ fixes P input/output pairs of the labeling map τ and $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$ makes at most $q_{\mathbb{G}}^{\text{on}} := q_{\mathbb{G}}^{\text{on}}(\lambda)$ queries to the generic group oracles and $q_{\mathcal{O}}^{\text{on}} := q_{\mathcal{O}}^{\text{on}}(\lambda)$ queries to the oracle $\mathcal{O}$. Then $\Pr[\text{QDBridgeChal}_{\mathcal{A}}^{\tau}(\lambda) = 1] \leq \varepsilon$, where

$$\varepsilon := \frac{3(q_{\mathbb{G}}^{\text{on}} + q_{\mathcal{O}}^{\text{on}})^2 + (5 + 2P)(q_{\mathbb{G}}^{\text{on}} + q_{\mathcal{O}}^{\text{on}}) + 4}{2p - 4P(3q_{\mathbb{G}}^{\text{on}} + q_{\mathcal{O}}^{\text{on}} + 1) - 2(3q_{\mathbb{G}}^{\text{on}} + q_{\mathcal{O}}^{\text{on}} + 1)^2 - 2(2q_{\mathbb{G}}^{\text{on}} + q_{\mathcal{O}}^{\text{on}})},$$

in the GGM of prime order p , where the randomness is taken over the selection of $x_1, \dots, x_N, \tau$ as well as any random coins of $\mathcal{A}_{\text{on}}^{\text{BF-GG}(P)}$.

Proof of Lemma 2. The proof works by maintaining a list $\mathcal{L}$ of tuples $(\tau(y), \mathbf{a}, b)$ where $\tau(y) \in \mathbb{G}$, $\mathbf{a} \in \mathbb{Z}_p^{\dim(\mathbf{x})}$, and $b \in \mathbb{Z}_p$ such that $y = \mathbf{a} \cdot \mathbf{x} + b$ for every oracle output $\tau(y)$. To bound the probability that the attacker wins the quadratic bridge-finding game, we consider the case where the event QDBridge occurs after the attacker makes $q_{\mathbb{G}}$ generic group queries and $q_{\mathcal{O}}$ queries to $\mathcal{O}$. Suppose that $\dim(\mathbf{x}) = j$ at the point in time when QDBridge occurs. We observe that $j \leq 2q_{\mathbb{G}} + q_{\mathcal{O}} + 1$ since $\dim(\mathbf{x})$ can be increased by at most 2 whenever $\mathcal{A}$ makes a generic group query and at most 1 whenever $\mathcal{A}$ makes a query to $\mathcal{O}$. We further observe that in this case, $|\mathcal{L}| \leq 3q_{\mathbb{G}} + q_{\mathcal{O}} + 1$ since for each query to $\mathcal{O}$, at most 1 item can be added to $\mathcal{L}$ whereas each generic group oracle query can add at most 3 times to $\mathcal{L}$ (exactly three in the case when $\mathcal{A}$ queries $\text{Mult}(\eta_1, \eta_2)$ on two fresh elements).

Now consider the event BRIDGE where the list $\mathcal{L}$ contains two distinct tuples $(\eta, \mathbf{a}, b)$ and $(\eta', \mathbf{a}', b')$ such that $\mathbf{a} \cdot \mathbf{x} + b = \mathbf{a}' \cdot \mathbf{x} + b'$. To upper bound $\Pr[\text{BRIDGE}]$, consider the event $\text{BRIDGE}_{<i}$ that the event BRIDGE has not occurred until the $(i-1)^{\text{th}}$ query. Conditioning on the event $\text{BRIDGE}_{<i}$, we are now interested in the event BRIDGE_i where the i^{th} query makes the event BRIDGE occur, i.e., a tuple $(\eta_i, \mathbf{a}_i, b_i)$ has been recorded to $\mathcal{L}$ and there exists a tuple of the form $(\cdot, \mathbf{a}, b)$ such that $\mathbf{a}_i \cdot \mathbf{x} + b_i = \mathbf{a} \cdot \mathbf{x} + b$. We can essentially view $\mathbf{x}$ sampled uniformly at random subject to some restrictions due to $\text{BRIDGE}_{<i}$, from which we know that for any distinct pair $(\eta, \mathbf{a}, b)$ and $(\eta', \mathbf{a}', b')$ we have $\mathbf{a} \cdot \mathbf{x} + b \neq \mathbf{a}' \cdot \mathbf{x} + b'$.

Let $r \leq \dim(\mathbf{x}) = j$ be an index such that $\mathbf{a}[r] - \mathbf{a}'[r] \neq 0$ and suppose that $x_r = \mathbf{x}[r]$ is the last value sampled. We also define two subsets $\mathcal{L}_0, \mathcal{L}_1 \subseteq \mathcal{L}$ where $\mathcal{L}_0 := \{(\eta, \mathbf{a}, b) \in \mathcal{L} : \mathbf{a} = \mathbf{0}\}$ and $\mathcal{L}_1 := \{(\eta, \mathbf{a}, b) \in \mathcal{L} : \mathbf{a} \neq \mathbf{0}\}$. Now we observe that $\mathcal{L}_0 \cup \mathcal{L}_1 = \mathcal{L}$ and if we pick two distinct pairs $(\eta_1, \mathbf{a}_1, b_1)$ and $(\eta_2, \mathbf{a}_2, b_2)$ both from $\mathcal{L}_0$ we are guaranteed to have that $\mathbf{a}_1 \cdot \mathbf{x} + b_1 \neq \mathbf{a}_2 \cdot \mathbf{x} + b_2$ since $\mathbf{a}_1 = \mathbf{a}_2 = \mathbf{0}$, which implies that we are guaranteed to not cause any bridge event. At this point, we can view x_r as being drawn uniformly at random from a set of at least $p - (|\mathcal{L}|^2 - |\mathcal{L}_0|^2) - (j-1)$ remaining values, subject to all of the restrictions. To see this, we observe the following:

- x_r must be distinct from $x_1, \dots, x_{r-1}$ since otherwise we would have had a bridge event, and
- each pair of distinct elements $(\eta_j, \mathbf{a}_j, b_j) \in \mathcal{L}$ and $(\eta_k, \mathbf{a}_k, b_k) \in \mathcal{L}$, we have the following cases:
 - if $(\eta_j, \mathbf{a}_j, b_j) \in \mathcal{L}_1$ and $(\eta_k, \mathbf{a}_k, b_k) \in \mathcal{L}_1$, then this pair eliminates at most one possible value of x_r , i.e., since η_j and η_k are distinct, we have $\mathbf{a}_j \cdot \mathbf{x} + b_j \neq \mathbf{a}_k \cdot \mathbf{x} + b_k$, which implies that $x_r \neq \left((b_k - b_j) - \sum_{i=1, i \neq j}^N (\mathbf{a}_k[i] - \mathbf{a}_j[i])x_i \right) (\mathbf{a}_j[r] - \mathbf{a}_k[r])^{-1}$,

- if $(\eta_j, \mathbf{a}_j, b_j) \in \mathcal{L}_1$ and $(\eta_k, \mathbf{a}_k, b_k) \in \mathcal{L}_0$, then this pair eliminates at most one possible value of x_r , i.e., since η_j and η_k are distinct, we have $\mathbf{a}_j \cdot \mathbf{x} + b_j \neq \mathbf{0} \cdot \mathbf{x} + b_k$, which implies that $x_r \neq \left((b_k - b_j) + \sum_{i=1, i \neq j}^N \mathbf{a}_j[i]x_i \right) \mathbf{a}_j[r]^{-1}$,
 - $(\eta_j, \mathbf{a}_j, b_j) \in \mathcal{L}_0$ and $(\eta_k, \mathbf{a}_k, b_k) \in \mathcal{L}_1$, then this pair eliminates at most one possible value of x_r , i.e., since η_j and η_k are distinct, we have $\mathbf{0} \cdot \mathbf{x} + b_j \neq \mathbf{a}_k \cdot \mathbf{x} + b_k$, which implies that $x_r \neq \left((b_k - b_j) - \sum_{i=1, i \neq j}^N \mathbf{a}_k[i]x_i \right) (-\mathbf{a}_k[r])^{-1}$, and
 - if both $(\eta_j, \mathbf{a}_j, b_j) \in \mathcal{L}_0$ and $(\eta_k, \mathbf{a}_k, b_k) \in \mathcal{L}_0$, then since both $\mathbf{a}_j = \mathbf{a}_k = \mathbf{0}$, it is a contradiction that $\mathbf{a}_j[r] - \mathbf{a}_k[r] \neq 0$ and therefore we do not eliminate any value of x_r .
- Hence, the total number of values that we remove when we draw x_r is at most $(r-1) + |\mathcal{L}_1|^2 + 2|\mathcal{L}_0||\mathcal{L}_1| \leq (j-1) + (|\mathcal{L}|^2 - |\mathcal{L}_0|^2)$.

We also observe that $|\mathcal{L}| \leq P + 3q_G^{\text{on}} + q_{\mathcal{O}}^{\text{on}} + 1$ and $|\mathcal{L}_0| \geq P$ since the list already contains P additional pre-fixed pairs and each generic group oracle query adds *at most* three new tuples to $\mathcal{L}$ — exactly three in the case that we query $\text{Mult}(\eta_1, \eta_2)$ on two fresh elements.

Thus, the probability that $\mathbf{a}_i \cdot \mathbf{x} + b_i = \mathbf{a} \cdot \mathbf{x} + b$ is at most

$$\frac{1}{p - (P + 3q_G^{\text{on}} + q_{\mathcal{O}}^{\text{on}} + 1)^2 + P^2 - (2q_G^{\text{on}} + 2q_{\mathcal{O}}^{\text{on}})}.$$

Union bounding over all tuples in $\mathcal{L}$, we have

$$\begin{aligned} & \Pr[\text{BRIDGE}] \\ &= \sum_{i \leq q_G^{\text{on}} + q_{\mathcal{O}}^{\text{on}}} \Pr[\text{BRIDGE}_i | \overline{\text{BRIDGE}}_{<i}] \\ &\leq \sum_{i \leq q_G^{\text{on}} + q_{\mathcal{O}}^{\text{on}}} \frac{1 + P + 3i}{p - (P + 3q_G^{\text{on}} + q_{\mathcal{O}}^{\text{on}} + 1)^2 + P^2 - (2q_G^{\text{on}} + 2q_{\mathcal{O}}^{\text{on}})} \\ &\leq \frac{3(q_G^{\text{on}} + q_{\mathcal{O}}^{\text{on}})^2 + (5 + 2P)(q_G^{\text{on}} + q_{\mathcal{O}}^{\text{on}})}{2p - 4P(3q_G^{\text{on}} + q_{\mathcal{O}}^{\text{on}} + 1)^2 - 2(2q_G^{\text{on}} + 2q_{\mathcal{O}}^{\text{on}})}. \end{aligned}$$

Now, consider the case where the event BRIDGE never occurs. Then we would like to upper bound the probability

$$\Pr \left[\text{QDBridgeChal}_{\mathcal{A}}^{\tau, \mathcal{O}}(\lambda) = 1 \mid \overline{\text{BRIDGE}} \right].$$

The argument works similarly; conditioning on the event $\overline{\text{BRIDGE}}$, we know that for any distinct pair $(\eta, \mathbf{a}, b)$ and $(\eta', \mathbf{a}', b')$ we have $\mathbf{a} \cdot \mathbf{x} + b \neq \mathbf{a}' \cdot \mathbf{x} + b'$. Let $r \leq \dim(\mathbf{x}) \leq 2q_G + q_{\mathcal{O}} + 1$ be an index such that $\mathbf{a}[r] - \mathbf{a}'[r] \neq 0$ and suppose that $x_r = \mathbf{x}[r]$ is the last value sampled. At this point, we can view x_r as being drawn uniformly at random from a set of at least $p - (|\mathcal{L}|^2 - |\mathcal{L}_0|^2) - (2q_G^{\text{on}} + q_{\mathcal{O}}^{\text{on}})$ remaining values as before. Now, the attacker wins if and only if s/he outputs a tuple $(i_1, i_2, \mathbf{a}, b)$ such that $x_{i_1}x_{i_2} = \mathbf{a} \cdot \mathbf{x} + b$. Since this equation is at most degree 2 in terms of x_r (in case $i_1 = i_2 = r$), there are at most 2 roots in $\mathbb{Z}_p$. Thus, we have that

$$\begin{aligned} & \Pr \left[\text{QDBridgeChal}_{\mathcal{A}}^{\tau, \mathcal{O}}(\lambda) = 1 \mid \overline{\text{BRIDGE}} \right] \\ &\leq \frac{2}{p - (|\mathcal{L}|^2 - |\mathcal{L}_0|^2) - (2q_G^{\text{on}} + q_{\mathcal{O}}^{\text{on}})} \end{aligned}$$

$$\leq \frac{2}{p - 2P(3q_G^{\text{on}} + q_O^{\text{on}} + 1)^2 - (2q_G^{\text{on}} + 2q_O^{\text{on}})}.$$

Taken together, we have

$$\begin{aligned} & \Pr \left[\text{QDBridgeChal}_{\mathcal{A}}^{\tau, \mathcal{O}}(\lambda) = 1 \right] \\ & \leq \Pr [\text{BRIDGE}] + \Pr \left[\text{QDBridgeChal}_{\mathcal{A}}^{\tau, \mathcal{O}}(\lambda) = 1 \mid \overline{\text{BRIDGE}} \right] \\ & \leq \frac{3(q_G^{\text{on}} + q_O^{\text{on}})^2 + (5 + 2P)(q_G^{\text{on}} + q_O^{\text{on}})}{2p - 4P(3q_G^{\text{on}} + q_O^{\text{on}} + 1)^2 - 2(2q_G^{\text{on}} + 2q_O^{\text{on}})} \\ & \quad + \frac{2}{p - 2P(3q_G^{\text{on}} + q_O^{\text{on}} + 1)^2 - (2q_G^{\text{on}} + 2q_O^{\text{on}})} \\ & = \frac{3(q_G^{\text{on}} + q_O^{\text{on}})^2 + (5 + 2P)(q_G^{\text{on}} + q_O^{\text{on}}) + 4}{2p - 4P(3q_G^{\text{on}} + q_O^{\text{on}} + 1) - 2(3q_G^{\text{on}} + q_O^{\text{on}} + 1)^2 - 2(2q_G^{\text{on}} + q_O^{\text{on}})}, \end{aligned}$$

which completes the proof. $\square$

G.3 Missing Proofs from Section 5.2

Reminder of Theorem 9. Let $\Pi = (\text{Gen}, \text{Encaps}, \text{Decaps})$ be PSEC-KEM and $p > 2^{2\lambda}$ be a prime number. Let $\mathcal{A} = \left(\mathcal{A}_{\text{pre}}^{\text{BF-RO+GG}(P_1, P_2)}, \mathcal{A}_{\text{on}}^{\text{BF-RO+GG}(P_1, P_2)} \right)$ be a pair of bit-fixing generic algorithms with a labeling map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ such that $\mathcal{A}_{\text{pre}}^{\text{BF-RO+GG}(P_1, P_2)}$ fixes $P_{1,1}$ (resp. $P_{1,2}, P_{1,3}$) input/output pairs of a random oracle $H_0 : \{0, 1\}^* \rightarrow \mathbb{Z}_p$ (resp. $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda, H_2 : \{0, 1\}^* \rightarrow \{0, 1\}^{\lambda_1}$), and P_2 input/output pairs of the map τ such that $P_{1,1} + P_{1,2} + P_{1,3} = P_1$ and $2P_1 + P_2 = P$, and the hint $\text{str}_{\tau, H_0, H_1, H_2}$ is only dependent on those P points. If $\mathcal{A}_{\text{on}}^{\text{BF-RO+GG}(P_1, P_2)}$ makes at most q_H^{on} (resp. $q_G^{\text{on}}, q_E^{\text{on}}$) queries to the random oracle (resp. generic group oracle, encapsulation oracle), then $\Pr \left[\text{KEM}_{\mathcal{A}_{\text{on}}, \text{str}_{\tau, H_0, H_1, H_2}}^{\tau, (H_0, H_1, H_2), \text{cpa}}(\lambda) = 1 \right] \leq \frac{1}{2} + \varepsilon$, with

$$\begin{aligned} \varepsilon = & \frac{3(q_G^{\text{on}} + 2q_E^{\text{on}})^2 + (5 + 2P)(q_G^{\text{on}} + 2q_E^{\text{on}}) + 4}{p - 2P(3q_G^{\text{on}} + 2q_E^{\text{on}} + 1) - (3q_G^{\text{on}} + 2q_E^{\text{on}} + 1)^2 - 2(q_G^{\text{on}} + q_E^{\text{on}})} \\ & + \frac{3(q_H^{\text{on}} + P)q_E^{\text{on}}}{2^{\lambda_1}} + \frac{(q_H^{\text{on}} + P)q_E^{\text{on}}}{p}, \end{aligned}$$

where the randomness is taken over the selection of τ and the random coins of $\mathcal{A}_{\text{on}}^{\text{BF-RO+GG}(P_1, P_2)}$.

Proof of Theorem 9. We use a hybrid argument to prove the CPA security of PSEC-KEM in the BF-ROM+GGM. The hybrids work the same as the proof of Theorem 16 during the online phase, except that the distinguisher $\mathcal{D} = (\mathcal{D}_{\text{pre}}, \mathcal{D}_{\text{on}})$ in the preprocessing phase gets the hint generated from $\mathcal{A}_{\text{pre}}^{\text{BF-RO+GG}(P_1, P_2)}$. In particular, $\mathcal{A}_{\text{pre}}^{\text{BF-RO+GG}(P_1, P_2)}$ fixes

- $P_{1,1}$ input/output pairs $(\{(h_{1,1}, H_0(h_{1,1})), \dots, (h_{1,P_{1,1}}, H_0(h_{1,P_{1,1}}))\})$ of H_0 ,
- $P_{1,2}$ input/output pairs $(\{(h_{2,1}, H_1(h_{2,1})), \dots, (h_{2,P_{1,2}}, H_1(h_{2,P_{1,2}}))\})$ of H_1 ,
- $P_{1,3}$ input/output pairs $(\{(h_{3,1}, H_2(h_{3,1})), \dots, (h_{3,P_{1,3}}, H_2(h_{3,P_{1,3}}))\})$ of H_2 , and

– P_2 input/output pairs $(\{(t_1, \tau(t_1)), \dots, (t_{P_2}, \tau(t_{P_2}))\})$ of τ .

Furthermore, for each fixed RO input/output pair $(h_j, H_i(h_j))$ for $i = 0, 1, 2$, $\mathcal{D}_{\text{pre}}$ first parses $h_j = I_j \| I'_j$ where I_j is a bitstring of length ℓ . If I_j is a fresh ℓ -bit string, i.e., $I_j \neq \tau(t_k)$ for all $k \in [P_2]$, then $\mathcal{D}_{\text{pre}}$ picks a value for $\tau^{-1}(I_j) = r_j$ where the labeling map τ will be sampled later. Note that this is possible because the labeling map τ has not been sampled yet. Finally, $\mathcal{D}_{\text{pre}}$ additionally fixes the input/output pairs of τ such as $\{(r_1, I_1), \dots, (r_{P_1}, I_{P_1})\}$. This leads to having $P_{1,1} + P_{1,2} + P_{1,3} = P_1$ fixed RO points and $P_1 + P_2$ fixed points of τ for the distinguisher, having in total $P_1 + (P_1 + P_2) = P$ fixed points as the hint for the online phase. For completeness, we restate the hybrids below.

Hybrid H_0 . This is the original CPA indistinguishability game $\text{KEM}_{\mathcal{A}_{\text{on, str}, \tau, H_0, H_1}}^{\tau, H_0, H_1, \text{cpa}, \text{BF-RO+GG}(P_1, P_2), \Pi}(\lambda)$ for PSEC-

KEM with the challenge bit $b = 0$. In particular, $\mathcal{D}_{\text{on}}$ has access to the encapsulation oracle $\text{Encaps}'_0(\cdot) := \text{Encaps}(\cdot, 1^\lambda)$, i.e., whenever $\mathcal{D}_{\text{on}}$ submits a query $\text{Encaps}'_0(\text{pk})$, $\mathcal{D}_{\text{on}}$ always gets the output $\text{Encaps}(\text{pk}, 1^\lambda)$.

Hybrid H_1 . This is the same security game with H_0 except that the encapsulation oracle $\text{Encaps}'_0(\cdot)$ is replaced with a modified oracle $\text{Encaps}'_1(\cdot)$. In $\text{Encaps}'_1(\cdot)$, $\tau(\alpha)$ and $\tau(\alpha x)$ (where x is the secret key) are replaced with random elements in $\tau(\mathbb{Z}_p)$ by querying the oracle $\mathcal{O}$ twice. See Figure 2 for the details.

Hybrid H_2 . This is the same security game with H_1 except that the encapsulation oracle $\text{Encaps}'_1(\cdot)$ is further replaced with a modified oracle $\text{Encaps}'_2(\cdot)$. $\text{Encaps}'_2(\cdot)$ is exactly the same as $\text{Encaps}'_1(\cdot)$ except that the key k is sampled uniformly at random from $\{0, 1\}^\lambda$.

Hybrid H_3 . This is the same security game with H_2 except that the encapsulation oracle $\text{Encaps}'_2(\cdot)$ is replaced with a modified oracle $\text{Encaps}'_3(\cdot) := \text{Encaps}_1(\cdot)$, which leads to the original CPA indistinguishability game $\text{KEM}_{\mathcal{A}_{\text{on, str}, \tau, H_0, H_1}}^{\tau, H_0, H_1, \text{cpa}, \text{BF-RO+GG}(P_1, P_2), \Pi}(\lambda)$ for PSEC-KEM with the challenge bit $b = 1$. In particular, $\text{Encaps}'_3(\cdot)$ is the same as $\text{Encaps}'_2(\cdot)$ but the random elements in $\tau(\mathbb{Z}_p)$ are reverted back to the honest computation $\tau(\alpha)$ and $\tau(\alpha x)$. This can also be interpreted as replacing an honest key k in $\text{Encaps}'_0(\cdot)$ with a uniformly random key of length λ . See Figure 2 for the details.

Indistinguishability of H_0 and H_1 . With a similar reasoning, we can argue that two hybrids are perfectly indistinguishable unless one of the three events occurs: for some round $i \leq q_E^{\text{on}}$,

- (1) the random oracle query $H_2(\tau(\alpha_i) \| \tau(\alpha_i x))$ has been made,
- (2) the adversary makes query r_i to H_0 (which is not one of the fixed points of H_0) but has not queried $(\tau(\alpha_i) \| \tau(\alpha_i x))$ to H_2 , or
- (3) r_i is one of the fixed points of H_0 .

We can argue in the same way as Theorem 16 that Case (1) leads to winning the quadratic bridge-finding game with preprocessing.

Let Lucky_1 be the event that Case (2) occurs. If r is not one of the fixed points of H_0 , then it is essentially the same as Case (2) in the proof of Theorem 16. Hence,

$$\Pr[\text{Lucky}_1] \leq \frac{q_H^{\text{on}} \cdot q_E^{\text{on}}}{2^{\lambda_1}}.$$

Let Lucky_2 be the event that Case (3) occurs. Since there are at most P fixed points, union bounding over q_E^{on} queries, we have

$$\Pr[\text{Lucky}_2] \leq \frac{P \cdot q_E^{\text{on}}}{2^{\lambda_1}}.$$

Taken all together with [Lemma 2](#), we have

$$\begin{aligned} & |\Pr[\mathcal{D}^{H_0} = 1] - \Pr[\mathcal{D}^{H_1} = 1]| \\ & \leq \Pr[\text{QDBridgeChal}_{\mathcal{D}}^{\tau}(\lambda) = 1] + \Pr[\text{Lucky}_1] + \Pr[\text{Lucky}_2] \\ & \leq \frac{3(q_G^{\text{on}} + q_O^{\text{on}})^2 + (5 + 2P)(q_G^{\text{on}} + q_O^{\text{on}}) + 4}{2p - 4P(3q_G^{\text{on}} + q_O^{\text{on}} + 1) - 2(3q_G^{\text{on}} + q_O^{\text{on}} + 1)^2 - 2(2q_G^{\text{on}} + q_O^{\text{on}})} \\ & \quad + \frac{q_H^{\text{on}} \cdot q_E^{\text{on}}}{2^{\lambda_1}} + \frac{P \cdot q_E^{\text{on}}}{2^{\lambda_1}} \\ & \leq \frac{3(q_G^{\text{on}} + 2q_E^{\text{on}})^2 + (5 + 2P)(q_G^{\text{on}} + 2q_E^{\text{on}}) + 4}{2p - 4P(3q_G^{\text{on}} + 2q_E^{\text{on}} + 1) - 2(3q_G^{\text{on}} + 2q_E^{\text{on}} + 1)^2 - 4(q_G^{\text{on}} + q_E^{\text{on}})} \\ & \quad + \frac{(q_H^{\text{on}} + P) \cdot q_E^{\text{on}}}{2^{\lambda_1}}, \end{aligned}$$

since $q_O^{\text{on}} \leq 2q_E^{\text{on}}$.

Indistinguishability of H_1 and H_2 . Again, we observe that two hybrids are perfectly indistinguishable unless the random oracle query $H_1(r_i)$ is queried (which is not one of the fixed points) or r_i is one of the fixed points of H_1 for some round i . If r_i was not one of the fixed points of H_1 , there are still several ways to retrieve r_i :

- if the random oracle query $H_2(\eta_{1,i} \parallel \eta_{2,i})$ was made that is not one of the fixed points of H_2 , or
- $\eta_{1,i} \parallel \eta_{2,i}$ is one of the fixed points of H_2 .

In either case, one can retrieve $r_i \leftarrow c_{1,i} \oplus H_2(\eta_{1,i} \parallel \eta_{2,i})$. Note that since the attacker only gets $(k_i, c_i = (\eta_{1,i}, c_{1,i}))$ as the output, s/he can first check if $\eta_{1,i}$ is the prefix of one of the fixed points of H_2 (let's say this fixed point is $\tilde{\eta}$) and check if $H_1(c_{1,i} \oplus H_2(\tilde{\eta})) \stackrel{?}{=} k_i$. If it matches then the attacker would have known that the pair (k_i, c_i) is from Encaps'_1 . Since there are at most P fixed points, union bounding over q_E^{on} queries we have

$$|\Pr[\mathcal{D}^{H_1} = 1] - \Pr[\mathcal{D}^{H_2} = 1]| \leq q_E^{\text{on}} \left(\frac{q_H^{\text{on}} + P}{p} + \frac{q_H^{\text{on}} + P}{2^{\lambda_1}} \right).$$

Indistinguishability of H_2 and H_3 . As we discussed in the proof of [Theorem 16](#), the distinguishing probability is the same as the distinguishing probability between H_0 and H_1 , which is

$$\begin{aligned} & |\Pr[\mathcal{D}^{H_2} = 1] - \Pr[\mathcal{D}^{H_3} = 1]| \\ & \leq \frac{3(q_G^{\text{on}} + 2q_E^{\text{on}})^2 + (5 + 2P)(q_G^{\text{on}} + 2q_E^{\text{on}}) + 4}{2p - 4P(3q_G^{\text{on}} + 2q_E^{\text{on}} + 1) - 2(3q_G^{\text{on}} + 2q_E^{\text{on}} + 1)^2 - 4(q_G^{\text{on}} + q_E^{\text{on}})} \\ & \quad + \frac{(q_H^{\text{on}} + P) \cdot q_E^{\text{on}}}{2^{\lambda_1}}. \end{aligned}$$

Putting all together, we have

$$\begin{aligned}
& \Pr \left[\text{KEM}_{\mathcal{A}_{\text{on}, \text{str}_{\tau, H_0, H_1}}^{\tau, H_0, H_1, \text{cpa}}, \Pi}^{\text{BF-RO+GG}(P_1, P_2)}(\lambda) = 1 \right] \\
& \leq \frac{1}{2} + |\Pr[\mathcal{D}^{H_0} = 1] - \Pr[\mathcal{D}^{H_3} = 1]| \\
& \leq \frac{1}{2} + \sum_{i=0}^2 |\Pr[\mathcal{D}^{H_i} = 1] - \Pr[\mathcal{D}^{H_{i+1}} = 1]| \\
& \leq \frac{3(q_G^{\text{on}} + 2q_E^{\text{on}})^2 + (5 + 2P)(q_G^{\text{on}} + 2q_E^{\text{on}}) + 4}{p - 2P(3q_G^{\text{on}} + 2q_E^{\text{on}} + 1) - (3q_G^{\text{on}} + 2q_E^{\text{on}} + 1)^2 - 4(q_G^{\text{on}} + q_E^{\text{on}})} \\
& \quad + \frac{3(q_H^{\text{on}} + P)q_E^{\text{on}}}{2^{\lambda_1}} + \frac{(q_H^{\text{on}} + P)q_E^{\text{on}}}{p} + \frac{1}{2}. \quad \square
\end{aligned}$$

Reminder of Theorem 10. Let $\Pi = (\text{Gen}, \text{Encaps}, \text{Decaps})$ be PSEC-KEM and $p > 2^{2\lambda}$ be a prime number. Let $\mathcal{A} = (\mathcal{A}_{\text{pre}}^{\text{BF-RO+GG}(P_1, P_2)}, \mathcal{A}_{\text{on}}^{\text{BF-RO+GG}(P_1, P_2)})$ be a pair of bit-fixing generic algorithms with a labeling map $\tau : \mathbb{Z}_p \rightarrow \mathbb{G}$ such that $\mathcal{A}_{\text{pre}}^{\text{BF-RO+GG}(P_1, P_2)}$ fixes $P_{1,1}$ (resp. $P_{1,2}, P_{1,3}$) input/output pairs of a random oracle $H_0 : \{0, 1\}^* \rightarrow \mathbb{Z}_p$ (resp. $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$, $H_2 : \{0, 1\}^* \rightarrow \{0, 1\}^{\lambda_1}$), and P_2 input/output pairs of the map τ such that $P_{1,1} + P_{1,2} + P_{1,3} = P_1$ and $2P_1 + P_2 = P$, and the hint $\text{str}_{\tau, H_0, H_1, H_2}$ is only dependent on those P points. If $\mathcal{A}_{\text{on}}^{\text{BF-RO+GG}(P_1, P_2)}$ makes at most q_H^{on} (resp. $q_G^{\text{on}}, q_E^{\text{on}}, q_D^{\text{on}}$) queries to the random (resp. generic group, encapsulation, decapsulation) oracle, then

$$\begin{aligned}
& \Pr \left[\text{KEM}_{\mathcal{A}_{\text{on}, \text{str}_{\tau, H_0, H_1, H_2}}^{\tau, (H_0, H_1, H_2), \text{cca}}, \Pi}^{\text{BF-RO+GG}(P_1, P_2)}(\lambda) = 1 \right] \leq \frac{1}{2} + \varepsilon, \text{ with} \\
& \varepsilon = \frac{3(q_G^{\text{on}} + 2q_E^{\text{on}})^2 + (5 + 2P)(q_G^{\text{on}} + 2q_E^{\text{on}}) + 4}{p - 2P(3q_G^{\text{on}} + 2q_E^{\text{on}} + 1) - (3q_G^{\text{on}} + 2q_E^{\text{on}} + 1)^2 - 2(q_G^{\text{on}} + q_E^{\text{on}})} \\
& \quad + \frac{2q_D^{\text{on}}(q_H^{\text{on}} + P) + 3q_E^{\text{on}}(q_H^{\text{on}} + q_D^{\text{on}} + P)}{2^{\lambda_1}} + \frac{(q_H^{\text{on}} + q_D^{\text{on}} + P)q_E^{\text{on}} + 2q_D^{\text{on}}}{p},
\end{aligned}$$

where the randomness is taken over the selection of τ and the random coins of $\mathcal{A}_{\text{on}}^{\text{BF-RO+GG}(P_1, P_2)}$.

Proof of Theorem 10. We use a hybrid argument to prove the CCA security of PSEC-KEM in the BF-ROM+GGM. The hybrids work the same as the proof of Theorem 17 during the online phase, except that the distinguisher $\mathcal{D} = (\mathcal{D}_{\text{pre}}, \mathcal{D}_{\text{on}})$ in the preprocessing phase gets the hint generated from $\mathcal{A}_{\text{pre}}^{\text{BF-RO+GG}(P_1, P_2)}$. In particular, $\mathcal{A}_{\text{pre}}^{\text{BF-RO+GG}(P_1, P_2)}$ fixes

- $P_{1,1}$ input/output pairs $(\{(h_{1,1}, H_0(h_{1,1})), \dots, (h_{1,P_{1,1}}, H_0(h_{1,P_{1,1}}))\})$ of H_0 ,
- $P_{1,2}$ input/output pairs $(\{(h_{2,1}, H_1(h_{2,1})), \dots, (h_{2,P_{1,2}}, H_1(h_{2,P_{1,2}}))\})$ of H_1 ,
- $P_{1,3}$ input/output pairs $(\{(h_{3,1}, H_2(h_{3,1})), \dots, (h_{3,P_{1,3}}, H_2(h_{3,P_{1,3}}))\})$ of H_2 , and
- P_2 input/output pairs $(\{(t_1, \tau(t_1)), \dots, (t_{P_2}, \tau(t_{P_2}))\})$ of τ .

Furthermore, for each fixed RO input/output pair $(h_j, H_i(h_j))$ for $i = 0, 1, 2$, $\mathcal{D}_{\text{pre}}$ first parses $h_j = I_j \| I'_j$ where I_j is a bitstring of length ℓ . If I_j is a fresh ℓ -bit string, i.e., $I_j \neq \tau(t_k)$ for all $k \in [P_2]$, then $\mathcal{D}_{\text{pre}}$ picks a value for $\tau^{-1}(I_j) = r_j$ where the labeling map τ will be sampled later. Note that this is possible because the labeling map τ has not been sampled yet. Finally, $\mathcal{D}_{\text{pre}}$

additionally fixes the input/output pairs of τ such as $\{(r_1, I_1), \dots, (r_{P_1}, I_{P_1})\}$. This leads to having $P_{1,1} + P_{1,2} + P_{1,3} = P_1$ fixed RO points and $P_1 + P_2$ fixed points of τ for the distinguisher, having in total $P_1 + (P_1 + P_2) = P$ fixed points as the hint for the online phase. For completeness, we restate the hybrids below, along with the changes due to the preprocessing setting.

Hybrid H_0 .

This is the original CCA indistinguishability game $\text{KEM}_{\mathcal{A}_{\text{on}, \text{str}_{\tau, H_0, H_1}}^{\tau, H_0, H_1, \text{cca}}, \Pi}^{\text{BF-RO+GG}(P_1, P_2)}(\lambda)$ for PSEC-KEM with the challenge bit $b = 0$. In particular, whenever $\mathcal{D}_{\text{on}}$ submits a query $\text{Encaps}_b(\text{pk})$, $\mathcal{D}$ always gets the output $\text{Encaps}(\text{pk}, 1^\lambda)$. $\mathcal{D}_{\text{on}}$ is additionally given access to $\text{Decaps}'_{\text{sk}}(\cdot)$ as defined in [Section 5](#).

Hybrid H_1 .

Hybrid H_1 works almost identically to H_0 , but it replaces the oracle $\text{Decaps}'_{\text{sk}}(\cdot)$ with an oracle named $\text{SimDecaps}_{\text{sk}}(\cdot)$. Let D_{H_0} (resp. D_{H_2}) be the database of input/output pairs that contains (1) the random oracle H_0 (resp. H_2) that has been queried by $\mathcal{D}_{\text{on}}$, and (2) the fixed points of H_0 (resp. H_2). Then for an input ciphertext c , $\text{SimDecaps}_{\text{sk}}(c)$ works as follows:

- If $c \notin \text{resp}_{\text{pk}}$ and if there exists a tuple (r, α, z, c_1) such that
 - $(r, \alpha) \in D_{H_0}$,
 - $((\tau(\alpha) \parallel \tau(\alpha x)), z) \in D_{H_2}$,
 - $c = (\tau(\alpha), c_1)$, and
 - $r = c_1 \oplus z$,
 then return $H_1(r)$.
- Otherwise, return $\perp$.

Indistinguishability of H_0 and H_1 . Two hybrids are perfectly indistinguishable unless one of the following bad events occurs for some round $i \leq q_{\text{D}}^{\text{on}}$:

- (1) $\text{Decaps}'_{\text{sk}}(c_i)$ does not return $\perp$ but $(r_i, \alpha_i) \notin D_{H_0}$ where the values $c_i = (c_i^1, c_i^2)$, $z_i = H_2(c_i^1 \parallel \text{Pow}(c_i^1, x))$, $r_i = c_i^2 \oplus z_i$, and finally $\alpha_i = H_0(r_i)$ are as computed by Decaps' .
- (2) $\text{Decaps}'_{\text{sk}}(c_i)$ does not return $\perp$ and $(r_i, \alpha_i) \in D_{H_0}$ and $((\tau(\alpha_i) \parallel \tau(\alpha_i x)), z_i) \notin D_{H_2}$ where the values $c_i = (c_i^1, c_i^2)$, $z_i = H_2(c_i^1 \parallel \text{Pow}(c_i^1, x))$, $r_i = c_i^2 \oplus z_i$, and finally $\alpha_i = H_0(r_i)$ are as computed by Decaps' .

Let event (1) be $\text{Lucky}_{1,i}$ and event (2) be $\text{Lucky}_{2,i}$ (those events are defined for round i). And let $\text{Lucky}_1 := \bigcup_{i=1}^{q_{\text{D}}^{\text{on}}} \text{Lucky}_{1,i}$ (resp. $\text{Lucky}_2 := \bigcup_{i=1}^{q_{\text{D}}^{\text{on}}} \text{Lucky}_{2,i}$) be the event that the corresponding bad event occurs for *some* round. First observe that if neither event Lucky_1 or Lucky_2 occurs then $\text{SimDecaps}(c_i)$ will perfectly simulate $\text{Decaps}'_{\text{sk}}(c_i)$. If $c_i \notin \text{resp}_{\text{pk}}$ then both oracles output $\perp$. Otherwise $\text{Decaps}'_{\text{sk}}(c_i)$ computes $c_i = (c_i^1, c_i^2)$, $z_i = H_2(c_i^1 \parallel \text{Pow}(c_i^1, x))$, $r_i = c_i^2 \oplus z_i$, and $\alpha_i = H_0(r_i)$. If $\text{Decaps}'_{\text{sk}}(c_i)$ outputs $H_1(r)$ instead of $\perp$ (i.e., verification that $\text{Pow}(g, \alpha_i) = c_i^1$ or equivalently $\alpha_i = \tau^{-1}(c_i^1)$ passes) then we can conclude that both $((\tau(\alpha_i) \parallel \tau(\alpha_i x)), z_i) \in D_{H_2}$ and $(r_i, \alpha_i) \in D_{H_0}$ since neither event $\text{Lucky}_{1,i}$ or $\text{Lucky}_{2,i}$ occurs. But in this case $\text{SimDecaps}(c_i)$ will also output $H_1(r)$. Similarly, if $\text{Decaps}'_{\text{sk}}(c_i)$ outputs $\perp$ then $\text{SimDecaps}(c_i)$ will also output $\perp$.

Thus, we observe that the distinguishing probability between two hybrids is upper-bounded by $\Pr[\text{Lucky}_1] + \Pr[\text{Lucky}_2]$.

It is easy to see that $\Pr[\text{Lucky}_{1,i}] = \Pr[H_0(r_i) = \tau^{-1}(c_i^1)] \leq \frac{1}{p}$ since $H_0(r_i) \in \mathbb{Z}_p$ can be viewed as a uniformly random integer that is selected after the query to Decaps' is fixed i.e., once $\tau^{-1}(c_i^1)$ is fixed. Union bounding over all q_D^{on} queries to Decaps' we have $\Pr[\text{Lucky}_1] \leq \frac{q_D^{\text{on}}}{p}$.

Similarly, to analyze $\Pr[\text{Lucky}_{2,i}]$ observe that $z_i = H_2(c_i^1 \parallel \text{Pow}(c_i^1, x))$ (and thus $r_i = c_i^2 \oplus z_i$) can be viewed as uniformly random strings that are selected after the query to Decaps' since we had $((\tau(\alpha_i) \parallel \tau(\alpha_i x)), z_i) \notin D_{H_2}$. However, for the event to occur we must also have $(r_i, \alpha_i) \in D_{H_0}$. Thus,

$$\Pr[\text{Lucky}_{2,i}] \leq \Pr[(r_i, \alpha_i) \in D_{H_0}] \leq \frac{|D_{H_0}|}{2^{\lambda_1}} \leq \frac{P + q_H^{\text{on}}}{2^{\lambda_1}}.$$

Union bounding over all q_D^{on} queries to Decaps' we have $\Pr[\text{Lucky}_2] \leq \frac{q_D^{\text{on}}(P + q_H^{\text{on}})}{2^{\lambda_1}}$. Taken together,

$$\begin{aligned} |\Pr[\mathcal{D}^{H_0} = 1] - \Pr[\mathcal{D}^{H_1} = 1]| &\leq \Pr[\text{Lucky}_1] + \Pr[\text{Lucky}_2] \\ &\leq \frac{q_D^{\text{on}}}{p} + \frac{q_D^{\text{on}}(q_H^{\text{on}} + P)}{2^{\lambda_1}}. \end{aligned}$$

Hybrid H_2 .

This is the same as Hybrid H_1 but with the challenge bit $b = 1$.

Indistinguishability of H_1 and H_2 . Intuitively, if an attacker $\mathcal{D}$ can distinguish between H_1 and H_2 , then $\mathcal{D}$ can also be used to win the CPA indistinguishability game for PSEC-KEM since $\mathcal{D}$ never queries the real decapsulation oracle — it only gets to query the simulated oracle. The algorithm $\mathcal{D}'$ will simply simulate $\mathcal{D}$ and use the SimDecaps oracle to respond to any query to Decaps. The advantage of $\mathcal{D}'$ in CPA indistinguishability game is equivalent to the advantage of distinguisher $\mathcal{D}$ in distinguishing H_1 and H_2 . If $\mathcal{D}$ makes q_H random oracle queries and q_D decapsulation queries, then $\mathcal{D}'$ will make at most $q_H' = q_H + q_D$ random oracle queries and 0 decapsulation queries.

Thus, with similar reasoning to [Theorem 17](#), distinguishing two hybrids can be upper bounded by the probability of winning the CPA indistinguishability game for PSEC-KEM in the BF-ROM+GGM, with $(q_H^{\text{on}})' = q_H^{\text{on}} + q_D^{\text{on}}$ random oracle queries. Hence, by [Theorem 9](#),

$$\begin{aligned} &|\Pr[\mathcal{D}^{H_1} = 1] - \Pr[\mathcal{D}^{H_2} = 1]| \\ &\leq \left| \Pr \left[\text{KEM}_{\mathcal{A}_{\text{BF-ROM+GGM}(P_1, P_2)}, \Pi}^{\tau, H_0, H_1, \text{cpa}}(\lambda) = 1 \right] - \frac{1}{2} \right| \\ &\leq \frac{3(q_G^{\text{on}} + 2q_E^{\text{on}})^2 + (5 + 2P)(q_G^{\text{on}} + 2q_E^{\text{on}}) + 4}{p - 2P(3q_G^{\text{on}} + 2q_E^{\text{on}} + 1) - (3q_G^{\text{on}} + 2q_E^{\text{on}} + 1)^2 - 2(q_G^{\text{on}} + q_E^{\text{on}})} \\ &\quad + \frac{3(q_H^{\text{on}} + q_D^{\text{on}} + P)q_E^{\text{on}}}{2^{\lambda_1}} + \frac{(q_H^{\text{on}} + q_D^{\text{on}} + P)q_E^{\text{on}}}{p}. \end{aligned}$$

Hybrid H_3 .

This is the same as Hybrid H_0 but with the challenge bit $b = 1$. That is, this is the original CCA indistinguishability game $\text{KEM}_{\mathcal{A}_{\text{on}, \text{str}, \tau, H_0, H_1}^{\tau, H_0, H_1, \text{cca}}^{\text{BF-RO+GG}(P_1, P_2)}, \Pi}(\lambda)$ for PSEC-KEM with the challenge bit $b = 1$.

Indistinguishability of H_2 and H_3 . The distinguishing probability between H_2 and H_3 is the same as the distinguishing probability between H_0 and H_1 since the only difference between H_2 and H_3 is that we replace the simulated oracle SimDecaps with the original oracle Decaps' . Note: Since the challenge bit is now $b = 1$ in H_2 and H_3 the encapsulation oracle will now generate random keys. However, because both oracles SimDecaps and Decaps' will reject outputs of the encapsulation oracle the change in the challenge bit will not affect the analysis. Hence,

$$\begin{aligned} |\Pr[\mathcal{D}^{H_2} = 1] - \Pr[\mathcal{D}^{H_3} = 1]| &= |\Pr[\mathcal{D}^{H_0} = 1] - \Pr[\mathcal{D}^{H_1} = 1]| \\ &\leq \frac{q_{\text{D}}^{\text{on}}}{p} + \frac{q_{\text{D}}^{\text{on}}(q_{\text{H}}^{\text{on}} + P)}{2^{\lambda_1}}. \end{aligned}$$

Putting all together, we have

$$\begin{aligned} \Pr \left[\text{KEM}_{\mathcal{A}_{\text{on}, \text{str}, \tau, H_0, H_1}^{\tau, H_0, H_1, \text{cca}}^{\text{BF-RO+GG}(P_1, P_2)}, \Pi}(\lambda) = 1 \right] &\leq \frac{1}{2} + |\Pr[\mathcal{D}^{H_0} = 1] - \Pr[\mathcal{D}^{H_3} = 1]| \\ &\leq \frac{1}{2} + \sum_{i=0}^2 |\Pr[\mathcal{D}^{H_i} = 1] - \Pr[\mathcal{D}^{H_{i+1}} = 1]| \\ &\leq \frac{1}{2} + \frac{3(q_{\text{G}}^{\text{on}} + 2q_{\text{E}}^{\text{on}})^2 + (5 + 2P)(q_{\text{G}}^{\text{on}} + 2q_{\text{E}}^{\text{on}}) + 4}{p - 2P(3q_{\text{G}}^{\text{on}} + 2q_{\text{E}}^{\text{on}} + 1) - (3q_{\text{G}}^{\text{on}} + 2q_{\text{E}}^{\text{on}} + 1)^2 - 2(q_{\text{G}}^{\text{on}} + q_{\text{E}}^{\text{on}})} \\ &\quad + \frac{2q_{\text{D}}^{\text{on}}(q_{\text{H}}^{\text{on}} + P) + 3q_{\text{E}}^{\text{on}}(q_{\text{H}}^{\text{on}} + q_{\text{D}}^{\text{on}} + P)}{2^{\lambda_1}} + \frac{(q_{\text{H}}^{\text{on}} + q_{\text{D}}^{\text{on}} + P)q_{\text{E}}^{\text{on}} + 2q_{\text{D}}^{\text{on}}}{p}. \quad \square \end{aligned}$$

H Operations in RPM/ICM

In the RPM, all parties have oracle access to a uniform permutation P and its inverse P^{-1} , i.e., $P^{-1}(P(x)) = x$. The RPM has been used as a framework to analyze key-alternating ciphers (e.g., AES [oST01]), Even-Mansour cipher [EM93], and sponge-based constructions such as SHA-3 [oSN14, Dwo15]. In the ICM which goes back to Shannon [Sha49], we assume that the block cipher is a random permutation for *every* key K . In particular, for any fixed key K , the function $E_K(x) := E(K, x)$ is a truly random permutations with inverse $E_K^{-1}(y) := E^{-1}(K, y)$, i.e., $E_K^{-1}(E_K(x)) = x$. In the ICM, all parties (honest parties and adversaries included) have oracle access to the block cipher $E(\cdot, \cdot)$ and its inverse $E^{-1}(\cdot, \cdot)$. If Alice and Bob share a secret key K then they can both evaluate the permutation $E_K(\cdot)$ and its inverse $E_K^{-1}(\cdot)$, but a party who does not have K will not be able to perform this computation using the block cipher oracles $E(\cdot, \cdot)$ or $E^{-1}(\cdot, \cdot)$.